ORCA / CMUX / TMUX · CLAUDE CODE · MATTPOCOCK SKILLS · GITHUB

티켓 무인 루프 가이드북

스킬로 쪼갠 GitHub 이슈를 Orca·cmux·tmux 워크트리에서
한 건씩 자동으로 구현·검증·병합하고, 멈추면 이어받는 방법

1. 개요
2. 워크트리와 백엔드 — Orca, cmux, tmux
3. 구성 요소
4. 환경 구축과 설치
5. 티켓 준비
6. 루프 실행
7. 티켓 한 건의 생애
8. 워커 안에서 일어나는 일
9. 실전 시나리오
10. 중단과 복구
11. 안전 장치와 운영 수칙
12. FAQ
13. 부록 — 치트시트, 프롬프트·스킬 원문, 스크립트 전문, 개정 이력

대상: GitHub 저장소를 쓰는 개발자·팀, macOS·Linux·Windows(WSL2 — 4장 끝 절). 특정 언어·프레임워크를 가정하지 않는다. 운영 설명은 Orca 백엔드 기준이고, cmux(macOS)와 tmux(어디서나)로도 같은 루프를 돌린다(2.5).

함께 배포하는 파일: `ticket-loop.sh` (부록 D에 전문 수록). 설치·업데이트는 `ticket-loop-kit` (`iwishash/ticket-loop-kit`)의 `install.sh` 와 `/setup-ticket-loop` 스킬로 한다. 예시 저장소 `acme/todo-app` 와 예시 출력은 설명용이다.

3판 2026-10-12 (2판 2026-10-11, 초판 2026-10-08) · 실제 저장소에서 티켓 21건을 돌린 실측을 반영했다(바뀐 곳은 부록 E). 2026-10-14 보강: cmux·tmux 백엔드(2.5), Windows(4장 끝). 2026-10-15: 터미널 로그 영어화(부록 E).

CLI 플래그와 응답 형식은 버전에 따라 다를 수 있으니 `orca --help` · `cmux --help` 로 확인한다(확인 버전 Orca 1.4.224, cmux 0.65, tmux 3.8).

1. 개요

1.1 해결하려는 문제

코딩 에이전트(Claude Code 등)는 잘 정의된 작업 하나를 사람보다 빨리 구현한다. 그런데 작업이 여러 개가 되면 병목은 에이전트가 아니라 **에이전트를 돌리는 사람과 작업 방식**에 생긴다. 대화형으로 한 세션에서 여러 작업을 이어 가면 다음 문제가 반복된다.

#	문제	증상
P1	**사람이 병목이다**	에이전트는 티켓 하나를 몇십 분이면 끝내지만, 다음 티켓을 주고 결과를 확인하고 PR을 여는 일에 사람이 계속 붙어 있어야 한다. 사람이 자리를 비우면(밤, 회의, 다른 일) 에이전트도 멈춘다. 에이전트 시간 5시간짜리 일이 달력으로는 며칠이 된다.
P2	**긴 세션은 오염된다**	한 세션에서 티켓을 이어 가면 앞 티켓의 가정, 임시 파일, 커밋하지 않은 변경이 다음 티켓에 섞인다. `/implement`가 세션당 티켓 하나를 전제로 하는 이유지만, 매번 `/clear`하고 다시 시작하는 일도 사람 몫이다.
P3	**한 체크아웃을 나눠 쓰면 사고가 난다**	사람과 에이전트, 또는 에이전트 둘이 같은 디렉터리에서 일하면 `git stash` · `--amend` · `checkout` 하나가 상대의 작업을 지운다. 테스트 결과도 누구 변경 때문인지 알 수 없다.
P4	**무엇이 끝났는지 대화 안에만 남는다**	어느 티켓이 어디까지 됐는지가 채팅 기록에만 있다. 다른 사람은 다시 읽어야 알고, 세션을 잃으면 같이 사라진다.
P5	**검증 없이 병합하게 된다**	에이전트가 "테스트 통과했다"고 말하면 믿고 합치기 쉽다. 사람이 PR마다 CI를 기다려 확인하는 것은 P1을 다시 만든다.
P6	**멈추면 처음부터다**	에이전트가 질문하고 멈추거나 세션이 끊기면, 어디까지 했는지 복원하기 어려워 다시 시작하게 된다.

이 방식은 위 여섯 가지를 다음 장치로 푼다. 각 장치는 뒤 장에서 자세히 다룬다.

장치	푸는 문제	장
티켓을 **GitHub 이슈 + 라벨 + 선행 관계**로 쓴다	P4 — 상태가 GitHub에만 있다. P1 — 루프가 다음 할 일을 스스로 고른다.	5
루프 스크립트가 **사람 대신 다음 티켓을 집어 워커에 준다**	P1 — 사람은 티켓을 준비해 두고 자리를 비운다.	6, 7
티켓마다 **새 워크트리 + 새 에이전트 프로세스**	P2 — 컨텍스트와 작업 디렉터리가 매번 비어 있다. P3 — 메인 체크아웃과 다른 워커를 건드리지 않는다.	2
CI 녹색일 때만 병합, 실패는 제한 횟수 수리 후 사람에게. 워커가 못 돌리는 무거운 검증은 **병합 직후 루프가 한 번** 돌린다	P5 — 병합 판정을 에이전트 말이 아니라 CI와 루프가 한다.	7, 11
멈추면 **워크트리를 남기고** 상태판에 표시	P6 — 어디서 왜 멈췄는지 그대로 남아 이어받을 수 있다.	10
스킬(`/to-tickets` · `tdd` · `ticket-review` · `pr`)이 **티켓 품질과 구현 규율**을 맞추고, 구현 절차는 **저장소가 소유한 `implement` 스킬 한 곳**에 둔다	P5의 바탕 — 테스트 먼저, 자체 리뷰, 근거가 있는 PR 본문. 사람이 직접 돌릴 때와 무인 루프가 같은 절차를 쓴다.	3, 8

1.2 이 방식이 맞지 않는 경우

- **결정이 남아 있는 작업.** 워커는 질문할 수 없다. 설계·사업 결정이 필요한 일은 먼저 사람이 결정해 티켓에 적어야 한다(`/grill-with-docs`가 이 단계를 돕는다).
- **크고 모호한 작업.** 한 PR 크기로 쪼개지지 않는 일은 루프에 넣지 않는다. 쪼개는 것이 `/to-tickets`의 일이다.
- **PR에서 CI가 돌지 않는 저장소.** 루프는 CI 결과로만 병합을 판단한다. CI가 없으면 자동 병합의 근거가 없다(`--no-merge`로 PR까지만 쓸 수는 있다). 워크플로에 경로 필터가 있어 일부 PR에서 CI가 돌지 않는다면 그 필터를 루프에 알려 준다(`CI_SKIP_REGEX` · `CI_SKIP_CMD`, 6장). 필터가 없으면 비워 둔다.
- **동시에 수십 개를 돌려야 하는 경우.** 이 루프는 직렬이다. 병렬이 필요하면 12장 FAQ의 `/implement-spec`을 본다.

1.3 작업 방식

이 가이드는 다음 작업 방식을 처음부터 갖추고 운영하는 방법을 다룬다.

1. 사람이 Claude Code와 대화하며 기능을 정리하고, **mattpocock 스킬**(`/to-spec`, `/to-tickets`, `/triage`)로 작업을 **GitHub 이슈 여러 개**로 쪼갠다. 이슈 사이의 순서는 GitHub의 선행 관계(blocked by)로 건다.
2. **루프 스크립트** `ticket-loop.sh`가 준비된 이슈를 순서대로 하나씩 집는다. 이슈마다 **워크트리**를 새로 만들고(Orca가 만들거나, cmux·tmux 백엔드면 루프가 git으로 만든다), 그 안에서 Claude Code를 띄워 `/implement`를 시킨다(워크트리와 백엔드는 2장).
3. 워커(그 Claude Code)가 테스트 우선으로 구현하고 자체 리뷰를 거쳐 PR을 연다. 루프는 CI가 통과하면 병합하고, (설정했다면) 병합된 기본 브랜치로 무거운 검증을 한 번 돌린 뒤, 워크트리를 지우고 다음 이슈로 간다.
4. 한 묶음이 끝나면 사람이 PR 본문과 루프 로그의 티켓별 요약 줄을 읽고 `/retro`로 환경(스킬·규칙·검사)을 고친다. 다음 묶음은 고친 환경에서 돈다.

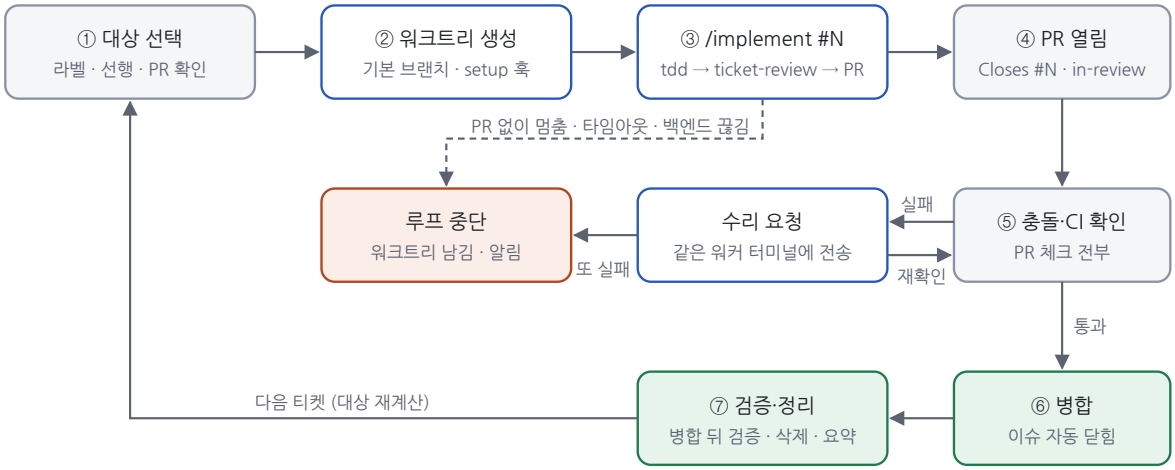


그림 1. 티켓 한 건의 흐름. 회색은 GitHub, 파랑은 백엔드(Orca·cmux·tmux)·Claude Code, 초록은 성공 경로, 빨강은 사람이 이어받는 지점이다.

1.4 세 가지 설계 원칙

원칙	뜻과 효과
티켓 하나 = 워크트리 하나 = 새 에이전트	메인 체크아웃을 건드리지 않고, 티켓마다 깨끗한 기본 브랜치에서 시작한다. 에이전트 컨텍스트도 매번 비어 있어서 앞 티켓의 가정이 다음 티켓으로 넘어가지 않는다.
상태는 GitHub에만 둔다	루프는 따로 상태 파일을 두지 않는다. 무엇이 끝났고 무엇이 남았는지는 이슈 라벨·열림 여부·PR로 판단한다. 그래서 **복구의 기본은 상태를 GitHub에 맞춰 놓고 루프를 다시 실행하는 것**이다(10장).
확실하면 병합하고, 애매하면 멈춘다	CI가 녹색일 때만 병합한다. 워커 질문, 수리 실패, 연결 끊김처럼 애매한 상황에서는 루프 전체를 멈추고 워크트리를 증거로 남긴다.

1.5 사람과 도구의 역할

누가	하는 일
사람	요구사항 정리, 티켓 승인, 라벨 관리, 멈춘 티켓 이어받기, 병합된 결과 사후 검수, 회고(`/retro`)로 환경 고치기
mattpocock 스킬	스펙·티켓 작성 규칙, 구현 규율(TDD), 자체 리뷰(Standards·Spec 두 축), 회고. 구현 절차(`implement`)와 PR 본문 양식(`pr`)은 저장소가 소유한 사본을 쓴다(3장)
백엔드 (Orca · cmux · tmux)	워크트리 격리, 에이전트 터미널 실행·관찰(Orca `tui-idle` / cmux·tmux는 에이전트 훅), 상태 표시, setup 훅(cmux·tmux는 루프의 `WORKTREE_SETUP_CMD`)
`ticket-loop.sh`	대상 선택, 워커 기동, PR·CI 대기, 수리 지시, 병합, 병합 뒤 검증, 정리, 티켓별 요약 로그
GitHub + CI	단 하나의 상태 저장소이자 병합 여부를 가르는 판정자

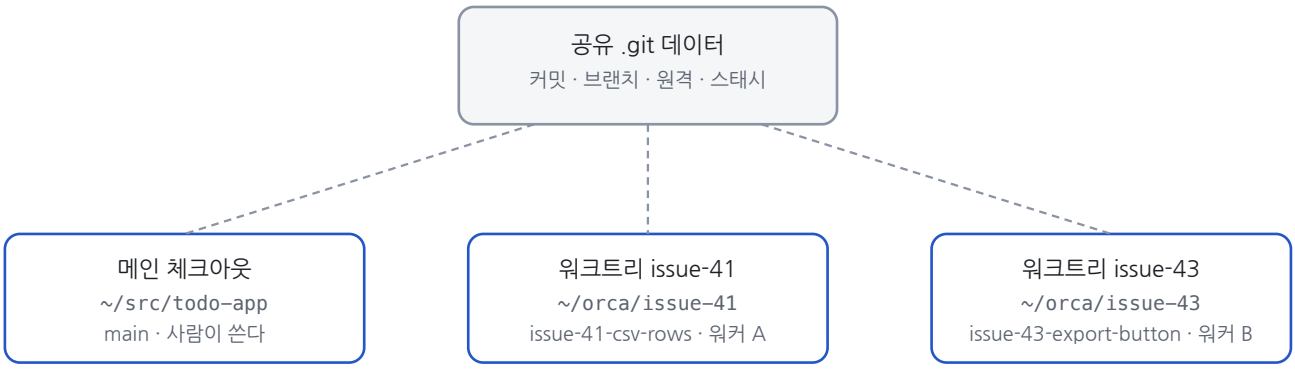
2. 워크트리와 백엔드 — Orca, cmux, tmux

워크트리는 git 기능이고, 워크트리를 만들고 그 안에서 에이전트 터미널을 돌리는 앱이 **백엔드**다. 이 책의 기본은 Orca(2.3·2.4)이고, macOS에서는 cmux로, 어떤 터미널 앱이나 SSH에서는 tmux로 같은 루프를 돌린다(2.5). 백엔드는 `.ticket-loop/loop.env`의 `BACKEND`(`orca` | `cmux` | `tmux`)로 고르고, 루프의 나머지(티켓 선택·PR·CI·수리·병합·병합 뒤 검증·기록·리포트)는 백엔드와 무관하게 같다.

2.1 git 워크트리란

보통 git 저장소 하나에는 작업 디렉터리가 하나 있고, 브랜치를 바꾸려면 `git checkout`으로 그 디렉터리의 파일을 통째로 갈아 끼운다. **워크트리**(`git worktree`)는 **같은 저장소에 작업 디렉터리를 여러 개** 두는 기능이다. 각 디렉터리는 서로 다른 브랜치를 체크아웃하고, 커밋·브랜치·원격 정보가 들어 있는 `.git` 데이터는 하나를 공유한다.

```
git worktree add ../todo-app-issue-41 -b issue-41 origin/main   # 새 디렉터리에 새 브랜치
git worktree list                                               # 디렉터리 ↔ 브랜치 대응
git worktree remove ../todo-app-issue-41                        # 디렉터리 삭제 (브랜치는 남는다)
git worktree prune                                              # 사라진 디렉터리의 등록 정보 정리
```



각 디렉터리가 다른 브랜치를 체크아웃한다 — 서로 파일을 건드리지 않는다

그림 2. 워크트리는 저장소 데이터를 공유하는 독립된 작업 디렉터리다. 같은 브랜치를 두 곳에 체크아웃할 수는 없다.

성질	뜻
파일시스템 격리	한 워크트리에서 파일을 고치거나 테스트가 임시 파일을 만들어도 다른 워크트리에는 보이지 않는다.
저장소 데이터 공유	커밋은 즉시 다른 워크트리에서 보인다(`git log`, `git merge`). 복제(clone)와 달리 원격을 두 번 받지 않고, 디스크도 작업 파일만큼만 더 쓴다.
브랜치당 한 곳	같은 브랜치를 두 워크트리에 동시에 체크아웃할 수 없다. 메인 체크아웃이 `main`을 쓰고 있으면 워커는 `main`을 체크아웃하지 못한다. 그래서 워커 프롬프트가 "`main`을 체크아웃하지 말고 `origin/main`에서 브랜치를 만들라"고 한다.
추적되지 않는 파일은 없다	새 워크트리에는 Git이 추적하는 파일만 있다. `.env`, `node_modules/`, 빌드 산출물은 없으므로 setup 훅으로 만든다(4장 9단계).

2.2 이 조합에서 워크트리가 하는 일

효과	설명
사람의 작업을 보호한다	워커는 메인 체크아웃을 전혀 건드리지 않는다. 루프가 도는 동안에도 사람은 자기 디렉터리에서 다른 브랜치 작업을 계속할 수 있다. 에이전트가 `git checkout`·`git stash`·`git reset`을 잘못 실행해도 피해는 그 워크트리 안에 머문다.
티켓마다 같은 출발점	워크트리는 매번 `origin/main` 최신 커밋에서 새로 만들어진다. 앞 티켓이 남긴 미커밋 변경, 임시 파일, 캐시가 다음 티켓에 섞이지 않는다. 재현 가능한 실행의 바탕이다.
에이전트 컨텍스트 격리	워크트리마다 Claude Code 프로세스가 새로 뜬다. 대화 맥락이 비어 있으니 앞 티켓에서 내린 잘못된 가정이 이어지지 않고, 컨텍스트 창이 티켓 하나에 온전히 쓰인다.
멈춘 자리가 그대로 남는다	루프가 멈추면 워크트리를 지우지 않는다. 커밋되지 않은 변경, 터미널 기록, 브랜치가 그 디렉터리에 그대로 있어 사람이 열어 보고 이어갈 수 있다(10장). 체크아웃 하나로 일했다면 다음 작업이 이 흔적을 덮어썼을 것이다.

필요하면 병렬로 넓힐 수 있다	이 루프는 직렬이지만, 워크트리 구조 자체는 동시에 여러 워커를 돌리는 데 쓰인다(`/implement-spec` , 12장 FAQ). 직렬에서 병렬로 갈 때 작업 방식을 바꿀 필요가 없다.
정리가 명확하다	티켓이 끝나면 디렉터리를 통째로 지운다. 반쯤 남은 상태가 없다.

> **비용.** 워크트리마다 의존성 설치(`npm ci` 등)가 다시 일어나므로 setup 훅 시간과 디스크를 쓴다. 패키지 관리자의 전역 캐시(pnpm store, npm cache 등)를 쓰면 대부분 줄어든다. 디스크가 걱정되면 루프가 끝난 워크트리를 바로 지우는 기본 동작을 유지하고, `--no-merge` 로 남긴 워크트리는 검토 후 바로 정리한다.

2.3 다른 도구와 비교했을 때 Orca의 강점

워크트리 자체는 git 기능이므로 어떤 도구로도 쓸 수 있다. 차이는 **"워크트리 만들기 → 에이전트 띄우기 → 상태 보기 → 지시 보내기 → 정리"를 얼마나 자동화할 수 있는가**에 있다. 무인 루프에는 이 다섯 가지가 전부 명령 한 줄로 되어야 한다.

필요한 것	**터미널 + `git worktree` 직접**	**일반 IDE + 에이전트 확장** (VS Code·Cursor·JetBrains 등)	**Orca**
워크트리 생성과 에이전트 기동	스크립트로 가능하나 디렉터리·브랜치·setup·에이전트 실행을 직접 엮어야 한다	창 하나가 체크아웃 하나다. 워커마다 창을 따로 열고 워크트리는 손으로 만든다	`worktree create --agent claude --prompt …` 한 번에 디렉터리·브랜치·setup 훅·에이전트 기동·첫 프롬프트까지
에이전트가 멈췄는지 알기	PTY 출력을 직접 파싱해야 한다	사람이 화면을 봐야 안다. 외부에서 물어볼 방법이 보통 없다	`terminal wait --for tui-idle` 가 "입력을 기다리는 중"을 신호로 준다. 루프의 "PR 없이 멈춤" 판정이 이 신호에 기대고 있다
에이전트에 지시 보내기	tmux `send-keys` 같은 우회가 필요하다	사람이 채팅창에 입력한다	`terminal send` 로 보내고 수락 여부를 JSON으로 받는다(충돌·CI 수리 지시)
작업 상태판	없음	창 제목과 git 상태 정도	워크트리마다 카드: 상태(`todo` → `completed`), 코멘트, 연결된 이슈·PR, 읽지 않음 표시. 밤새 돌린 뒤 어디서 멈췄는지 사이드바에서 바로 보인다
스크립트 친화성	높음(전부 셸)	낮음. 확장 API는 있어도 에이전트 세션을 외부에서 제어하는 표면은 드물다	거의 모든 명령이 `--json` 을 지원하고 `ok` / `error` 구조가 일정하다
여러 종류의 에이전트	각자 실행 방법을 알아야 한다	확장마다 다르다. 보통 한 제품에 묶인다	Claude Code, Codex, OpenCode, Pi 등을 같은 방식으로 띄운다. 워커를 바꿔도 루프는 그대로다
setup 훅	직접 작성	없음(태스크 설정으로 흉내)	`orca.yaml` 의 `scripts.setup` 이 워크트리마다 실행된다. `ORCA_ROOT_PATH` · `ORCA_WORKTREE_PATH` 로 메인 체크아웃의 파일을 복사한다
이어받기	디렉터리로 `cd` 해서 에이전트를 다시 띄운다	그 디렉터리를 새 창으로 연다	사이드바 카드를 클릭하면 그 워크트리의 터미널이 그대로 열린다. 워커와 나눈 대화가 보인다

요약하면, IDE는 **사람이 한 체크아웃에서 코드를 보는 도구**이고, Orca는 **여러 워크트리에서 도는 에이전트를 관리하는 도구**다. 이 루프가 Orca에서 가져다 쓰는 것은 결국 세 가지다. 한 번에 만드는 워크트리, `tui-idle` 신호, JSON으로 돌아오는 상태. cmux와 tmux에는 첫째가 없어 루프가 git으로 대신하고, 둘째는 Claude Code 훅이 보고하는 상태로 채운다(2.5).

2.4 Orca를 쓸 때 알아 둘 제약

- **데스크톱 앱이 켜져 있어야 한다.** CLI는 실행 중인 앱에 명령을 보낸다. 서버·CI에서 헤드리스로 돌리는 용도가 아니다. 앱이 꺼지면 루프는 멈추고(R4), 워크트리는 디스크에 남는다.
- **CLI 플래그는 버전을 탄다.** `--agent` · `--prompt` · `--setup` 같은 플래그는 비교적 최근에 들어왔다. `orca worktree create --help` 로 확인하고, 없으면 워크트리를 만든 뒤 `terminal create` · `terminal send` 로 나눠 한다.

- **응답 형식도 버전을 탄다 — exit 코드만 믿지 않는다.** Orca 1.4.223의 `terminal wait`는 기다리다 시간이 다 되면 `{"ok": false, "error": {"code": "timeout"}}`과 함께 **exit 1**을 돌려준다. exit 코드만 보면 "터미널에 닿지 못함"과 구별되지 않아, 초판 스크립트는 10분 넘게 일하던 워커를 두고 루프를 멈췄다. 2판 스크립트는 JSON의 `error.code`로 "아직 일하는 중"과 "닿지 못함"을 가른다(부록 D `terminal_idle`). Orca를 올린 뒤에는 `--max 1`로 한 번 지켜본다.
- **Linux에서는 `orca-ide`.** 4장 3단계 참고.
- **`tui-idle`은 추정 신호다.** 화면 출력이 멈추고 입력 프롬프트가 보이는 상태를 감지한다. 긴 테스트가 출력 없이 돌면 잠시 idle로 보이고, PR을 여는 순간에는 GitHub에 PR이 보이기 전에 idle이 먼저 잡힌다(실제 운영에서 매 티켓 관찰). 루프가 "기동 후 2분 제외, idle 뒤 30초 쉬고 PR 재확인, 화면이 바뀌었으면 횟수 초기화, 연속 3회"로 판정하는 이유다.
- **에이전트 기동이 가끔 실패한다.** Orca가 셸에 기동 명령을 넣을 때 앞 글자가 빠져(`laude …`) 셸 프롬프트에 멈춘 일이 있었다(연속 4건). 루프는 배너가 90초 안에 보이지 않으면 한 번 다시 띄운다.
- **훅 설정은 저장소 파일보다 앱 설정이 먼저다.** 저장소에 `orca.yaml`을 커밋해도 앱 설정의 명령 출처가 기본값("Local only")이면 무시된다(4장 9단계).

2.5 cmux와 tmux 백엔드

Orca 없이도 루프는 돈다. 루프가 백엔드에 시키는 일은 아홉 가지뿐이고(만들기·지우기·프롬프트 보내기·셸 입력·Ctrl-C·idle 대기·화면 읽기·상태 표시·시작 점검), 그 아홉 개를 cmux와 tmux로도 구현해 두었다. **cmux**는 macOS 전용 터미널 앱이다(Swift·AppKit, `brew install --cask cmux`). 워크스페이스마다 셸을 두고 사이드바에 lane(`todo` · `working` · `needs-attention` · `review` · `done`)을 보여 주며, Claude Code를 자기 래퍼로 띄워 훅을 붙인다. **tmux**는 터미널 멀티플렉서라 에이전트를 전혀 모른다. 그래서 루프가 Claude Code 훅 설정을 직접 써서 `--settings`로 붙이고, 훅이 상태를 파일에 쓰게 한다. iTerm·Warp·Ghostty·Windows Terminal 같은 터미널 앱은 바깥에서 창을 조종할 CLI가 없어 백엔드가 아니다 — 그 안에서 tmux 백엔드를 돌린다. 두 백엔드 모두 **워크트리를 만드는 명령과 idle까지 블로킹하는 명령이 없으므로** 루프가 그 둘을 직접 한다.

루프가 하는 일	Orca	cmux	tmux
워크트리 만들기	`orca worktree create`가 디렉터리·브랜치·`orca.yaml` setup을 한 번에	루프가 `git worktree add`로 `WORKTREE_DIR`(기본 `~/.cache/ticket-loop/<owner-repo>/worktrees/issue-N`)에 만들고, `loop.env`의 `WORKTREE_SETUP_CMD`(의존성 설치·env 복사)를 그 안에서 돌린다	
에이전트 띄우기	같은 호출의 `--agent claude --prompt …`	`cmux workspace create --cwd … --command "claude … \"$(cat 프롬프트파일)\""`. 워크스페이스 셸의 PATH 앞에 cmux의 claude 래퍼가 있어 훅이 자동으로 붙는다	`TMUX_SESSION`(기본 `ticket-loop`, tmux 안에서 돌리면 그 세션)에 창을 열고 셸에 `claude … --settings <훅 설정> "$(cat 프롬프트파일)"`을 친다. 에이전트가 죽어도 셸이 남아 재기동을 받는다
멈췄는지 알기	`terminal wait --for tui-idle`(화면 추정, 블로킹)	`cmux rpc surface.input_state`의 `lifecycle`(`running` · `idle` · `needsInput` · `unknown`)을 5초마다 읽는다	훅이 `~/.cache/ticket-loop/<owner-repo>/tmux/state/issue-N`에 쓴 한 단어(`UserPromptSubmit`→running, `Stop`→idle, `Notification`→needsInput, `SessionEnd`→exited)를 5초마다 읽는다. 창이 사라지면 "닿지 못함"
지시 보내기 / 화면 읽기	`terminal send` / `terminal read`	`cmux paste --surface H --submit "…"`(입력란에 초안이 있으면 거부). 15초 안에 `lifecycle`이 idle에서 벗어나거나 프롬프트가 화면에 찍히면 수락된 것 / `cmux read-screen --surface H --lines 25`. 보내기 전에 `agent-hibernation wake`	`tmux send-keys -l` + `Enter`. 보내기 전에 상태 파일에 `sent`를 쓰고 15초 안에 running이 되면 수락된 것 / `capture-pane -p` 마지막 25줄
상태판	워크스페이스 상태 + 코멘트 + PR 연결 + 읽지 않음	lane(`working` → `review` → `done`, 중단은 `todo`) + `cmux pr` + `cmux log`	창 이름 뒤에 `[working]`·`[review]`·`[done]`·`[needs-attention]`. `tmux attach -t ticket-loop`로 본다
정리	`orca worktree rm --force`	`cmux workspace close ID --force`(에이전트 프로세스가 살아 있어 `--force` 필요) 뒤 `git worktree remove --force`	`tmux kill-window` 뒤 `git worktree remove --force`

실행 조건	Orca 앱 실행 중, 저장소 등록	**루프를 cmux 터미널 안에서 실행**한다(소켓 모드 기본값 `cmuxOnly` — 밖에서는 `cmux identify` 가 거부된다). Settings의 Claude Code 통합이 켜져 있어야 한다	`tmux` 만 있으면 된다. 어느 터미널 앱, SSH 세션에서도. `AGENT=claude|codex` 만 받는다(idle을 훅으로 알기 때문)

사람이 느끼는 차이는 세 가지다. 워크트리가 저장소 밖 캐시 디렉터리에 생기고, setup 훅 자리에 `WORKTREE_SETUP_CMD` 가 있으며, 이어받을 때 Orca 카드 대신 cmux 사이드바의 워크스페이스나 tmux 창(`issue-N`)을 연다. 이 책에서 "Orca 터미널을 연다"는 cmux에서는 "그 워크스페이스를 연다", tmux에서는 `tmux attach -t ticket-loop` 뒤 그 창으로 읽는다(10장, 부록 A).

- **메인 체크아웃을 Claude Code가 신뢰하고 있어야 한다.** Claude Code는 새 워크트리의 폴더 신뢰를 **메인 체크아웃 경로**로 판정한다. 신뢰 전이면 워커가 신뢰 대화상자에 멈추고 훅도 프롬프트도 돌지 않는다(더미 저장소 실측). cmux·tmux의 시작 점검이 `~/.claude.json` 의 `projects[<메인 체크아웃>].hasTrustDialogAccepted` 를 보고 아니면 멈춘다 — 메인 체크아웃에서 `claude` 를 한 번 열어 "Yes, I trust this folder"를 고른다. Orca는 워크스페이스마다 신뢰를 기록해 주므로 보지 않는다.
- **워크트리 준비 명령은 `loop.env` 에 커밋한다.** 예: `WORKTREE_SETUP_CMD="${WORKTREE_SETUP_CMD:-pnpm install --frozen-lockfile --prefer-offline}"` . env 파일 복사까지 하려면 `&& cp \"\$LOOP_ROOT_PATH/.env.local\" .` 처럼 잇는다 — 루프가 새 워크트리 안에서 `LOOP_ROOT_PATH` (메인 체크아웃)· `LOOP_WORKTREE_PATH` 를 환경변수로 주고 `bash -c` 로 실행한다(`$` 는 `loop.env` 를 읽을 때 풀리지 않도록 `\$` 로 적는다). 실패하면 그 티켓에서 멈춘다. 키트의 `npx … init` 에서 Orca 아닌 백엔드를 고르면 이 줄의 자리를 만들어 둔다.
- **`Stop` 훅은 메인 에이전트의 턴이 끝날 때마다 온다.** 백그라운드 서브에이전트(`ticket-review`)를 기다리는 동안 한 번 idle로 보인다(tmux 실측 1회, "worker idle, no PR (1/3)"). 서브에이전트가 돌아오면 Claude Code가 스스로 턴을 열어 running이 되고, 공통부는 화면이 바뀌면 연속 횟수를 0부터 세므로 멈추지 않는다. cmux의 `lifecycle` 도 같은 훅에 기대므로 같은 성질이다. 수리 대기도 0.9.3부터 같은 규칙이다(7장 4단계).
- **cmux에서 프롬프트를 붙여 넣은 직후의 `lifecycle` 은 직전 턴의 값이다.** 대기 중이던 워커에 붙여 넣으면 새 턴이 시작돼 훅이 running을 보고하기 전까지 `idle` 이 그대로 돌아온다. 0.9.2까지는 보내자마자 대기 상태를 읽어, rental-portal #251(2026-10-20)에서 CI 수리 요청 5초 뒤 "수리 뒤 새 커밋 없음"으로 멈췄다 — 워커는 2분 뒤 수리 커밋을 push했고, 그사이 뜬 수습 세션과 같은 PR 브랜치에 함께 push했다. 0.9.3부터 cmux 보내기는 tmux처럼 수락을 확인한다(`SEND_ACCEPT_SEC`).
- **cmux에서 `lifecycle` 이 `unknown` 만 돌아오면** Settings의 Claude Code 통합이 꺼져 있거나 래퍼를 거치지 않은 claude가 뜬 것이다. 워커는 멀쩡히 일하는데 루프는 기동 유예(2분) 뒤 idle 대기를 되풀이하다(진행 로그의 커밋 줄은 찍히는데 PR이 열린 뒤에도 "PR #… 열림"으로 넘어가지 않는다) 티켓 제한 시간에 타임아웃으로 멈춘다. `cmux rpc surface.input_state '{"surface_id":"…"}'` 로 직접 확인한다. cmux CLI의 `--json` 은 명령 뒤에 둔다(앞에 두면 `OK workspace:N` 한 줄, cmux 0.65 실측).
- **실측.** cmux 0.65: 실제 저장소 티켓 1건(2026-10-12)을 워크트리 생성 → pnpm 설치 → PR → CI 8개 작업 → 병합 → 정리·기록까지 벽시계 7분(수리 0회, idle 오판 0회). tmux 3.8: 더미 저장소 티켓 1건(2026-10-13)을 아무 터미널에서 `--max 1` 로 4분(수리 0회). 둘 다 Orca 쪽 티켓 21건 실측에 비하면 적다. 수리 경로(충돌·CI 실패)는 cmux에서 한 번(rental-portal #251, CI 수리) 지나갔고 위의 묵은 idle을 드러냈다. tmux에서는 아직이다 — `--max 1` 로 먼저 본다.
- **OS.** cmux는 macOS 전용이다. tmux는 macOS·Linux·WSL 어디서나 같다 — Windows는 WSL2 + tmux가 권장 구성이다(4장 끝절). Linux·WSL에서는 코드 경로만 있고 실제 티켓은 아직 돌리지 않았다.

3. 구성 요소

사용하는 mattpocock 스킬

스킬	언제	역할
`/setup-matt-pocock-skills`	저장소당 1회 (키트 없이 할 때)	이슈 트래커(GitHub), 분류 라벨, 도메인 문서 위치를 정해 `docs/agents/` 아래 설정 파일과 `CLAUDE.md` 의 `## Agent skills` 블록을 만든다. 다른 엔지니어링 스킬이 이 설정을 읽는다. 키트 0.6부터는 설치기가 `docs/agents/issue-tracker.md` 템플릿을 놓고 `/setup-ticket-loop` 가 블록을 쓰므로 이 스킬은 들이지 않는다.
`/grill-with-docs` , `/grill-me`	기획	요구사항을 집요하게 캐물어 결정되지 않은 부분을 드러낸다. 용어집·ADR도 함께 다듬는다.
`/to-spec`	기획	대화를 스펙 이슈 하나로 정리해 트래커에 올린다(`ready-for-agent` 라벨이 붙는다, 5장 주의 참고).
`/to-tickets`	분해	스펙을 독립적으로 검증 가능한 세로 조각(vertical slice) 티켓으로 쪼갠다. 선행 관계는 GitHub 기본 기능(blocked by)으로 걸고, `ready-for-agent` 를 붙여 발행한다.
`/triage`	상시	들어온 이슈를 분류해 상태 라벨을 붙이고, 에이전트가 바로 일할 수 있는 브리프를 쓴다.
`/implement` **저장소 소유**	워커	티켓을 읽고 구현한다. 선행 확인 → 브랜치 → `tdd` → 검증 → 커밋 → `ticket-review` 반영 → PR → 라벨까지 **절차 전체가 이 파일 하나**에 있다. upstream 사본을 저장소에 복사해 고쳐 쓴다(부록 B).
`tdd`	워커	정한 테스트 지점(seam)에서 red → green 사이클로 구현한다.
`ticket-review` (upstream 이름 `code-review`)	워커	기준점 이후 변경을 Standards(저장소 코딩 표준 + 코드 스멜 기준)와 Spec(이슈 요구와 일치하는가) 두 축으로, 병렬 서브에이전트가 검토한다. Claude Code 내장 `/code-review` 와 이름이 겹쳐 바꿔 들인다(4장 5단계).
`pr` **저장소 소유**	워커	PR 본문 양식. upstream 양식(Summary · Evidence · Merge Danger)은 사람이 여는 PR용이다. 루프 PR은 사람이 **나중에** 인수 조건·테스트 지점·검증 못 한 것·남은 판단만 골라 읽으므로 그 순서로 고정한 사본을 쓴다(부록 B).
`/retro`	묶음 끝	세션 기록과 루프 로그를 읽고 환경(탐색 단서, 자동 검사, 코딩 표준, 도구 비용) 개선 후보를 낸다. 루프 운영의 피드백 고리다.

> **왜 `implement` · `pr` 을 저장소가 소유하나.** 초판은 절차를 루프의 워커 프롬프트(7개 규칙)에 적었다. 그러자 사람이 대화형으로 `/implement` 를 쓸 때와 무인 워커가 받는 절차가 둘로 갈라졌고, 한쪽만 고치는 일이 생겼다. 2판은 절차를 `implement` 스킬 한 곳에 두고, 프롬프트에는 "질문하지 마라·막히면 코멘트하고 멈춰라"처럼 **무인 실행에서만 참인 것**만 남긴다(8장).

나머지 구성 요소

구성 요소	역할
백엔드 — Orca (데스크톱 앱 + CLI), cmux (macOS), tmux	Orca는 여러 코딩 에이전트를 각자의 git 워크트리에서 나란히 돌리는 오케스트레이터다. 루프는 CLI로 워크트리를 만들고(`worktree create --agent claude --prompt …`), 터미널이 입력을 기다리는 상태인지 확인하며 (`terminal wait --for tui-idle`), 수리 지시를 보낸다(`terminal send`). 워크스페이스 카드 상태(`todo` · `in-progress` · `in-review` · `completed`)와 코멘트도 갱신한다. cmux·tmux 백엔드에서는 루프가 git으로 워크트리를 만들고 셸에 `claude …` 를 띄우며, Claude Code 훅이 보고하는 상태로 입력 대기를 판정한다(2.5).
Claude Code (또는 Codex)	백엔드가 워크트리마다 띄우는 워커. Claude Code는 저장소의 `CLAUDE.md` · `.claude/skills/` · `.claude/settings.json` 을, Codex는 `AGENTS.md` · `.agents/skills/` · `.codex/rules/` 를 읽는다. 키트는 한 사본을 두 경로에서 보이게 배치한다(3.3).
gh / **jq** / **git**	이슈·PR 조회, CI 대기, 병합, 라벨 정리, JSON 처리, 브랜치 정리
CI (예: GitHub Actions)	PR마다 도는 체크. 루프가 병합 여부를 판단하는 유일한 기준이다. 체크가 하나도 없으면 루프는 병합하지 않는다(저장소 CI의 경로 필터에 걸리는 PR만 예외 — `CI_SKIP_REGEX` · `CI_SKIP_CMD`).
`ticket-loop.sh`	위 전부를 엮는 bash 스크립트(부록 D). 저장소의 `scripts/` 에 두고 쓴다.

저장소 파일	`.agents/skills/` (벤더링한 스킬· `implement` · `pr` 정본 — `.claude/skills/` 는 그 링크), `AGENTS.md` (`## Agent skills` 블록 정본 — `CLAUDE.md` 는 `@AGENTS.md` 로 가져온다), `.claude/settings.json` (`deny`)· `.codex/rules/` (같은 금지의 Codex 판), `orca.yaml` (Orca setup 훅 — cmux·tmux는 `.ticket-loop/loop.env` 의 `WORKTREE_SETUP_CMD`), `.ticket-loop/worker-rules.md` (무인 전용 규칙), `docs/agents/` (트래커·라벨·도메인 설정), 코딩 표준 문서. 워커는 `origin/main` 의 것만 보므로 **바꾸면 먼저 병합**한다.

라벨과 백엔드 상태가 바뀌는 시점

시점	GitHub 이슈	Orca 워크스페이스	cmux lane	tmux 창 이름
대기	**ready-for-agent**	—	—	—
워크트리 생성	워커가 자기를 담당자로 단다	`in-progress` · "티켓 루프: /implement N 진행 중"	`working`	`issue-N [working]`
PR 열림	**in-review** 추가, 인수 조건 체크	`in-review` · PR 번호 연결	`review` · PR 연결 · 로그	`[review]`
병합	닫힘(`Closes #N`), **in-review** 제거	`completed` → 워크트리 삭제	`done` → 워크스페이스 닫고 워크트리 삭제	`[done]` → 창 닫고 워크트리 삭제
루프 중단	그대로	`todo` + 읽지 않음 점 · "루프 중단: … 확인 필요"	`todo` · 로그 "루프 중단: …"	`[needs-attention]`

4. 환경 구축과 설치

아래 순서대로 한 번 해 두면 된다. 각 단계 끝의 "확인"이 통과하면 다음 단계로 간다. 예시는 macOS(Homebrew)와 Ubuntu 기준이다.

> **키트로 줄이기.** 1~3단계(도구·로그인·백엔드 설치)를 마쳤다면 나머지는 `ticket-loop-kit` 이 대부분 한다. 저장소 루트에서 `npx github:iwishash/ticket-loop-kit init` 을 실행하면 백엔드(Orca/cmux/tmux)와 에이전트를 묻고 `.ticket-loop/loop.env` 에 `BACKEND=` 줄을 쓴 뒤 4·5·7단계, 2단계의 git 자격 증명, 10단계의 `deny` , 13단계의 파일 배치를 끝낸다(npx 없이: `curl -fsSL https://raw.githubusercontent.com/iwishash/ticket-loop-kit/main/install.sh | bash`). 이어서 Claude Code에서 `/setup-ticket-loop` (6·8·9·11·12단계)를 실행하고 PR을 병합한다. 사람이 손으로 할 것은 9단계의 Orca 앱 설정뿐이다(cmux·tmux면 그것도 없고, 메인 체크아웃에서 `claude` 를 한 번 열어 폴더를 신뢰해 두는 것만 남는다 — 2.5). 아래 단계는 키트가 하는 일의 설명이자, 키트 없이 할 때의 절차다. **Windows**는 이 장 끝의 절을 먼저 읽는다 — 아래 단계를 전부 WSL 안에서 한다.

1 **기본 도구** — git, GitHub CLI, jq, Node.js(스킬 설치용 `npx`), Claude Code.

```
# macOS
brew install git gh jq node
# Ubuntu
sudo apt install -y git jq nodejs npm   # gh는 cli.github.com 안내에 따라 apt 저장소 추가 후 설치

# Claude Code (공식 설치 안내를 따른다. npm 예)
npm install -g @anthropic-ai/claude-code
claude          # 처음 한 번 실행해 로그인
```

확인: `git --version && gh --version && jq --version && claude --version`

2 **GitHub 로그인** — 루프가 이슈 편집, PR 병합, 원격 브랜치 삭제를 하므로 쓰기 권한이 필요하다.

```
gh auth login          # GitHub.com → HTTPS → 브라우저 로그인
gh auth status         # 로그인 계정과 토큰 범위 확인
gh auth setup-git      # git push도 gh 토큰을 쓰게 한다(아래 주의)
```

> **무인 `git push` 가 출력 없이 멈출 수 있다.** macOS의 시스템·Homebrew git 설정에는 `credential.helper = osxkeychain` 이 들어 있다. 무인 워커 세션은 키체인 확인 창을 띄우지 못해 `git push` 가 아무 출력 없이 기다린다(`git fetch` 는 정상이라 알아채기 어렵다). 루프에서는 워커가 "PR 직전 idle"로 보이다가 멈춤 판정을 받는다. 전역이면 `gh auth setup-git` , 저장소 범위로만 하려면 메인 체크아웃에서 `git config --local credential.helper '!gh auth git-credential'` (워크트리가 설정을 공유한다). 이미 멈춘 push는 죽이고 `GIT_TERMINAL_PROMPT=0 git -c credential.helper= -c 'credential.helper=!gh auth git-credential' push -u origin <브랜치>` 로 다시 한다.

3 Orca 설치와 CLI 등록

```
# macOS
brew install --cask stablyai/orca/orca
# Linux: onorca.dev/download 또는 GitHub Releases의 AppImage
#        (Arch: yay -S stably-orca-bin)
```

Orca를 실행하고 **Settings → General → Orca CLI**에서 CLI를 등록한다.

> **Linux 주의.** Linux에서 `orca`는 보통 GNOME 스크린리더(`/usr/bin/orca`)를 가리킨다. Orca 터미널 밖에서는 `orca-ide`를 쓴다. `ticket-loop.sh`는 Linux에서 기본으로 `orca-ide`를 고르고, `ORCA_CLI` 환경변수로 바꿀 수 있다. 이 문서의 `orca`는 Linux에서는 `orca-ide`로 읽는다.

확인: `orca repo list --json`이 JSON을 돌려준다(앱이 꺼져 있으면 `orca open --json`으로 켠다).

> **Orca 대신 cmux나 tmux를 쓸 때.** 이 단계와 4단계가 다음으로 바뀐다. 어느 쪽을 쓸지는 `loop.env`의 `BACKEND`다(2.5). 두 백엔드 모두 저장소 등록(4단계)은 없고, setup 훅(9단계)은 `loop.env`의 `WORKTREE_SETUP_CMD`로 대신하며, 메인 체크아웃을 Claude Code가 신뢰해 두어야 한다.
>
> ```
> # tmux — macOS / Ubuntu·WSL
> brew install tmux            # 또는 sudo apt install -y tmux
> tmux -V                      # 확인. 루프는 ticket-loop 세션을 스스로 만든다(TMUX_SESSION으로 바꾼다)
>
> # cmux — macOS만
> brew tap manaflow-ai/cmux && brew install --cask cmux   # 또는 github.com/manaflow-ai/cmux/releases 의 DMG
> # cmux를 실행하고 Settings에서 Claude Code 통합을 켠다(훅이 lifecycle을 보고한다)
> cmux socket-status --json    # cmux 터미널 안에서. configured_mode 가 cmuxOnly 면 루프도 cmux 터미널 안에서 돌린다
>
> # 둘 다: 메인 체크아웃에서 한 번
> cd ~/src/todo-app && claude   # "Yes, I trust this folder" 를 고르고 /exit
> ```
>
> cmux CLI는 PATH의 `cmux`, 없으면 `/Applications/cmux.app/Contents/Resources/bin/cmux`를 루프가 찾는다(`CMUX_CLI`로 바꾼다).

4 저장소를 Orca에 등록 — 루프는 git remote(`github.com/소유자/저장소`)로 Orca 저장소를 찾는다.

```
cd ~/src/todo-app
orca repo add --path "$(pwd)" --json
orca repo list --json | jq '.result.repos[] | {id, key: .gitRemoteIdentity.canonicalKey}'
```

확인: 목록에 `"key": "github.com/acme/todo-app"`가 보인다. cmux·tmux 백엔드는 등록이 없다 — 이 단계를 건너뛴다.

5 **mattpocock 스킬 설치** — 아래 두 방법 중 **하나만** 쓴다. 둘 다 설치하면 스킬이 두 벌이 된다.

```
# (가) Claude Code 플러그인 – 자동 업데이트
claude plugin install mattpocock-skills@claude-plugins-official

# (나) 저장소에 파일로 복사 – 팀이 같은 버전을 쓰도록 커밋할 수 있다, 수동 업데이트
npx skills@latest add mattpocock/skills -a claude-code   # 설치할 스킬을 물으면 전부 고른다
npx skills@latest update      # 나중에 업데이트할 때
```

무인 루프에서는 **(나)**를 권한다. 워커 워크트리는 저장소를 그대로 체크아웃하므로, 커밋된 `.claude/skills/` 가 모든 워커에 똑같이 적용된다. 루프 도중 자동 업데이트로 스킬 동작이 바뀌지도 않는다. 플러그인 방식(가)은 루프를 돌리는 컴퓨터 사용자 환경에 설치돼 있어야 하고, 클라우드 세션 같은 다른 환경에서는 버전이 달라질 수 있다.

(나)로 들인 뒤 세 가지를 손본다. 직접 하기 번거로우면 이 셋을 하는 동기화 스크립트를 저장소에 두고, upstream 커밋·버전을 매니페스트 파일(예: `.agents/skills/.mattpocock.json`)에 남긴다.

할 일	이유
`code-review` → `ticket-review` 로 이름 바꾸기(다른 스킬 본문의 참조도 함께)	Claude Code에 내장 `/code-review` 명령이 있다. 같은 이름이면 워커가 어느 쪽을 부르는지 흔들린다.
`implement` · `pr` 을 **저장소 소유**로 표시하고 동기화에서 빼기	저장소의 브랜치 규칙·검증 명령·PR 양식을 넣어 고쳐 쓰는 파일이다(부록 B). 동기화가 덮어쓰면 절차가 사라진다. upstream이 바뀌면 변경 내역을 보고 손으로 옮긴다.
버전은 태그가 아니라 **커밋**으로 기록하기	upstream 기본 브랜치에는 마지막 태그 뒤 아직 출시되지 않은 변경이 쌓여 있다. "1.3.1"만 적으면 실제 내용과 다를 수 있다.

확인: Claude Code에서 `/` 를 누르면 `/implement`, `/tdd`, `/ticket-review` 가 보이고, 같은 스킬이 두 벌 뜨지 않는다.

키트가 하는 방식(0.6·0.7). 루프는 스킬에 기대지 않는다. 워커에게 `/implement N` (Codex는 `$implement N`)을 보낼 뿐이고, 저장소 소유 `implement` 가 `tdd` 와 `ticket-review` 를 부른다(Claude Code는 Skill 도구로, Codex는 `$tdd` 의 SKILL.md를 읽어서). **배치(0.7):** 스킬 정본은 `.agents/skills/<이름>` (Codex가 읽는 자리)이고 `.claude/skills/<이름>` 은 `../../.agents/skills/<이름>` 심볼릭 링크다 — `npx skills add` 와 같은 배치라 한 사본을 두 에이전트가 본다. 0.6 이하가 `.claude/skills/` 에 실제 디렉터리로 놓은 것은 설치기와 동기화 스크립트가 `git mv` 로 옮기고 링크를 남긴다. 금지 명령은 `.claude/settings.json` 의 `deny` (Claude Code)와 `.codex/rules/ticket-loop.rules` (Codex, trusted 프로젝트에서만)에 같은 내용으로 둔다. 그래서 키트의 `scripts/sync-mattpocock-skills.sh` 는 (나)를 두 묶음으로 좁혀 한다 — **core**(`tdd`, `code-review` → `ticket-review`)는 항상, **workflow**(`grill-with-docs` · `to-spec` · `to-tickets` · `retro` 와 그것들이 부르는 `grilling` · `domain-modeling` · `writing-for-agents`)는 새 설치면 들이되 `mattpocock-skills` 플러그인이 그 저장소에 켜져 있으면 들이지 않고(이 머신에서는 플러그인이 준다), 재설치면 지난 선택을 잇는다 — `--workflow-skills` / `--no-workflow-skills` 가 우선. 나머지 upstream 스킬은 새 설치에는 들이지 않는다(원하면 플러그인으로). 다만 지난 설치(0.5 이하는 26개 전부)가 이미 놓은 것은 사용자가 쓰고 있을 수 있으니 지우지 않고 **kept**로 함께 동기화한다 — `--prune-skills` 를 줄 때만 지운다. 들이기 전에 같은 스킬이 다른 경로로 있는지 본다 — Claude Code는 중복을 경고하지 않기 때문이다. 플러그인이 이 저장소에 켜져 있으면 선택기에 둘 다 보이고(`/tdd` 와 `/mattpocock-skills:tdd` — 루프는 벤더링한 쪽을 돈다, 알림만), `~/.claude/skills/` 의 같은 이름은 프로젝트 사본을 가려 커밋한 버전이 돌지 않으며, `npx skills` 가 둔 `skills-lock.json` ·심볼릭 링크는 두 도구가 같은 디렉터리를 덮어쓰므로 설치를 멈춘다. `bash scripts/sync-mattpocock-skills.sh --check` 로 따로 볼 수 있다.

6 **저장소 설정** — 저장소 루트에서 Claude Code를 열고 실행한다. 키트로 설치했으면 이 단계는 건너뛴다: `docs/agents/issue-tracker.md` 는 설치기가 템플릿으로 놓고(아래 "티켓 작업 관례" 포함), `## Agent skills` 블록은 `/setup-ticket-loop` 가 `AGENTS.md` 에 쓰고 `CLAUDE.md` 에 `@AGENTS.md` 한 줄을 둔다(Codex는 `AGENTS.md` 만 읽는다).

```
/setup-matt-pocock-skills
```

질문에는 이렇게 답한다: 이슈 트래커 → **GitHub**, 분류 라벨 → 기본값(**needs-triage** **needs-info** **ready-for-agent** **ready-for-human** **wontfix**), 도메인 문서 → 단일 컨텍스트(루트 `GLOSSARY.md` + `docs/adr/`). 결과로 다음 파일이 생긴다.

```
docs/agents/issue-tracker.md   # gh로 이슈를 읽고 쓰는 규칙
docs/agents/triage-labels.md   # 역할 → 실제 라벨 이름
docs/agents/domain.md          # 용어집·ADR 위치
CLAUDE.md                      # "## Agent skills" 블록 추가 (키트: AGENTS.md 에 두고 CLAUDE.md 는 @AGENTS.md)
```

`docs/agents/issue-tracker.md` 끝에 트래커 관례를 한 단락 덧붙인다. 구현 절차 자체는 `implement` 스킬(부록 B)에 두고, 여기에는 트래커에 관한 사실만 둔다.

```
## 티켓 작업 관례
- 마일스톤 = 스펙 이슈 1건 + 하위 이슈(sub-issue) 티켓 `T<NN>: …`.
- 티켓마다 최신 기본 브랜치에서 `issue-<N>-<slug>` 브랜치를 만든다.
- PR 본문은 `Closes #<N>`으로 시작한다. PR을 열면 이슈에 `in-review` 라벨을 붙인다.
- 선행 관계는 GitHub 기본 기능(blocked by)으로 건다. 시작 가능 여부는 선행 이슈 목록
  (`gh api repos/{owner}/{repo}/issues/N/dependencies/blocked_by`)에서 열린 것을 세어 판단한다.
  요약 값 `issue_dependencies_summary.blocked_by`는 선행 이슈가 닫힌 뒤에도 한동안 옛 값을 준다.
```

같은 단계에서 `CLAUDE.md` 를 **짧게** 유지한다. 워커는 티켓마다 새로 뜨면서 `CLAUDE.md` 를 매번 처음부터 읽는다. 코딩 관례·아키텍처·스택 설명은 별도 문서(예: `CODING_STANDARDS.md` , `docs/agents/architecture.md`)로 빼고 `CLAUDE.md` 에는 "필요할 때 읽을 곳" 포인터만 둔다. 코딩 표준 문서는 `ticket-review` 의 Standards 축이 읽는다.

7 **GitHub 라벨 만들기** — 이미 있으면 `--force` 가 색·설명만 갱신한다.

```
gh label create ready-for-agent --color 0E8A16 --description "에이전트가 바로 구현 가능" --force
gh label create ready-for-human --color FBCA04 --description "사람이 구현"           --force
gh label create needs-info      --color D93F0B --description "정보 부족"             --force
gh label create needs-triage    --color C5DEF5 --description "분류 필요"             --force
gh label create wontfix         --color FFFFFF --description "하지 않음"             --force
gh label create in-review       --color 5319E7 --description "PR 열림"               --force
```

8 **CI 준비** — PR에서 도는 체크가 최소 하나 있어야 한다. 루프는 PR에 체크가 15분 안에 나타나지 않으면 병합하지 않고 멈춘다. Node 프로젝트 예시:

```
# .github/workflows/ci.yml
name: CI
on: { pull_request: { branches: [main] } }
jobs:
  check:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 22, cache: npm }
      - run: npm ci
      - run: npm run lint
      - run: npm test
```

브랜치 보호에서 필수 리뷰를 켜 두면 `gh pr merge`가 거절된다. 그때는 `--no-merge`로 운영하거나 보호 규칙을 조정한다.

문서만 바뀐 PR은 CI를 건너뛰고 싶다면 워크플로에 경로 필터를 두고, 같은 판정을 루프에도 알려 준다. 알려 주지 않으면 루프는 붙지 않는 체크를 15분 기다리다 멈춘다(R7).

```
# .github/workflows/ci.yml
on:
  pull_request:
    branches: [main]
    paths-ignore: ["docs/**", ".claude/**", ".ticket-loop/**", "**/*.md"]

# 루프에 같은 필터를 알려 준다(.ticket-loop/loop.env) — 바뀐 파일이 전부 이 정규식에 맞으면 체크 없이 병합한다
CI_SKIP_REGEX="${CI_SKIP_REGEX:-^(docs/|\.claude/|\.ticket-loop/)|\.md$}"
```

이 단계는 워크플로에 경로 필터를 두기로 한 저장소에만 해당한다. 필터를 두지 않으면 루프는 모든 PR에서 체크를 기다리고, 두 값은 비워 둔다. 두 목록은 손으로 맞춘다. 테스트가 문서를 문자열로 읽는다면(예: 문서 내용을 검사하는 테스트) 그 문서는 제외 목록에서 빼야 한다. `paths`에 포함·제외가 순서대로 섞여 정규식으로 못 쓰면, 워크플로 파일을 읽어 같은 판정을 내는 저장소 스크립트(바뀐 파일 목록을 stdin으로 받아 `run` / `skip` 출력)를 두고 `CI_SKIP_CMD`에 적는다.

9 **워크트리 setup 훅 (`orca.yaml`)** — 새 워크트리에는 Git에 없는 파일(`.env` 등)과 의존성이 없다. 저장소 루트의 `orca.yaml`에 setup 스크립트를 두면 Orca가 워크트리를 만들 때마다 실행한다. 이때 `ORCA_ROOT_PATH`(메인 체크아웃)와 `ORCA_WORKTREE_PATH`(새 워크트리) 환경변수를 쓸 수 있다.

```
# orca.yaml
scripts:
  setup: |
    npm ci
    # 개발용 비밀값은 메인 체크아웃에서 복사한다 (운영 값이 든 파일은 두지 않는다)
    [ -f "$ORCA_ROOT_PATH/.env.local" ] && cp "$ORCA_ROOT_PATH/.env.local" "$ORCA_WORKTREE_PATH/.env.local" || true
# 에이전트가 setup이 끝난 뒤에 시작하도록
setupAgentStartupPolicy: wait-for-setup
```

cmux·tmux 백엔드는 `orca.yaml`을 읽지 않는다. 같은 일을 `.ticket-loop/loop.env`의 `WORKTREE_SETUP_CMD`로 한다. 루프가 `git worktree add` 직후 새 워크트리 안에서 `LOOP_ROOT_PATH`·`LOOP_WORKTREE_PATH`를 주고 실행하며, 실패하면 그 티켓에서 멈춘다(2.5). 예: `WORKTREE_SETUP_CMD="${WORKTREE_SETUP_CMD:-npm ci && cp \"\$LOOP_ROOT_PATH/.env.local\" . || true}"`.

앱 설정을 바꿔야 이 파일이 쓰인다. Orca의 **Settings → Projects → <저장소> → Worktree Hooks → Advanced (Command source & orca.yaml)**에서 Command Source를 **"orca.yaml only"**로 고르고, 같은 섹션의 **"Wait for setup to complete before starting agent"**를 켠다(설정 검색창에 `orca.yaml`을 치면 바로 나온다). 기본값 "Local only"는 저장소의 `orca.yaml`을 무시하고 앱에 저장된 로컬 명령만 쓴다.

운영 자격 증명은 넣지 않는다. 워커는 사람 확인 없이 명령을 실행한다. 메인 체크아웃의 `.env*`에는 개발·테스트용 값만 둔다. 더 안전하게 하려면 복사 전에 내용을 검사한다. 예: DB 주소가 개발용 호스트 하나만 가리킬 때만 복사하고, 다른 호스트가 섞이면 복사하지 않는다.

10 **Claude Code 무인 권한** — 워커가 권한 확인 창에서 멈추면 루프는 "PR 없이 멈춤"으로 판단한다. 실제 운영에서 Orca는 Claude Code 워커를 `claude --dangerously-skip-permissions`로 띄웠다(Orca 에이전트 설정에서 확인한다). cmux·tmux 백엔드는 처음부터 루프의 `AGENT_RESTART_CMD`(기본값이 같은 명령)로 띄운다. 이 모드에서는 권한 창이 뜨지 않는 대신 **`deny` 목록이 유일한 방어선**이다. 저장소의 `.claude/settings.json` 예시:

```
{
  "permissions": {
    "deny": [
      "Bash(gh pr merge:*)",
      "Bash(git push --force:*)", "Bash(git push -f:*)",
      "Bash(git push origin main:*)",
      "Read(./.env.production)"
    ]
  }
}
```

`deny`는 대화형 세션에도 적용된다. 사람이 병합할 때는 터미널에서 직접 `gh pr merge`를 실행한다(Claude Code 안이면 `! gh pr merge …`). 권한 모드를 쓰지 않는 실행기라면 `allow`에 테스트·린트·`gh issue`·`gh pr create` 같은 명령을 미리 넣고, `--max 1` 첫 실행 때 워커가 어디서 묻는지 보고 보강한다.

11 **워커 규칙 파일 (`.ticket-loop/worker-rules.md`) — 무인 워커에게만** 필요한 프로젝트 규칙을 적는다. 사람이 대화형으로 쓸 때도 참인 절차는 `implement` 스킬(12단계)에 둔다. 루프는 이 파일이 있으면 워커 프롬프트에 "이 규칙도 따르라"를 덧붙인다. 커밋해 두어야 워크트리에도 생긴다.

```
# 워커 규칙 (ticket-loop)
- 검증은 두 층이다. 수시에는 타입 검사(`npm run typecheck`)와 단일 테스트 파일
  (`npm test -- <파일>`)만 돌린다. 전체 묶음(`npm run verify` = lint → 전체 테스트 → build)은
  첫 커밋 직전 1회, 리뷰 반영 커밋 직전 1회만 돌린다.
- 통합 테스트(tests/integration/)·E2E는 전체를 돌리지 않는다 — 루프가 병합 뒤 돌린다. 이번 티켓에서
  추가·수정한 그 파일만 PR을 열기 직전에 `npm run test:integration -- <파일>`로 1회 돌리고,
  파일과 결과를 PR 본문 "검증"에 적는다. 손대지 않았으면 "없음"이라고 적는다.
- DB 마이그레이션을 새로 만들면 번호가 기본 브랜치의 마지막 번호 다음인지 확인한다.
  개발 DB에만 적용하고 결과를 PR 본문에 적는다. 운영 DB에는 연결하지 않는다.
- 프로젝트 의존성을 쓰는 임시 스크립트는 워크트리 안 `scripts/`에 두고 커밋 전에 지운다
  (저장소 밖 임시 폴더에서는 모듈 해석이 실패한다).
- 화면 확인용 스크린샷은 작게(폭 400px 이하) 한 장만 찍는다.
- CI를 통과시키려고 테스트를 건너뛰거나 지우지 않는다.
```

> **검증을 두 층으로 나누는 이유.** 규칙이 "수시로 테스트, 마지막에 전체"뿐이던 실제 운영에서 워커는 티켓당 전체 테스트를 1~9번, build를 2~8번 돌렸다. 시간과 토큰이 그만큼 든다. lint·전체 테스트·build를 한 명령(`verify`)으로 묶고 "몇 번, 언제"를 숫자로 정해 두면 줄어든다. 간헐적으로 실패하는 테스트가 있으면 "단독으로 다시 돌려 판별하라"처럼 판별법도 여기에 적는다.

12 **`implement`·`pr` 스킬을 저장소에 맞추기** — 5단계에서 저장소 소유로 남긴 두 스킬을 고친다. 부록 B의 예시를 바탕으로 기본 브랜치 이름, 검증 명령, 마이그레이션 규칙, PR 본문 양식을 이 저장소의 것으로 바꾼다. 무인이든 대화형이든 `/implement N` 한 번으로 "브랜치 → TDD → 검증 → 리뷰 → PR → 라벨"이 같은 순서로 일어나야 한다.

확인: 메인 체크아웃에서 작은 티켓 하나로 `/implement N`을 대화형으로 돌려 본다. 질문이 나오는 지점이 곧 무인 워커가 멈출 지점이다. 답을 스킬이나 `worker-rules.md`에 적는다.

13 **루프 스크립트 배치와 드라이런**

```
mkdir -p scripts && cp /경로/ticket-loop.sh scripts/ && chmod +x scripts/ticket-loop.sh
echo '/.ticket-loop/*.log' >> .gitignore      # 루프 로그는 커밋하지 않는다(규칙 파일만 커밋)
git add .gitignore orca.yaml .claude .ticket-loop/loop.env .ticket-loop/worker-rules.md scripts/ticket-loop.sh docs/agent
s CLAUDE.md
git commit -m "chore: 티켓 무인 루프 환경" && git push
bash scripts/ticket-loop.sh --dry-run
```

확인: 저장소, 기준 브랜치, 라벨이 한 줄 찍히고 대상 목록(또는 "No tickets to process")이 나온다. 오류가 나면 표 4-1을 본다.

Windows — WSL2 + tmux 백엔드

루프 스크립트는 bash이고 git·gh·jq·Claude Code를 Linux 방식으로 부른다. 네이티브 Windows(cmd·PowerShell·Git Bash)는 지원하지 않는다 — 설치기와 루프가 bash이고, `.claude/skills/` 아래가 심볼릭 링크이며, Git Bash에는 tmux가 없다. Windows에서는 **WSL2(Ubuntu) 안에서 tmux 백엔드**로 돌린다. WSL 안에서는 Linux 절차가 그대로이고, tmux 백엔드는 GUI 앱 없이 터미널 하나로 끝나 경계를 넘는 호출이 없다. Windows용 Orca도 WSL 경로의 저장소를 다룰 수 있지만, 호출마다 PowerShell 브리지와 `wsl.exe`를 오가고 WSL 통합 이슈가 아직 열려 있어(2026-10 기준) 이 책은 권하지 않는다. 이 절의 명령은 Ubuntu 기준이고, 실측은 아직 macOS뿐이다(README의 OS 표). 반드시 `--max 1`로 먼저 한 건을 본다.

W1 **WSL2와 Ubuntu** — PowerShell(관리자)에서. Windows 11 또는 Windows 10 22H2 이상.

```
wsl --install -d Ubuntu     # WSL2 + Ubuntu. 재부팅 뒤 Ubuntu 창에서 사용자 이름·암호를 만든다
wsl --update
wsl --status                # 기본 버전이 2인지
```

확인: Windows Terminal에 Ubuntu 탭이 생기고 bash 프롬프트가 뜬다. 이후 단계는 전부 그 탭(WSL 안)에서 한다.

W2 **WSL 안에 도구** — 1·2단계의 Ubuntu 절차에 tmux가 더해진다. 저장소는 **WSL 파일시스템**(`~/src/…`)에 둔다. `/mnt/c/…`는 파일 접근이 느리고 권한·파일 감시가 달라 설치와 테스트가 몇 배 걸린다.

```
sudo apt update && sudo apt install -y git jq tmux rsync lsof curl build-essential
# gh: cli.github.com 의 apt 저장소 안내대로 설치
# Node: nvm 으로 LTS 설치 → npm install -g @anthropic-ai/claude-code → claude 로 한 번 로그인
gh auth login && gh auth setup-git     # WSL엔 키체인이 없다 — gh 도우미가 무인 push를 맡는다(2단계 주의)
git clone https://github.com/acme/todo-app ~/src/todo-app
cd ~/src/todo-app && claude             # "Yes, I trust this folder" 를 고르고 /exit — cmux·tmux의 시작 점검이 본다(2.5)
```

확인: `git --version && gh auth status && jq --version && claude --version && tmux -V`. `git config --get-all credential.helper`에 `!gh auth git-credential`이 보인다.

W3 **키트 설치와 백엔드 선택** — 4장의 키트 절차를 WSL 안에서 그대로 한다. 백엔드는 `tmux`.

```
npx github:iwishash/ticket-loop-kit init --backend tmux   # 또는 대화형으로 tmux 를 고른다
cat .ticket-loop/loop.env                      # BACKEND="${BACKEND:-tmux}" 와 WORKTREE_SETUP_CMD 자리
# 이어서 Claude Code에서 /setup-ticket-loop → PR 병합. Orca 앱 설정(9단계)은 없다
```

확인: `bash scripts/ticket-loop.sh --dry-run`이 "backend tmux"를 찍고 대상 목록(또는 "No tickets to process")을 낸다. "Claude Code does not trust … yet"이면 W2의 마지막 줄.

W4 **루프는 tmux 안에서** — Windows Terminal의 Ubuntu 탭에서. 탭을 닫거나 터미널 세션이 끊겨도 tmux 서버와 그 안의 루프·워커는 살아남고, 프로세스가 남아 있는 동안 WSL도 배포판을 끄지 않는다. 루프를 tmux 세션 안에서 돌리면 워커 창도 그 세션에 열려 한 화면에서 보인다.

```
cd ~/src/todo-app
tmux new -s ticket-loop                        # 세션 만들기(이미 있으면 tmux attach -t ticket-loop)
bash scripts/ticket-loop.sh --parent 40 --max 1   # 첫 실행은 한 건. 워커 창 issue-N 이 같은 세션에 열린다
# 떼기: Ctrl-b 뒤 d  /  붙기: tmux attach -t ticket-loop  /  창 이동: Ctrl-b n·p  /  목록: tmux ls
```

잠자기 방지는 `caffeinate`·`systemd-inhibit` 대신 Windows 전원 옵션에서 절전을 끈다. 노트북을 덮으면 WSL VM도 함께 멈추고 깨어난 뒤 CI 대기 같은 네트워크 호출이 실패해 루프가 죽을 수 있다(R1).

W5 **macOS·Linux와 다른 점**

- **알림이 없다.** WSL에는 `notify-send`가 없어 티켓 완료·중단 알림이 뜨지 않는다. 로그(`.ticket-loop/loop-MMDD.log`)와 tmux 창 이름의 `[working]`·`[review]`·`[done]`·`[needs-attention]`으로 본다.
- **리포트 열기.** WSLg가 있는 Windows 11은 `DISPLAY`가 있어 루프가 `xdg-open`을 부른다. Windows 브라우저로 열리게 하려면 `sudo apt install wslu`(`xdg-open`이 `wslview`로 간다). 아니면 `REPORT_OPEN=0`으로 끄고 `explorer.exe docs/loop-reports`로 연다.
- **줄 끝.** 저장소를 WSL 안에서만 다루면 CRLF 문제가 없다. Windows 쪽 에디터로 같은 파일을 열어 저장하지 않는다.
- **Windows용 Orca를 굳이 쓰려면** 설치 뒤 Settings → Terminal의 기본 셸을 WSL로, 프로젝트는 `\\wsl.localhost\Ubuntu\…` 경로로 등록하고, Settings → General → Orca CLI의 WSL 등록으로 배포판에 `orca-ide`를 둔다(Orca가 연 WSL 터미널에는 자동으로 들어간다). 루프는 WSL 안에서 `BACKEND=orca`로 돈다. 검증하지 않았다.

표 4-1. 설치 확인 체크리스트

✓	항목	실패할 때 메시지 / 조치
☐	백엔드 CLI(`orca` / `orca-ide`, `cmux` 또는 `tmux`)·gh·jq·git이 PATH에 있다	"orca(-ide) 이(가) 없습니다" → 3단계 CLI 등록. "cmux CLI … 가 없습니다" → `CMUX_CLI`. "tmux … 가 없습니다" → 설치
☐	(cmux·tmux) 메인 체크아웃을 Claude Code가 신뢰한다	"Claude Code가 … 를 아직 신뢰하지 않습니다" → 메인 체크아웃에서 `claude`를 한 번 열어 신뢰(2.5)
☐	(cmux) 루프를 cmux 터미널 안에서 띄운다	"cmux 소켓에 닿지 못했습니다" → cmux 터미널에서 실행(`cmux socket-status --json`)
☐	(Orca) 저장소가 Orca에 등록돼 있다	"Orca에 … 저장소가 없습니다" → 4단계
☐	스킬과 `docs/agents/` 설정이 **기본 브랜치에 병합돼** 있다	워커가 "트래커 설정이 없다"고 하면 6단계 후 push. 워커는 `origin/main`의 파일만 본다

☐	`implement` · `pr` 이 저장소 소유 사본이고, `code-review` 가 `ticket-review` 로 바뀌었다	5·12단계
☐	라벨 6개가 있다	`gh label list`
☐	PR에서 CI가 돈다(워크플로에 경로 필터가 있으면 `CI_SKIP_REGEX` 또는 `CI_SKIP_CMD` 를 같은 판정으로 맞췄다)	빈 PR을 하나 열어 체크가 뜨는지 본다
☐	워크트리 setup이 성공한다	Orca 앱 설정이 "orca.yaml only"인지 보고, 워크트리를 직접 하나 만들어 setup 출력을 확인한다. cmux·tmux는 `loop.env` 의 `WORKTREE_SETUP_CMD` 를 메인 체크아웃에서 그대로 실행해 본다
☐	무인 세션의 `git push` 가 멈추지 않는다	`git config --get-all credential.helper` 에 `osxkeychain` 만 있으면 2단계 주의대로 바꾼다
☐	워커가 권한을 묻지 않는다	`--max 1` 첫 실행 때 Orca 터미널을 지켜본다

5. 티켓 준비

루프가 집는 조건

자동 선택 모드는 다음을 **모두** 만족하는 열린 이슈만 집는다.

조건	설명
`ready-for-agent` 라벨 (`READY_LABEL`)	사람이 할 일은 `ready-for-human` , 정보가 부족하면 `needs-info` 로 두면 루프가 건드리지 않는다.
(선택) 범위 라벨 (`--label` , `SCOPE_LABEL`)	예: `--label bug` , `--label v2.0` . 이번 실행을 특정 묶음으로 좁힌다.
(선택) 스펙의 하위 이슈 (`--parent` , `SPEC_ISSUE`)	예: `--parent 40` . 스펙 이슈 하나에 딸린 티켓만 돈다. 마일스톤 하나를 돌릴 때 가장 많이 쓴다.
`in-review` 라벨이 **없음**	이미 PR 단계인 티켓은 뺀다.
담당자 **없음**	워커는 시작할 때 자기를 담당자로 단다. 진행 중이던 티켓, 사람이 맡은 티켓을 다시 집지 않는다.
하위 이슈 **없음**	하위 이슈를 가진 이슈는 스펙으로 보고 건너뛴다. `/to-spec` 이 스펙 이슈에도 `ready-for-agent` 를 붙이기 때문이다.
열린 선행 이슈 0개	선행 이슈 목록(`/issues/N/dependencies/blocked_by`)에서 `state == "open"` 인 것을 센다. 요약 값 `issue_dependencies_summary.blocked_by` 는 선행 이슈가 닫힌 뒤에도 한동안 옛 값을 돌려준 적이 있어 쓰지 않는다.
head 브랜치가 `issue-N` 이거나 `issue-N-` 로 시작하는 열린 PR 없음	사람이 먼저 PR을 연 티켓과 겹치지 않게 한다. `issue-1020` 은 #102의 PR이 아니다.

처리 순서

제목에서 `ORDER_REGEX` (기본 `^T(?<n>[0-9]+)` , 예 `T03: …`)로 순번을 뽑아 오름차순으로 정렬한다. 순번이 없는 이슈는 그 뒤에 이슈 번호 순으로 온다. 그리고 **티켓 하나를 끝낼 때마다 대상을 다시 계산**한다. 그래서 T04가 T02·T03에 막혀 있어도, 둘이 병합되면 다음 계산에서 T04가 풀려 이어서 처리된다.

스킬로 티켓을 만들 때의 요령

- `/to-tickets` **에 제목 형식과 부모를 지정한다.** "제목은 `T01: …` 형식, 스펙 #40의 하위 이슈로" 하고 요청하면 처리 순서가 분명해지고 `--parent 40` 으로 묶어 돌릴 수 있다.
- **스펙 이슈의** `ready-for-agent` **는 떼지 않아도 된다.** 루프는 하위 이슈가 있는 이슈를 건너뛴다(초판은 떼어야 했다). 떼어도 문제없다.
- **선행 관계를 GitHub 기능으로 건다.** 본문에 "#41 다음"이라고만 쓰면 루프는 모른다. to-tickets가 선행 관계를 걸었는지 이슈 오른쪽 "Relationships"에서 확인한다.
- **결정은 티켓을 만들기 전에 커밋한다.** `/grill-with-docs` 에서 나온 용어·ADR을 기본 브랜치에 병합해 두어야 워커가 본다. 워커는 대화 맥락 없이 `origin/main` 만 읽는다.

무인 실행에 맞는 티켓

조건	이유
인수 조건이 체크박스 목록이다	워커가 이 목록으로 테스트 지점을 고르고, PR을 열 때 충족한 항목을 체크한다.
결정이 이미 내려져 있다	워커는 질문할 수 없다. 결정이 필요한 티켓은 `ready-for-human` 이나 `needs-info` 로 둔다.
PR 하나 크기다	구현·리뷰·CI 수리까지 기본 4시간(`TICKET_TIMEOUT_SEC`) 안에 끝나야 한다.
세로 조각이다	각 티켓이 혼자서 테스트로 검증된다(to-tickets의 기본 원칙). 다른 티켓이 끝나야 테스트가 통과하는 티켓은 무인 병합에 맞지 않는다.
용어·결정의 근거가 저장소에 있다	`GLOSSARY.md` , `docs/adr/` . 워커는 대화 맥락 없이 저장소만 본다.

> **번호를 직접 지정하면 일부 조건을 건너뛴다.** `ticket-loop.sh 42` 처럼 번호를 주면 라벨·담당자는 보지 않는다. 대신 선행 이슈가 열려 있거나 PR이 이미 있으면, 그 티켓만 건너뛰지 않고 **루프 전체를 멈춘다**.

급한 티켓 끼워 넣기. 제목을 `T00: …`으로 하고 **ready-for-agent**를 붙인다. 루프는 지금 티켓을 끝낸 뒤 대상을 다시 계산하므로 그 티켓을 다음에 집는다. `--parent`로 돌리는 중이면 그 스펙의 하위 이슈로도 건다.

6. 루프 실행

```
bash scripts/ticket-loop.sh --parent 40 --dry-run   # 스펙 #40의 티켓 중 지금 집을 수 있는 것
bash scripts/ticket-loop.sh --parent 40 --max 1     # 1건만 시험
bash scripts/ticket-loop.sh --parent 40             # 스펙 #40 전부 (끝낼 때마다 대상 재계산)
bash scripts/ticket-loop.sh                         # 준비된 티켓 전부
bash scripts/ticket-loop.sh --label bug --max 3     # 라벨로 좁혀 3건
bash scripts/ticket-loop.sh 41 45                   # 지정한 이슈만, 적은 순서대로(닫힌 이슈는 건너뛰고, 스펙 이슈면 멈춘다)
bash scripts/ticket-loop.sh --no-merge 44           # CI 통과까지만, 병합은 사람이
POST_MERGE_CMD="npm ci && npm run test:e2e" POST_MERGE_COPY=".env.local" POST_MERGE_PORT=3000 \
  bash scripts/ticket-loop.sh --parent 40           # 병합마다 E2E를 기본 브랜치로 한 번
bash scripts/ticket-loop.sh --parent 40             # 끝난 스펙이면 "No tickets to process" 뒤 리포트만 다시 만든다
```

옵션	동작
`N ...`	처리할 이슈 번호. 생략하면 자동 선택한다.
`--parent N`	스펙 이슈 #N의 하위 이슈만 대상으로 한다.
`--label L`	`ready-for-agent` 에 더해 라벨 L도 요구한다.
`--dry-run`	대상만 출력하고 끝낸다. 잠금도 잡지 않고 로그 파일도 쓰지 않는다.
`--max N`	N건 처리 후 멈춘다.
`--no-merge`	CI 통과 후 병합하지 않고 워크트리를 남긴 채 다음 티켓으로 간다. 그 티켓에 막힌 티켓은 병합 전까지 풀리지 않는다.
`--skip-post-merge`	`POST_MERGE_CMD` 가 설정돼 있어도 이번 실행에서는 돌리지 않는다(포트를 비울 수 없을 때 등).
`--model M`	이번 실행의 워커 모델(`AGENT_MODEL` 보다 우선). 기동 명령 뒤에 `--model M` 을 붙여 워커·다시 띄우기·무인 수습 세션이 모두 같은 모델로 뜬다. cmux·tmux만 된다 — Orca에서는 시작 때 멈춘다(12장 FAQ).

환경변수	기본값	의미
`BACKEND`	`orca`	`orca` | `cmux` | `tmux` . 워크트리·에이전트 터미널·idle 판정·상태 표시를 누가 하나(2.5). 보통 `loop.env` 에 둔다
`ORCA_CLI`	`orca` (Linux·WSL `orca-ide`)	Orca CLI 실행 파일. Orca가 연 터미널의 `ORCA_CLI_COMMAND` 가 있으면 그것을 먼저 쓴다
`CMUX_CLI`	PATH의 `cmux` → 앱 번들 안의 `cmux`	cmux CLI 실행 파일
`TMUX_CLI`	`tmux`	tmux 실행 파일
`TMUX_SESSION`	tmux 안이면 그 세션, 밖이면 `ticket-loop`	(tmux) 워커 창을 열 세션. 없으면 만든다
`WORKTREE_DIR`	`~/.cache/ticket-loop/<owner-repo>/worktrees`	(cmux·tmux) 루프가 워크트리를 둘 디렉터리. Orca는 자기 설정 위치에 둔다
`WORKTREE_SETUP_CMD`	(없음)	(cmux·tmux) 새 워크트리에서 에이전트를 띄우기 전에 돌릴 명령 — `orca.yaml` setup에 해당. `LOOP_ROOT_PATH` · `LOOP_WORKTREE_PATH` 를 준다
`SPEC_ISSUE`	(없음)	`--parent` 기본값
`READY_LABEL`	`ready-for-agent`	대상 라벨
`SCOPE_LABEL`	(없음)	함께 요구할 라벨(`--label` 기본값)
`IN_REVIEW_LABEL`	`in-review`	PR이 열린 티켓의 라벨
`BRANCH_PREFIX`	`issue-`	이슈 브랜치 접두사. 워커 브랜치 `issue-N-slug` 와 루프가 만드는 워크트리 브랜치 `issue-N` 둘 다 앞에 붙는다
`BASE_BRANCH`	저장소 기본 브랜치	워크트리 기준이자 PR 대상
`ORDER_REGEX`	`^T(?<n>[0-9]+)`	제목에서 순번을 뽑는 정규식(그룹 이름 `n`)

`WORKER_RULES_FILE`	`.ticket-loop/worker-rules.md`	워커에게 따르게 할 프로젝트 규칙
`AGENT`	`claude`	`claude` | `codex`. Orca 에이전트 ID이자 기록의 agent 필드. 아래 네 값의 기본을 정하는 프리셋이다 — 다른 값이면 네 값을 직접 적는다(비어 있으면 시작 때 멈춘다). `codex`는 프리셋만 있고 아직 실측 전이다(12장 FAQ)
`AGENT_RESTART_CMD`	`claude --dangerously-skip-permissions` / `codex --dangerously-bypass-approvals-and-sandbox`	셸에서 에이전트를 띄울 명령. 워커 프롬프트가 인자로 붙는다. Orca는 기동 실패 때만, cmux·tmux는 처음부터 이 명령으로 띄운다(tmux는 뒤에 훅 설정 인자를 붙인다). Orca가 워커를 띄우는 방식과 같게 맞춘다
`AGENT_MODEL`	(없음)	워커 모델. 비우면 에이전트의 평소 기본 모델이다. 주면 기동 명령 뒤에 `AGENT_MODEL_FLAG <모델>`을 붙인다. `--model` 이 이긴다. cmux·tmux만 — Orca는 에이전트를 직접 띄우고 인자를 Orca 설정에서만 받으므로 시작 때 멈춘다
`AGENT_MODEL_FLAG`	`--model` / `--model`	모델을 고르는 기동 명령 인자. 다른 `AGENT`면 직접 적는다(비어 있는데 모델을 주면 시작 때 멈춘다)
`AGENT_BANNER_REGEX`	`Claude Code|Opus|Sonnet|Fable|Haiku / OpenAI Codex|gpt-[0-9]`	에이전트가 떴는지 확인할 화면 문구(ERE)
`SKILL_PREFIX`	`/` / `$`	워커 프롬프트에서 스킬을 부르는 접두사 — `/implement N`(Claude Code) 또는 `$implement N`(Codex)
`TICKET_TIMEOUT_SEC`	`14400` (4시간)	티켓 한 건 전체 제한 시간
`MERGE_METHOD`	`merge`	`merge` | `squash` | `rebase`
`REPAIR_ROUNDS`	`1`	충돌·CI 수리를 합친 허용 횟수
`CI_SKIP_REGEX`	(없음)	저장소 CI에 경로 필터가 있을 때 그 필터를 루프에 알려 주는 값. 바뀐 파일이 전부 이 정규식(ERE)에 맞으면 체크가 안 붙는 게 정상인 PR로 보고 체크 없이 병합한다. 루프의 규칙이 아니라 워크플로의 `paths-ignore`를 따라 하는 것이다 — 필터가 없으면 비운다(4장 8단계)
`CI_SKIP_CMD`	(없음)	같은 목적의 명령. 순서 규칙이 있는 `paths` 목록처럼 정규식 하나로 못 쓰는 필터용. 바뀐 파일 목록을 stdin으로 받아 `run` / `skip`을 출력하는 저장소 스크립트를 적는다. 있으면 정규식보다 먼저 보고, 실패하면 체크를 기다린다
`POST_MERGE_CMD`	(없음 = 끔)	병합 뒤 기본 브랜치에서 한 번 돌릴 명령. 실패하면 루프가 멈춘다. 왜 도는지(`TICKET_LOOP_POST_MERGE_WHY`: `every` · `spec_end` · `run_end`)와 스펙을 끝낸 병합을 덮는지(`TICKET_LOOP_POST_MERGE_SPEC_COMPLETE`: 스펙 번호 또는 빈 값)를 env로 받는다 — 스펙 끝에만 무거운 검증을 돌리는 식이다(7장 6단계)
`POST_MERGE_RETRY_CMD`	(없음 = 재실행 없음)	`POST_MERGE_CMD`가 실패했을 때 딱 한 번 돌릴 재실행 명령. 같은 검증 워크트리에서 `POST_MERGE_FAILED_LOG`(첫 실행 로그의 절대 경로)를 받아 돌고, 출력은 `post-merge-N-retry.log`. 통과하면 루프는 계속 가고 이슈 코멘트·Slack에 플레이크 의심을 남긴다. 실패한 테스트를 못 찾으면 다시 돌리지 않고 비0으로 끝나야 한다. `POST_MERGE_CMD`와 같은 env(`TICKET_LOOP_POST_MERGE_WHY` · `TICKET_LOOP_POST_MERGE_SPEC_COMPLETE`)를 첫 실행과 같은 값으로 받는다(7장 6단계)
`POST_MERGE_EVERY`	4	병합 뒤 검증을 병합 몇 개마다 돌리나. 4면 네 번째 병합 뒤에 앞의 넷을 한 번에 검증한다. 그 전이라도 티켓의 스펙이 끝나거나 이번 실행의 마지막 티켓이면 돌린다. 1이면 병합마다, 0이면 스펙·실행이 끝날 때만. 아직 검증하지 않은 병합은 `.git/ticket-loop-post-merge.json`에 남는다. 묶음이 재실행까지 실패하면 `POST_MERGE_RETRY_CMD`로 처음 실패하는 병합을 찾는다(7장 6단계)
`POST_MERGE_COPY`	(없음)	그 전에 메인 체크아웃에서 복사할 파일(공백 구분), 예 `.env.local`. 검증 워크트리를 정리한 뒤에 놓으므로, 메인 체크아웃에서 지운 파일은 옛 사본도 남지 않는다

`POST_MERGE_KEEP`	(없음 = 깨끗한 체크아웃)	검증 워크트리를 정리할 때 남길 경로 패턴(공백 구분, `.gitignore` 꼴 — 검증 워크트리 루트 기준, 슬래시 없으면 어느 깊이든, `/` 로 시작하면 루트만), 예 `node_modules`. 의존성 디렉터리를 남겨 다시 설치하는 시간을 아낄 때만 쓴다. 검증마다 커지는 빌드 캐시(`.next` 등)는 넣지 않는다 — 다시 쌓인다(7장 6단계)
`POST_MERGE_PORT`	(없음)	그 명령이 쓰는 포트. 지정하면 비기를 최대 10분 기다린다
`POST_MERGE_WORKTREE`	`~/.cache/ticket-loop/<owner-repo>/post-merge`	병합 뒤 검증을 돌릴 전용 워크트리. 저장소 밖에 두고 재사용하되, 검증마다 시작할 때 기본 브랜치 끝 그대로 되돌린다(추적 파일의 수정은 버리고, 추적 안 된·무시된 파일은 `POST_MERGE_KEEP` 에 맞는 것 말고 지운다). 메인 체크아웃을 가리키면 루프가 시작하지 않고(종료 코드 2), 이미 있는 그 자리가 이 저장소의 연결 워크트리가 아니거나 브랜치에 올라 있으면 정리하지 않는다
`LOG_FILE`	`.ticket-loop/loop-MMDD.log`	진행 로그. 복구와 회고의 1차 자료다. `/dev/null` 이면 끈다. 상대 경로는 저장소 루트 기준이고 시작 때 절대 경로로 굳힌다(`RECORDS_FILE` · `REPORT_DIR` 도 같다) — 병합 뒤 검증 로그 `post-merge-N.log` 는 이 파일과 같은 디렉터리에 생긴다. 터미널에는 같은 줄을 단계별 색으로 보여 준다(티켓 제목 굵은 자주, 단계 전환 청록, 성공 초록, 되풀이·수리·건너뜀 노랑, 중단 굵은 빨강, 시각과 워커 화면 꼬리는 흐리게) — stderr가 터미널이 아니거나 `NO_COLOR` 가 있으면 맨글자고, 파일에는 언제나 색 코드 없이 쓴다. 실행 맨 앞에는 키트 머리줄 `── ticket-loop-kit 0.9.6 (커밋) ──…` (굵은 주황)을 한 줄 찍는다 — 같은 날 로그에 이어 붙는 실행의 경계이고, 그 실행을 어느 키트 버전이 돌렸는지 남긴다(`.ticket-loop/kit.json` , 없으면 `unknown`)
`PROGRESS_SEC`	`60`	PR·수리를 기다리는 동안 워크트리를 git으로 들여다보는 간격. 브랜치 생성(`issue-N` → `issue-N-slug`), 첫 파일 수정, 새 커밋(제목 그대로, 오래된 것부터)을 그때 바로 한 줄씩 찍는다 — 워크트리를 만든 뒤 PR까지 수십 분 동안 루프 터미널이 조용해 멈춘 것처럼 보이던 구간이다. 백엔드·에이전트와 무관하고 네트워크를 쓰지 않는다. `0` 이면 끄고 idle 대기 상한(10분)마다만 돌아온다
`HEARTBEAT_SEC`	`300`	커밋은 없지만 커밋 안 된 변경 파일 목록이 지난 줄과 달라졌을 때 "working — Nm elapsed, M uncommitted files: …(이름 셋까지)" 한 줄을 찍는 최소 간격. 목록이 같으면(또는 비어 있으면) 시간이 아무리 지나도 찍지 않는다 — 같은 내용을 되풀이하지 않는다. 이슈를 읽고 TDD로 도는 첫 수십 분은 커밋이 없어도 무엇을 만지는지 보인다
`RECORDS_FILE`	`.ticket-loop/tickets.jsonl`	티켓마다 한 줄씩 남기는 기록(벽시계와 그 단계별 몫 — 구현·수리·CI·병합 뒤 검증, 수리 횟수, 병합 뒤 검증 결과, 모델, 에이전트 토큰, 그 실행의 로그 파일). 기계별 원자료라 커밋하지 않는다(`.gitignore` 에 `/.ticket-loop/*.jsonl`). 스펙 리포트가 읽는다(7장 9단계)
`REPORT_DIR`	`docs/loop-reports`	`--parent` 실행이 끝나면 `spec-<N>.html` 을 쓰는 디렉터리. 리포트는 사람이 PR로 커밋한다
`REPORT_OPEN`	`1`	리포트를 쓴 뒤 기본 브라우저로 연다(macOS `open` , Linux `xdg-open`). `0` 이면 끔. SSH 세션·화면 없는 Linux에서는 값과 무관하게 열지 않는다
`REPORT_DECISIONS_HEADING`	없음(모으지 않음)	PR 본문에서 사람이 봐야 할 절의 마크다운 제목(글자 그대로, 예: `사람 확인 필요`). 지정하면 스펙 리포트가 각 티켓 PR 본문에서 그 절을 뽑아 머리의 목록으로 모으고(티켓·PR 링크와 절 본문 그대로), KPI에 건수를, 절이 있는 행에 표시를 단다. 절이 없거나 본문이 빈 PR은 목록에 없다. 저장소마다 절 이름이 다르므로 키트는 이름만 받는다 — `pr` 스킬에 그 절을 두고, 워커가 스펙과 다르게 정했거나 스펙이 사람에게 남긴 결정을 내렸을 때만 적게 한다(기록용 "남은 판단 사항"과 분리). 7장 9단계
`REPORT_DECISIONS_NONE`	`없음¦none¦N/A`	그 절에 "내용 없음"으로 적는 말들(`¦` 구분, 대소문자 무시). 절 본문에서 목록 기호(`-` · `*`)와 끝 마침표를 걷은 것이 이 가운데 하나와 같으면(`- 없음` , `없음.`) 빈 절처럼 목록·건수·표시에서 뺀다. `- 없음 — 다만 …` 처럼 뒤에 내용이 더 붙으면 남는다. `pr` 스킬이 "없으면 '없음'"이라고 쓰게 하는 저장소에서, 사람이 볼 건수가 '없음'뿐인 PR에 묻히지 않게 한다. 7장 9단계
`MODEL_REGEX`	Claude Code 상태줄의 `Opus 5.5` 꼴 / Codex 기동 블록의 `gpt-5.1-codex` 꼴	워커 터미널 화면에서 모델 이름을 뽑는 정규식(ERE). 못 찾으면 리포트에 "not recorded"(Codex는 상태줄 설정에 따라 모델이 안 보일 수 있다)

`SLACK_WEBHOOK_URL`	없음(Slack 끔)	Slack Incoming Webhook URL. 루프 시작·티켓 완료·멈춤·루프 종료를 그 채널에 올린다. 시작에는 대상 수·백엔드·에이전트·키트 버전과 대상 목록이 붙는다. 멈춤에는 사유와 로그 끝 12줄(워커 화면 꼬리 포함)이 붙는다. 잠금을 잡은 실행만 보낸다(`--dry-run` , 이미 도는 루프와 겹친 실행은 안 보낸다). 비밀 값이라 커밋하는 `loop.env` 가 아니라 `~/.config/ticket-loop/loop.env` 에 둔다(아래)
`SLACK_MENTION`	없음	멈춤 알림 앞에 붙일 멘션, 예 `<@U012AB3CD>` (자기 멤버 ID) 또는 `<!channel>` . 채널 알림을 "멘션만"으로 두고도 멈출 때만 폰이 울린다. Ctrl-C로 멈춘 것에는 붙이지 않는다
`STOPPED_LABEL`	`loop-stopped`	티켓 처리 중 멈추면 그 이슈에 코멘트(사유·머신·PR·워크트리·루프 자신의 로그 끝 30줄)와 함께 붙이는 라벨. 없으면 만든다. 기록용이다 — 루프는 본인 계정으로 쓰므로 GitHub는 본인에게 알리지 않고, 대상 선택은 이 라벨을 보지 않는다. 루프가 그 티켓을 병합하면 뗀다
`LOOP_TRIAGE`	`1`	멈출 일이 생기면 멈추기 전에 수습 세션을 띄운다(10.6). `0` 이면 전처럼 바로 멈춘다
`TRIAGE_MAX`	`1`	한 실행에서 수습 세션을 띄우는 최대 횟수. 다 쓰면 그 뒤의 멈춤은 수습 없이 멈춘다. 같은 멈춤(종류·이슈)에는 언제나 한 번뿐이다
`TRIAGE_TIMEOUT_SEC`	`3600`	수습 세션 한 번의 제한 시간. 넘기면 판정 없이 멈춘다
`TRIAGE_LABEL`	`loop-triage`	수습 세션이 낸 수정 PR에 루프가 붙이는 라벨. 없으면 만든다. 사람이 나중에 몰아서 본다

저장소마다 고정할 값은 `.ticket-loop/loop.env` 에 `CI_SKIP_REGEX="${CI_SKIP_REGEX:-^docs/}"` , `POST_MERGE_PORT="${POST_MERGE_PORT:-3000}"` 꼴로 적어 커밋한다. 루프가 시작할 때 읽고, 실행할 때 준 환경변수가 이긴다. 키트의 `/setup-ticket-loop` 가 이 파일을 만든다. 컴퓨터마다 다른 값과 비밀 값(`SLACK_WEBHOOK_URL`)은 커밋하지 않는 `~/.config/ticket-loop/loop.env` (`XDG_CONFIG_HOME` 을 따른다)에 같은 꼴로 둔다. 저장소 `loop.env` 보다 먼저 읽어 그쪽이 이기고, 그 컴퓨터의 모든 저장소에 적용된다. 저장소 `loop.env` 에 Slack Webhook URL이 있으면 시작할 때 경고한다.

스크립트 내부 상수

상수	값	의미
`IDLE_WAIT_MS`	10분	워커가 입력 대기 상태가 되기를 한 번 기다리는 상한. PR·수리 대기에서는 진행 로그 간격 `PROGRESS_SEC` (1분)마다 돌아와 워크트리를 보고 다시 기다린다(`PROGRESS_SEC=0` 이면 이 값대로)
`IDLE_POLL_SEC`	5초	(cmux·tmux) 에이전트 상태를 다시 읽는 간격
`SEND_ACCEPT_SEC`	15초	(cmux·tmux) 프롬프트를 보낸 뒤 에이전트가 새 턴을 시작하는지 (running) 0.5초마다 보는 상한. 넘기면 수락되지 않은 것으로 보고 멈춘다 (cmux는 프롬프트가 화면에 찍혔으면 수락으로 본다)
`IDLE_STRIKES`	3	PR 없이 연속 3번 대기 상태면 "멈춤"으로 본다. 수리 뒤 새 커밋 없이 연속 3번 대기 상태여도 수리가 끝난 것으로 보고 CI를 다시 본다 — 재실행만 한 수리가 통과하면 이어 가고, 아니면 "새 커밋 없음"으로 멈춘다. 화면이 바뀌면 처음부터 센다
`STARTUP_GRACE_SEC`	120	기동 직후 2분은 대기 상태를 세지 않는다(프롬프트 처리 전에도 대기로 잡힌다)
`AGENT_START_WAIT_SEC`	90	에이전트 배너가 보이기를 기다리는 상한. 넘기면 한 번 다시 띄우고, 또 넘기면 멈춘다
`CI_APPEAR_TIMEOUT_SEC`	15분	PR에 체크가 나타나기를 기다리는 상한
`GH_TRIES` · `GH_RETRY_SEC`	3 · 15초	GitHub 일시 장애(5xx·연결 오류)에 걸리면 첫 시도를 포함해 세 번까지, 15초·30초 쉬며 다시 해 본다. 조회(이슈·PR·체크·GET `api` ·GraphQL 조회)와 병합(7장 5단계)만이다 — 코멘트·라벨·이슈 닫기는 응답만 실패하고 처리됐을 수 있어 다시 하지 않는다. 오래 도는 `--watch` 도 감싸지 않고 결과를 다시 읽는다(7장 4단계)
`PR_SETTLE_SEC`	2분	PR을 찾은 뒤 워커의 이슈 정리를 기다리는 상한. 이슈에 `IN_REVIEW_LABEL` 이 붙거나 워커가 대기 상태면 바로 넘어간다(7장 2단계)

`REPAIR_SETTLE_SEC`	5분	수리 중 대기 상태가 끝내 안 잡힐 때, 새 커밋이 push된 뒤 head가 이만큼 그대로이고 워크트리가 깨끗하면 수리가 끝난 것으로 본다(7장 4단계)
`PORT_WAIT_SEC`	10분	`POST_MERGE_PORT` 가 비기를 기다리는 상한. 기다리는 동안 30초마다 다시 본다 — 로그 한 줄 뒤 조용해도 멈춘 게 아니다(7장 6단계, R10)

어디서 실행하나

루프 자체는 평범한 셸 프로세스다. 워커 터미널은 Orca가 관리하지만, 루프 셸은 그렇지 않으므로 루프를 띄운 터미널이 닫히면 루프도 죽는다(워커는 계속 일한다 — R1). 권장은 **메인 체크아웃의 Orca 터미널**에서 실행하는 것이다. Orca 앱이 켜져 있는 동안 그 터미널이 유지되고, 루프와 워커들이 한 화면에서 보이며, Orca가 꺼지면 어차피 루프도 멈춘다(2.4). 별도 터미널 앱에서 돌릴 때만 tmux가 필요하다. **cmux 백엔드는 선택이 없다**: 소켓 모드 기본값(`cmuxOnly`) 때문에 cmux 터미널 안에서만 돈다. **tmux 백엔드**는 어느 터미널 앱이나 SSH에서 돌고, 루프를 tmux 세션 안에서 띄우면 워커 창이 같은 세션에 열려 한 화면이 된다(Windows는 4장 끝 절).

```
# 권장: Orca 사이드바에서 메인 체크아웃(루트 워크트리)을 열고 그 터미널에서
bash scripts/ticket-loop.sh --parent 40          # 로그는 기본으로 .ticket-loop/loop-MMDD.log
# 또는 CLI로 메인 체크아웃에 터미널을 하나 띄워 실행
orca terminal create --worktree id:<repoId>::<메인 체크아웃 경로> \
  --command "bash scripts/ticket-loop.sh --parent 40" --json

# cmux 백엔드: cmux 터미널 안에서 (loop.env에 BACKEND가 있으면 환경변수는 생략)
BACKEND=cmux bash scripts/ticket-loop.sh --parent 40

# tmux 백엔드: 아무 터미널에서. 세션 안에서 띄우면 워커 창이 같은 세션에 열린다
tmux new -s ticket-loop                          # 떼기 Ctrl-b d, 붙기 tmux attach -t ticket-loop
BACKEND=tmux bash scripts/ticket-loop.sh --parent 40

# Orca 백엔드를 외부 터미널 앱(iTerm, GNOME Terminal 등)에서 돌릴 때만
tmux new -s loop                                 # 창을 닫아도 세션이 남는다

# 노트북이면 어느 쪽이든 잠자기 방지를 앞에 붙인다
caffeinate -is bash scripts/ticket-loop.sh       # macOS
systemd-inhibit bash scripts/ticket-loop.sh      # Linux (WSL은 Windows 전원 옵션)
```

로그는 `[YYYY-MM-DD HH:MM:SS]` 형식으로 시작한다. macOS(`osascript`)나 Linux(`notify-send`)에서는 티켓 완료·루프 중단·종료 때 데스크톱 알림이 뜬다. 같은 저장소에서는 루프가 하나만 돌도록 잠금(`.git/ticket-loop.lock`)을 건다. 루프를 도는 중에 스크립트 파일이 바뀌면 bash가 엉뚱한 줄을 읽으므로, 스크립트를 고칠 일이 있으면 사본으로 돌린다.

7. 티켓 한 건의 생애

1. **워크트리 생성.** 기본 브랜치를 fetch한 뒤 다음 한 번의 호출로 워크트리를 만들고 Claude Code를 띄워 워커 프롬프트(부록 B)를 보낸다. `orca.yaml` setup 훅이 먼저 돈다.

   ```
   orca worktree create --repo id:<repoId> --name issue-N --no-parent \
     --base-branch origin/main --issue N --agent claude --prompt "…" --comment "…" --json
   ```

   응답에서 워크트리 ID(`<repoId>::<경로>`)와 에이전트 터미널 핸들을 받아 둔다. 워크스페이스 상태는 `in-progress`가 된다. cmux·tmux 백엔드에서는 같은 일을 루프가 직접 한다 — `git worktree add` → `WORKTREE_SETUP_CMD` → cmux 워크스페이스 또는 tmux 창에 `claude …` 기동(2.5). 이어서 터미널 화면에 에이전트 배너(`AGENT_BANNER_REGEX`)가 보이는지 90초까지 확인하고, 안 보이면 입력 줄을 지우고 `AGENT_RESTART_CMD "<워커 프롬프트>"`로 한 번 다시 띄운다. 그래도 안 뜨면 멈춘다.

2. **PR 대기.** head 브랜치가 `issue-N`과 같거나 `issue-N-`로 시작하는 열린 PR이 생길 때까지 반복한다. 워커는 `implement` 스킬대로 `issue-N-slug`를 만들지만, 루프가 만든 워크트리 브랜치 `issue-N`에 그대로 커밋해 PR을 열어도 같은 티켓의 PR이다 — rental-portal #102 실측(0.8.5): 워커가 `issue-102`로 PR #120을 열었는데 루프는 `issue-102-`만 봐서 못 찾고 idle 3회 뒤 "PR 없이 멈춤"으로 죽었다. `issue-1020`처럼 번호가 이어지는 브랜치는 다른 티켓이다.

   - **진행 로그.** 기다리는 동안 `PROGRESS_SEC`(1분)마다 워크트리를 git으로 들여다본다 — 브랜치가 `issue-N-slug`로 바뀌면 "branch …", 처음 파일이 수정되면 "first file changes — a.ts, b.ts, c.ts +1 more", 새 커밋이 들어오면 "commit `abc1234` feat: … #N"을 바로 한 줄씩. 그 밖에는 변경 파일 목록이 지난 줄과 달라졌을 때만, `HEARTBEAT_SEC`(5분)에 한 번까지 "working — Nm elapsed, M uncommitted files: …"을 남긴다. 목록이 그대로면 조용하다 — 같은 줄을 되풀이하지 않는다. 워커 화면을 읽어 도구 호출을 뽑지는 않는다(에이전트마다 형식이 다르다). 이 줄들은 터미널과 `LOG_FILE`에 함께 간다.
   - PR이 생기면 워커가 이슈 정리(인수 조건 체크박스, 맨 끝에 **in-review** 라벨 — `implement` 스킬의 마지막 단계)를 마치기를 기다린다. 이슈에 `IN_REVIEW_LABEL`이 붙었거나 워커가 대기 상태면 바로, 아니면 2분(`PR_SETTLE_SEC`)에서 넘어간다. 기다리는 까닭은 CI 경로 필터에 걸려 체크 없이 바로 병합하는 PR이다 — 기다리지 않으면 워커보다 먼저 병합·이슈 닫힘·라벨 제거가 일어난다. 0.9.1까지는 대기 상태 하나만 10분까지 기다렸다. rental-portal 스펙 #141(cmux)에서 lifecycle이 빈 값으로 이어져 13건 중 3건이 10분을 다 썼고(라벨은 PR 생성 8~12초 뒤에 붙어 있었다), CI는 그사이 끝나 2~3초로 찍혔다. `IN_REVIEW_LABEL`을 바꾼 저장소는 `implement` 스킬의 라벨 이름도 같이 바꿔야 라벨로 끝난다.
   - PR 없이 워커가 대기 상태면(기동 후 2분 제외) 30초 쉬고 **PR부터 다시 본다**. PR을 여는 순간에는 GitHub에 PR이 보이기 전에 대기 상태가 먼저 잡히기 때문이다. 그래도 PR이 없으면 터미널 화면을 지난번과 비교해, 바뀌었으면 그사이 일한 것이므로 횟수를 0으로 되돌리고, 그 위에 1회를 센다. 연속 3회면 **PR 없이 멈춤**으로 루프를 멈춘다. 워커가 계속 출력 중이면 횟수를 0으로 되돌린다.
   - **idle 오판**은 일하던 워커를 대기 상태로 본 횟수다(티켓 요약·기록). 다시 보니 PR이 열려 있을 때 PR이 대기 상태보다 먼저 열렸으면 일을 마친 워커라 세지 않고, 나중에 열렸으면 센다. PR 없이 센 대기 뒤에 워커가 다시 일했으면(화면이 바뀜, 대기가 풀림, PR을 엶) 그 대기들도 센다. 연속 3회로 멈추면 그 연속 대기는 오판이 아니다. 0.9.1까지는 "다시 보니 PR이 열려 있음"을 다 셌다 — 대개 PR을 연 뒤 대기에 든, 일을 마친 워커였다.
   - 백엔드 터미널에 닿지 못하면(앱 종료, tmux 창 사라짐 등) 즉시 멈춘다(cmux·tmux는 5번 연속 조회 실패일 때). 대기 상태를 기다리다 시간이 다 된 것(워커가 아직 일하는 중 — 진행 로그를 켜 두면 1분, 끄면 10분마다)은 닿지 못한 것과 구별해 계속 기다린다(2.4). 타임아웃도 아닌 이상한 응답(사람이 Orca에서 그 워크트리를 열어 세션이 다시 붙을 때 생긴다)은 터미널 읽기를 10초 간격으로 5번 다시 해 보고, 전부 실패할 때만 닿지 못한 것으로 본다. 티켓 제한 시간을 넘기면 타임아웃으로 멈춘다.

3. **충돌 확인.** PR의 병합 가능 여부(`mergeable`)를 본다. 계산 중(`UNKNOWN`)이면 10초 간격으로 최대 6번 다시 본다. 충돌(`CONFLICTING`)이면 같은 워커 터미널에 "기본 브랜치를 병합해 충돌을 해결하고, 테스트·lint 후 push하라"고 보낸다.

4 **CI 대기.** 저장소 CI의 경로 필터에 걸리는 PR(`CI_SKIP_CMD`가 `skip`, 또는 바뀐 파일이 전부 `CI_SKIP_REGEX`에 맞음)이면 체크를 기다리지 않고 병합으로 넘어간다(병합 뒤 검증도 건너뛴다). 그 밖에는 체크가 나타나기를 최대 15분 기다린 뒤 `gh pr checks --watch --fail-fast`로 끝까지 본다. `--watch`는 GitHub API 오류 한 번에도 체크 실패와 같은 코드(1)로 끝나므로, 실패로 끝나면 체크 상태(`bucket`)를 다시 읽어 실패·취소한 체크가 있을 때만 실패로 본다 — 다 통과했으면 통과, 아직 도는 체크가 있으면 세 번까지 다시 본다. 실패하면 워커에게 "`gh pr checks` · `gh run view --log-failed`로 원인을 보고 고쳐 push하라, 같은 커밋의 다른 job은 통과했고 러너·네트워크 오류처럼 코드와 무관하면 코드를 바꾸지 말고 `gh run rerun <run-id> --failed`로 실패한 job만 다시 돌려라"고 보낸다. 수리 뒤에도 PR의 head 커밋이 그대로면(새 push 없음) CI를 한 번 더 본다 — 재실행이 도는 중이면 끝까지 보고, 통과면 충돌·CI 확인부터 이어 가고, 그래도 실패면 "수리 뒤 새 커밋 없음"으로 멈춘다. 0.9.4까지는 head만 비교해, 워커가 재실행만으로 녹색을 만든 PR에서도 멈췄다(rental-portal #258 — self-hosted 러너 둘의 설치 경합). 충돌·CI 수리는 합쳐서 `REPAIR_ROUNDS`회까지다. 수리가 끝났는지는 워커의 대기 상태로 보는데, 대기 상태여도 PR에 새 커밋이 아직 없으면 바로 끝내지 않는다 — 화면이 그대로인 대기를 연달아 3번(`IDLE_STRIKES`, 그 사이 30초) 볼 때만 끝이고, 그 전에 커밋이 들어오면 끝, 화면이 바뀌거나 워커가 다시 일하면 처음부터 센다(PR 대기와 같은 규칙). 수리 프롬프트 직후의 묵은 대기 상태(2.5)와 백그라운드 서브에이전트를 기다리는 동안의 대기를 수리 끝으로 읽지 않기 위해서다. 0.9.2까지는 첫 대기에서 끝내, 수리 요청 5초 만에 "새 커밋 없음"으로 멈춘 일이 있다. 대기가 끝내 잡히지 않으면(cmux lifecycle이 빈 값으로 이어질 때) 새 커밋이 push된 뒤 head가 5분(`REPAIR_SETTLE_SEC`) 그대로이고 워크트리에 커밋 안 된 변경이 없을 때 끝난 것으로 본다. 0.9.1까지는 티켓 제한 시간까지 기다렸다.

요약·기록의 CI 소요는 PR head 체크들의 GitHub 시각 — 가장 이른 시작부터 가장 늦은 끝까지 — 이다(못 읽으면 루프가 기다린 시간). 루프가 CI 대기를 시작한 때부터 재면, PR을 늦게 알아챈 티켓에서 이미 끝난 CI가 2~3초로 찍힌다.

5 **병합.** `gh pr merge --<MERGE_METHOD>`. GitHub 일시 장애(5xx, 연결 오류, "Base branch was modified")로 실패하면 15초·30초 쉬고 세 번까지(`GH_TRIES`) 다시 해 보고, 실패할 때마다 먼저 PR이 이미 병합됐는지 본다 — 서버는 병합을 끝냈는데 응답만 502였을 수 있다. 그 밖의 거절(필수 리뷰 같은 브랜치 보호, 권한)은 바로 멈추고(멈춤 종류 `merge_failed`, 사유에 `gh` 오류 첫 줄) 사람이 R3로 이어받는다. 5초 뒤에도 이슈가 열려 있으면 직접 닫고 코멘트를 남긴다. `in-review`를 지운다. 이어서 이 티켓이 속한 스펙 이슈(`--parent` 값, 없으면 GitHub의 부모 이슈)의 하위 이슈 목록을 세어 모두 닫혔으면 스펙 이슈도 코멘트와 함께 닫는다. 요약 값 `sub_issues_summary`는 쓰지 않는다(선행 이슈 요약처럼 늦게 바뀔 수 있다). 하위 이슈가 하나라도 열려 있거나 스펙이 이미 닫혔으면 아무것도 하지 않는다 — 같은 검사를 `--parent` 실행이 끝날 때 한 번 더 하므로, 마지막 티켓을 사람이 닫은 스펙도 다음 실행에서 닫힌다.

6 **병합 뒤 검증 (선택).** `POST_MERGE_CMD` 가 있으면 병합된 기본 브랜치를 저장소 밖 전용 워크트리에 받아(기본 브랜치 끝 그대로 되돌리고, `POST_MERGE_COPY` 파일을 복사하고, `POST_MERGE_PORT` 가 비기를 기다린 뒤) 그 명령을 한 번 돌린다. 로그는 `LOG_FILE` 과 같은 디렉터리의 `post-merge-N.log` (기본 저장소 루트의 `.ticket-loop/post-merge-N.log`)이고, "post-merge check started" 줄과 실패 ERROR 줄에 그 절대 경로를 적는다. 실패하면 로그 꼬리 40줄을 흐리게 보여 주고, 이 티켓의 정리와 요약까지 마친 뒤 루프를 멈춘다. 기본 브랜치가 깨진 채로 다음 티켓을 쌓지 않기 위해서다(R10).

검증 워크트리 정리. 검증 워크트리는 저장소마다 하나이고, 처음 한 번 만든 뒤 다시 쓴다. 검증이 시작할 때마다 루프는 그 워크트리를 기본 브랜치 끝과 똑같이 되돌린다 — 강제 체크아웃으로 추적 파일의 수정을 버리고, 추적 안 된 파일과 무시된 파일(중첩 git 저장소·빈 디렉터리 포함)을 지운다. 그래서 검증은 CI처럼 깨끗한 체크아웃에서, 병합된 그 파일 그대로 돈다. 지난 검증이 남긴 빌드 캐시·생성물이 이번 결과를 바꾸지 않고, 지난 검증이 고쳐 둔 추적 파일 때문에 체크아웃이 실패하지도 않는다. 남길 것은 `POST_MERGE_KEEP` 에 `.gitignore` 꼴 패턴으로 적는다(기본 없음). 의존성 디렉터리를 남겨 다시 설치하는 시간을 아끼는 식이다: `POST_MERGE_KEEP="${POST_MERGE_KEEP:-node_modules}"` . 검증마다 커지는 빌드 캐시(`.next` 등)는 넣지 않는다 — 넣으면 다시 쌓인다. `POST_MERGE_COPY` 파일은 정리 뒤에 복사하므로 메인 체크아웃에서 지운 파일은 옛 사본도 남지 않는다. 정리는 검증이 시작할 때 한 번뿐이다. 첫 실행·재실행·이분 탐색 사이에는 하지 않고(재실행은 첫 실행이 설치·빌드한 것을 그대로 쓴다), 검증이 끝날 때도 하지 않는다 — 실패한 검증의 산출물(서버 로그, 빌드 결과)은 멈춤 사유와 멈춤 기록이 가리키는 그대로 다음 검증이 시작할 때까지 남는다. 그래서 검증 워크트리는 "남긴 것 + 검증 한 번이 만드는 것"보다 커지지 않는다. 지운 것이 있으면 시작 줄 앞에 "cleaned the check worktree — removed N untracked or ignored paths" 한 줄을 남긴다(크기는 재지 않는다). 지우지 못하면(권한 등) 경고 한 줄 뒤 검증을 그대로 돌린다. 안전장치가 있다. `POST_MERGE_WORKTREE` 가 메인 체크아웃을 가리키면 루프는 시작하지 않는다(종료 코드 2). 이미 있는 그 자리가 이 저장소의 연결 워크트리가 아니거나 브랜치에 올라 있으면 정리하지 않고 검증을 실패로 끝낸다(did not start) — 루프는 검증 워크트리를 늘 브랜치 없이(detached) 만들고 옮기므로, 브랜치에 올라 있으면 사람이나 워커의 작업 트리다. 멈춘 검증 워크트리를 조사하다 브랜치로 옮겼으면 `git switch --detach` 로 되돌려 둔다. 0.9.22까지는 체크아웃만 옮겨 다시 써서, rental-portal의 검증 워크트리가 57GB까지 컸다(`.next/dev/cache/turbopack` 만 51GB). 저장소 밖(`~/.cache/ticket-loop/…`)에 있어 디스크가 찰 때까지 아무도 몰랐다.

재실행 (선택). 무거운 스위트는 그 병합과 무관하게 아무 테스트나 가끔 시간 초과로 깨진다(rental-portal의 44분 통합 테스트는 7회 중 2회, 매번 다른 테스트 하나). `POST_MERGE_RETRY_CMD` 가 있으면 멈추기 전에 포트 대기를 다시 지나 같은 검증 워크트리에서 그 명령을 딱 한 번 돌린다. 첫 실행 로그 경로가 `POST_MERGE_FAILED_LOG` 로 가므로 실패한 테스트만 골라 다시 돌릴 수 있다. 통과하면 결과는 "passed on retry in Ns (first run failed in Ms)"로 요약·기록에 남고, 이슈에 "병합 뒤 검증 재시도 통과 — 플레이크 의심" 코멘트(머신, 두 로그 파일 이름, 소요 시간 — 로그 본문은 올리지 않는다)와 멘션 없는 Slack 알림이 간다. `loop-stopped` 라벨은 붙이지 않는다. 재실행도 실패하면 두 로그 경로를 적고 멈춘다. 키트는 테스트 도구를 모르므로 무엇을 다시 돌릴지는 저장소의 스크립트가 정한다. vitest라면 이렇게 커밋해 두고(검증 워크트리는 병합된 기본 브랜치다) `POST_MERGE_RETRY_CMD="${POST_MERGE_RETRY_CMD:-pnpm install --frozen-lockfile --prefer-offline && bash scripts/post-merge-retry.sh}"` 로 건다. 바뀐 게 없으면 설치는 몇 초다 — 묶음 실패의 원인 병합을 찾으려고 지난 커밋에서 스크립트를 다시 돌릴 때 의존성을 맞춘다(아래 묶어 돌리기).

```
# scripts/post-merge-retry.sh — 첫 실행 로그에서 실패한 테스트 파일만 다시 돌린다
files=$(perl -pe 's/\e\[[0-9;]*m//g' "$POST_MERGE_FAILED_LOG" \
  | grep -oE 'FAIL +[^ ]+\.test\.ts' | awk '{print $2}' | sort -u)
[ -n "$files" ] || { echo "no failed test files found — not retrying"; exit 1; }
pnpm exec vitest run --config vitest.integration.config.ts $files
```

실패한 파일을 못 찾으면(설치·환경 점검 단계에서 실패) 다시 돌리지 않고 실패로 끝내는 것이 요점이다 — 진짜 깨진 기본 브랜치가 재실행에 묻히지 않게. 로그에 색 코드가 섞여 있고 macOS 기본 `sed` 는 `\x1b` 를 못 알아들어 `perl` 로 지운다.

묶어 돌리기 (선택). 병합 뒤 검증은 누적이다 — 기본 브랜치 끝에서 도는 검증은 그 전의 병합을 모두 덮는다. 그래서 병합 `POST_MERGE_EVERY` 개(기본 4)마다 한 번만 돌려도 검증을 거치지 않는 병합은 없다. 1이면 전처럼 병합마다 돌고, 0이면 개수로는 돌지 않고 스펙·실행이 끝날 때만 돈다. 루프는 한 번에 한 티켓이라 병합 직전 기본 브랜치는 워커가 테스트한 그 상태이고, 워커가 관련 테스트를 PR 전에 이미 돌렸다면 병합마다 도는 전체 스위트가 더해 주는 것은 "관련 테스트 선별이 놓친 테스트"뿐이다. 실측(rental-portal, 키트 0.9.5): 통합 테스트가 티켓마다 47~52분, 스펙 #248의 12티켓에서 9.8시간이었고, 워커가 PR 전에 관련 통합 테스트를 돌리기 시작한 뒤 34회 동안 잡은 회귀가 없었다.

- **언제 돌리나.** 아직 검증하지 않은 병합이 `POST_MERGE_EVERY` 개가 되면(0이면 이 조건은 없다), 또는 그 전이라도 티켓의 스펙이 끝났거나(루프가 스펙 이슈를 닫음) 이번 실행의 마지막 티켓이면(`--max` , 번호 지정, 자동 선택이면 다음 대상이 없을 때) 그 티켓의 병합 뒤에 돌린다. 그 밖의 티켓은 요약에 "post-merge check deferred (k/N pending)"로 남는다. 루프가 끝날 때 남은 병합이 있으면 한 번 더 돈다(`post-merge-N-end.log`). 남은 병합이 없어도 이번 실행이 닫은 스펙에 그 번호를 받은 검증이 없었으면 한 번 돈다(아래 "상황에 따라 다른 검증"). CI가 붙지 않는 문서 PR은 세지 않는다 — 다만 스펙을 끝낸 문서 PR은 넣는다. 창이 비어 있어도(`POST_MERGE_EVERY=1`) 스펙 끝의 검증이 돌게 하려는 것이다.
- **결과.** 검증이 다른 병합까지 덮었으면 결과에 ", covering #a #b …"가 붙고, 통과하면 그 커밋이 다음 묶음의 시작이 된다. 아직 검증하지 않은 병합과 마지막으로 통과한 커밋은 `.git/ticket-loop-post-merge.json` 에 남는다 — 루프가 멈췄다 다시 실행돼도 묶음

이 이어지고, 시작 줄에 "not checked yet"으로 알린다.

- **어느 병합이 깨뜨렸나.** 묶는 대가는 이것을 모르는 것이다. 묶음이 재실행까지 실패하면 루프가 같은 검증 워크트리에서 `POST_MERGE_RETRY_CMD`(실패한 테스트만)를 마지막으로 통과한 커밋에서 한 번 돌리고(통과해야 탐색이 성립한다), 그 뒤의 병합 커밋들을 이분 탐색한다. 병합 넷이면 세 번, 한 번에 1~2분이다. 찾은 병합("first failing merge: #13 (PR #23, commit …)")은 결과·멈춤 사유와 멈춤 기록의 `post_merge_window`에 실려 무인 수습에 넘어간다. 시작 커밋에서도 실패하면 "not found"로 적는다 — 플레이크·환경, 또는 재실행 명령이 테스트를 못 고른 것이다. 로그는 `post-merge-N-bisect-<커밋>.log`. 재실행 명령이 없으면 탐색하지 않는다 — 전체 스위트로 탐색하면 묶어서 아낀 시간을 도로 쓴다. 지난 커밋에서 돌리므로 설치된 의존성에 기대는 재실행 명령이면 앞에 설치를 붙인다(`pnpm install --frozen-lockfile --prefer-offline && …`).
- **잃는 것.** 깨진 기본 브랜치 위에 다음 티켓이 최대 `POST_MERGE_EVERY`−1개 쌓인다. 그 깨짐은 관련 테스트가 못 잡은 것이라 다음 워커도 대개 알아채지 못하고, 고칠 때 그 위에 쌓인 변경까지 봐야 할 수 있다. 무거운 스위트가 짧으면(몇 분) `POST_MERGE_EVERY=1`로 병합마다 돌리는 편이 낫다. 0은 이 쌓임이 스펙 전체로 커진다.

상황에 따라 다른 검증 (선택). 평소에는 가벼운 검증을, 스펙이 끝날 때만 무거운 검증을 돌리고 싶을 때가 있다. 루프는 `POST_MERGE_CMD`·`POST_MERGE_RETRY_CMD`에 env 둘을 넘긴다. `TICKET_LOOP_POST_MERGE_WHY`는 왜 도는지다 — `every`(병합 `POST_MERGE_EVERY`개), `spec_end`(스펙 끝), `run_end`(이번 실행의 마지막 티켓·남은 티켓 없음·루프 끝). `TICKET_LOOP_POST_MERGE_SPEC_COMPLETE`는 이 검증이 덮는 병합 가운데 스펙(`--parent` 또는 티켓의 부모)을 끝낸 것이 있으면 그 스펙 번호(드물게 여럿이면 공백 구분), 없으면 빈 값이다. 둘을 따로 보는 까닭은 루프가 창이 찼는지를 먼저 보기 때문이다 — `POST_MERGE_EVERY=1`이면 스펙의 마지막 병합 뒤 검증도 이유가 `every`다. 스펙을 끝낸 병합은 아직 검증하지 않은 병합과 함께 `.git/ticket-loop-post-merge.json`에 남는다. 그래서 스펙 끝의 검증이 실패해 멈춘 뒤 다시 실행한 루프의 검증도 같은 값을 받는다. 루프 끝에서 처음 닫히는 스펙(사람이 마지막 티켓을 닫은 경우 등)은 스펙을 닫은 뒤 남은 병합을 검증하므로 그 검증이 스펙 번호를 받는다. 남은 병합이 없을 때도 있다 — `POST_MERGE_EVERY=1`이면 병합마다 검증이 통과해 창이 빈다. 그래도 그 스펙 번호를 싣고 통과한 검증이 이번 실행에 없었으면 기본 브랜치 끝에서 한 번 돈다(이유 `spec_end`, 로그 `post-merge-<스펙>-end.log`). 실패하면 무인 수습을 거쳐 스펙 이슈로 멈추고, 스펙 번호가 상태 파일에 남아 다시 실행한 루프의 검증이 같은 스펙 번호를 받는다. 사람이 스펙 이슈까지 닫은 경우는 루프가 알 수 없어 돌지 않는다. 재실행·이분 탐색·수습 뒤 재검증도 첫 실행과 같은 값을 받아, 실패한 검증과 같은 조건(같은 DB 등)으로 다시 돈다. 받은 값은 검증 로그마다 첫 줄(`# ticket-loop: TICKET_LOOP_POST_MERGE_WHY="…" TICKET_LOOP_POST_MERGE_SPEC_COMPLETE="…"`)과 티켓 기록(`post_merge_why`·`post_merge_spec_complete`)에 남고, 스펙 리포트는 결과 뒤 괄호에 적는다(예: "post-merge check passed, covering #61 #62 (every, spec #60 complete)"). 예 — `POST_MERGE_EVERY=1`로 병합마다 돌리되, 스펙을 끝낸 병합을 덮는 검증만 공유 DB로:

```
# scripts/post-merge.sh — POST_MERGE_CMD="${POST_MERGE_CMD:-bash scripts/post-merge.sh}"
if [ -n "$TICKET_LOOP_POST_MERGE_SPEC_COMPLETE" ]; then
  export DATABASE_URL="$SHARED_TEST_DATABASE_URL"   # 스펙 끝: 공유 DB(느리다)
fi                                                  # 평소: 로컬 DB
pnpm run test:integration
```

포트 대기는 멈춤이 아니다. 포트가 차 있으면 루프는 점유 프로세스와 기한을 한 줄 적고, 30초마다 다시 보며 조용히 기다린다. 대개 메인 체크아웃에서 사람이 띄워 둔 개발 서버다(실측: 아침에 띄운 `npm run dev`가 남아 4분을 기다렸다). 그 서버를 끄면 30초 안에 알아서 검증을 시작한다 — 루프를 다시 실행하지 않는다. 10분이 지나면 "port … did not free up"으로 멈춘다.

```
[2026-10-12 15:36:03] #67: PR #75 merged
[2026-10-12 15:36:04] #67: port 3000 in use (node pid 48213) — waiting for it to free up until 15:46:04. Stop that server and the loop continues right away
[2026-10-12 15:41:44] #67: post-merge check started (…/post-merge, log /Users/me/rental-portal/.ticket-loop/post-merge-67.log)
```

> **왜 워커가 아니라 루프가 돌리나.** 실제 서버·공유 개발 DB·고정 포트를 쓰는 통합·E2E 테스트는 워커가 돌리면 다른 워커나 사람의 개발 서버와 부딪힌다. 그래서 워커 규칙은 "돌리지 말고 미실행이라고 적어라"였다. 그 결과 실제 운영에서 PR 16건이 모두 "로컬에서 확인 필요"로 병합됐고, 어느 단계에서도 통합 테스트가 돌지 않아 기본 브랜치의 통합 스위트가 깨진 채 배포까지 갔다. 루프는 직렬이므로 병합 직후가 부딪힘 없이 돌릴 수 있는 유일한 지점이다. 다만 "전체를 돌리지 마라"가 "한 번도 돌리지 마라"가 되면 안 된다 — 워커가 새로 쓴 통합 테스트가 병합 뒤 검증에서 처음 돌아 깨지면 루프가 그 자리에서 멈춘다(실측). 그래서 워커 규칙은 자기가 추가·수정한 무거운 테스트 파일만 PR 직전에 1회 돌리게 한다(1~2분). 파일 하나는 포트·DB 부딪힘이 거의 없고, 전체 스위트는 여전히 루프 몫이다.

7 **정리와 요약.** 워크스페이스를 `completed`로 표시하고 `orca worktree rm --force`로 지운다(cmux: `workspace close --force`, tmux: `kill-window`, 둘 다 뒤에 `git worktree remove --force`). 이어서 `git worktree prune`을 하고, 로컬의 이슈 브랜치와 Orca가 워크트리용으로 만든 브랜치(설정에 따라 `issue-N` 또는 `<git 사용자>/issue-N`, 워크트리를 만든 직후 기록해 둔다), 원격의 이슈 브랜치를 지운다. 마지막으로 로그에 한 줄을 남긴다.

```
[2026-10-08 22:36:14] #42 summary: wall-clock 54m, PR #46 (picked up after 19s), CI 258s, repairs 0, idle false positives 0, post-merge check passed in 212s
```

"picked up after"는 PR 생성부터 루프가 다음 단계(충돌·CI 확인)로 넘어가기까지다. 몇십 초를 넘으면 2단계의 정리 대기가 상한까지 간 것이다. GitHub이 준 생성 시각이 지금보다 늦으면 지금으로 잘라 0초가 된다. 미국 서머타임이 끝나는 날 태평양 01:00-02:00(PDT)에 생긴 사건은 GitHub API가 1시간 뒤로 돌려준다. 1분 넘게 늦으면 `PR #N: GitHub reports it created at …, …s from now — using now` 경고가 남는다.

이 줄들이 `/retro`의 1차 자료다. 오래 걸린 티켓, 수리가 잦은 티켓, idle 오판이 많은 티켓을 찾아 원인을 본다.

같은 내용에 모델(터미널 화면에서 읽은 `MODEL_REGEX` 결과)과 시작·종료 시각을 더한 한 줄을 `RECORDS_FILE`(JSONL)에도 덧붙인다. `--no-merge`로 끝난 티켓은 `no-merge`, 처리 중 루프가 멈춘 티켓은 `stopped`(멈춘 이유 포함)로 남는다. 병합 뒤 검증 결과(`post_merge`)는 루프가 병합한 티켓에만 있다 — 병합 전에 멈춘 티켓에는 없다. 검증을 돌린 티켓에는 그 검증 명령이 받은 env(`post_merge_why`, 스펙을 끝낸 병합을 덮었으면 `post_merge_spec_complete` — 6단계)도 남는다. `log`는 그 티켓을 처리한 실행의 로그 파일이다. 로그는 날짜별 파일이라, 멈췄다가 다른 날 이어 간 스펙은 로그가 여러 파일로 나뉜다. 로그는 사람이 읽고, 기록은 리포트와 회고 안내가 읽는다.

`impl_sec` · `repair_sec` · `ci_wait_sec` · `post_merge_sec`는 경과 시간(`wall_sec`)을 단계별로 나눈 값이다(0.9.8부터, 초).

- **구현**: 에이전트가 뜬 때부터 PR이 열린 때(GitHub 시각, 지금보다 늦으면 지금)까지.
- **수리**: 충돌·CI 수리 프롬프트를 보낸 때부터 수리가 끝났다고 본 때까지, 모든 회차.
- **CI**: 첫 회차는 PR이 열린 때부터 결과까지다. 루프가 PR을 알아채기 전에도 CI는 돌기 때문이다. 수리 뒤 회차는 루프가 CI를 다시 보기 시작한 때부터 센다.
- **병합 뒤 검증**: 검증 명령이 돈 시간. 재실행과 무인 수습 뒤 재검증을 포함하고, 포트 대기는 뺀다.

나머지(워크트리 준비, PR을 알아채기, 병합·정리, 포트 대기, 무인 수습)는 어느 단계에도 넣지 않는다. 멈춘 티켓은 멈춘 때까지 센다. `ci_sec`는 전과 같이 통과한 마지막 회차의 GitHub 체크 시간이다.

`tokens`는 그 티켓에 든 에이전트 토큰이다(0.9.8부터, Claude Code 워커만). 워커와 그 티켓의 무인 수습 세션이 남긴 Claude Code 세션 기록(`~/.claude/projects/<작업 폴더>/`, 서브에이전트 포함)에서 응답마다 남는 사용량을 더한다. 값은 네 가지다.

- `input` : 캐시를 거치지 않은 입력.
- `cache_write` : 캐시에 쓴 입력.
- `cache_read` : 캐시에서 읽은 입력.
- `output` : 출력.

한 응답이 여러 줄로 남으므로 응답 ID로 한 번만 센다. 세션 기록을 못 찾으면 필드가 없다. Codex 워커는 세션 기록 형식이 달라 아직 세지 않는다.

```
{"spec":40,"issue":42,"title":"T02: …","result":"merged","started":"2026-10-08T12:42:10Z","finished":"2026-10-08T13:36:14Z","wall_sec":3244,"impl_sec":2731,"repair_sec":0,"ci_wait_sec":262,"post_merge_sec":212,"repairs":0,"idle_false_positives":0,"agent":"claude","log":".ticket-loop/loop-1008.log","pr":46,"pr_pickup_sec":19,"ci_sec":258,"post_merge":"passed in 212s","tokens":{"input":2412,"output":98714,"cache_write":516287,"cache_read":17724963},"model":"Opus 5.5"}
```

8 **다음 티켓.** 자동 선택이면 대상을 다시 계산한다. 이번 실행에서 이미 처리한 티켓은 GitHub 반영이 늦어도 다시 집지 않는다. 대상이 없거나 `--max`에 닿으면 "done: N tickets"를 남기고 끝난다. 이번 실행에서 끝난(하위 이슈가 모두 닫혀 루프가 닫은) 스펙이 있으면 그 앞에 회고를 권하는 안내 한 줄을 남긴다 — `/retro`가 읽을 자료(로그, 스펙 리포트, 병합된 PR 본문의 "남은 판단 사항"·"검증하지 못한 것")를 적어 둔다. 로그는 이번 실행의 것만이 아니라 그 스펙의 티켓이 지나간 로그 전부다 — 루프가 멈췄다가 다른 날 이어 가 스펙을 끝냈으면 `loop-1020.log, loop-1021.log`처럼 여럿을 적는다(기록의 `log`, 그 필드가 없는 0.9.6 이하 기록은 로그 파일의 티켓 머리줄로 찾는다). 권고일 뿐 루프가 회고를 돌리지는 않는다. `/retro`는 사람이 후보를 보고 고르는 대화형 스킬이고, 환경을 고치는 일은 검토를 거쳐 PR로 가는 것이 맞기 때문이다(11장).

9 **스펙 리포트 (`--parent` 일 때).** 루프가 정상 종료하면("done", "No more tickets to process", "No tickets to process") `REPORT_DIR/spec-<N>.html` 한 장을 쓰고 기본 브라우저로 연다(`REPORT_OPEN=0` 이면 열지 않는다). 스펙의 하위 티켓마다 한 행이다. 표의 고정 문구(열 이름·상태 배지·요약 라벨)는 영어이고, GitHub에서 가져온 이슈·PR 제목과 기록의 중단 사유는 원문 그대로 둔다.

열	내용과 출처
Status	Merged · Merged after stop(루프가 멈춘 뒤 루프 밖에서 병합 — 마지막 기록이 `stopped` 이고 PR 병합이 그보다 늦다) · PR open · In progress(담당자 있음) · Stopped(기록의 `stopped`) · Not started · Closed without PR — GitHub 이슈·PR 상태와 기록을 합쳐 판정
Result brief	PR 제목(링크), PR 본문의 인수 조건 체크 수(Acceptance criteria `- [x]` / 전체), 중단 사유(Stop reason — 마지막 기록이 `stopped` 면 병합된 티켓에도 보인다), 수리 횟수, 병합 뒤 검증 결과(post-merge check passed / failed / passed on retry / skipped / did not start / deferred, 묶음이면 ", covering #a #b …"), 멈춘 뒤 병합까지 걸린 시간(merged outside the loop … after the stop), 시도 횟수. 걸린 시간은 아래 열에 있으므로 여기에는 결과만 적는다
Implementation	구현 시간 + 워커의 수리 시간(7단계의 `impl_sec` + `repair_sec`). 수리가 있었으면 "incl. repair …"로 그 몫을 적는다. 0.9.7 이하 기록은 PR 생성 시각 – 시작으로 어림한다. 수리가 있었으면 그 시간을 알 수 없어 "excl. repairs"라고 표시한다. PR 없이 멈춘 지난 기록은 경과 전부를 구현으로 본다
CI	모든 회차의 CI 시간(`ci_wait_sec`). CI 경로 필터로 체크가 붙지 않는 PR은 "none"("path excluded"). 0.9.7 이하 기록은 통과한 마지막 회차(`ci_sec`)이고, 수리가 있었으면 "last run only"라고 표시한다
Post-merge check	병합 뒤 검증 명령(`POST_MERGE_CMD` , 예시 저장소에서는 통합 테스트)이 돈 시간(`post_merge_sec`). 재실행·이분 탐색 포함, 포트 대기 제외. 묶어 돌리면(`POST_MERGE_EVERY`) 그 검증을 돌린 티켓에 들고, 앞 티켓은 "—"이다. 0.9.7 이하 기록은 결과 문자열의 초를 더한다(재실행이면 첫 실행 + 재실행)
Elapsed	기록의 벽시계(워크트리 생성 → 병합·정리). • 아래 막대: 경과 시간을 앞의 세 열(열 이름 앞 점의 색)과 회색 나머지로 나눈다. 회색은 워크트리 준비, PR을 알아채기, 병합·정리, 포트 대기, 무인 수습이다. • 나머지가 1분 이상이면 "other …"라고 적는다. • 멈춘 뒤 병합된 티켓: 멈출 때까지의 루프 시간이고 "loop, until the stop"이라고 표시한다. • 기록이 없는 티켓(다른 PC·사람이 처리): PR 생성 → 병합 시간으로 대신하고 "PR created→merged"라고 표시한다. 막대는 없다.
Tokens	기록의 `tokens` 를 보인다. • 첫 줄: 네 값의 합. • 아래: out(출력), new in(캐시 밖 입력 + 캐시 쓰기), cached(캐시에서 읽은 입력). • 정확한 값은 마우스를 올리면 보인다. 기록에 없는 티켓(0.9.7 이하 등)은 리포트를 그릴 때 그 기록의 시작~끝 시각으로 세션 기록에서 다시 센다. 이때는 cmux·tmux의 `WORKTREE_DIR` 로 찾으므로, Orca로 돈 지난 티켓과 세션 기록이 지워진 티켓(Claude Code는 기본 30일 뒤 지운다)은 "—"다
Model	기록의 모델. 없으면 "not recorded"
Finished	기록의 종료 시각. 기록이 없거나 멈춘 뒤 병합된 티켓은 PR 병합 시각

머리에는 스펙 완료 여부(하위 티켓 전부 닫힘), 병합 건수, 기록 있는 티켓의 벽시계 합계, 구현·CI·병합 뒤 검증 시간의 합계, 토큰 합계(출력 토큰), 쓰인 모델이 요약된다. `REPORT_DECISIONS_HEADING` 을 지정하면(예: `사람 확인 필요`) 그 아래에 **사람이 봐야 할 절 모음**이 붙는다 — 각 티켓 PR 본문에서 제목이 글자 그대로 같은 마크다운 절을 뽑아 티켓·PR 링크와 함께 원문 그대로 보이고, KPI에 건수를, 절이 있는 행의 Result brief에 표시(⚑)를 단다. 절이 없거나 비어 있는 PR, 본문이 `없음` · `- 없음` · `N/A.` 처럼 없음 표시뿐인 PR(`REPORT_DECISIONS_NONE`)은 나오지 않고, 하나도 없으면 블록도 없다. 무인 루프의 워커는 스펙과 다르게 정했거나 스펙이 사람에게 남긴 결정을 내린 것을 PR 본문에 적는데, 0.9.15까지는 그것을 보려면 병합된 PR을 하나씩 열어야 했다(rental-portal 스펙 #313: 14개 PR의 "남은 판단 사항" 116줄 가운데 사람이 정할 것은 두 건). 리포트 로그 줄과 회고 안내 줄에도 건수가 붙는다. 같은 스펙은 같은 파일을 덮어쓰므로, 끝난 스펙의 리포트를 다시 뽑으려면 `--parent` 로 한 번 더 돌리면 된다(티켓이 없으면 리포트만 만들고 끝난다). 생성에 실패해도 루프 결과는 바뀌지 않고(로그 한 줄), 이전 리포트는 그대로 남는다. 리포트는 jq 하나로 그리므로 의존 도구가 늘지 않는다. 표를 더 풍부하게 만들고 싶으면(PR 본문의 "검증" 절까지) 저장소 스크립트로 분리한다 — 예시 저장소는 TypeScript 생성기(`scripts/loop-report.ts`)와 단위 테스트로 뽑아 갔다.

> **왜 로그가 아니라 리포트인가.** 로그의 `summary:` 줄은 한 실행 안에서만 이어지고, 루프가 죽었다 다시 돌면 흩어진다. 스펙 단위 리포트는 실행 횟수와 무관하게 GitHub의 하위 이슈 목록을 뼈대로 삼고 기록을 붙이므로, 사후 검수(9장 S1 7단계)와 `/retro` 가 티켓 묶음 하나를 한 표에서 본다.

> **구현 메모.** 루프 안에서 실행하는 `gh` · `orca` · `git` 이 표준 입력을 읽으면 남은 티켓 목록을 삼켜 버린다. 그래서 대상 목록은 변수에 담고, 티켓마다 표준 입력을 닫고(`</dev/null`) 실행한다. 스크립트를 고칠 때 이 구조를 유지한다.

8. 워커 안에서 일어나는 일

워커는 Orca 워크트리 안에서 도는 평범한 Claude Code 세션이다. 받는 프롬프트는 `/implement N`에 **무인 실행에서만 참인 문장 몇 개**를 붙인 것이다(부록 B-1). 할 일의 순서는 프롬프트가 아니라 저장소의 `implement` 스킬이 정한다.

어디에	무엇을	왜 거기에
워커 프롬프트 루프가 만든다	질문하지 말고 스스로 결정해 끝까지. PR은 병합하지 마라. 진행이 불가능하면(열린 선행 이슈 포함) 이슈에 이유를 코멘트하고 PR 없이 멈춰라. 규칙 파일도 따르라.	사람이 대화형으로 `/implement`를 쓸 때는 참이 아닌 것들이다. 질문하면 대기 상태가 되고 루프가 멈춘다. 멈출 때 남긴 코멘트가 사람이 이어받는 출발점이다.
`implement` 스킬 저장소 소유	티켓 읽기·상태 확인 → 열린 선행 이슈 확인(목록 기준) → `origin/<base>`에서 `issue-N-<slug>` 브랜치·담당자 지정 → 용어집·ADR 읽기 → `tdd`(seam은 스스로 정함) → 2단 검증 → 커밋(`#N`) → `ticket-review` 반영 → `pr` 양식으로 PR → 인수 조건 체크·`in-review` → 병합하지 않음	무인이든 대화형이든 같은 절차다. 한 곳에 두어야 고칠 때 둘이 갈라지지 않는다.
`worker-rules.md`	검증 명령과 횟수, 돌리지 말 무거운 테스트, 마이그레이션 규칙, 임시 파일·스크린샷 규칙, 간헐 실패 판별법	프로젝트마다 다르고 자주 바뀐다. 회고 결과가 가장 많이 들어가는 곳이다.
`CLAUDE.md`·**코딩 표준**·`docs/agents/`	저장소 사실, 코딩 관례, 트래커 관례	워커는 저장소만 본다. 대화 맥락이 없다.

프롬프트를 줄인 이유 (초판과 다른 점)

초판의 워커 프롬프트는 위 `implement` 절차를 7개 규칙으로 다시 적었다. 실제로 돌려 보니 다음 문제가 생겼다.

- **둘이 갈라진다.** 리뷰 스킬 이름을 바꾸거나 PR 양식을 고칠 때 스킬과 프롬프트 중 한쪽만 고쳐졌다. 워커는 둘이 어긋나면 어느 쪽을 따를지 스스로 정한다.
- **대화형과 무인이 다르게 동작한다.** 사람이 손으로 `/implement`를 돌렸을 때는 프롬프트의 규칙이 없으니, 브랜치·라벨 처리가 달라졌다.
- **매 티켓 같은 긴 문장을 다시 보낸다.** 터미널 한 줄 입력이라 길수록 깨지기 쉽다(재기동 경로에서 따옴표가 문제가 된다).

그래서 절차는 스킬로 옮기고, 프롬프트에는 "사람이 없다"는 사실에서 나오는 문장만 남겼다. 스킬에서 `tdd`의 seam 확인 단계를 건너뛰라고 명시하는 것도 같은 이유다. 원래 `tdd`는 seam을 사용자와 합의하는데, 그 확인을 스킬이 직접 대체해 두면 무인이든 대화형이든 질문 없이 진행된다.

> 워커가 반복해서 같은 곳에서 멈추거나 같은 실수를 하면, 그 답을 어디에 적을지 위 표로 고른다. 무인일 때만 필요하면 `worker-rules.md`, 사람이 할 때도 필요하면 `implement` 스킬, 리뷰가 잡아야 할 관례면 코딩 표준 문서, 기계적으로 잡을 수 있으면 린트·테스트 같은 자동 검사다.

9. 실전 시나리오

예시 저장소 `acme/todo-app` (Node 웹 앱)에 4장 설치를 마친 상태에서 시작한다. 출력은 설명용 예시다.

S1 새 기능을 기획부터 병합까지 — "할 일 목록 CSV 내보내기"

1 요구사항 다듬기 (메인 체크아웃, 대화형 Claude Code)

```
/grill-with-docs 할 일 목록을 CSV로 내보내는 기능. 필터 적용 상태 그대로, 최대 1만 행.
```

열 순서, 날짜 형식, 수식 주입 방어, 권한 같은 결정이 나오면 그 자리에서 정한다. 용어는 `GLOSSARY.md`, 되돌리기 어려운 결정은 `docs/adr/`에 남는다. 브랜치로 커밋해 PR로 기본 브랜치에 병합한다. 워커는 원격 기본 브랜치에서 시작하므로 **병합까지 해야** 워커가 본다(기본 브랜치 직접 push는 4장 10단계 `deny`가 막는다).

2 스펙 이슈 만들기

```
/to-spec
```

스펙 이슈 #40이 생긴다. 이 이슈에도 `ready-for-agent`가 붙지만, 하위 이슈가 생기면 루프가 알아서 건너뛴다.

3 티켓으로 쪼개기

```
/to-tickets #40 — 제목은 "T01: …" 형식, #40의 하위 이슈로 만들어 줘.
```

제안된 목록과 선행 관계를 검토해 승인하면 다음처럼 발행된다.

이슈	제목	선행
#41	T01: 필터 조건으로 할 일을 CSV 행으로 직렬화	—
#42	T02: `GET /api/todos/export` 응답	#41
#43	T03: 목록 화면 "내보내기" 버튼	#41
#44	T04: 1만 행 초과 시 안내와 문서 갱신	#42, #43

4 점검

```
gh issue list --label ready-for-agent
bash scripts/ticket-loop.sh --parent 40 --dry-run
```

```
— ticket-loop-kit 0.9.6 (435e70e) ————————————————
[2026-10-08 21:00:03] skip #42 (T02: GET /api/todos/export 응답): blocked by an open issue
[2026-10-08 21:00:04] skip #43 (T03: 목록 화면 "내보내기" 버튼): blocked by an open issue
[2026-10-08 21:00:04] skip #44 (T04: 1만 행 초과 시 안내와 문서 갱신): blocked by an open issue
[2026-10-08 21:00:04] repository acme/todo-app / backend Orca / base branch main / label ready-for-agent / sub-issue
s of spec #40
[2026-10-08 21:00:04] post-merge check: off
[2026-10-08 21:00:04] targets (all blocking issues currently closed):
  #41   T01: 필터 조건으로 할 일을 CSV 행으로 직렬화
```

지금 집을 수 있는 건 #41뿐이다. 나머지는 #41이 병합되면 차례로 풀린다.

5 한 건만 시험 실행 — 처음이거나 규칙을 바꾼 직후라면 꼭 한다.

```
bash scripts/ticket-loop.sh --parent 40 --max 1
```

Orca 사이드바에 `issue-41` 카드가 `in-progress`로 나타난다. 터미널을 열어 워커가 권한을 묻지 않는지, 테스트를 먼저 쓰는지, PR 본문이 기대한 모양인지 본다. 막힌 곳이 있으면 `implement` 스킬, `.claude/settings.json`, `worker-rules.md` 중 맞는 곳을 고쳐 기본 브랜치에 병합한다(8장 끝 표).

6 나머지 무인 실행

```
# 메인 체크아웃의 Orca 터미널에서 (로그는 .ticket-loop/loop-1008.log)
caffeinate -is bash scripts/ticket-loop.sh --parent 40
```

```
[2026-10-08 21:42:10] === #42 T02: GET /api/todos/export 응답 ===
[2026-10-08 21:42:12] #42: worktree r-7f3::/Users/me/orca/issue-42 (/Users/me/orca/issue-42) / terminal term-19
[2026-10-08 21:44:15] #42: branch issue-42-todos-export
[2026-10-08 21:49:16] #42: first file changes — src/app/api/todos/export/route.ts, tests/api/todos-export.test.ts
[2026-10-08 21:54:17] #42: working — 12m elapsed, 2 uncommitted files: src/app/api/todos/export/route.ts, tests/api/
todos-export.test.ts
[2026-10-08 22:08:20] #42: commit 3f9c2a1 feat: GET /api/todos/export CSV 응답 #42
[2026-10-08 22:26:35] #42: commit b71e0d4 fix: 리뷰 반영 — 빈 목록 헤더만 #42
[2026-10-08 22:31:40] #42: PR #46 opened (created 22:31:21, picked up after 19s)
[2026-10-08 22:31:41] #42: waiting for CI (PR #46)
[2026-10-08 22:36:02] #42: CI passed
[2026-10-08 22:36:05] #42: PR #46 merged
[2026-10-08 22:36:14] #42 summary: wall-clock 54m, PR #46 (picked up after 19s), CI 258s, repairs 0, idle false posi
tives 0, post-merge check skipped
[2026-10-08 22:36:12] === #43 T03: 목록 화면 "내보내기" 버튼 ===
...
[2026-10-09 00:58:31] done: 3 tickets
```

7 사후 검수와 마무리

```
open docs/loop-reports/spec-40.html                # 스펙 리포트: 티켓별 상태·브리프·경과·모델 한 표 (7장 9단계)
gh pr list --state merged --limit 10
grep "summary:" .ticket-loop/loop-1008.log          # 티켓별 한 줄
gh issue close 40 --comment "하위 티켓 #41–#44 병합으로 완료"
git switch -c report-spec-40 && git add docs/loop-reports/spec-40.html && git commit -m "docs: 스펙 #40 루프 리포트"
&& gh pr create --fill
```

리포트에서 중단·수리·오래 걸린 티켓을 먼저 고르고, PR마다 본문의 "남은 판단 사항"과 "검증하지 못한 것"을 읽고, 필요하면 후속 티켓을 만든다(`/to-tickets` 또는 직접 이슈 작성 → `ready-for-agent`). 그다음 `/retro` 에 로그와 PR들을 주고 환경 개선 후보를 받는다. 고친 스킬·규칙·검사는 PR로 병합해 다음 묶음부터 쓴다.

S2 버그 백로그 정리

1. 새로 들어온 이슈를 분류한다: `/triage 새 이슈들 정리해 줘` . 재현과 수정 방향이 분명한 버그는 브리프와 함께 `bug` + `ready-for-agent` 가 된다. 애매한 것은 `needs-info` 로 간다.
2. 버그만 골라 보기: `bash scripts/ticket-loop.sh --label bug --dry-run`
3. 몇 건씩 끊어 처리한다: `bash scripts/ticket-loop.sh --label bug --max 3`
4. 버그 수정 PR은 재현 테스트가 "실패 → 통과"로 바뀐 근거가 "테스트 지점"·"검증" 절에 있는지 본다.

S3 위험한 변경은 사람이 병합

DB 스키마, 인증, 결제처럼 되돌리기 어려운 티켓은 CI 통과까지만 자동으로 하고 병합은 사람이 한다.

```
bash scripts/ticket-loop.sh --no-merge 51          # CI 녹색까지, 워크트리는 남는다
gh pr view 52 --web                                # 사람이 검토 (PR 번호는 로그에 찍힌다)
gh pr merge 52 --merge                             # 승인 후 병합 → #51 닫힘
gh issue edit 51 --remove-label in-review
orca worktree rm --worktree id:<워크트리ID> --force --json   # ID는 로그 또는 orca worktree list --json
git worktree prune; git branch -D issue-51 issue-51-<slug>; git push origin --delete issue-51-<slug>
```

이런 티켓을 평소에도 자동 병합에서 빼고 싶으면 `ready-for-human` 으로 두거나, 별도 라벨(예: `needs-human-merge`)을 붙여 `SCOPE_LABEL` 실행을 나눈다.

S4 야간 무인 실행과 아침 점검

1. 퇴근 전: `--dry-run`으로 대상을 확인하고, 메인 체크아웃의 Orca 터미널에서 잠자기 방지를 붙여 실행한다. Orca 앱은 켜 둔다. 병합 뒤 검증이 포트를 쓰면 그 포트의 개발 서버를 꺼 둔다 — 실행 전에 `lsof -nP -iTCP:3000 -sTCP:LISTEN`으로 확인한다(낮에 띄운 서버가 남아 있기 쉽다). 켜 둔 채 두면 병합마다 10분을 기다리다 멈춘다.
2. 도중에 급한 티켓을 끼워 넣으려면 그 이슈에 **ready-for-agent**와 앞선 순번 제목(예: `T00: …`)을 준다. 루프는 지금 티켓을 끝낸 뒤 대상을 다시 계산하므로 그 티켓을 다음에 집는다.
3. 아침 점검:

```
tail -n 30 .ticket-loop/loop-1008.log                    # 마지막 줄이 "done"인가 "ERROR"인가
grep "summary:" .ticket-loop/loop-1008.log               # 티켓별 한 줄
gh pr list --state merged --limit 20                   # 밤새 병합된 것
gh issue list --label in-review                        # PR 단계에 남은 것
orca worktree list --json ¦ jq -r '.. ¦ .id? // empty ¦ strings ¦ select(contains("::"))'   # 남은 워크트리
open docs/loop-reports/spec-40.html                    # "완료"로 끝났으면 스펙 리포트가 갱신돼 있다
```

4. "ERROR"로 끝났다면 10장 절차를 따른다. 마지막 줄이 "포트 … 사용 중"이면 멈춘 게 아니라 기다리는 중이다(R10).

10. 중단과 복구

10.1 복구의 원칙

- **루프에는 기억이 없다.** 다시 실행하면 GitHub 상태만 보고 대상을 정한다. 복구는 "멈춘 티켓을 GitHub에서 정리된 상태(끝남 / PR 단계 / 미착수)로 만든 뒤 루프를 다시 실행"하는 일이다.
- **워크트리는 증거다.** 루프는 멈출 때 워크트리를 지우지 않는다. 워커의 커밋, 터미널 기록, 이슈 코멘트가 남아 있다. 확인하기 전에 지우지 않는다.
- **진행 중이던 티켓을 정리하지 않고 재실행하지 않는다.** 워커가 담당자를 달아 둔 티켓은 루프가 건너뛰므로 같은 일을 두 번 하지는 않는다. 하지만 그 티켓에 막힌 티켓은 계속 풀리지 않고, 담당자를 달기 전에 멈춘 티켓은 같은 이름의 워크트리를 다시 만들려다 실패한다. 먼저 PR 단계까지 마무리하거나 폐기한다.

10.2 상태 진단

루프는 어떻게 멈추든(10.3의 멈춤 조건, 터미널 창 닫힘, Ctrl-C, `kill`, 네트워크가 끊겨 `git fetch`·`gh`가 실패하는 등 `set -e`에 걸린 명령 — GitHub 조회의 짧은 5xx·연결 오류는 그 전에 세 번까지 다시 해 본다) 한 번 보고한다 — 데스크톱과 Slack(`SLACK_WEBHOOK_URL`) 알림, 그리고 처리 중이던 티켓 이슈에 사유·PR·워크트리·루프 로그 끝을 적은 코멘트와 `loop-stopped` 라벨. 자리를 비웠다면 Slack 알림이나 `gh issue list --label loop-stopped`의 마지막 코멘트부터 본다. `kill -9`·전원 꺼짐·잠자기는 스크립트가 아무것도 보내지 못해 잡지 못한다. 루프를 일부러 멈출 때는 Ctrl-C나 `kill <루프 pid>`면 된다 — 보고(몇 초)를 마치고 끝나며, 워커 터미널은 그대로 남으니 10.4 절차로 정리한다. 라벨은 루프가 그 티켓을 병합하면 떨어지고, 사람이 마무리했으면 손으로 뗀다(남아도 대상 선택에는 영향이 없다).

멈출 때마다 **멈춤 기록** `.ticket-loop/stops/<id>.json`(사유·이슈·PR·로그 경로·워크트리·다시 실행할 명령, 기계별)이 남고 이슈 코멘트에 그 id가 적힌다. 그 저장소의 에이전트 세션에서 `/loop-triage <id>`(Codex는 `$loop-triage`)를 부르면 그 기록부터 진단을 이어받는다. 무인 수습(10.6)이 이미 돌았다면 같은 id의 `.verdict.json`에 판정과 요약이 있고, 멈춤 사유 끝에 "— triage (분류): 요약"이 붙어 있다.

멈춘 티켓 번호는 로그의 마지막 `=== #N …` 줄에서 찾는다. 아래 함수를 셸에 붙여 넣거나 `scripts/ticket-state.sh`로 저장해 두고 `ticket_state N`으로 한눈에 본다.

```
ticket_state() {
  local n="$1" b="${BRANCH_PREFIX:-issue-}$1"   # 워커 브랜치 issue-N-slug 또는 워크트리 브랜치 issue-N. issue-N0은 아니다
  echo "== 이슈";  gh issue view "$n" --json state,labels,assignees \
    -q '"\(.state) ¦ 라벨: \([.labels[].name]¦join(", ")) ¦ 담당: \([.assignees[].login]¦join(", "))"'
  echo "== 열린 선행 이슈"; gh api "repos/{owner}/{repo}/issues/$n/dependencies/blocked_by" \
    --jq '.[] ¦ select(.state == "open") ¦ "  #\(.number) \(.title)"'
  echo "== PR";    gh pr list --state all --limit 100 --json number,state,headRefName \
    ¦ jq -r --arg b "$b" '.[] ¦ select(.headRefName == $b or (.headRefName¦startswith($b + "-"))) ¦ "#\(.number) \(.state) \(.h
eadRefName)"'
  echo "== 원격 브랜치"; git ls-remote --heads origin "refs/heads/${b}" "refs/heads/${b}-*"
  echo "== 워크트리";    git worktree list ¦ grep -E "/${BRANCH_PREFIX:-issue-}$n( ¦-)" ¦¦ echo "(없음)"
  echo "== 루프 잠금";   ls -d "$(git rev-parse --path-format=absolute --git-common-dir)/ticket-loop.lock" 2>/dev/null ¦¦ echo
"(없음)"
}
```

10.3 상태 판정표

#	이슈	PR	워크트리	뜻	할 일
A	열림, 담당 없음	없음	없음	아직 시작 전	그대로 루프 실행
B	열림, 담당 있음	없음	있음	구현 중이거나 PR 없이 멈춤	워커 터미널 확인 → R2 (아직 일하는 중이면 PR이 열리기를 기다렸다가 E로)
C	열림	없음	없음, 원격 브랜치 있음	워크트리만 지워짐(사람이 지웠거나 디스크 정리)	원격 브랜치에서 워크트리를 다시 만들어 이어가거나(R2-a″) 브랜치를 지우고 A로
D	열림, in-review	열림, CI 실패·충돌·미확인	있음	PR 단계에서 중단	R3
E	열림, in-review	열림, CI 녹색	있음	병합만 남음(`--no-merge` 포함)	R3의 4-6단계(병합·정리)
F	닫힘	병합됨	있음	병합 후 정리 실패	R5

G	열림	병합됨	—	본문에 `Closes` 누락	`gh issue close N`, `in-review` 제거, 워크트리·브랜치 정리(R5) → 루프 재실행
H	열림	닫힘(병합 안 됨)	있음/없음	버린 시도	R2-b(처음부터 다시)

표의 PR·원격 브랜치는 head가 `issue-N-<slug>` 인 것과 `issue-N` 인 것 둘 다다. 워커가 워크트리 브랜치에 그대로 커밋했으면 원격에는 `issue-N` 만 있고, 아래 절차의 `git push origin --delete issue-N-<slug>` 는 그 이름으로 바꿔 읽는다(`gh pr view <PR> --json headRefName`).

10.4 중단 지점별 복구 절차

R1 루프 프로세스가 죽었다 (Ctrl-C, 루프 터미널 닫힘, 노트북 잠자기)

워커는 Orca 안에서 계속 일하고 있을 수 있다. 루프만 없어진 상태다.

1. **잠금 확인.** Ctrl-C나 오류로 끝났다면 잠금이 이미 풀려 있다. 강제 종료·전원 차단이면 남아 있을 수 있다. `pgrep -fl ticket-loop` 로 실행 중인 루프가 없음을 확인한 뒤 `rmdir .git/ticket-loop.lock`.
2. **멈춘 티켓 찾기.** 로그의 마지막 `=== #N` 줄을 보고 `ticket_state N` 을 실행한다.
3. **판정표대로 처리.** 대개 B(아직 구현 중)나 D·E(PR 단계)다. B라면 Orca 터미널에서 워커가 아직 일하는지 본다. 일하는 중이면 PR이 열리기를 기다린다.
4. **재실행.** 그 티켓이 끝난 상태(F·G → 정리 후)나 PR 단계(D·E)가 되면 `bash scripts/ticket-loop.sh` 로 다시 시작한다. PR 단계에 남겨 둔 채 재실행해도 루프는 PR이 열린 티켓을 건너뛰므로 안전하다. 다만 그 티켓에 막힌 티켓은 병합 전까지 풀리지 않으니, 가능하면 R3로 병합까지 끝내고 재실행한다.

R2 워커가 PR 없이 멈췄다 ("워커가 PR 없이 멈췄습니다" / 타임아웃)

먼저 로그에 찍힌 워커 출력 마지막 25줄과 **이슈 코멘트**(`gh issue view N --comments`)를 읽는다. 원인은 대개 질문, 권한 창, 진행 불가 판단, 너무 큰 티켓 중 하나다. 워커 꼬리가 셸 프롬프트(`$`)라면 에이전트가 뜨지 않은 것이다(R9). 꼬리가 `git push` 에서 출력 없이 멈춰 있으면 자격 증명 확인 창을 기다리는 것이다(4장 2단계 주의). 그다음 세 갈래 중 하나를 고른다.

갈래	절차
a. 워커를 이어서 일하게 한다 워커 판단이 대체로 맞고 답 하나가 빠졌을 때	Orca에서 `issue-N` 워크트리 터미널을 열어 질문에 답하거나 지시를 입력한다. 권한 창이면 허용한다(그리고 4장 10단계 허용 목록에 추가). PR이 열리면 E/D 상태가 되고, R3로 마무리한 뒤 재실행한다. CLI로 보낼 때: `orca terminal send --terminal <핸들> --text "…" --enter --json`. cmux: 사이드바에서 `issue-N` 워크스페이스를 열거나 `cmux paste --surface <핸들> --submit "…"`. tmux: `tmux attach -t ticket-loop` 뒤 `issue-N` 창에서 직접 입력
a′. 같은 워크트리에서 새 워커로 이어간다 워크트리는 있는데 터미널이 없어졌을 때(R4)	`orca terminal create --worktree id:<워크트리ID> --command "claude --dangerously-skip-permissions" --json` 로 에이전트를 다시 띄운다. `terminal wait --for tui-idle` 로 준비를 확인한 뒤 다음을 보낸다: "이슈 #N 작업을 이어서 해라. `git log origin/main..HEAD` 와 `git status` 로 진행 상황을 파악하고, 남은 인수 조건을 구현한 뒤 implement 스킬의 나머지 단계(리뷰·PR·라벨)를 따라 PR까지 열어라. 질문하지 마라." cmux·tmux: 워크스페이스·창이 남아 있으면 그 셸에서 `claude --dangerously-skip-permissions` 를 다시 띄우고, 없으면 `cd ~/.cache/ticket-loop/<owner-repo>/worktrees/issue-N` 에서 띄운다(tmux면 `tmux new-window -t ticket-loop -n issue-N -c 그경로`).

a″. 원격 브랜치에서 워크트리를 다시 만든다
워크트리가 사라지고 push된 브랜치만 남았을 때(C 상태)
	`orca worktree create --repo id:<repoId> --name issue-N --no-parent \`
	`  --base-branch origin/issue-N-<slug> --issue N --agent claude --json \`
	`  --prompt "이슈 #N 작업을 이어서 해라. 먼저 \`git checkout issue-N-<slug>\`로 원격 브랜치를 받고, \`git log origin/main..HEAD\`로 진행 상황을 파악한 뒤 남은 인수 조건을 구현하고 PR까지 열어라. 질문하지 마라."`
	
	push되지 않은 작업은 이미 잃었다. 워크트리 안의 커밋이 push 전이었다면 그만큼은 다시 해야 한다.

b. 처음부터 다시
방향이 틀렸거나 티켓을 고쳐야 할 때
	`gh pr close <PR> --delete-branch 2>/dev/null ¦¦ true  # PR이 있었다면`
	`git push origin --delete issue-N-<slug> 2>/dev/null ¦¦ true  # PR의 head 브랜치 이름. 워커가 워크트리 브랜치에 그대로 커밋했으면 issue-N`
	`gh issue edit N --remove-label in-review --remove-assignee @me`
	`orca worktree rm --worktree id:<워크트리ID> --force --json`
	`#  cmux: cmux workspace close <workspace_id> --force / tmux: tmux kill-window -t issue-N — 그 뒤 둘 다`
	`#  git worktree remove --force ~/.cache/ticket-loop/<owner-repo>/worktrees/issue-N`
	`# 이슈 본문 보강(결정 추가, 범위 축소, 쪼개기) 후`
	`bash scripts/ticket-loop.sh N`

c. 사람에게 넘긴다 결정이 필요하거나 무인에 맞지 않을 때	`gh issue edit N --remove-label ready-for-agent --add-label ready-for-human`. 워크트리는 사람이 이어 쓰거나 지운다. 이 티켓에 막힌 다른 티켓은 이 티켓이 닫힐 때까지 기다린다. 나머지는 `bash scripts/ticket-loop.sh` 로 계속한다.

R3 PR은 열렸는데 병합되지 않았다 (CI 실패·충돌 수리 소진, 병합 거절, 루프가 먼저 죽음)

1. 원인을 본다: `gh pr checks <PR>`, `gh run view --log-failed`, `gh pr view <PR> --json mergeable,mergeStateStatus`. CI가 녹색인데 "failed to merge"로 멈췄으면 사유 괄호의 `gh` 오류를 본다 — `BLOCKED`는 브랜치 보호(리뷰 승인), `BEHIND`는 기본 브랜치를 PR 브랜치에 합쳐 push, `CLEAN`이면 일시 장애가 지나간 것이니 3단계로.
2. 워크트리에서 고친다. 직접 고쳐도 되고, 워커 터미널에 지시해도 된다. 충돌이면 `git fetch origin && git merge origin/main`. 테스트·lint 후 같은 브랜치에 push한다.
3. CI를 끝까지 본다: `gh pr checks <PR> --watch --fail-fast`.
4. 병합: `gh pr merge <PR> --merge`. 이슈가 닫혔는지 확인하고, 안 닫혔으면 `gh issue close N`.
5. 정리: `gh issue edit N --remove-label in-review`, `orca worktree rm --worktree id:<ID> --force --json`, `git worktree prune`, `git branch -D issue-N issue-N-<slug>`, `git push origin --delete issue-N-<slug>` (PR의 head 브랜치. `issue-N`이면 그 이름).
6. 재실행: `bash scripts/ticket-loop.sh`.

> CI를 통과시키려고 테스트를 건너뛰거나 지우지 않는다. 빈 커밋으로 CI를 다시 돌리지도 않는다. 실패는 원인을 고친다. 정말 일시적인 실패(러너 손실 등)라면 GitHub에서 그 작업만 한 번 다시 실행한다.

R4 백엔드 앱이 꺼졌다 / 컴퓨터가 재부팅됐다 ("… 터미널 … 에 닿지 못했습니다")

1. Orca를 다시 켠다(`orca open --json`). cmux면 앱을 열고 루프도 cmux 터미널 안에서 다시 띄운다(2.5). tmux면 서버가 죽은 것이다(재부팅, `tmux kill-server`) — `tmux ls`로 확인한다. 워크트리는 디스크에 그대로 있다. 다만 워커 프로세스와 그 대화 맥락은 사라졌다.
2. `ticket_state N`과 워크트리 안의 `git status`·`git log`로 어디까지 됐는지 본다. 커밋되지 않은 변경도 남아 있을 수 있다.
3. PR이 이미 있으면 R3로 간다. 없으면 R2-a′(새 워커로 이어가기)나 R2-b(처음부터)를 고른다.
4. 재부팅이었다면 루프 잠금이 남았을 수 있다(R1 1단계).

R5 병합은 됐는데 정리가 안 됐다 ("워크트리 삭제 실패 — 수동 정리 필요")

이슈가 닫혔으므로 루프를 다시 실행해도 안전하다. 남은 것만 지운다: `orca worktree rm --worktree id:<ID> --force --json` (cmux: `cmux workspace close <ID> --force`, tmux: `tmux kill-window -t issue-N`, 그 뒤 `git worktree remove --force <경로>`), `git worktree prune`, `git branch -D issue-N issue-N-<slug>` (두 번째는 워커 브랜치, 첫 번째는 루프가 워크트리용으로 만든 브랜치. 워커가 첫 번째에 그대로 커밋했으면 하나뿐이다), `git push origin --delete issue-N-<slug>` (PR의 head 브랜치 이름으로), `gh issue edit N --remove-label in-review`.

R6 GitHub 인증 만료·네트워크 오류로 루프가 죽었다

`gh auth status` → 필요하면 `gh auth login`. 그다음 R1과 같다. 마지막 티켓의 상태를 진단하고, 판정표대로 정리한 뒤 재실행한다. 루프가 병합 직후에 죽었다면 F 상태일 가능성이 크다.

R7 "체크가 등록되지 않았습니다"

워크플로가 PR 이벤트에서 돌지 않는 경우다(트리거 브랜치 불일치, Actions 비활성, 경로 필터). 워크플로에 경로 필터(`paths` / `paths-ignore`)를 두었다면 루프의 `CI_SKIP_REGEX`·`CI_SKIP_CMD`가 같은 판정을 내는지 본다. 워크플로는 건너뛰는데 루프는 기다리면 이 메시지가 난다. 원인을 고쳐 기본 브랜치에 반영한다. 그 PR은 기본 브랜치를 병합해 push하면 체크가 붙는다(정말 CI가 필요 없는 PR이었다면 사람이 병합한다). 그 뒤 R3로 마무리한다.

R8 "다른 루프가 실행 중입니다"

정말 다른 루프가 돌고 있으면 끝나기를 기다린다(같은 저장소에서 두 루프를 돌리지 않는다). `pgrep -fl ticket-loop` 결과가 비어 있으면 강제 종료로 남은 잠금이므로 `rmdir "$(git rev-parse --git-common-dir)/ticket-loop.lock"`.

R9 "에이전트가 터미널 … 에서 뜨지 않습니다"

워크트리는 만들어졌지만 에이전트가 두 번 다 뜨지 않았다. Orca에서 그 워크트리 터미널을 열어 무엇이 찍혔는지 본다. 흔한 원인은 에이전트(Claude Code·Codex) 미설치·로그인 만료, setup 훅 실패(`setupAgentStartupPolicy: wait-for-setup` 이면 setup이 끝나야 에이전트가 뜬다), 에이전트 배너가 `AGENT_BANNER_REGEX` 와 맞지 않는 경우(다른 에이전트나 배너 형식 변경)다. 원인을 고친 뒤 워크트리를 지우고(R2-b) 그 티켓부터 다시 돌린다. cmux·tmux에서 워커는 떠서 일했는데 루프가 티켓 타임아웃으로 멈췄다면 R9가 아니라 훅 상태가 오지 않은 경우다 — cmux는 Settings의 Claude Code 통합, tmux는 워커가 `--settings` 훅 설정으로 떴는지(`~/.cache/ticket-loop/<owner-repo>/tmux/state/issue-N` 에 값이 쓰이는지) 본다(2.5). `WORKTREE_SETUP_CMD` 실패는 "WORKTREE_SETUP_CMD 실패"로 따로 멈추며, 메인 체크아웃에서 그 명령을 돌려 재현한다.

R10 "병합 뒤 검증이 실패했습니다" / "포트 … 이 비지 않았다"

PR은 이미 병합돼 이슈가 닫혔고 워커 워크트리도 정리됐다(F 상태와 같다). 기본 브랜치가 무거운 검증을 못 넘긴 것이다. ERROR 줄에 적힌 로그(`LOG_FILE` 과 같은 디렉터리의 `post-merge-N.log`, 기본은 저장소 루트의 `.ticket-loop/post-merge-N.log` — 루프가 이미 꼬리 40줄을 찍어 두었다)를 읽고, 검증 워크트리(`~/.cache/ticket-loop/<owner-repo>/post-merge`)에서 실패한 테스트 하나만 다시 돌려 재현한다. 수정은 새 브랜치 + PR로 한다(티켓이면 `T00:` 으로 끼워 넣는다). 병합한 뒤 루프를 다시 실행한다.

- **"…, and again on retry"**면 `POST_MERGE_RETRY_CMD` 의 재실행까지 실패한 것이다. ERROR 줄의 두 로그(`post-merge-N.log`, `post-merge-N-retry.log`)를 본다. 재실행 로그가 "실패한 테스트를 못 찾았다"면 첫 실행이 설치·환경 단계에서 깨진 것이고, 재실행에서 다른 테스트가 시간 초과로 깨졌다면 플레이크일 가능성이 크다 — 그때는 고칠 것 없이 루프를 그대로 다시 실행한다. 같은 테스트가 두 번 다 깨졌다면 위 절차대로 수정 PR을 낸다.
- **"(merges checked together: #a #b … — first failing merge: #N …)"**가 붙었다면 묶어 돌린 검증이다(`POST_MERGE_EVERY`). 원인 후보는 그 병합들이고, 루프가 실패한 테스트만 다시 돌려 찾은 첫 실패 병합부터 본다(검증 로그 옆 `post-merge-N-bisect-<커밋>.log`). "first failing merge not found"면 마지막으로 통과한 커밋에서도 실패한 것이니 플레이크·환경부터 의심한다. 그 병합들은 다음 실행의 검증이 다시 덮는다.
- **이번 티켓 탓이 아닐 수 있다.** 공유 개발 DB의 데이터가 바뀌어 테스트 전제가 깨졌을 수 있다("X가 없다"를 전제로 0건을 단언하는 테스트 등). 제품 버그로 단정하기 전에 전제부터 확인한다.
- **"포트 … 이 비지 않았다"**면 루프가 도는 동안 그 포트에 띄운 개발 서버를 끄고 다시 실행하거나, 이번에는 `--skip-post-merge` 로 돈다. 재실행하면 그 티켓의 검증은 건너뛰어지고 다음 티켓 병합 뒤 검증이 그 변경까지 함께 본다.
- **아직 "ERROR"가 아니고 마지막 줄이 "포트 … 사용 중(… pid …) — 비기를 …까지 기다린다"**면 멈춘 게 아니다. 루프는 30초마다 포트를 다시 본다. `lsof -nP -iTCP:<포트> -sTCP:LISTEN` 으로 점유 프로세스를 찾아(대개 메인 체크아웃의 개발 서버) 끄면 30초 안에 검증이 시작된다 — 루프를 죽이거나 다시 실행하지 않는다. 기다리는 동안 로그가 조용한 것은 정상이다.

10.5 따라 해 보기 — S1 도중 두 번 멈췄을 때

상황 1 T03(#43) 진행 중 노트북을 덮었다

아침에 Orca의 메인 체크아웃 터미널을 보니 루프가 끝나 있다(잠자기로 셸이 멈췄다가 깨어나면서 Orca 호출이 실패했다). 로그 끝:

```
[2026-10-08 23:05:12] === #43 T03: 목록 화면 "내보내기" 버튼 ===
[2026-10-08 23:05:14] #43: worktree r-7f3::/Users/me/orca/issue-43 (/Users/me/orca/issue-43) / terminal term-23
```

1 잠금과 프로세스 확인

```
pgrep -fl ticket-loop ¦¦ echo "루프 없음"
ls -d .git/ticket-loop.lock 2>/dev/null && rmdir .git/ticket-loop.lock
```

2 상태 진단

```
ticket_state 43
```

```
== 이슈
OPEN ¦ 라벨: ready-for-agent, in-review ¦ 담당: me
== PR
#48 OPEN issue-43-export-button
== 원격 브랜치
9c1e…   refs/heads/issue-43-export-button
== 워크트리
/Users/me/orca/issue-43  9c1e2ab [issue-43-export-button]
== 루프 잠금
(없음)
```

판정표 D 또는 E다. 루프가 죽은 사이에도 워커가 PR #48을 열었다.

3 CI 확인 후 병합·정리 (R3)

```
gh pr checks 48 --watch --fail-fast && gh pr merge 48 --merge
gh issue view 43 --json state -q .state          # CLOSED
gh issue edit 43 --remove-label in-review
orca worktree rm --worktree "id:r-7f3::/Users/me/orca/issue-43" --force --json
git worktree prune; git branch -D issue-43 issue-43-export-button
git push origin --delete issue-43-export-button
```

4 재개 — #43이 닫혔으므로 #44의 선행이 모두 풀렸다.

```
bash scripts/ticket-loop.sh --parent 40
```

상황 2 T04(#44) 워커가 질문하고 멈췄다

```
[2026-10-09 09:58:40] #44: worker idle, no PR (3/3)
[2026-10-09 09:58:40] #44: worker output tail ---
  1만 행 초과 시 동작이 티켓에 두 가지로 읽힙니다: (1) 처음 1만 행만 내보내고 안내,
  (2) 내보내기를 거부하고 필터를 좁히라고 안내. 어느 쪽으로 할까요?
[2026-10-09 09:58:41] ERROR: #44: worker stopped without a PR (a question or a failure?). Check the issue-44 worktree in O
rca.
```

1 결정한다. 티켓이 모호했던 것이므로 **이슈에도 결정을 남긴다**. 그래야 사후 검수 때 근거가 보인다.

```
gh issue comment 44 --body "결정: 1만 행 초과면 거부하고 필터를 좁히라고 안내한다(부분 내보내기 안 함)."
```

2. R2-a: Orca에서 `issue-44` 터미널을 열고 답한다.

   ```
   (2)로 해라. 이슈 #44 코멘트에도 적어 두었다. 질문하지 말고 PR까지 끝내라.
   ```

3. PR이 열리면 R3로 CI를 확인하고 병합·정리한다. 남은 대상이 없으면 루프를 다시 돌릴 필요도 없다. `bash scripts/ticket-loop.sh --dry-run` 이 "No tickets to process"라고 하면 끝이다.

4. 재발 방지: 같은 종류의 모호함이 또 나오면 `/to-tickets` 단계에서 걸러지도록, 스펙이나 `GLOSSARY.md`·ADR에 결정을 남긴다.

10.6 무인 수습 — 루프가 먼저 고쳐 보는 범위

사람이 멈춤을 알아채고 붙기까지의 시간이 멈춤 자체보다 길다(rental-portal spec #96: 병합 뒤 검증 실패로 15:43에 멈췄고, 고친 PR은 테스트 기대값 3줄이었는데 루프가 다시 돈 것은 19:08). 그래서 `LOOP_TRIAGE=1`(기본)이면 아래 네 경우에 루프가 멈추기 전에 **수습 세션**을 띄운다. 수습 세션은 워커와 같은 길로 새 워크트리(`loop-triage-N-<시각>`)에 띄운 에이전트이고, 할 일은 기본 브랜치에 커밋된 `loop-triage` 스킬이 정한다 — 멈춤 기록을 읽고 원인을 분류(project·flake·environment·loop-bug·needs-human)해 고치거나 진단한 뒤 판정 파일(`resume` 또는 `halt`)을 쓴다. 루프는 잠금을 쥔 채 그 파일을 기다린다. Slack에는 "수습 시작"과 결과가 간다.

멈출 일	수습 세션이 하는 일	resume이면 루프가
병합 뒤 검증 실패(재실행까지 실패)	실패를 원인 PR의 diff와 대조해 테스트 기대값이나 코드를 고치고, 실패했던 테스트 파일만 돌려 확인한 뒤 기본 브랜치로 수정 PR. 무관한 시간 초과면 다시 돌려 보고 플레이크로 판정	수정 PR에 `loop-triage` 라벨을 붙이고 CI 뒤 병합, 병합된 기본 브랜치로 검증을 다시 돌려(`post-merge-N-after-triage.log`) 통과하면 다음 티켓
수리를 다 쓴 CI 실패·충돌, 새 커밋 없이 끝난 수리 뒤에도 CI 실패	티켓 PR의 브랜치에 고쳐 push. 번들 크기 같은 임계값은 티켓 변경으로 정당하게 늘어난 경우에만 근거를 PR 코멘트로 남기고 올린다. CI 인프라 오류면 실패한 job만 다시 돌린다	그 PR의 충돌·CI 확인부터 이어 간다
`WORKTREE_SETUP_CMD` 실패(cmux·tmux)	준비 명령을 재현하고 저장소 스크립트·lockfile을 고쳐 수정 PR, 일시 장애면 수정 없이	수정 PR을 병합하고(그 병합이 아직 구현 전인 티켓 이슈를 닫았으면 다시 연다) 워커 워크트리를 기본 브랜치로 당긴 뒤(메인 체크아웃도 기본 브랜치면 fast-forward — 로컬 변경을 덮는 merge는 git이 거부한다) 준비를 다시 돌린다
워커가 PR 없이 멈춤	진단만 — 이슈 코멘트·워커 화면·워커 워크트리로 무엇에 막혔는지 요약한다. 질문에 답하거나 대신 구현하지 않는다	(언제나 멈춘다)

이어 가는 곳은 멈춘 그 자리다. 처음부터 다시 실행하면 PR이 열렸거나 담당자가 붙은 진행 중 티켓을 자동 선택이 건너뛰기 때문이다. 판정이 `halt`이거나, 판정 없이 끝나거나(제한 시간, idle 연속), 수정 PR의 CI가 실패하거나, 이어 간 뒤 같은 곳에서 또 실패하면 루프는 판정 요약을 사유 끝에 붙여 멈추고(Slack 멘션), 수습 세션의 워크트리는 조사용으로 남긴다. 한 실행에 `TRIAGE_MAX` 번(기본 1), 같은 멈춤에는 한 번만 띄운다.

수습 세션이 하지 않는 것: 메인 체크아웃을 고치지 않는다. 키트 관리 파일(`scripts/ticket-loop.sh` 등)을 고치지 않는다 — 루프 버그는 키트 버그 보고서(`<id>.kit-bug.md`, 멈춤 기록 옆)를 쓰고 멈춘다. 공개일 수 있는 키트 저장소에 직접 이슈를 만들지 않는 것은 이 저장소의 로그·코드가 밖으로 나가지 않게 하려는 것이다. 근거 없이 테스트를 약하게 만들지 않는다(지우기, 건너뛰기, 단정 완화, 시간 제한만 늘리기). PR을 병합하지 않는다 — 병합은 루프가 CI 뒤에 한다. 이 머신의 프로세스·설정, 운영 환경, 비밀 값은 건드리지 않는다.

수습 세션을 띄우지 않는 멈춤: 티켓 타임아웃, 터미널 끊김, 에이전트 기동 실패, 병합 실패, 신호(창 닫힘·Ctrl-C·`kill`), `set -e`에 걸린 명령 — 어디서든 안전하게 이어 갈 수 있는 자리가 아니다. 병합 실패 가운데 GitHub 일시 장애는 루프가 멈추기 전에 다시 해 보고(7장 5단계), 남는 것은 필수 리뷰 같은 브랜치 보호의 거절이다 — 사람에게 넘기려고 켜 두는 것이라(12장 FAQ "병합 전에 꼭 사람이 보고 싶다") 수습 세션이 할 일이 없다. 키트 설치 PR이 병합되기 전(기본 브랜치에 `loop-triage` 스킬이 없을 때)에도 띄우지 않고, 시작 줄에 그렇게 적는다. 이 경우들은 멈춤 기록만 남으니 사람이 `/loop-triage <id>`로 이어받는다.

11. 안전 장치와 운영 수칙

루프가 지키는 것

- **CI가 녹색일 때만 병합한다.** 체크가 하나라도 실패하면(`--fail-fast`) 병합하지 않는다. 체크가 아예 없어도 병합하지 않는다. 예외는 저장소 CI의 경로 필터에 걸리는 PR(`CI_SKIP_REGEX` · `CI_SKIP_CMD`)뿐이다.
- **기본 브랜치가 깨지면 쌓지 않는다.** 병합 뒤 검증이 실패하면 다음 티켓으로 가지 않는다. 무인 수습(10.6)이 고친 뒤에도 병합된 기본 브랜치로 검증을 다시 통과해야 이어 간다.
- **무인 수습은 한 번, 고친 것은 표시한다.** 한 실행에 `TRIAGE_MAX` 번(기본 1), 같은 멈춤에 한 번. 수습 세션이 낸 PR에는 `loop-triage` 라벨이, 이슈에는 "무인 수습 — 이어 감" 코멘트가, 스펙 리포트에는 결과가 남는다.
- **수리 횟수에 상한이 있다.** 기본 1회 고쳐 보게 하고, 그래도 안 되면 사람에게 넘긴다.
- **워커는 병합하지 않는다.** 프롬프트로 지시하고 권한으로도 막는다(4장 10단계).
- **멈추면 증거를 남긴다.** 워크트리, 워커 출력 꼬리, Orca 카드의 읽지 않음 표시와 코멘트.
- **저장소당 루프 하나.** 잠금 디렉터리로 동시 실행을 막는다.
- **한 실행 안에서 같은 티켓을 두 번 처리하지 않는다.** 담당자가 있는 티켓도 집지 않는다.
- **남긴 기록으로 회고할 수 있다.** 티켓마다 요약 한 줄과, 멈췄다면 워커 출력 꼬리가 로그에 남는다. 기록 파일에는 모델까지 남고, 스펙 단위로는 리포트 HTML 한 장이 남는다.

사람이 지킬 것

하지 말 것	이유
같은 저장소에서 루프를 두 개 돌리기(다른 컴퓨터 포함)	잠금은 한 체크아웃 안에서만 통한다. 두 루프가 같은 티켓을 집을 수 있다.
루프가 도는 동안 기본 브랜치에 직접 커밋	진행 중인 PR에 예상하지 못한 충돌을 만든다. 사람 작업도 PR로 올린다.
메인 체크아웃 `.env` 에 운영 자격 증명 두기	setup 훅이 워커에 복사하고, 워커는 확인 없이 명령을 실행한다.
CI를 통과시키려고 테스트를 건너뛰거나 지우기	무인 병합의 유일한 판정 기준을 스스로 무너뜨린다.
결정이 필요한 티켓에 `ready-for-agent` 붙이기	워커는 질문할 수 없다. 멈추거나 추측한다.
진행 중인 티켓을 정리하지 않고 루프 재실행	같은 티켓을 다시 집는다(10.1).
확인하지 않고 멈춘 워크트리 지우기	커밋되지 않은 작업과 원인 단서가 사라진다.
루프가 도는 동안 `POST_MERGE_PORT` 에 개발 서버 띄우기(루프를 시작하기 전에 이미 떠 있는 것도 포함)	병합 뒤 검증이 10분 기다리다 멈춘다. 그 사이 로그는 "포트 … 사용 중" 한 줄 뒤 조용하다(R10). 시작 전에 `lsof -nP -iTCP:<포트> -sTCP:LISTEN` 으로 비어 있는지 본다.
스킬·규칙 파일을 고치고 병합하지 않은 채 루프 실행	워커는 `origin/main` 의 파일만 본다. 고친 내용이 적용되지 않는다.

권장 운영 리듬

1. 기획: `/grill-with-docs` → 결정 문서 병합 → `/to-spec` → `/to-tickets` (하위 이슈, `T<NN>`)
2. 준비 확인: `--parent <스펙> --dry-run`
3. 규칙을 바꾼 직후에는 `--max 1` 로 지켜보기
4. 무인 실행: 메인 체크아웃의 Orca 터미널 + 잠자기 방지
5. 점검: 로그 끝, `summary:` 줄, 남은 워크트리, `in-review` 이슈 → 멈췄으면 10장
6. 사후 검수: 스펙 리포트(`docs/loop-reports/spec-<N>.html`)로 묶음 전체를 한 표에서 보고 → PR의 "남은 판단 사항"·"검증하지 못한 것" → 후속 티켓, 리포트 커밋(PR). 스펙 이슈는 하위 이슈가 모두 닫히면 루프가 닫는다(7장 5단계) — 사람은 닫혔는지만 본다
7. 회고: `/retro` → 스킬·`worker-rules.md`·자동 검사·스크립트를 PR로 고친다 → 다음 묶음

12. FAQ

Q. 병렬로 여러 티켓을 동시에 돌릴 수는 없나?
이 루프는 의도적으로 직렬이다. 앞 티켓이 병합된 기본 브랜치에서 다음 티켓을 시작하므로 충돌과 중복 구현이 적다. 병렬이 필요하면 mattpocock의 `/implement-spec` 을 쓴다. 이 스킬은 하나의 통합 브랜치에서 실행 가능한 티켓들을 서브에이전트로 동시에 처리한다. 대신 병합 시점과 리뷰를 사람이 더 챙겨야 한다.

Q. 병합 전에 꼭 사람이 보고 싶다.
`--no-merge` 로 돌린다(S3). 또는 브랜치 보호에서 필수 리뷰를 켠다. 그러면 `gh pr merge` 가 거절돼 루프가 멈추고, 그 지점부터 R3로 이어받는다.

Q. E2E·통합 테스트처럼 무거운 검증은?
CI에서 돌릴 수 있으면 CI에 넣는 편이 가장 낫다. 병합 전에 걸러진다. 실제 개발 DB·고정 포트가 필요해 CI에 못 넣는다면, 워커에게는 전체를 돌리지 말되 자기가 추가·수정한 그 테스트 파일만 PR 직전에 1회 돌리고 결과를 PR "검증"에 적으라고 `worker-rules.md` 에 쓰고, 전체는 루프의 `POST_MERGE_CMD` 로 `POST_MERGE_EVERY` 개 병합마다(기본 4, 1이면 병합마다) 한 번 돌린다(7장 6단계). "미실행"이라고만 적고 아무 데서도 돌리지 않으면 깨진 채로 쌓이고(실제로 그랬다), 워커가 새로 쓴 파일을 한 번도 돌리지 않으면 병합 뒤 검증에서 처음 깨져 루프가 멈춘다(그것도 그랬다).

Q. 묶음 하나가 어떻게 끝났는지 한눈에 보려면?
`--parent` 실행이 끝나면 루프가 `docs/loop-reports/spec-<N>.html` 을 쓴다(7장 9단계). 티켓마다 상태·결과 브리프·구현·CI·병합 뒤 검증 시간·경과 시간·토큰·모델이 한 표다. 루프가 도중에 죽어 리포트가 없으면 같은 `--parent` 로 한 번 더 돌린다 — 남은 티켓을 이어서 하고 끝에 리포트를 쓰며, 남은 티켓이 없으면 리포트만 만들고 끝난다. 원자료는 `.ticket-loop/tickets.jsonl` 과 GitHub라서, 기록이 없는 티켓(이 기능 이전, 다른 PC)도 PR 시각으로 채워진다.

Q. 워커가 시간과 토큰을 너무 많이 쓴다.
실측에서 가장 컸던 것은 반복 검증과 큰 화면 캡처였다. ① 검증을 두 층으로 나눠 전체 묶음을 정해진 두 번만 돌린다(4장 11단계). ② 스크린샷은 작게 한 장. ③ `CLAUDE.md` 를 포인터로 줄인다. 워커는 티켓마다 새로 읽는다. ④ "새 API를 추가하면 이 테스트와 이 문서도 바꾼다" 같은 함께 바꿀 목록을 문서로 두고 `implement` 가 먼저 읽게 하면, 실패를 하나씩 겪으며 찾는 왕복이 줄어든다. 스펙 리포트(7장 9단계)의 구현 시간과 토큰 열로 어느 티켓이 오래 걸리고 많이 썼는지 보고 `/retro` 로 원인을 찾는다.

Q. 워커 모델이나 Claude Code 옵션을 바꾸려면?
`worktree create --agent claude` 는 Orca에 설정된 실행기를 쓰고, 호출마다 플래그를 넘기지 않는다. 모델·권한 같은 설정은 저장소의 `.claude/settings.json` 이나 Orca의 에이전트 설정(에이전트 기본 인자)에서 바꾼다. 그래서 Orca 백엔드에서 `--model` · `AGENT_MODEL` 을 주면 루프가 시작하지 않는다 — 루프가 다시 띄운 워커만 모델이 달라지기 때문이다. cmux·tmux 백엔드는 루프의 `AGENT_RESTART_CMD` 로 띄우므로 모델은 `--model opus` (이번 실행만)나 `loop.env` 의 `AGENT_MODEL` 로 고르고, 다른 플래그는 `AGENT_RESTART_CMD` 에 넣는다. 리포트의 모델 칸은 설정값이 아니라 워커 화면에서 읽은 이름이다.

Q. GitLab이나 다른 트래커에서도 쓸 수 있나?
스킬은 GitLab·로컬 파일 트래커도 지원한다. 하지만 이 루프 스크립트는 `gh` 와 GitHub의 선행 관계 API를 쓴다. 다른 트래커에서는 대상 선택(`candidates` , `blocked_by`), PR 조회, 병합 함수를 그 트래커 CLI로 바꿔야 한다.

Q. Codex 같은 다른 에이전트를 워커로 쓸 수 있나?
`loop.env` 에 `AGENT="${AGENT:-codex}"` 를 적으면 프리셋이 기동 명령(`codex --dangerously-bypass-approvals-and-sandbox` — Claude의 `--dangerously-skip-permissions` 에 해당. 더 좁은 `--sandbox workspace-write` 는 네트워크를 막아 push· `gh pr create` 가 안 된다), 배너·모델 정규식, 스킬 접두사(`$implement N`)를 바꾼다. 워커 프롬프트와 수리 프롬프트는 특정 에이전트의 도구 이름을 쓰지 않는다. 저장소 배치도 두 에이전트 공용이다 — 스킬 정본은 `.agents/skills/` (Codex가 읽는 자리)이고 `.claude/skills/` 는 링크, `## Agent skills` 블록은 `AGENTS.md` 에 있고 `CLAUDE.md` 가 가져오며, 금지 명령은 `.claude/settings.json` 의 `deny` 와 `.codex/rules/ticket-loop.rules` 양쪽에 있다(3장 note). 설치기는 `--agent codex` (npx: `--agent` 또는 질문)를 받아 `loop.env` 에 `AGENT` 를 쓰고, Claude Code에만 있는 플러그인 · `~/.claude/skills` 검사를 건너뛴다. 그래도 아직 "지원"이라고는 하지 않는다 — idle 판정(Orca `tui-idle` , cmux shim의 lifecycle 훅), `$implement` 호출, 수리 프롬프트 재전송, `.codex/rules/ticket-loop.rules` 가 로드되는지(`codex execpolicy check --rules .codex/rules/ticket-loop.rules -- gh pr merge 12`)와 `--dangerously-bypass-approvals-and-sandbox` 아래에서도 유지되는지는 Codex가 설치된 컴퓨터에서 실측하지 않았다. `.codex/rules` 는 Codex가 그 저장소의 `.codex/` 를 신뢰할 때만 읽으니, 병합 금지의 진짜 방어선은 GitHub 브랜치 보호다.

Q. 클라우드(웹) Claude Code에서 루프를 돌릴 수 있나?
아니다. 루프는 로컬의 백엔드(Orca·cmux 앱, 또는 tmux)와 로컬 `gh` 로그인을 전제로 한다. 원격 서버에 SSH로 들어가 tmux 백엔드로 돌리는 것은 된다 — 그 서버에 git·gh·jq·tmux·Claude Code가 있고 메인 체크아웃을 신뢰해 두면 된다(2.5).

Q. Orca 대신 다른 터미널 앱을 쓸 수 있나?
cmux(macOS)와 tmux는 백엔드로 들어 있다(2.5, `BACKEND=cmux|tmux`). iTerm·Warp·Ghostty·Windows Terminal 같은 앱은 밖에서 창을

조종할 CLI가 없어 백엔드가 아니고, 그 안에서 tmux 백엔드를 돌린다. 다른 오케스트레이터를 붙이려면 루프가 백엔드에 시키는 아홉 가지 일(`<backend>_check` , `_worktree_create` , `_worktree_remove` , `_send` , `_type` , `_interrupt` , `_wait_idle` , `_tail` , `_set_status`)을 그 CLI로 구현한다(부록 D "백엔드" 절). 핵심은 "에이전트가 입력을 기다리는 중"을 바깥에서 알 수 있느냐다 — tmux 백엔드는 그 신호를 에이전트 훅으로 만들었다.

Q. Windows에서 되나?
WSL2(Ubuntu) 안에서 tmux 백엔드로 돌린다. 설치 순서는 4장 끝의 Windows 절(W1~W5). 네이티브 Windows는 지원하지 않고 cmux는 macOS 전용이다. Linux·WSL 실측은 아직 없으니 `--max 1` 로 먼저 본다.

Q. 루프가 만든 PR의 품질이 들쭉날쭉하다.
대개 티켓 품질 문제다. 인수 조건이 구체적인지, 결정이 문서에 있는지, 티켓이 PR 하나 크기인지 본다. 반복되는 지적은 저장소의 코딩 표준 문서(`CODING_STANDARDS.md` 등)로 옮긴다. `ticket-review` 의 Standards 축이 그 문서를 읽는다. 패턴이 기계적이면(금지된 API, import 모양, 파일 위치) 문서보다 린트 규칙이나 테스트로 만드는 편이 확실하다.

13. 부록

A. 명령 치트시트

하고 싶은 일	명령
대상 미리보기	`bash scripts/ticket-loop.sh --parent 40 --dry-run`
한 건만 시험	`bash scripts/ticket-loop.sh --parent 40 --max 1`
스펙 하나 전부 (잠자기 방지)	`caffeinate -is bash scripts/ticket-loop.sh --parent 40`
준비된 것 전부	`bash scripts/ticket-loop.sh`
특정 티켓만	`bash scripts/ticket-loop.sh 41 45`
버그만 3건	`bash scripts/ticket-loop.sh --label bug --max 3`
병합은 사람이	`bash scripts/ticket-loop.sh --no-merge`
병합마다 E2E	`POST_MERGE_CMD="npm ci && npm run test:e2e" bash scripts/ticket-loop.sh`
티켓별 요약 보기	`grep "summary:" .ticket-loop/loop-MMDD.log`
스펙 리포트 보기 / 다시 만들기	`open docs/loop-reports/spec-40.html` / `bash scripts/ticket-loop.sh --parent 40` (남은 티켓이 없으면 리포트만)
포트 점유 확인(루프 시작 전·대기 중)	`lsof -nP -iTCP:3000 -sTCP:LISTEN`
열린 선행 이슈 수	`gh api repos/OWNER/REPO/issues/N/dependencies/blocked_by --jq '[.[] | select(.state == "open")] | length'`
티켓 상태 한눈에	`ticket_state N` (10.2)
PR 체크 / 실패 로그	`gh pr checks PR --watch` / `gh run view --log-failed`
Orca 워크트리 목록	`orca worktree list --json`
워커 터미널 읽기 / 보내기	`orca terminal read --terminal H --json` / `orca terminal send --terminal H --text "…" --enter --json`
새 에이전트로 이어가기	`orca terminal create --worktree id:ID --command "claude --dangerously-skip-permissions" --json`
워크트리 삭제	`orca worktree rm --worktree id:ID --force --json` 후 `git worktree prune`
(cmux) 워커 화면 읽기 / 보내기 / 상태	`cmux read-screen --surface H --lines 25` / `cmux paste --surface H --submit "…"` / `cmux rpc surface.input_state '{"surface_id":"H"}'`
(tmux) 워커 창 보기 / 상태 파일	`tmux attach -t ticket-loop` (떼기 `Ctrl-b d`) / `cat ~/.cache/ticket-loop/OWNER-REPO/tmux/state/issue-N`
(cmux·tmux) 워크스페이스·창과 워크트리 삭제	`cmux workspace close ID --force` 또는 `tmux kill-window -t issue-N` 후 `git worktree remove --force ~/.cache/ticket-loop/OWNER-REPO/worktrees/issue-N`
남은 잠금 해제	`rmdir "$(git rev-parse --git-common-dir)/ticket-loop.lock"`
멈춘 push 다시 하기	`GIT_TERMINAL_PROMPT=0 git -c credential.helper= -c 'credential.helper=!gh auth git-credential' push -u origin BR`

B-1. 워커 프롬프트 원문 (N=41)

```
/implement 41 — 무인 루프(ticket-loop.sh)에서 실행 중이다. 사람이 답할 수 없으니 질문(AskUserQuestion)하지 말고 스스로 결정해 끝까지 진행해라. PR은 병합하지 마라 — 루프가 CI 뒤에 병합한다. 진행이 불가능하면(열린 선행 이슈 포함) 이슈 #41에 이유를 코멘트로 남기고 PR 없이 멈춰라. 저장소의 .ticket-loop/worker-rules.md 규칙도 따라라.
```

한 줄로 보낸다. TUI에서는 줄바꿈이 곧 제출이다. 마지막 문장은 규칙 파일이 있을 때만 붙는다. 절차는 아래 B-2 스킬에 있다. 프로젝트에 따라 "스키마를 바꾸면 개발 DB에만 적용하라, 운영에는 연결하지 마라"처럼 환경 판단 결과를 한 문장 더 붙일 수 있다(스크립트의 `worker_prompt`).

B-2. `implement` 스킬 예시 (`.agents/skills/implement/SKILL.md`, 저장소 소유 — `.claude/skills/implement`는 그 링크)

upstream `implement`를 바탕으로 무인 실행에 필요한 절차를 채운 것이다. `main`, `npm run verify`, 라벨 이름을 저장소에 맞게 바꾼다.

```
---
name: implement
description: "Implement a piece of work based on a spec or set of tickets."
disable-model-invocation: true
---

<!-- Project-owned: the skill sync skips this file. Based on upstream mattpocock/skills
`implement` (<version>, <commit>); when upstream changes it, port the change by hand. -->

Implement the work described by the user in the spec or tickets. The tracker is described
in `docs/agents/issue-tracker.md`.

If the user passes only a number `N`, it is GitHub issue #N. Run
`gh issue view N --json number,title,body,labels,state,comments` and state its title back.
Stop if it is closed, or labelled `needs-info`, `ready-for-human` or `wontfix`. If the issue
is a sub-issue, read the parent spec too.

Before starting, check the issue has no open blockers: count the open ones in the blocker
list (`gh api repos/{owner}/{repo}/issues/N/dependencies/blocked_by --jq '[.[] | select(.state == "open")] | length'`
is `0`). Do not use `issue_dependencies_summary.blocked_by`: it keeps an old count for a
while after a blocker closes. If it has open blockers, stop and name them.

Create the branch from the remote tip, never by checking out `main` (another worktree may
hold it): `git fetch origin && git checkout -b issue-<N>-<slug> origin/main`.
Assign the issue to yourself (`gh issue edit N --add-assignee @me`).

Read `GLOSSARY.md` and the ADRs in `docs/adr/` that touch the area, and use the glossary's
terms in code, tests and the PR.

Call the Skill tool with "tdd" where possible. Do not ask the user to confirm the seams:
this overrides the confirmation step in `tdd`. Choose the seams yourself from the ticket's
acceptance criteria and the existing tests around the code you touch, then proceed. List
the seams you tested in the pull request body.

While working, run only the typecheck and single test files. Run the full bundle
`npm run verify` (lint, full test suite, build) exactly twice: once right before the first
commit and once right before the review-fix commit.

Heavy tests (integration under `tests/integration/`, E2E) are not part of the bundle and you
never run the whole suite: the loop runs it on the base branch after merge. If this ticket
adds or changes any of those test files, run only those files once, right before opening
the pull request, with `npm run test:integration -- <file>`, and put the files and the
result in the pull request body's Verification section. If you touched none, say so there.

Commit your work to the issue branch, mentioning `#N` in the commit message. Then call the
Skill tool with "ticket-review", with `origin/main` as the fixed point and issue #N as the
spec. Fix every hard violation and every Spec finding in a follow-up commit; list the
remaining judgement calls in the pull request body.

Push the branch and open a pull request into `main` with `gh pr create --base main`. Call
the Skill tool with "pr" for the body's shape. Tick the met acceptance criteria in the issue
body and add the `in-review` label to the issue. Do not merge the pull request.
```

B-3. `pr` 스킬 예시 (`.agents/skills/pr/SKILL.md`, 저장소 소유)

```
---
name: pr
description: "Use when writing a PR body."
---

Write the PR body in this order. Skip no section; write "없음" when a section is empty.

```markdown
Closes #N

## 인수 조건
- [x] <이슈의 인수 조건을 그대로, 충족한 것만 체크>
```
```

````
- [ ] <충족하지 못한 것과 이유>

## 테스트 지점(seam)
- `tests/<file>` — <무엇을 고정하는지 한 줄>

## 검증
- `npm run verify` <결과>, 마이그레이션 <적용/없음>
- 손댄 통합 테스트 파일(`npm run test:integration -- <파일>`): <파일별 결과, 또는 "없음">

## 검증하지 못한 것
- 통합·E2E 전체 미실행 — 루프가 병합 뒤 실행 / 브라우저 수동 확인 없음 / ...

## 남은 판단 사항
- <리뷰가 남긴 판단, 스펙에 없어 택한 해석>
```

Add `## 되돌리기` only for a one-way door (schema data change, auth, money, production
config): say what reverting the PR cannot undo and what to do instead.
Keep each bullet to one or two sentences. No screenshots unless the ticket is visual.
````

C. 수리 프롬프트 원문

```
[충돌] PR #P 이 main과 충돌한다. `git fetch origin && git merge origin/main`으로 충돌을 해결하고 테스트·lint를 다시 돌린 뒤 같은 브랜치에 push해라. 질문하지 말고 끝까지 해라.
[CI]  PR #P 의 CI가 실패했다. `gh pr checks P` 와 `gh run view --log-failed` 로 원인을 보고 고친 뒤 테스트·lint를 다시 돌리고 같은 브랜치에 push해라. 테스트를 건너뛰거나 지우지 마라. 질문하지 말고 끝까지 해라.
```

마이그레이션처럼 번호가 겹칠 수 있는 파일이 있으면 충돌 프롬프트에 "번호가 겹치면 네 것을 지우고 다음 번호로 다시 만들어라"를 더한다.

D. `ticket-loop.sh` 전문 (3판)

이 문서와 함께 배포되는 파일과 같다. 저장소의 `scripts/ticket-loop.sh`로 복사해 쓴다. 의존 도구: bash, git 2.31 이상, gh, jq, 백엔드 CLI(Orca, cmux 또는 tmux), (병합 뒤 검증에 포트를 쓰면) lsof.

```
#!/usr/bin/env bash
set -euo pipefail

# ticket-loop.sh — GitHub 이슈 티켓을 워크트리에서 한 건씩 무인 처리한다. 워크트리와 에이전트 터미널은 백엔드(Orca, cmux 또는 tmux)가 맡는다.
# 가이드: 「티켓 무인 루프 가이드북」 3판 (ticket-loop-kit docs/). 이 파일은 ticket-loop-kit이 관리한다 — 고치면 키트에서 고친다.
#
# 티켓 하나당:
#   1. 기본 브랜치에서 워크트리를 새로 만들고 에이전트(Claude Code 또는 Codex)에 `/implement N`(Codex는 `$implement N`)을 보낸다
#      (티켓마다 새 프로세스라 컨텍스트가 비어 있다)
#   2. 워커가 `<BRANCH_PREFIX>N-<slug>`(또는 워크트리 브랜치 `<BRANCH_PREFIX>N` 그대로) 브랜치로 PR을 열 때까지 기다린다
#   3. CI를 기다리고, 충돌·CI 실패는 같은 워커에 REPAIR_ROUNDS회까지 고치게 한다
#   4. PR을 병합한다 → 본문의 `Closes #N`으로 이슈가 닫힌다. 그 티켓이 속한 스펙 이슈의 하위 이슈가 모두 닫혔으면 스펙 이슈도 닫는다
#   5. (선택) 병합된 기본 브랜치로 POST_MERGE_CMD를 한 번 돌린다 — 워커가 못 돌리는 무거운 검증. 실패하면 멈춘다
#      (POST_MERGE_RETRY_CMD가 있으면 멈추기 전에 한 번 재실행해 본다 — 통과하면 플레이크 의심으로 기록하고 계속 간다).
#      병합 POST_MERGE_EVERY개마다(기본 4), 그리고 스펙이나 실행이 끝날 때 한 번 돌린다 — 그 사이 병합은 다음 검증이 덮는다.
#      1이면 병합마다, 0이면 스펙·실행이 끝날 때만
#   6. 워크트리·브랜치를 지우고, 티켓 요약 한 줄을 로그에, 기록 한 줄을 RECORDS_FILE에 남기고, 다음 티켓으로 간다
#      (자동 선택이면 대상 재계산)
#   7. `--parent` 실행이 정상 종료하면 하위 이슈가 모두 닫힌 스펙 이슈를 닫고(아직 열려 있으면), 스펙 리포트 HTML(티켓별 결과·구현·CI·
#      병합 뒤 검증 시간·경과 시간·모델 표)을 REPORT_DIR/spec-<N>.html에 쓰고 브라우저로 연다(REPORT_OPEN=0이면 열지 않음)
#   8. 이번 실행에서 끝난 스펙이 있으면 회고를 권하는 안내 한 줄을 남긴다(/retro로 읽을 자료 안내). 권고일 뿐 아무것도 실행하지 않는다 —
#      회고는 사람이 후보를 보고 고르는 단계라 무인으로 돌리지 않는다
# 그 사이 멈출 일이 생기면(병합 뒤 검증 실패, 수리를 다 쓴 CI 실패·충돌, 워크트리 준비 실패, PR 없이 멈춘 워커) 멈추기 전에 수습 세션을
# 띄운다(LOOP_TRIAGE, 아래 "무인 수습" 절). 고쳐지면 멈춘 자리에서 이어 가고, 못 고치면 진단을 실어 멈춘다
#
# 사용법:
#   ticket-loop.sh                    # 준비된 티켓 전부
#   ticket-loop.sh --parent 40        # 스펙 이슈 #40의 하위 이슈만
#   ticket-loop.sh --label bug        # 라벨로 좁히기(SCOPE_LABEL과 같다)
```
```

```
#   ticket-loop.sh 12 15              # 지정한 이슈만, 적은 순서대로(라벨은 보지 않는다. 닫힌 이슈는 건너뛰고, 스펙 이슈면 시
작 전에 멈춘다)
#   ticket-loop.sh --dry-run          # 대상 목록만 출력
#   ticket-loop.sh --max 1            # 최대 1건
#   ticket-loop.sh --no-merge 12      # CI 통과까지만, 병합은 사람이
#   ticket-loop.sh --skip-post-merge  # POST_MERGE_CMD를 이번 실행에서 끈다
#   ticket-loop.sh --model opus       # 워커 모델을 이번 실행에만 정한다(AGENT_MODEL과 같다, cmux·tmux만)
#
# 전제: 백엔드 준비(Orca: 앱 실행·CLI 등록 / cmux: cmux 터미널 안에서 실행하거나 소켓 모드를 열어 둠 / tmux: tmux 설치), `gh au
th status` 로그인,
# 저장소 안에서 실행.
# 진행 로그는 기본으로 .ticket-loop/loop-MMDD.log에 덧붙인다(.gitignore에 /.ticket-loop/*.log 권장).
# PR을 기다리는 동안은 워크트리를 PROGRESS_SEC마다 들여다봐 브랜치 생성·첫 파일 수정·새 커밋을 바로, 그 사이 변경 파일 목록이
달라졌으면
# HEARTBEAT_SEC마다 "working" 한 줄을 찍는다. 지난번과 같은 내용이면 찍지 않는다.
# 터미널·로그 파일에 찍는 문구는 영어다(워커 프롬프트·GitHub 코멘트·백엔드 상태 코멘트는 그대로).
# 터미널에는 단계별로 색을 입힌다(티켓 제목·단계·성공·주의·오류). stderr가 터미널이 아니거나 NO_COLOR가 있으면 맨글자고,
# 로그 파일에는 언제나 색 코드 없이 쓴다. 실행 맨 앞에는 키트 이름·버전 머리줄을 한 줄 찍는다(.ticket-loop/kit.json — 설치기가
쓴다).
# 티켓마다 "#N summary: …" 한 줄(벽시계·PR과 그 PR을 알아채기까지·CI 소요·수리·idle 오판·병합 뒤 검증)을 남겨 /retro가 읽는다.
# 같은 내용 + 모델을 RECORDS_FILE(JSONL)에도 남기고, 스펙 리포트가 GitHub의 PR과 합쳐 표로 만든다.
# 저장소별 설정은 .ticket-loop/loop.env에 둔다(있으면 읽는다). 그 파일은 `VAR="${VAR:-값}"` 꼴로 써서
# 실행할 때 준 환경변수가 이기게 한다. 예: CI_SKIP_REGEX="${CI_SKIP_REGEX:-^docs/}" POST_MERGE_PORT="${POST_MERGE_PORT:-3000}"
# 기계별 설정과 비밀 값(SLACK_WEBHOOK_URL)은 커밋하지 않는 ~/.config/ticket-loop/loop.env(XDG_CONFIG_HOME을 따른다)에 같은 꼴로
둔다.
# 저장소 loop.env보다 먼저 읽어 그쪽이 이긴다(실행할 때 준 환경변수는 둘 다 이긴다).
#
# 환경변수(선택):
#   BACKEND             orca ¦ cmux ¦ tmux (기본 orca). 워크트리 생성·에이전트 터미널·idle 판정·상태 표시를 누가 하나. 아래 "백
엔드" 절
#   ORCA_CLI            Orca CLI 실행 파일 (기본: ORCA_CLI_COMMAND → Linux면 orca-ide → orca)
#   CMUX_CLI            cmux CLI 실행 파일 (기본: PATH의 cmux → /Applications/cmux.app/Contents/Resources/bin/cmux)
#   TMUX_CLI            tmux 실행 파일 (기본 tmux)
#   TMUX_SESSION        tmux 백엔드가 워커 창을 열 세션 (기본: tmux 안에서 돌리면 그 세션, 밖이면 ticket-loop — 없으면 만든다)
#   WORKTREE_DIR        cmux·tmux 백엔드가 워크트리를 둘 디렉터리 (기본 ~/.cache/ticket-loop/<owner-repo>/worktrees). Orca는 자
기 자리에 둔다
#   WORKTREE_SETUP_CMD  cmux·tmux 백엔드가 새 워크트리에서 에이전트를 띄우기 전에 돌릴 명령(의존성 설치·env 복사 — Orca의 orca.
yaml setup에
#                       해당). LOOP_ROOT_PATH·LOOP_WORKTREE_PATH를 환경변수로 준다. 예 "pnpm install && cp \"$LOOP_ROOT_PATH/.e
nv.local\" ."
#   SPEC_ISSUE          --parent 기본값
#   SCOPE_LABEL         함께 요구할 라벨(--label 기본값)
#   READY_LABEL         대상 라벨 (기본 ready-for-agent)
#   IN_REVIEW_LABEL     PR이 열린 티켓 라벨 (기본 in-review)
#   BRANCH_PREFIX       이슈 브랜치 접두사 (기본 issue-)
#   BASE_BRANCH         기준 브랜치 (기본: 저장소 기본 브랜치)
#   ORDER_REGEX         제목에서 순번을 뽑는 정규식, 이름 있는 그룹 n (기본 ^T(?<n>[0-9]+))
#   WORKER_RULES_FILE   워커가 추가로 따를 규칙 파일 (기본 .ticket-loop/worker-rules.md)
#   AGENT               claude ¦ codex (기본 claude). Orca가 띄울 에이전트 ID이자 기록의 agent 필드. 기동 명령·배너·스킬 접두사
·모델 정규식·
#                       화면 분류 정규식 둘의 기본을 정하는 프리셋이다(아래 "에이전트 프리셋" 절). 다른 값이면 기동 명령·배너는
직접 적어야 한다
#   AGENT_RESTART_CMD   에이전트를 셸에서 띄우는 명령. 프롬프트는 인자로 붙는다 (claude: claude --dangerously-skip-permissions,
#                       codex: codex --dangerously-bypass-approvals-and-sandbox). Orca는 뜨지 않았을 때만, cmux·tmux는 처음부터
이 명령으로 띄운다(tmux는 훅 설정 인자를 뒤에 붙인다)
#   AGENT_MODEL         워커 모델 (기본 비움 = 에이전트의 평소 기본 모델). --model이 이긴다. 주면 기동 명령 뒤에 "AGENT_MODEL_F
LAG <모델>"을
#                       붙인다 — 워커, 다시 띄우기, 무인 수습 세션이 모두 같은 명령을 쓴다. cmux·tmux만 된다: Orca는 에이전트를
직접 띄우고
#                       인자는 Orca 설정(에이전트 기본 인자)에서만 받으므로 시작 때 멈춘다(이슈 #50)
#   AGENT_MODEL_FLAG    모델을 고르는 기동 명령 인자 (claude·codex: --model). 다른 AGENT면 직접 적는다 — 비어 있는데 모델을 주
면 시작 때 멈춘다
#   AGENT_BANNER_REGEX  에이전트가 떴는지 확인할 화면 문구(ERE) (claude: Claude Code 배너·모델명, codex: "OpenAI Codex"·gpt- 모
델명)
#   SKILL_PREFIX        워커 프롬프트에서 스킬을 부르는 접두사 (claude: / → /implement N, codex: $ → $implement N)
#   TICKET_TIMEOUT_SEC  티켓 한 건 제한 시간 (기본 14400 = 4시간)
#   MERGE_METHOD        merge ¦ squash ¦ rebase (기본 merge)
#   REPAIR_ROUNDS       충돌·CI 수리 횟수 합계 (기본 1)
#   CI_SKIP_REGEX       저장소 CI가 경로 필터(paths / paths-ignore)로 일부 PR에서 돌지 않을 때, 그 필터를 루프에 알려 주는 값.
#                       바뀐 파일이 전부 이 정규식(ERE)에 맞으면 체크가 안 붙는 게 정상인 PR로 보고 체크 없이 병합한다.
#                       루프가 정하는 규칙이 아니라 워크플로의 필터를 따라 하는 것이다 — 필터가 없으면 비워 둔다 (기본 없음 =
모든 PR에서 체크를 기다린다)
```

```
#   CI_SKIP_CMD          같은 목적의 명령 — 순서 규칙이 있는 paths 목록처럼 정규식 하나로 못 쓰는 필터용. 바뀐 파일 목록을 stdin
으로
#                        (한 줄에 하나) 받아 CI가 돌면 "run", 전부 제외 경로면 "skip"을 출력한다. 저장소 루트에서 실행되고, 있으
면
#                        CI_SKIP_REGEX보다 먼저 본다. 실패하거나 다른 출력이면 체크를 기다리는 쪽으로 간다
#   POST_MERGE_CMD       병합 뒤 기본 브랜치에서 돌릴 명령, 예 "npm ci && npm run test:e2e" (기본 없음 = 끔). 검증을 상황에 따라
다르게
#                        돌릴 수 있게 env 둘을 받는다(POST_MERGE_RETRY_CMD도 같다): TICKET_LOOP_POST_MERGE_WHY — 왜 도는지, ever
y(병합
#                        POST_MERGE_EVERY개) · spec_end(스펙 끝) · run_end(이번 실행의 마지막 티켓·남은 티켓 없음·루프 끝).
#                        TICKET_LOOP_POST_MERGE_SPEC_COMPLETE — 이 검증이 덮는 병합 가운데 스펙(--parent 또는 티켓의 부모)을 끝
낸 것이
#                        있으면 그 스펙 번호(드물게 여럿이면 공백 구분), 없으면 빈 값. 이유와 따로 본다 — POST_MERGE_EVERY=1이면
스펙 끝의
#                        검증도 every다. 병합이 모두 검증을 거친 뒤에 스펙이 닫혀도(사람이 마지막 티켓을 닫음·하위 이슈 반영 지
연)
#                        그 번호를 싣고 통과한 검증이 이번 실행에 없었으면 루프 끝에 기본 브랜치로 한 번 돈다 — spec_end와 그 번
호.
#                        재실행·이분 탐색·수습 뒤 재검증은 첫 실행과 같은 값이다. 예: 스펙 끝에만 무거운 DB로
#                        [ -n "$TICKET_LOOP_POST_MERGE_SPEC_COMPLETE" ]. 받은 값은 검증 로그 첫 줄과 RECORDS_FILE에 남는다
#   POST_MERGE_RETRY_CMD POST_MERGE_CMD가 실패했을 때 딱 한 번 돌려 볼 재실행 명령 (기본 없음 = 재실행 없이 멈춤). 같은 검증 워
크트리에서
#                        POST_MERGE_FAILED_LOG(첫 실행 로그의 절대 경로)를 받아 돈다 — 그 로그에서 실패한 테스트만 골라 다시 돌
리는 식이다.
#                        출력은 post-merge-N-retry.log. 통과하면 계속 가고 이슈 코멘트·Slack에 플레이크 의심을 남긴다. 실패한 테
스트를
#                        찾지 못하면(설치·환경 단계에서 실패) 다시 돌리지 말고 비0으로 끝내야 한다 — 진짜 깨진 기본 브랜치를 덮
지 않게
#   POST_MERGE_EVERY     POST_MERGE_CMD를 병합 몇 개마다 돌리나 (기본 4 — 네 번째 병합 뒤에 앞의 넷을 한 번에 검증한다. 1이면 병
합마다,
#                        0이면 개수로는 돌리지 않는다). 그 전이라도 티켓의 스펙이 끝나거나(스펙 이슈를 닫음) 이번 실행의 마지막
티켓이면
#                        돌린다 — 0이면 그때만 돈다. 검증은 그때까지의 병합을 모두
#                        덮으므로 빠지는 병합은 없고, 잃는 것은 "어느 병합이 깨뜨렸나"다 — 묶음이 실패하고 POST_MERGE_RETRY_CMD
가 있으면
#                        그 명령으로 마지막 통과 뒤의 병합 커밋을 이분 탐색해 처음 실패하는 병합을 찾아 멈춤 사유·무인 수습에 넘
긴다.
#                        아직 검증하지 않은 병합과 그 번호를 받은 검증을 아직 통과하지 못한 스펙 완료는 .git/ticket-loop-post-me
rge.json에
#                        남아 멈췄다 다시 실행해도 이어진다
#   POST_MERGE_COPY      POST_MERGE_CMD 전에 메인 체크아웃에서 복사할 파일들(공백 구분), 예 ".env.local". 검증 워크트리를 정리한
뒤에 놓는다 —
#                        메인 체크아웃에서 지운 파일은 검증 워크트리에도 남지 않는다
#   POST_MERGE_KEEP      검증 워크트리를 정리할 때 남길 경로 패턴(공백 구분, .gitignore 꼴 — 검증 워크트리 루트 기준, 슬래시 없
으면 어느 깊이든,
#                        /로 시작하면 루트만), 예 "node_modules" (기본 없음 = CI처럼 깨끗한 체크아웃). 의존성 디렉터리를 남겨 다
시 설치하는
#                        시간을 아낄 때만 쓴다. 검증마다 커지는 빌드 캐시(.next 등)는 넣지 않는다 — 넣으면 다시 쌓인다(#47)
#   POST_MERGE_PORT      POST_MERGE_CMD가 쓰는 포트. 지정하면 비기를 기다린다(기본 없음)
#   POST_MERGE_WORKTREE POST_MERGE_CMD를 돌릴 워크트리 (기본 ~/.cache/ticket-loop/<owner-repo>/post-merge, 저장소 밖). 처음 한
번 만들고 다시 쓰되,
#                        검증마다 시작할 때 기본 브랜치 끝으로 되돌린다 — 추적 파일의 수정은 버리고, 추적 안 된 파일과 무시된 파
일은
#                        POST_MERGE_KEEP에 맞는 것 말고 지운다. 재실행·이분 탐색 사이와 검증이 끝날 때는 정리하지 않는다(실패한
검증의
#                        산출물은 다음 검증까지 남는다). 메인 체크아웃을 가리키면 시작하지 않고(종료 코드 2), 이미 있는 그 자리
가 이 저장소의
#                        연결 워크트리가 아니거나 브랜치에 올라 있으면(루프는 검증 워크트리를 늘 브랜치 없이 쓴다 — 브랜치에 올
라 있으면 사람이나
#                        워커의 작업 트리다) 정리하지 않고 검증을 실패로 끝낸다(did not start)
#   LOG_FILE             진행 로그 파일 (기본 .ticket-loop/loop-MMDD.log, /dev/null이면 끔). 상대 경로는 저장소 루트 기준이고 시
작 때
#                        절대 경로로 굳힌다 — 병합 뒤 검증 로그(post-merge-N.log)도 같은 디렉터리에 둔다
#   PROGRESS_SEC         PR·수리를 기다리는 동안 워크트리(git)를 들여다보는 간격 (기본 60초). 브랜치 생성·첫 파일 수정·새 커밋
(제목 그대로)을
#                        그때 바로 한 줄씩 찍는다. 0이면 끄고 전처럼 idle 대기 상한(10분)마다만 돌아온다
#   HEARTBEAT_SEC        커밋은 없지만 변경 파일 목록이 지난 줄과 달라졌을 때 "working — Nm elapsed, N uncommitted files …" 한
줄을 찍는 최소 간격
#                        (기본 300초). 목록이 같으면 아무리 지나도 찍지 않는다 — 같은 내용을 되풀이하지 않는다
#   TERMINAL_UNRESPONSIVE_SEC  워커 터미널을 읽지 못하는데 백엔드 앱(cmux·Orca)의 프로세스는 살아 있을 때(앱이 멈춤) 응답을 기
다리는 상한
#                        (기본 600초, 경과 시간). 기다리는 동안 1분마다 한 줄, 응답이 돌아오면 한 줄을 찍고 이어 간다. 넘기면 터
```

```
미널 끊김으로
#                       멈춘다(#61). 터미널이나 앱 프로세스가 없으면 기다리지 않고 아래 두 설정대로 다시 읽어 본 뒤 멈춘다
#   TERMINAL_RETRY      터미널 화면을 읽지 못할 때(터미널·앱이 없음) 연달아 다시 읽어 보는 횟수 (기본 5). 다 실패하면 터미널 끊
김으로 멈춘다
#   TERMINAL_RETRY_SEC  그 사이 쉬는 간격 (기본 10초)
#   RECORDS_FILE        티켓별 기록 JSONL (기본 .ticket-loop/tickets.jsonl, 기계별 — 커밋하지 않는다). 상대 경로는 저장소 루트
기준
#   REPORT_DIR          스펙 리포트 HTML 디렉터리 (기본 docs/loop-reports — 리포트는 사람이 PR로 커밋). 상대 경로는 저장소 루트
기준
#   REPORT_OPEN         리포트를 쓴 뒤 브라우저로 여는지 (기본 1). 0이면 안 연다. SSH 세션·화면 없는 Linux에서는 값과 무관하게
안 연다
#   REPORT_DECISIONS_HEADING  PR 본문에서 사람이 봐야 할 절의 마크다운 제목(글자 그대로, 예: "사람 확인 필요"). 지정하면 리포트
가 각 티켓 PR 본문에서
#                       그 절을 뽑아 머리의 목록(PR 링크 포함)으로 모으고, 절이 있는 행에 표시를 단다. 절이 없거나 빈 PR은 목록
에 없다.
#                       기본 비움 = 모으지 않는다(지금까지와 같다). 저장소마다 절 이름이 다르므로 키트는 이름만 받는다(이슈 #2
5)
#   REPORT_DECISIONS_NONE  그 절에 "내용 없음"으로 적는 말들(글자 그대로 — 정규식이 아니다, "¦" 구분, 대소문자 무시) (기본 "없
음¦none¦N/A").
#                       절 본문에서 목록 기호(-·*)와 끝 마침표를 걷은 것이 이 가운데 하나와 같으면("- 없음", "없음.") 빈 절처럼
목록·건수·표시에서
#                       뺀다. 뒤에 내용이 더 붙으면 남는다(이슈 #39)
#   MODEL_REGEX         워커 터미널에서 모델 이름을 뽑는 정규식(ERE) (claude: 상태줄·배너의 "Opus 5.5" 꼴, codex: "gpt-5.1-code
x" 꼴).
#                       못 찾으면 리포트에 not recorded
#   USAGE_LIMIT_REGEX   워커 화면 끝 25줄에서 사용량 한도 문구를 찾는 정규식(ERE, 대소문자를 가리지 않는다) (claude: "usage lim
it reached¦
#                       (weekly¦[0-9]+-hour) limit reached¦(hit¦reached) your( [a-z0-9-]+)?( [a-z0-9-]+)? limit", codex·그 외:
없음).
#                       idle인 워커의 화면에 맞으면 strike를 세지 않고 usage_limit로 바로 멈춘다 — 무인 수습은 띄우지 않는다(같
은 한도에 걸린다).
#                       비우면 감지하지 않는다. 아래 "화면 분류" 절
#   PERMISSION_PROMPT_REGEX  워커 화면 끝에서 권한 확인 창 문구를 찾는 정규식(ERE) (claude: "Do you want to proceed\?", codex·
그 외: 없음). 그 줄이 든
#                       화면이 PROMPT_STALL_SEC(300초) 동안 글자 그대로면 idle 신호가 없어도 worker_no_pr로 멈춘다. 루프는 확인
창에 답하지 않는다.
#                       비우면 감지하지 않는다
#   SLACK_WEBHOOK_URL   Slack Incoming Webhook URL (기본 없음 = Slack 끔). 루프 시작·티켓 완료·멈춤·루프 종료를 그 채널에 올린
다.
#                       잠금을 잡은 실행만 보낸다(--dry-run, 이미 도는 루프와 겹친 실행은 안 보낸다). 비밀 값이다 — 커밋하는 저
장소
#                       loop.env가 아니라 ~/.config/ticket-loop/loop.env나 환경변수에 둔다
#   SLACK_MENTION       멈춤 알림 앞에 붙일 멘션, 예 "<@U012AB3CD>"(자기 멤버 ID) 또는 "<!channel>" (기본 없음). 채널 알림을
"멘션만"으로
#                       두고도 멈출 때만 폰이 울린다. Ctrl-C로 멈춘 것(사람이 앞에 있다)에는 붙이지 않는다
#   STOPPED_LABEL       루프가 멈춘 티켓에 붙이는 라벨 (기본 loop-stopped, 없으면 만든다). 기록용이다 — 대상 선택은 이 라벨을
보지 않고,
#                       루프가 그 티켓을 병합하면 뗀다
#   LOOP_TRIAGE         멈출 일이 생기면 수습 세션을 띄우는지 (기본 1). 0이면 전처럼 바로 멈춘다. 아래 "무인 수습" 절
#   TRIAGE_MAX          한 실행에서 수습 세션을 띄우는 최대 횟수 (기본 1). 다 쓰면 그 뒤의 멈춤은 수습 없이 멈춘다
#   TRIAGE_TIMEOUT_SEC  수습 세션 한 번의 제한 시간 (기본 3600 = 1시간)
#   TRIAGE_LABEL        수습 세션이 낸 수정 PR에 붙이는 라벨 (기본 loop-triage, 없으면 만든다) — 사람이 나중에 몰아서 본다
#
# 멈추는 조건(루프 종료, 워크트리는 조사용으로 남긴다):
#   - 워커가 PR 없이 멈춤(질문·실패, 확인 창에 PROMPT_STALL_SEC 머묾) / 워커가 사용량 한도에 걸림(usage_limit — 수습 없음) / 티
켓 타임아웃 /
#     백엔드 터미널 끊김(터미널·앱이 없음, 앱이 TERMINAL_UNRESPONSIVE_SEC 넘게 응답하지 않음 — 사유가 셋을 가르고, 워커가 루프
밖에서 PR을 열 수
#     있으니 다시 돌리기 전에 보라고 적는다) / 에이전트 기동 실패
#   - 수리 REPAIR_ROUNDS회 뒤에도 CI 실패 또는 충돌, 수리 뒤 새 커밋이 없고 CI도 그대로 실패(재실행만 한 수리가 통과하면 이어
간다)
#   - 병합 뒤 POST_MERGE_CMD 실패(POST_MERGE_RETRY_CMD가 있으면 재실행까지 실패), 또는 POST_MERGE_PORT가 10분 안에 비지 않음.
#     여러 병합을 묶어 검증했으면(POST_MERGE_EVERY) 사유에 묶음과 이분 탐색으로 찾은 처음 실패하는 병합을 적는다
#     (기다리는 동안은 로그 한 줄 뒤 조용하다 — 멈춤이 아니다)
# 자동 선택은 선행 이슈가 열렸거나, PR이 이미 있거나, 담당자가 있거나, 하위 이슈를 가진(스펙) 이슈를 건너뛴다.
#
# 멈추면 — 위 조건, 그리고 터미널 창 닫힘(HUP·PIPE)·Ctrl-C·kill(TERM)·set -e에 걸린 명령(네트워크가 끊겨 git fetch·gh가 실패하
는 등) —
# 어떻게 끝나든 한 번만 보고한다: 진행 중 티켓을 stopped로 기록하고, 그 이슈에 코멘트(사유·머신·PR·워크트리·루프 로그 끝)와
# STOPPED_LABEL을 남기고(GitHub 기록 — 루프는 본인 계정으로 쓰므로 GitHub는 본인에게 알리지 않는다. 알림이 아니라 기록이다),
# 데스크톱과 Slack(SLACK_WEBHOOK_URL)에 알린다. Slack에는 루프 시작·티켓 완료·루프 종료도 간다.
# kill -9·전원 꺼짐·잠자기처럼 스크립트가 아무것도 보내지 못하는 경우는 잡지 못한다.
```

```
# 멈출 때마다 멈춤 기록(.ticket-loop/stops/<id>.json — 사유·이슈·PR·로그 경로·워크트리·다시 실행할 명령, 기계별)을 남긴다.
# 사람은 그 저장소의 에이전트 세션에서 /loop-triage <id>(Codex는 $loop-triage)로 그 기록부터 이어받는다.
#
# 무인 수습(LOOP_TRIAGE=1, 기본): 아래 네 경우에는 멈추기 전에 멈춤 기록을 쓰고, 워커처럼 새 워크트리에 에이전트를 띄워
# `/loop-triage <기록>`을 시킨 뒤 판정 파일(<id>.verdict.json — resume ¦ halt)을 기다린다. Slack에는 "수습 시작"과 결과가 간다.
#   - 병합 뒤 검증 실패(재실행까지 실패) → 수습 세션이 수정 PR을 내면 루프가 CI 뒤 병합하고, 병합된 기본 브랜치로 검증을 다시 돌려
#     통과하면 이어 간다(플레이크로 판정하면 수정 없이 검증만 다시 돌린다)
#   - 수리를 다 쓴 CI 실패·충돌, 새 커밋 없이 끝난 수리 뒤에도 CI 실패 → 수습 세션이 티켓 PR 브랜치에 고쳐 push하면(또는 CI 플레이크면 실패한 job을 다시 돌리면)
#     CI 대기부터 이어 간다
#   - WORKTREE_SETUP_CMD 실패(cmux·tmux) → 수정 PR을 병합하면 워크트리를 기본 브랜치로 당기고(메인 체크아웃도 기본 브랜치면 fast-forward)
#     준비를 다시 돌린다
#   - 워커가 PR 없이 멈춤 → 진단만 한다. 판단이 필요한 질문이 많아 답하지 않고, 요약을 실어 멈춘다
# 판정이 halt이거나, 판정 없이 끝나거나(제한 시간·idle), 수정 PR의 CI가 실패하거나, 이어 간 뒤 같은 곳에서 또 실패하면 진단을 사유에 붙여
# 멈춘다(수습 세션의 워크트리는 남긴다). 한 실행에 TRIAGE_MAX번까지, 같은 멈춤에는 한 번만. 수습 세션은 기본 브랜치에 커밋된
# loop-triage 스킬을 쓰므로 키트 설치 PR이 병합되기 전에는 띄우지 않는다. 그 밖의 멈춤(사용량 한도, 타임아웃, 터미널 끊김, 에이전트 기동 실패,
# 병합 실패, 신호, set -e)은 수습 없이 멈춘다 — 어디서든 안전하게 이어 갈 수 있는 자리가 아니다. GitHub 일시 장애(5xx·연결 오류)는
# 읽기 호출과 병합 모두 멈추기 전에 GH_TRIES번까지 다시 해 본다. 필수 리뷰 같은 브랜치 보호의 거절은 사람에게 넘기는 자리다.

ROOT="$(git rev-parse --show-toplevel 2>/dev/null ¦¦ pwd)"   # 저장소 루트. 도구 검사 뒤 git으로 다시 확정한다
LOOP_ENV="$ROOT/.ticket-loop/loop.env"
# 기계별 설정·비밀 값은 커밋하지 않는 사용자 파일에 둔다. 저장소 loop.env보다 먼저 읽어 이긴다(둘 다 VAR="${VAR:-값}" 꼴)
USER_LOOP_ENV="${XDG_CONFIG_HOME:-$HOME/.config}/ticket-loop/loop.env"
# shellcheck source=/dev/null
[[ ! -f "$USER_LOOP_ENV" ]] ¦¦ source "$USER_LOOP_ENV"
# shellcheck source=/dev/null
[[ ! -f "$LOOP_ENV" ]] ¦¦ source "$LOOP_ENV"

BACKEND="${BACKEND:-orca}"
if [[ -n "${ORCA_CLI:-}" ]]; then :
elif [[ -n "${ORCA_CLI_COMMAND:-}" ]]; then ORCA_CLI="$ORCA_CLI_COMMAND"
elif [[ "$(uname -s)" == "Linux" ]]; then ORCA_CLI="orca-ide"   # Linux의 orca는 GNOME 스크린리더다
else ORCA_CLI="orca"
fi
if [[ -z "${CMUX_CLI:-}" ]]; then
  if command -v cmux >/dev/null 2>&1; then CMUX_CLI="cmux"; else CMUX_CLI="/Applications/cmux.app/Contents/Resources/bin/cmux";
fi
fi
TMUX_CLI="${TMUX_CLI:-tmux}"
TMUX_SESSION="${TMUX_SESSION:-}"
WORKTREE_DIR="${WORKTREE_DIR:-}"
WORKTREE_SETUP_CMD="${WORKTREE_SETUP_CMD:-}"
PARENT="${SPEC_ISSUE:-}"
CLOSED_SPECS=""   # 이번 실행에서 닫은 스펙 이슈 번호들(끝날 때 회고 안내에 쓴다)
SCOPE_LABEL="${SCOPE_LABEL:-}"
READY_LABEL="${READY_LABEL:-ready-for-agent}"
IN_REVIEW_LABEL="${IN_REVIEW_LABEL:-in-review}"
BRANCH_PREFIX="${BRANCH_PREFIX:-issue-}"
BASE_BRANCH="${BASE_BRANCH:-}"
ORDER_REGEX="${ORDER_REGEX:-^T(?<n>[0-9]+)}"
WORKER_RULES_FILE="${WORKER_RULES_FILE:-.ticket-loop/worker-rules.md}"
AGENT="${AGENT:-claude}"
AGENT_MODEL="${AGENT_MODEL:-}"
TICKET_TIMEOUT_SEC="${TICKET_TIMEOUT_SEC:-14400}"
MERGE_METHOD="${MERGE_METHOD:-merge}"
REPAIR_ROUNDS="${REPAIR_ROUNDS:-1}"
CI_SKIP_REGEX="${CI_SKIP_REGEX:-}"
CI_SKIP_CMD="${CI_SKIP_CMD:-}"
POST_MERGE_CMD="${POST_MERGE_CMD:-}"
POST_MERGE_RETRY_CMD="${POST_MERGE_RETRY_CMD:-}"
POST_MERGE_EVERY="${POST_MERGE_EVERY:-4}"
POST_MERGE_COPY="${POST_MERGE_COPY:-}"
POST_MERGE_KEEP="${POST_MERGE_KEEP:-}"
POST_MERGE_PORT="${POST_MERGE_PORT:-}"
POST_MERGE_WORKTREE="${POST_MERGE_WORKTREE:-}"
LOG_FILE="${LOG_FILE:-.ticket-loop/loop-$(date +%m%d).log}"
RECORDS_FILE="${RECORDS_FILE:-.ticket-loop/tickets.jsonl}"
```

```
REPORT_DIR="${REPORT_DIR:-docs/loop-reports}"
REPORT_OPEN="${REPORT_OPEN:-1}"
REPORT_DECISIONS_HEADING="${REPORT_DECISIONS_HEADING:-}"
REPORT_DECISIONS_NONE="${REPORT_DECISIONS_NONE:-없음¦none¦N/A}"
PROGRESS_SEC="${PROGRESS_SEC:-60}"
HEARTBEAT_SEC="${HEARTBEAT_SEC:-300}"
SLACK_WEBHOOK_URL="${SLACK_WEBHOOK_URL:-}"
SLACK_MENTION="${SLACK_MENTION:-}"
STOPPED_LABEL="${STOPPED_LABEL:-loop-stopped}"
LOOP_TRIAGE="${LOOP_TRIAGE:-1}"
TRIAGE_MAX="${TRIAGE_MAX:-1}"
TRIAGE_TIMEOUT_SEC="${TRIAGE_TIMEOUT_SEC:-3600}"
TRIAGE_LABEL="${TRIAGE_LABEL:-loop-triage}"
# 파일 경로는 시작 때 한 번 저장소 루트 기준 절대 경로로 굳힌다(loop.env나 환경변수로 상대 경로를 줘도 같다). 루프는 저장소
# 루트에서 돌지만, 병합 뒤 검증처럼 다른 디렉터리로 cd 한 서브셸에서 여는 파일은 그 디렉터리 기준이 된다 — 실측(0.8.3,
# rental-portal #101)에서 post-merge-N.log가 검증 워크트리 안에 생기고 루프는 저장소 루트에서 꼬리를 찾지 못했다.
# /dev/null(로그 끔)은 절대 경로라 그대로다. 검증 워크트리(POST_MERGE_WORKTREE)도 굳힌다 — 루프를 하위 디렉터리에서 띄우면 시작 검사와
# 워크트리 만들기(git -C 저장소 루트)는 루트 기준, 정리(git -C 검증 워크트리)는 지금 디렉터리 기준으로 풀려 서로 다른 곳을 볼 수 있다
abs_path() { case "$1" in /*) printf '%s' "$1" ;; *) printf '%s/%s' "$ROOT" "$1" ;; esac; }
LOG_FILE="$(abs_path "$LOG_FILE")"
RECORDS_FILE="$(abs_path "$RECORDS_FILE")"
REPORT_DIR="$(abs_path "$REPORT_DIR")"
[[ -z "$POST_MERGE_WORKTREE" ]] ¦¦ POST_MERGE_WORKTREE="$(abs_path "$POST_MERGE_WORKTREE")"
# ----- 에이전트 프리셋 -----
# AGENT가 기동 명령·배너·모델 정규식·스킬 접두사·화면 분류 정규식(사용량 한도·권한 확인 창)의 기본값을 정한다. loop.env나 환경변수로 준 값이 이긴다.
#   claude  Claude Code. 스킬은 /이름. 모델명은 상태줄·배너의 "Opus 5.5" 꼴. 사용량 한도는 "You've reached your … limit"(#10)·"You've hit your
#           weekly limit"·"Usage limit reached"(#52 — 2026-11 실측, 문장 첫머리가 대문자다), 이전 판의 "Claude usage limit reached"·"5-hour limit
#           reached"·"Weekly limit reached"(키트 실측 아님). "Context limit reached"(맥락 한도)는 맞지 않게 둔다. 권한 확인 창은
#           "Do you want to proceed?"(screen_kind — idle strike 앞에서 본다)
#   codex   Codex CLI. 스킬은 $이름 — Codex는 .claude/skills/ 를 읽지 않고 .agents/skills/ 를 읽는다(배치는 키트 다음 단계). 배너 "OpenAI Codex",
#           모델명 "gpt-5.1-codex" 꼴(상태줄 설정에 따라 안 보일 수 있다 → not recorded). --dangerously-bypass-approvals-and-sandbox(=--yolo)는
#           Claude의 --dangerously-skip-permissions에 해당한다 — 더 좁은 --sandbox workspace-write는 네트워크를 막아 push·gh pr create가 안 된다.
#           .claude/settings.json의 deny(gh pr merge)는 Codex에 안 걸린다 — 병합 금지는 프롬프트와 GitHub 브랜치 보호에 기댄다
#           사용량 한도·확인 창 문구는 실측 전이라 비워 둔다(감지하지 않음) — 실측 뒤 loop.env의 USAGE_LIMIT_REGEX·PERMISSION_PROMPT_REGEX로 켠다
#   그 외   프리셋 없음. AGENT_RESTART_CMD·AGENT_BANNER_REGEX(·AGENT_MODEL_FLAG·MODEL_REGEX·SKILL_PREFIX·USAGE_LIMIT_REGEX·PERMISSION_PROMPT_REGEX)를 loop.env에 적는다.
#           기동 명령·배너가 비어 있으면 시작 때 멈춘다
# 기본값("${VAR:-…}")에는 }를 쓰지 않는다 — bash는 기본값을 처음 나오는 }에서 끊고 나머지를 글자 그대로 뒤에 붙인다. 0.9.25의 한도 정규식
# "( [a-z0-9-]+){0,2} limit"이 "{0,2 limit}"으로 깨져 grep이 매번 rc=2(invalid repetition count)를 냈고 한도 감지가 꺼졌다(#57). 횟수 반복은
# ?를 겹쳐 풀어 쓴다
case "$AGENT" in
  claude)
    AGENT_RESTART_CMD="${AGENT_RESTART_CMD:-claude --dangerously-skip-permissions}"
    AGENT_MODEL_FLAG="${AGENT_MODEL_FLAG:---model}"
    AGENT_BANNER_REGEX="${AGENT_BANNER_REGEX:-Claude Code¦Opus¦Sonnet¦Fable¦Haiku}"
    MODEL_REGEX="${MODEL_REGEX:-(Opus¦Sonnet¦Haiku¦Fable¦Mythos)[[:space:]]+[0-9]+(\.[0-9]+)?}"
    SKILL_PREFIX="${SKILL_PREFIX:-/}"
    USAGE_LIMIT_REGEX="${USAGE_LIMIT_REGEX:-usage limit reached¦(weekly¦[0-9]+-hour) limit reached¦(hit¦reached) your( [a-z0-9-]+)?( [a-z0-9-]+)? limit}"
    PERMISSION_PROMPT_REGEX="${PERMISSION_PROMPT_REGEX:-Do you want to proceed\?}" ;;
  codex)
    AGENT_RESTART_CMD="${AGENT_RESTART_CMD:-codex --dangerously-bypass-approvals-and-sandbox}"
    AGENT_MODEL_FLAG="${AGENT_MODEL_FLAG:---model}"
    AGENT_BANNER_REGEX="${AGENT_BANNER_REGEX:-OpenAI Codex¦gpt-[0-9]}"
    MODEL_REGEX="${MODEL_REGEX:-gpt-[0-9]+(\.[0-9]+)?(-[[:alnum:]]+)*}"
    SKILL_PREFIX="${SKILL_PREFIX:-\$}"
    USAGE_LIMIT_REGEX="${USAGE_LIMIT_REGEX:-}"
    PERMISSION_PROMPT_REGEX="${PERMISSION_PROMPT_REGEX:-}" ;;
  *)
    AGENT_RESTART_CMD="${AGENT_RESTART_CMD:-}"
```

```
    AGENT_MODEL_FLAG="${AGENT_MODEL_FLAG:-}"
    AGENT_BANNER_REGEX="${AGENT_BANNER_REGEX:-}"
    MODEL_REGEX="${MODEL_REGEX:-}"
    SKILL_PREFIX="${SKILL_PREFIX:-/}"
    USAGE_LIMIT_REGEX="${USAGE_LIMIT_REGEX:-}"
    PERMISSION_PROMPT_REGEX="${PERMISSION_PROMPT_REGEX:-}" ;;
esac
# ----- 에이전트 프리셋 끝 -----
IDLE_WAIT_MS=600000       # idle 1회 대기 상한(Orca: terminal wait 한 번, cmux: 폴링 한 바퀴). PR·수리 대기는 PROGRESS_SEC(진행 로그)마다 돌아온다
IDLE_POLL_SEC=5           # cmux·tmux 백엔드가 에이전트 상태(lifecycle / 훅 파일)를 다시 읽는 간격
SEND_ACCEPT_SEC=15        # cmux·tmux 백엔드: 프롬프트를 보낸 뒤 에이전트가 새 턴을 시작하는지(running) 0.5초마다 보는 상한
TERMINAL_RETRY="${TERMINAL_RETRY:-5}"           # 터미널 화면을 읽지 못할 때(터미널·앱이 없음) 다시 해 보는 횟수. 연달아 다 실패하면 터미널 끊김으로 멈춘다
TERMINAL_RETRY_SEC="${TERMINAL_RETRY_SEC:-10}"   # 그 사이 쉬는 간격
TERMINAL_UNRESPONSIVE_SEC="${TERMINAL_UNRESPONSIVE_SEC:-600}"   # 앱에 닿지 못하지만 앱 프로세스는 있을 때(앱이 멈춤) 응답을 기다리는 상한(경과 시간).
                          # 넘기면 터미널 끊김으로 멈춘다(#61 — 2026-11 실측에서 cmux 앱이 1~2분 멈췄다). 위 셋은 loop.env·환경변수로 바꾼다
TERMINAL_UNRESPONSIVE_LOG_SEC=60   # 앱이 응답하지 않는 동안 경과를 한 줄씩 찍는 간격
IDLE_STRIKES=3            # PR 없이 연속 idle이면 멈춤으로 본다
IDLE_RECHECK_SEC=30
SESSION_LOG_FRESH_SEC=30  # idle로 보여도 Claude Code 세션 기록(*.jsonl)이 이 안에 갱신됐으면 아직 일하는 중 — strike를 세지 않는다(session_active_recently)
PROMPT_STALL_SEC=300      # 권한 확인 창 문구(PERMISSION_PROMPT_REGEX)가 든 화면이 이만큼 글자 그대로면 idle 신호가 없어도 확인 창에 멈춘 워커로 보고
                          # worker_no_pr로 멈춘다(screen_check). 화면이 바뀌면 다시 센다. 2026-10 실측(#3, tmux): idle 신호가 비어 27분을 조용히 기다렸다
STARTUP_GRACE_SEC=120     # 기동 직후에는 프롬프트 처리 전에도 idle로 잡힌다
AGENT_START_WAIT_SEC=90   # 에이전트 배너가 보이기까지 기다리는 상한(1회), 넘기면 한 번 다시 띄운다
CI_APPEAR_TIMEOUT_SEC=900
CI_NO_CHECKS=0            # 마지막 ci_passed 실패가 '체크가 등록되지 않음'(실패한 job 없음)이었는가 — repair_ci가 진단 프롬프트를 고른다
GH_TRIES=3                # GitHub 일시 장애(5xx·연결 오류)에 걸린 gh 읽기 호출(gh 함수)과 병합(merge_pr)을 다시 해 보는 횟수(첫 시도 포함).
GH_RETRY_SEC=15           # 그 사이 GH_RETRY_SEC×회차씩 쉰다(15초·30초)
PICK_RETRY_SEC=300        # 티켓 사이의 다음 대상 조회(pick_targets)가 gh 재시도까지 실패할 때 다시 해 보는 총 시간. 넘기면 멈춘다(could not list tickets)
PICK_RETRY_EVERY_SEC=60   # 그 사이 다시 해 보는 간격
PICK_RECHECK_SEC=30       # 티켓 사이의 다음 대상 조회가 성공하면서 후보를 하나도 돌려주지 않으면(0행) 이만큼 쉬고 한 번 더 조회한다 — GitHub가 성공한
                          # 빈 응답을 준 것을 "남은 티켓 없음"으로 읽지 않게(#22). 두 번째도 0행이면 그때 확정한다(--parent면 스펙의 열린 하위 이슈 수와 맞춰 본다)
PORT_WAIT_SEC=600         # POST_MERGE_PORT가 비기를 기다리는 상한. 기다리는 동안은 30초마다 다시 본다(멈춤이 아니다)
REOPEN_WATCH_SEC=2        # 수습 수정 PR을 병합한 뒤 티켓 이슈가 닫히는지 다시 보는 간격(5번 — triage_keep_issue_open)
PR_SETTLE_SEC=120         # PR을 찾은 뒤 워커의 이슈 정리(체크박스·IN_REVIEW_LABEL)를 기다리는 상한. 라벨이 붙거나 idle이면 바로 넘어간다(pr_settle)
REPAIR_SETTLE_SEC=300     # 수리 중 idle이 끝내 안 잡힐 때: 새 커밋이 push된 뒤 head가 이만큼 그대로이고 워크트리가 깨끗하면 수리가 끝난 것으로 본다

DRY_RUN=0
NO_MERGE=0
POST_MERGE=1
MAX=0
TICKETS=()
ORIG_ARGS=("$@")   # 멈춤 기록의 "다시 실행할 명령"
TICKET_ARGS=()     # ORIG_ARGS에서 번호 인자의 자리 — 다시 실행할 명령이 처리한 번호를 뺄 때(#46)

while [[ $# -gt 0 ]]; do
  case "$1" in
    --dry-run) DRY_RUN=1 ;;
    --no-merge) NO_MERGE=1 ;;
    --skip-post-merge) POST_MERGE=0 ;;
    --max) MAX="$2"; shift ;;
    --parent) PARENT="${2#\#}"; shift ;;
    --label) SCOPE_LABEL="$2"; shift ;;
    --model) [[ $# -ge 2 && -n "$2" && "$2" != -* ]] || { echo "--model needs a model name (e.g. --model opus)" >&2; exit 2; }
             AGENT_MODEL="$2"; shift ;;
    --help|-h) awk 'NR > 2 && /^#/ { on = 1; sub(/^# ?/, ""); print; next } on { exit }' "$0"; exit 0 ;;
    -*) echo "unknown option: $1" >&2; exit 2 ;;
    *) TICKETS+=("${1#\#}"); TICKET_ARGS+=($(( ${#ORIG_ARGS[@]} - $# ))) ;;
  esac
```

```
  shift
done
TICKETS_GIVEN=0; [[ ${#TICKETS[@]} -eq 0 ]] || TICKETS_GIVEN=1   # 번호 지정 실행 — 시작 검사가 닫힌 번호를 모두 빼도 자동 선택으로 바뀌지 않게
# ----- 워커 모델 -----
# 모델은 기동 명령의 인자로만 고른다 — 워커·다시 띄우기·무인 수습이 모두 agent_launch_cmd를 거치므로 한 곳에서 붙이면 된다.
# Orca는 에이전트를 `worktree create --agent`로 직접 띄우고 인자는 앱 설정(agentDefaultArgs)에서만 받는다. AGENT_RESTART_CMD는
# 배너가 안 보일 때만 쓰므로, 받아 두면 첫 기동과 다시 띄운 워커의 모델이 달라진다 — 시작 때 멈춘다(이슈 #50)
if [[ -n "$AGENT_MODEL" ]]; then
  [[ "$BACKEND" != orca ]] || { echo "--model / AGENT_MODEL is not supported on the Orca backend: Orca launches the agent itself and takes its arguments only from Orca settings (agent default args). Add '${AGENT_MODEL_FLAG:---model} ${AGENT_MODEL}' there, or use BACKEND=cmux|tmux." >&2; exit 2; }
  [[ -n "$AGENT_MODEL_FLAG" ]] || { echo "AGENT=${AGENT} has no model argument preset. Set AGENT_MODEL_FLAG in .ticket-loop/loop.env (e.g. --model)." >&2; exit 2; }
  AGENT_RESTART_CMD="$AGENT_RESTART_CMD $AGENT_MODEL_FLAG $(printf %q "$AGENT_MODEL")"
fi
# ----- 워커 모델 끝 -----
[[ -n "$POST_MERGE_CMD" ]] || POST_MERGE=0
[[ "$POST_MERGE_EVERY" =~ ^(0|[1-9][0-9]*)$ ]] || { echo "POST_MERGE_EVERY must be 0 or a positive integer (got '${POST_MERGE_EVERY}')" >&2; exit 2; }
# 병합 뒤 검증은 시작할 때마다 검증 워크트리를 기본 브랜치 끝으로 되돌리고 추적 안 된·무시된 파일을 지운다(pm_clean). 그 자리가 메인 체크아웃
# (또는 루프를 띄운 체크아웃)이면 사람의 작업 파일이 지워지므로 시작하지 않는다 — 실제 경로로 비교한다(상대 경로·심볼릭 링크로 줘도)
if [[ $POST_MERGE -eq 1 && -n "$POST_MERGE_WORKTREE" ]]; then
  pm_wt_real="$(cd "$POST_MERGE_WORKTREE" 2>/dev/null && pwd -P || true)"
  for pm_wt_main in "$ROOT" "$(git -C "$ROOT" worktree list --porcelain 2>/dev/null | sed -n '1s/^worktree //p')"; do
    [[ -z "$pm_wt_real" || -z "$pm_wt_main" || "$pm_wt_real" != "$(cd "$pm_wt_main" 2>/dev/null && pwd -P)" ]] \
      || { echo "POST_MERGE_WORKTREE must not be the main checkout or the checkout the loop runs from (got '${POST_MERGE_WORKTREE}') — the post-merge check resets its worktree and deletes the untracked and ignored files there" >&2; exit 2; }
  done
fi

# 터미널 색 — 단계가 한눈에 갈리도록. 터미널일 때만 칠하고, NO_COLOR(https://no-color.org)나 TERM=dumb이면 끈다.
# 아래 exec가 stderr를 tee로 바꾸기 전에 판정해야 한다. 파이프로 받는 쪽과 로그 파일은 맨글자만 본다.
# C_BANNER(시작 머리줄)는 256색 주황(208) — 16색에는 주황이 없고, 나머지 단계 색(보라·청록·초록·노랑·빨강) 어느 것과도 겹치지 않는다.
# 테마 팔레트가 아니라 고정 색이라 터미널 테마가 바뀌어도 같은 주황이다(macOS Terminal·iTerm2·Ghostty/cmux·tmux·Orca 모두 256색)
if [[ -t 2 && -z "${NO_COLOR:-}" && "${TERM:-}" != dumb ]]; then
  C_RESET=$'\e[0m'; C_DIM=$'\e[2m'; C_TITLE=$'\e[1;35m'; C_STEP=$'\e[36m'; C_OK=$'\e[32m'; C_WARN=$'\e[33m'; C_ERR=$'\e[1;31m'
  C_BANNER=$'\e[1;38;5;208m'
else
  C_RESET=""; C_DIM=""; C_TITLE=""; C_STEP=""; C_OK=""; C_WARN=""; C_ERR=""; C_BANNER=""
fi
# 로그 파일로 가는 줄에서 색 코드(SGR, ESC[…m)만 지운다. 그 밖의 이스케이프는 손대지 않는다 — bash 3.2(macOS)도 되는 문자열 처리
strip_ansi() {
  local l pre rest code
  while IFS= read -r l || [[ -n "$l" ]]; do
    pre=""; rest="$l"
    while [[ "$rest" == *$'\e['* ]]; do
      pre+="${rest%%$'\e['*}"; rest="${rest#*$'\e['}"; code="${rest%%m*}"
      if [[ "$rest" == *m* && "$code" != *[!0-9';']* ]]; then rest="${rest#*m}"; else pre+=$'\e['; fi
    done
    printf '%s\n' "$pre$rest"
  done
}

# --dry-run은 로그를 남기지 않는다(목록 확인만). 실행 로그는 다음 /retro가 읽는 1차 자료다.
# LOG_START_BYTES: 이 실행이 시작할 때의 로그 파일 크기. 로그는 날짜별 한 파일에 이어 쓰므로 멈춤 보고에 싣는 "로그 끝"을 이 실행의
# 줄로 한정한다(run_log) — 0.8.8 실측에서 같은 날 앞 실행의 ERROR 줄이 다음 실행의 Slack 멈춤 알림에 섞여 나왔다
LOG_START_BYTES=0
if [[ $DRY_RUN -eq 0 && "$LOG_FILE" != "/dev/null" ]]; then
  mkdir -p "$(dirname "$LOG_FILE")"
  [[ ! -f "$LOG_FILE" ]] || LOG_START_BYTES=$(( $(wc -c < "$LOG_FILE") ))
  if [[ -n "$C_RESET" ]]; then exec 2> >(tee >(strip_ansi >> "$LOG_FILE") >&2)
  else exec 2> >(tee -a "$LOG_FILE" >&2); fi
fi

# log [title|step|ok|warn|err] 메시지 — 첫 인자가 단계 이름이면 그 색으로(터미널), 없으면 맨글자. 시각은 흐리게.
```

```
#   title 티켓 시작(앞에 빈 줄) / step 단계 전환(워크트리·PR·CI) / ok 성공·요약 / warn 되풀이·수리·건너뜀 / err 중단
log() {
  local lvl="" c=""
  case "${1:-}" in title¦step¦ok¦warn¦err) [[ $# -lt 2 ]] ¦¦ { lvl="$1"; shift; } ;; esac   # 메시지가 따로 있을 때만 단계로 본다
  case "$lvl" in title) c="$C_TITLE"; printf '\n' >&2 ;; step) c="$C_STEP" ;; ok) c="$C_OK" ;; warn) c="$C_WARN" ;; err) c="$C_ERR" ;; esac
  printf '%s[%s]%s %s%s%s\n' "$C_DIM" "$(date '+%Y-%m-%d %H:%M:%S')" "$C_RESET" "$c" "$*" "${c:+$C_RESET}" >&2
}
# 설치된 키트 버전 — "0.9.6 (435e70e)". .ticket-loop/kit.json은 설치기가 쓴다. 없으면(키트 저장소에서 돌리거나 손으로 복사) unknown
kit_version() {
  local kitjson="${ROOT}/.ticket-loop/kit.json" v c
  v="$(jq -r '.version // empty' "$kitjson" 2>/dev/null ¦¦ true)"; c="$(jq -r '.commit // empty' "$kitjson" 2>/dev/null ¦¦ true)"
  printf '%s%s' "${v:-unknown}" "${c:+ (${c:0:7})}"
}
# 시작 머리줄 "━━ ticket-loop-kit 0.9.6 (435e70e) ━━…" — 실행 맨 앞에 한 줄(앞에 빈 줄). 로그는 날짜별 한 파일에 이어 쓰므로 실행의
# 경계가 되고, /retro·멈춤 진단에서 어느 키트 버전이 낸 줄인지 가른다. 폭은 BANNER_COLS로 고정 — 좁은 워커 창에서도 접히지 않게 짧게 둔다
BANNER_COLS=56
kit_banner() {
  local fill n=$(( BANNER_COLS - ${#1} - 4 ))
  [[ $n -ge 3 ]] ¦¦ n=3
  printf -v fill '%*s' "$n" ''
  printf '\n%s━━ %s %s%s\n' "$C_BANNER" "$1" "${fill// /━}" "${C_BANNER:+$C_RESET}" >&2
}
# 워커 화면·검증 로그 꼬리처럼 통째로 보여 주는 출력은 흐리게, 들여 써서 루프 로그와 가른다. $1은 줄 앞에 붙일 글자(선택)
dim_lines() { local l; while IFS= read -r l ¦¦ [[ -n "$l" ]]; do printf '%s%s%s\n' "$C_DIM" "${1:-  }" "$l" "$C_RESET" >&2; done; }
# ----- 알림 · 멈춤 보고 -----
# 알림은 데스크톱(자리에 있을 때)과 Slack(SLACK_WEBHOOK_URL — 자리를 비웠을 때) 두 곳으로 간다. 어느 쪽이 실패해도 루프 결과는 바뀌지 않는다.
# STOP_ISSUE: 멈추면 GitHub 기록을 남길 티켓. CUR_N(티켓 기록)은 병합 기록을 쓰면 비지만, 그 뒤 병합 뒤 검증이 실패해도 그 티켓의 멈춤이다
REPO_URL=""; STOP_ISSUE=""; STOP_REPORTED=0; LOCK_HELD=0; BG_PID=""
# STOP_KIND: 멈춤 기록의 종류(post_merge_failed·ci_failed·setup_failed·worker_no_pr·usage_limit·merge_failed·signal·unexpected_exit, 그 밖은 비워 other).
# STOP_RECORD: 이번 멈춤에 이미 쓴 멈춤 기록(무인 수습이 쓴 것) — stop_report가 같은 멈춤을 두 번 기록하지 않게
STOP_KIND=""; STOP_RECORD=""
fmt_epoch() { date -r "$1" '+%H:%M:%S' 2>/dev/null ¦¦ date -d "@$1" '+%H:%M:%S'; }   # macOS / Linux

# notify 제목 본문 [종류 [꼬리]] — 종류 start ¦ done ¦ flaky ¦ triage ¦ resumed ¦ stop ¦ interrupt ¦ finish(Slack 아이콘·멘션),
# 꼬리는 Slack에만 코드 블록으로 붙는다
notify() {
  if command -v osascript >/dev/null 2>&1; then
    # 문구를 AppleScript 소스에 끼우지 않고 인자로 넘긴다 — 티켓 제목·멈춤 사유의 따옴표가 스크립트를 깨지 않게
    osascript -e 'on run argv' -e 'display notification (item 2 of argv) with title (item 1 of argv)' -e 'end run' "$1" "$2" >/dev/null 2>&1 ¦¦ true
  elif command -v notify-send >/dev/null 2>&1; then
    notify-send "$1" "$2" >/dev/null 2>&1 ¦¦ true
  fi
  slack_post "$@"
}

# Slack Incoming Webhook. 본문의 #N은 이슈·PR 링크가 된다(GitHub의 /issues/N은 PR이면 /pull/N으로 넘긴다). &·<·>는 Slack 규칙대로 이스케이프한다.
# 멈춤(stop)에만 SLACK_MENTION을 앞에 붙인다. 잠금을 잡은 실행만 보낸다 — --dry-run과, 이미 도는 루프와 겹쳐 시작하자마자 멈춘 실행이
# "루프가 멈췄다"고 알려 도는 루프를 의심하게 하지 않도록
slack_post() {
  [[ -n "$SLACK_WEBHOOK_URL" && $LOCK_HELD -eq 1 ]] ¦¦ return 0
  local payload
  payload="$(jq -cn --arg title "$1" --arg body "${2:-}" --arg kind "${3:-}" --arg tail "${4:-}" --arg mention "$SLACK_MENTION" \
      --arg repo "${OWNER_REPO:-${ROOT##*/}}" --arg url "$REPO_URL" --arg host "$(hostname -s 2>/dev/null ¦¦ hostname)" '
    def esc: gsub("&"; "&amp;") ¦ gsub("<"; "&lt;") ¦ gsub(">"; "&gt;");
    def link: if $url == "" then . else gsub("#(?<n>[0-9]+)"; "<\($url)/issues/\(.n)¦#\(.n)>") end;
    ({start: ":arrow_forward:", done: ":white_check_mark:", flaky: ":warning:", triage: ":hammer_and_wrench:", resumed: ":recycle:",
      stop: ":octagonal_sign:", interrupt: ":octagonal_sign:"}[$kind] // ":checkered_flag:") as $icon
```

```
    ¦ {text: ("\($icon) *\($title ¦ esc)* · \($repo ¦ esc) · \($host ¦ esc)\n"
        + (if $kind == "stop" and $mention != "" then "\($mention) " else "" end)
        + ($body ¦ esc ¦ link)
        + (if $tail != "" then "\n```\n\($tail ¦ esc)\n```" else "" end))}')" ¦¦ return 0
  curl -fsS -m 15 -H 'Content-Type: application/json' --data "$payload" "$SLACK_WEBHOOK_URL" </dev/null >/dev/null 2>&1 \
    ¦¦ log warn "Slack notification failed — check SLACK_WEBHOOK_URL"
}

die() { log err "ERROR: $*"; stop_report "$*"; exit 1; }

# 이 실행이 로그 파일에 쓴 부분(LOG_START_BYTES 뒤). 같은 날 앞 실행의 줄은 빠진다
run_log() { tail -c "+$(( ${LOG_START_BYTES:-0} + 1 ))" "$LOG_FILE" 2>/dev/null ¦¦ true; }

# 멈춤 기록 — /loop-triage가 읽는 JSON 한 파일(ROOT/.ticket-loop/stops/<id>.json, 기계별 — 커밋하지 않는다). 무인 수습을 띄울 때 한 번,
# 수습 없이 멈출 때는 stop_report가 한 번 쓴다 — 사람이 나중에 /loop-triage로 이어받을 자리다. 경로를 stdout에 쓴다.
# 로그는 경로와 이 실행이 쓰기 시작한 바이트(loop_from_byte)만 적는다 — 내용은 수습 세션이 그 머신에서 직접 읽는다.
# $1 종류(STOP_KIND), $2 사유, $3 unattended(루프가 수습 세션을 띄운다) ¦ manual(아무도 수습하지 않았다)
write_stop_record() {
  local kind="$1" reason="$2" mode="$3" dir="${ROOT}/.ticket-loop/stops" id file i from
  local kitjson="${ROOT}/.ticket-loop/kit.json" rerun="bash scripts/ticket-loop.sh" wt="" log="" pm_window=null
  local args=(${ORIG_ARGS[@]+"${ORIG_ARGS[@]}"}) nums=(${TICKET_ARGS[@]+"${TICKET_ARGS[@]}"}) tickets=(${TICKETS[@]+"${TICKETS[@]}"})
  id="$(date '+%Y%m%d-%H%M%S')-${STOP_ISSUE:-0}"
  file="${dir}/${id}.json"
  # 다시 실행할 명령: 옵션은 처음 그대로. 번호 지정 실행이면 번호는 아직 끝내지 않은 것만 — 멈춘 티켓부터(#46). 이미 병합한 번호를 남기면 다시
  # 돌릴 때 닫힌 이슈부터 다시 보고, --no-merge로 PR만 연 번호는 "PR already open"에 멈춘다. 루프 끝(모든 번호 뒤)에서 멈췄으면 마지막 번호를,
  # 시작 검사가 번호를 모두 건너뛴 실행(모두 닫힘)이면 처음 번호를 그대로 둔다 — 번호가 하나도 없으면 자동 선택이 된다
  for (( i = 0; i < ${#args[@]}; i++ )); do
    [[ " ${nums[*]-} " != *" ${i} "* ]] ¦¦ continue
    rerun+=" $(printf '%q' "${args[$i]}")"
  done
  if [[ ${#tickets[@]} -gt 0 ]]; then
    from="${TICKET_IDX:-0}"; [[ $from -lt ${#tickets[@]} ]] ¦¦ from=$(( ${#tickets[@]} - 1 ))
    for (( i = from; i < ${#tickets[@]}; i++ )); do rerun+=" $(printf '%q' "${tickets[$i]}")"; done
  else
    for i in ${nums[@]+"${nums[@]}"}; do rerun+=" $(printf '%q' "${args[$i]}")"; done
  fi
  [[ -z "${WT_PATH:-}" ¦¦ ! -d "${WT_PATH:-}" ]] ¦¦ wt="$WT_PATH"
  [[ "${LOG_FILE:-/dev/null}" == /dev/null ]] ¦¦ log="$LOG_FILE"
  ! declare -F pm_window_json >/dev/null ¦¦ pm_window="$(pm_window_json 2>/dev/null ¦¦ echo null)"
  mkdir -p "$dir" ¦¦ return 1
  jq -n --arg id "$id" --arg kind "${kind:-other}" --arg mode "$mode" --arg reason "$reason" --arg created "$(date -u '+%Y-%m-%dT%H:%M:%SZ')" \
    --arg repo "${OWNER_REPO:-}" --arg base "${BASE_BRANCH:-}" --arg issue "${STOP_ISSUE:-}" --arg title "${CUR_TITLE:-}" \
    --arg spec "${PARENT:-}" --arg pr "${CUR_PR:-}" --arg wt "$wt" --arg main "${MAIN_WT:-$ROOT}" \
    --arg log "$log" --argjson from "${LOG_START_BYTES:-0}" --arg pm_log "${POST_MERGE_LOG:-}" --arg pm_retry_log "${POST_MERGE_RETRY_LOG:-}" \
    --arg pm_wt "${POST_MERGE_WT:-}" --arg pm_cmd "${POST_MERGE_CMD:-}" --arg pm_retry_cmd "${POST_MERGE_RETRY_CMD:-}" \
    --arg setup_cmd "${WORKTREE_SETUP_CMD:-}" --arg rerun "$rerun" --arg verdict "${dir}/${id}.verdict.json" \
    --arg agent "${AGENT:-}" --arg backend "${BACKEND:-}" --argjson pm_window "${pm_window:-null}" \
    --arg kit_version "$(jq -r '.version // empty' "$kitjson" 2>/dev/null ¦¦ true)" --arg kit_repo "$(jq -r '.kit // empty' "$kitjson" 2>/dev/null ¦¦ true)" '
    def num: if . == "" then null else tonumber end;
    def str: if . == "" then null else . end;
    {id: $id, kind: $kind, mode: $mode, created_at: $created, reason: $reason,
     repo: ($repo ¦ str), base_branch: ($base ¦ str), issue: ($issue ¦ num), title: ($title ¦ str), spec: ($spec ¦ num), pr: ($pr ¦ num),
     worker_worktree: ($wt ¦ str), main_checkout: $main,
     logs: {loop: ($log ¦ str), loop_from_byte: $from, post_merge: ($pm_log ¦ str), post_merge_retry: ($pm_retry_log ¦ str)},
     post_merge_worktree: ($pm_wt ¦ str), post_merge_window: $pm_window,
     commands: {post_merge: ($pm_cmd ¦ str), post_merge_retry: ($pm_retry_cmd ¦ str), worktree_setup: ($setup_cmd ¦ str)},
     rerun: $rerun, verdict_file: $verdict, agent: ($agent ¦ str), backend: ($backend ¦ str),
     kit: {version: ($kit_version ¦ str), repo: ($kit_repo ¦ str)}}' > "$file" ¦¦ return 1
  printf '%s' "$file"
}

# 멈춤 보고는 한 번만 — die, 신호, 예기치 않은 종료가 모두 여기로 온다. 진행 중 티켓을 stopped로 기록하고(스펙 리포트), 멈춤 기록을
# 남기고(무인 수습이 이미 남겼으면 그것), 그 이슈에 GitHub 기록을 남기고, 알린다. Slack에는 로그 끝 12줄을 붙인다 — 워커 화면
```

```
꼬리가
# 들어 있어 "왜 멈췄나"가 폰에서 보인다. $1 사유, $2 stop ¦ interrupt(Ctrl-C — 사람이 앞에 있으니 멘션하지 않는다)
stop_report() {
  [[ $STOP_REPORTED -eq 0 ]] ¦¦ return 0
  STOP_REPORTED=1
  local reason="$1" kind="${2:-stop}" tail=""
  [[ -z "${CUR_N:-}" ]] ¦¦ record_ticket "stopped" "$reason"
  if [[ $LOCK_HELD -eq 1 && -z "$STOP_RECORD" ]]; then
    STOP_RECORD="$(write_stop_record "$STOP_KIND" "$reason" manual 2>/dev/null)" ¦¦ STOP_RECORD=""
  fi
  if [[ $LOCK_HELD -eq 1 && "$LOG_FILE" != "/dev/null" && -f "$LOG_FILE" ]]; then
    sleep 1   # 로그 파일은 tee가 비동기로 쓴다 — 방금 찍은 ERROR 줄·워커 화면이 파일에 들어오게
    tail="$(run_log ¦ tail -n 12)"
  fi
  [[ -z "$STOP_ISSUE" ]] ¦¦ github_stop_record "$STOP_ISSUE" "$reason"
  local title="Ticket loop stopped"
  [[ "$STOP_KIND" != usage_limit ]] ¦¦ title="Ticket loop stopped — worker hit the usage limit"   # 수습 없이 멈춘 한도 — 제목
에서 바로 보이게
  notify "$title" "$reason" "$kind" "$tail"
}

# 멈춘 티켓의 GitHub 기록 — 이슈에 코멘트(사유·머신·PR·워크트리·루프 로그 끝)를 달고 STOPPED_LABEL을 붙인다. 알림이 아니라 기록
이다:
# 루프는 사용자 계정(gh auth)으로 쓰므로 GitHub는 본인에게 알리지 않는다(알림은 Slack). 대상 선택은 이 라벨을 보지 않고, 루프가
그 티켓을
# 병합하면 뗀다. 코멘트에 싣는 로그는 루프 자신의 줄([시각]으로 시작)뿐이다 — 들여 쓴 워커 화면·검증 로그는 공개 저장소에 올리
지 않는다
github_stop_record() {
  local n="$1" reason="$2" lines="" now body tilde='~'
  now="$(date +%s)"
  [[ "$LOG_FILE" == "/dev/null" ¦¦ ! -f "$LOG_FILE" ]] ¦¦ lines="$(run_log ¦ grep -E '^\[[0-9]{4}-[0-9]{2}-[0-9]{2} ' ¦ tail -n
30 ¦¦ true)"
  body="**ticket-loop 중단** — ${reason}"$'\n\n'
  body+="- 머신 \`$(hostname -s 2>/dev/null ¦¦ hostname)\` · 백엔드 ${BACKEND_NAME:-$BACKEND} · 에이전트 ${AGENT}${CUR_MODEL:+
(${CUR_MODEL})}"$'\n'
  [[ -z "${CUR_STARTED:-}" ]] ¦¦ body+="- 티켓 시작 $(fmt_epoch "$CUR_STARTED") → 중단 $(fmt_epoch "$now") ($(( (now - CUR_STAR
TED) / 60 ))분)"$'\n'
  [[ -z "${CUR_PR:-}" ]] ¦¦ body+="- PR #${CUR_PR}"$'\n'
  [[ -z "${WT_PATH:-}" ¦¦ ! -d "${WT_PATH:-}" ]] ¦¦ body+="- 워크트리 \`${WT_PATH/#"$HOME"/$tilde}\` — 조사용으로 남겨 둠"$'\n'
  [[ "$LOG_FILE" == "/dev/null" ]] ¦¦ body+="- 루프 로그 \`${LOG_FILE#"$ROOT"/}\` (그 머신에만 있다)"$'\n'
  [[ -z "${PM_PENDING:-}" ]] ¦¦ ! declare -F pm_list >/dev/null \
    ¦¦ body+="- 아직 병합 뒤 검증을 통과하지 않은 병합: $(pm_list) — 다음 실행의 검증이 함께 덮는다"$'\n'
  [[ -z "$STOP_RECORD" ]] ¦¦ body+="- 멈춤 기록 \`${STOP_RECORD#"$ROOT"/}\` (그 머신에만 있다) — 이어받으려면 그 저장소의 에이
전트 세션에서 \`${SKILL_PREFIX:-/}loop-triage $(basename "$STOP_RECORD" .json)\`"$'\n'
  [[ -z "$lines" ]] ¦¦ body+=$'\n'"<details><summary>루프 로그 마지막 $(grep -c . <<<"$lines")줄</summary>"$'\n\n```text\n'"${l
ines}"$'\n```\n\n</details>\n'
  gh label create "$STOPPED_LABEL" --color B60205 --description "ticket-loop가 이 티켓에서 멈춤(기록용)" </dev/null >/dev/null
2>&1 ¦¦ true
  gh issue comment "$n" --body "$body" </dev/null >/dev/null 2>&1 ¦¦ log warn "#${n}: failed to post the stop record to GitHub"
  gh issue edit "$n" --add-label "$STOPPED_LABEL" </dev/null >/dev/null 2>&1 ¦¦ log warn "#${n}: failed to add the ${STOPPED_LA
BEL} label"
}

# 병합 뒤 검증이 재실행에서 통과했을 때의 기록. 루프는 계속 가지만 같은 플레이크가 되풀이되는지 나중에 볼 수 있게 이슈에 코멘트
를 달고
# 알린다(멘션 없이 — 사람을 부를 일은 아니다). 멈춤이 아니라 STOPPED_LABEL은 붙이지 않는다. github_stop_record처럼 검증 로그 본
문은
# 공개 저장소에 올리지 않고 파일 이름만 적는다. $1 이슈, $2 첫 실행 로그, $3 재실행 로그, $4 첫 실행 초, $5 재실행 초
post_merge_flaky_record() {
  local n="$1" first_log="$2" retry_log="$3" first_sec="$4" retry_sec="$5" body
  body="**ticket-loop 병합 뒤 검증 재시도 통과** — 플레이크 의심"$'\n\n'
  body+="- 첫 실행 ${first_sec}초 만에 실패 → 재실행(\`POST_MERGE_RETRY_CMD\`) ${retry_sec}초 만에 통과. 루프는 멈추지 않고 계
속 갔다"$'\n'
  [[ -z "${CUR_PR:-}" ]] ¦¦ body+="- PR #${CUR_PR} 병합 뒤"$'\n'
  body+="- 머신 \`$(hostname -s 2>/dev/null ¦¦ hostname)\` · 검증 로그 \`${first_log##*/}\`(첫 실행), \`${retry_log##*/}\`(재실
행) — 그 머신에만 있다"$'\n'
  gh issue comment "$n" --body "$body" </dev/null >/dev/null 2>&1 ¦¦ log warn "#${n}: failed to post the post-merge retry recor
d to GitHub"
  notify "Post-merge check passed on retry" "#${n}: first run failed in ${first_sec}s, retry passed in ${retry_sec}s — likely f
laky (logs ${first_log##*/}, ${retry_log##*/})" flaky
}
```

```
# 무인 수습이 고쳐 이어 갈 때의 기록 — 그 이슈에 코멘트(멈춘 이유·분류·요약·조치)를 달고 멘션 없이 알린다. 멈춤이 아니라 STOPPE
D_LABEL은
# 붙이지 않는다. 멈춘 이유는 루프 자신의 문구라 로컬 경로가 섞일 수 있어 홈을 ~로 줄인다.
# $1 이슈(0이면 코멘트 없음) $2 멈춘 이유 $3 분류 $4 요약 $5 조치(영어 — 알림) $6 조치(한국어 — 코멘트)
triage_resume_record() {
  local n="$1" body tilde='~' prefix=""
  if [[ "$n" != 0 ]]; then
    prefix="#${n}: "
    body="**ticket-loop 무인 수습 — 이어 감**"$'\n\n'
    body+="- 멈춘 이유: ${2//"$HOME"/$tilde}"$'\n'
    body+="- 분류 \`$3\` — $4"$'\n'
    body+="- 조치: $6. 루프는 멈춘 자리에서 이어 갔다"$'\n'
    body+="- 머신 \`$(hostname -s 2>/dev/null || hostname)\`${STOP_RECORD:+ · 멈춤 기록 \`${STOP_RECORD#"$ROOT"/}\` (그 머신에
만 있다)}"$'\n'
    gh issue comment "$n" --body "$body" </dev/null >/dev/null 2>&1 || log warn "#${n}: failed to post the triage record to Git
Hub"
  fi
  notify "Triage fixed it — loop continues" "${prefix}$4 ($5)" resumed
}

# 모든 종료가 지나는 길(EXIT). die와 정상 종료는 따로 처리했고, 그 밖의 비정상 종료 — set -e에 걸린 명령(네트워크가 끊겨 git fe
tch·gh가
# 실패하는 등) — 는 걸린 명령을 사유로 보고한다(trap 안의 BASH_COMMAND는 trap이 걸린 때의 명령이다). 잠금은 이 실행이 잡았을 때
만 푼다
on_exit() {
  local rc=$? cmd="${BASH_COMMAND//$'\n'/ }"
  set +e
  if [[ $rc -ne 0 && $STOP_REPORTED -eq 0 ]]; then
    trap '' PIPE
    [[ ${#cmd} -le 200 ]] || cmd="${cmd:0:200}…"
    log err "ERROR: unexpected exit (status ${rc}) at: ${cmd}"
    STOP_KIND=unexpected_exit
    stop_report "${STOP_ISSUE:+#${STOP_ISSUE}: }unexpected exit (status ${rc}) at: ${cmd}"
  fi
  [[ $LOCK_HELD -eq 0 ]] || rmdir "$LOCK_DIR" 2>/dev/null
  [[ -z "${WAIT_STOP_FILE:-}" ]] || rm -f "$WAIT_STOP_FILE"
  [[ -z "${REACH_STATE_DIR:-}" ]] || rm -rf "$REACH_STATE_DIR"
}

# 신호로 끝날 때 — 터미널 창을 닫음(HUP), Ctrl-C(INT), kill(TERM), 로그를 받던 tee가 죽은 뒤 stderr 쓰기(PIPE). bash는 신호에도
EXIT trap을
# 돌리지만 그 안의 $?가 0이라 정상 종료와 구별되지 않아 따로 잡는다. PIPE를 잡지 않으면 tee가 먼저 죽은 상태에서 log 한 줄에 루
프가
# 141로 죽고 EXIT trap도 돌지 않아 보고 없이 잠금까지 남는다(실험으로 확인). 보고하는 동안은 PIPE를 무시한다 — 이 줄은 로그 파
일에
# 못 남을 수 있지만 티켓 기록·GitHub·Slack에는 남는다. trap은 자식 프로세스에 이어지지 않아 파이프의 SIGPIPE 동작은 그대로다.
# kill -9·전원·잠자기는 잡을 수 없다
on_signal() {
  local sig="$1" why kind=stop
  trap '' PIPE; trap - HUP INT TERM
  set +e
  [[ -z "$BG_PID" ]] || kill_tree "$BG_PID"   # bg_wait로 기다리던 것(PR·CI 대기, 검증 명령)이 루프 없이 남지 않게
  case "$sig" in
    HUP)  why="terminal closed (SIGHUP)" ;;
    INT)  why="interrupted (Ctrl-C)"; kind=interrupt ;;
    PIPE) why="log output closed (SIGPIPE — terminal gone?)" ;;
    *)    why="terminated (SIG${sig})" ;;
  esac
  log err "ERROR: ${why}"
  STOP_KIND=signal
  stop_report "${STOP_ISSUE:+#${STOP_ISSUE}: }${why}" "$kind"
  case "$sig" in HUP) exit 129 ;; INT) exit 130 ;; PIPE) exit 141 ;; *) exit 143 ;; esac
}

# 오래 기다리는 일(PR 대기, CI 대기, 병합 뒤 검증·워크트리 준비 명령, 수리 중 idle 대기)은 백그라운드로 돌리고 wait로 기다린다.
# bash는 포그라운드 자식이 끝날 때까지 trap을 미룬다 — 0.8.8 실측(더미 저장소, cmux)에서 루프 pid에만 보낸 SIGHUP이 PR 대기
# 서브셸이 끝난 2분 뒤에야 처리됐다. wait는 신호를 받으면 바로 돌아와 on_signal이 돈다(터미널 창 닫힘·Ctrl-C는 프로세스 그룹
# 전체에 가서 원래도 됐지만, 다른 터미널의 `kill <루프 pid>`는 PR이 열릴 때까지 먹히지 않았다). 자식은 set +e로 돈다 —
# 전에 커맨드 치환으로 돌던 wait_for_pr와 같다(bash는 커맨드 치환에서 -e를 끈다). 종료 코드는 그대로 돌려준다
bg_wait() {
  local rc=0
  { set +e; "$@"; } &
```

```
  BG_PID=$!
  wait "$BG_PID" || rc=$?
  BG_PID=""
  return $rc
}
# 프로세스와 그 자손을 끝낸다(TERM). 자식 목록을 먼저 읽고 부모를 끝내 새 자식이 생기지 않게 한 뒤 자식으로 내려간다
kill_tree() {
  local kids k
  kids="$(pgrep -P "$1" 2>/dev/null || true)"
  kill "$1" 2>/dev/null || true
  for k in $kids; do kill_tree "$k"; done
}
# ----- 알림 · 멈춤 보고 끝 -----

for tool in gh jq git; do
  command -v "$tool" >/dev/null 2>&1 || die "${tool} is not installed."
done

# ---------- GitHub 일시 장애 ----------
# GitHub는 공지 없이 짧게 5xx·연결 오류를 낸다(이슈 #7: CI 통과 직후 병합 한 번이 502). 루프의 gh 읽기가 그 한 번을 그대로 받으면 멈추거나
# (set -e — 병합 뒤 브랜치 조회, 수리 전 head) 틀린 값으로 읽는다(빈 대상 목록 → "No more tickets"로 끝남). 호출마다 막는 대신 gh를 함수로
# 감싸 읽기 호출만 GH_TRIES번까지 다시 해 본다. 쓰기(병합·코멘트·라벨·이슈 닫기)는 다시 하지 않는다 — 응답만 실패하고 서버는 처리했을 수
# 있어 두 번 들어간다(병합은 merge_pr가 상태를 확인하며 따로 다시 한다). 오래 도는 --watch도 감싸지 않는다(ci_passed가 결과를 다시 읽는다).
# 도구 검사(command -v) 뒤에 둔다 — 함수가 먼저 있으면 gh가 없어도 검사를 통과한다
GH_TRANSIENT_RE='status code: 5[0-9]{2}|HTTP 5[0-9]{2}|bad gateway|service unavailable|gateway time-?out|timeout|timed out|something went wrong while executing your query|error connecting to|dial tcp|connection (reset|refused)|tls handshake|unexpected eof|no such host'
# 다시 해도 되는 gh 호출인가 — 조회 하위 명령, 메서드·필드 없는 REST api(필드가 있으면 POST다), mutation이 아닌 GraphQL
gh_read_only() {
  local a
  case "$1 ${2:-}" in
    "pr view"|"pr list"|"pr diff"|"issue view"|"issue list"|"repo view") ;;
    "pr checks") [[ " $* " != *" --watch "* ]] || return 1 ;;
    "api graphql") [[ "$*" != *mutation* ]] || return 1 ;;
    "api "*) for a in "$@"; do case "$a" in -X|--method|--method=*|-f|-F|--field|--field=*|--raw-field|--raw-field=*|--input|--input=*) return 1 ;; esac; done ;;
    *) return 1 ;;
  esac
}
gh() {
  local out rc try errf pause
  gh_read_only "$@" || { command gh "$@"; return; }
  errf="$(mktemp "${TMPDIR:-/tmp}/ticket-loop-gh.XXXXXX")"
  for (( try = 1; ; try++ )); do
    rc=0; out="$(command gh "$@" 2>"$errf")" || rc=$?
    if [[ $rc -eq 0 || $try -ge $GH_TRIES ]] || ! grep -Eqi "$GH_TRANSIENT_RE" "$errf"; then
      [[ -z "$out" ]] || printf '%s\n' "$out"
      cat "$errf" >&2; rm -f "$errf"
      return "$rc"
    fi
    pause=$(( try * GH_RETRY_SEC ))
    log warn "gh $1 ${2:-}: transient GitHub error (${try}/${GH_TRIES}) — retrying in ${pause}s ($(grep -m1 . "$errf"))"
    sleep "$pause"
  done
}
[[ -n "$AGENT_RESTART_CMD" && -n "$AGENT_BANNER_REGEX" ]] \
  || die "AGENT=${AGENT} has no preset (claude | codex). Set AGENT_RESTART_CMD and AGENT_BANNER_REGEX in .ticket-loop/loop.env."

ROOT="$(git rev-parse --show-toplevel)"
MAIN_WT="$(git worktree list --porcelain | sed -n '1s/^worktree //p')"
# 루프는 저장소당 하나만 — 두 루프가 같은 티켓을 집지 않게 한다
LOCK_DIR="$(cd "$ROOT" && git rev-parse --path-format=absolute --git-common-dir)/ticket-loop.lock"
# 아직 병합 뒤 검증을 거치지 않은 병합(POST_MERGE_EVERY) — 잠금처럼 .git 안이라 커밋되지 않고, 루프가 멈췄다 다시 실행돼도 이어진다
PM_STATE_FILE="${LOCK_DIR%/*}/ticket-loop-post-merge.json"
if [[ $DRY_RUN -eq 0 ]]; then
  # 멈춤은 어떻게 끝나든 한 번 보고한다(on_exit·on_signal). 잠금을 못 잡고 멈추면 on_exit는 잠금을 지우지 않는다(다른 루프의
```

```
것)
  trap on_exit EXIT; trap 'on_signal HUP' HUP; trap 'on_signal INT' INT; trap 'on_signal TERM' TERM; trap 'on_signal PIPE' PIPE
  mkdir "$LOCK_DIR" 2>/dev/null ¦¦ die "Another loop is running (${LOCK_DIR}). If no loop is running (pgrep -fl ticket-loop), r
emove that directory."
  LOCK_HELD=1
  REACH_STATE_DIR="$(mktemp -d "${TMPDIR:-/tmp}/ticket-loop-terminal.XXXXXX")"   # 백엔드 대기가 서브셸을 넘어 남기는 터미널 상
태(reach_*) — on_exit가 지운다
fi

OWNER_REPO="$(gh repo view --json nameWithOwner -q .nameWithOwner)"
REPO_URL="$(gh repo view --json url -q .url 2>/dev/null ¦¦ true)"   # Slack 본문의 #N 링크(GitHub Enterprise도 맞게)
[[ -n "$BASE_BRANCH" ]] ¦¦ BASE_BRANCH="$(gh repo view --json defaultBranchRef -q .defaultBranchRef.name)"

# ---------- GitHub 조회 ----------

# 조회 넷(blocked_by·sub_issue_count·open_pr_for·candidates)은 gh가 실패하면(GH_TRIES번 재시도 뒤에도) 출력 없이 비0으로 끝난다
— 실패를
# "선행 이슈 0개"·"PR 없음"·"빈 목록"으로 읽어 막힌 티켓을 시작하거나 "No more tickets"로 끝나지 않게(이슈 #8). 출력을 변수에
먼저 받는다 —
# 파이프 뒤의 awk·jq가 종료 코드를 덮지 않게
# 열린 선행 이슈 수. issue_dependencies_summary.blocked_by는 선행 이슈가 닫힌 뒤에도
# 최대 ~20분 옛 값을 돌려준 실측이 있어, 목록에서 열린 것만 센다.
# 예외 하나: 일시 장애가 아닌 404(Not Found)는 이슈 의존성 API가 없는 환경(옛 GHES)이다 — 조회 실패가 아니라 "막힘 없음"(0)으로
보고 실행당
# 한 번만 경고한다. 그 밖(재시도 뒤의 5xx·연결 오류·401·403)은 비0. 경고 여부는 변수가 아니라 표시 파일이다 — blocked_by는 명령
치환(서브셸)
# 안에서 불리므로 변수는 후보 사이에 남지 않는다. 파일이 비어 있으면 아직 경고하지 않은 것
BLOCKED_BY_UNAVAILABLE_WARNED="$(mktemp "${TMPDIR:-/tmp}/ticket-loop-blocked-by-404.XXXXXX")"
blocked_by() {
  local counts errf rc=0
  errf="$(mktemp "${TMPDIR:-/tmp}/ticket-loop-blocked.XXXXXX")"
  counts="$(gh api "repos/${OWNER_REPO}/issues/$1/dependencies/blocked_by" --paginate \
    --jq '[.[] ¦ select(.state == "open")] ¦ length' 2>"$errf")" ¦¦ rc=$?
  if [[ $rc -ne 0 ]] && grep -Eqi 'HTTP 404¦Not Found' "$errf"; then
    rm -f "$errf"
    [[ -s "$BLOCKED_BY_UNAVAILABLE_WARNED" ]] ¦¦ { log warn "dependencies API not available (HTTP 404) — treating tickets as no
t blocked"; echo 1 > "$BLOCKED_BY_UNAVAILABLE_WARNED"; }
    echo 0; return 0
  fi
  cat "$errf" >&2; rm -f "$errf"   # gh 래퍼의 재시도 경고·마지막 오류는 그대로 보인다
  [[ $rc -eq 0 ]] ¦¦ return 1
  awk '{s += $1} END {print s + 0}' <<<"$counts"
}

# 하위 이슈 수 — 스펙 이슈(/to-spec이 ready-for-agent를 붙인다)를 티켓처럼 집지 않게 한다
sub_issue_count() {
  local total
  total="$(gh api "repos/${OWNER_REPO}/issues/$1" --jq '.sub_issues_summary.total // 0')" ¦¦ return 1
  printf '%s\n' "$total"
}

# 스펙 $1의 하위 이슈 — 모든 상태, 페이지를 합친 JSON 배열 [{number, state, labels: [이름]}]. 조회 실패면 출력 없이 비0
spec_sub_issues() {
  local pages
  pages="$(gh api "repos/${OWNER_REPO}/issues/$1/sub_issues" --paginate --jq '[.[] ¦ {number, state, labels: [.labels[].nam
e]}]')" ¦¦ return 1
  jq -sc 'add // []' <<<"$pages"
}

# 티켓 사이의 대상 조회가 재확인까지 빈 목록(후보 0행)일 때 스펙 $1과 맞춰 본다(pick_next_targets, #22). 빈 목록이 맞으면 출력
없이 0, 믿을 수
# 없으면 그 이유 한 줄을 내고 0, 조회 실패면 출력 없이 비0. 스펙 요약(sub_issues_summary — 목록과 다른 끝점 issues/N)에 열린 하
위 이슈가 남아
# 있을 때만 의심하고, 그때 (a) sub_issues 끝점이 요약의 total보다 적게 돌려줬거나(빈·부분 응답), (b) 돌려준 것 가운데 열려 있고
# READY_LABEL(+SCOPE_LABEL)이 붙고 IN_REVIEW_LABEL이 없는 것이 있으면(그것은 issue list에 있었어야 한다 — 막혔거나 담당자·PR이
있어도 skip 줄로
# 남지 0행이 되지 않는다) 믿지 않는다. 열린 하위 이슈가 전부 ready 라벨이 없거나(needs-info·ready-for-human …) in-review면 빈
목록이
# 맞다 — 요약의 열린 수만 보면 그런 스펙은 끝에 늘 멈춤으로 끝나고 루프 끝 검증도 건너뛴다
spec_empty_list_check() {
  local summary kids total completed
  summary="$(gh api "repos/${OWNER_REPO}/issues/$1" --jq '"\(.sub_issues_summary.total // 0) \(.sub_issues_summary.completed //
```

```
0)"')" ¦¦ return 1
  read -r total completed <<<"$summary"
  [[ "${total:-}" =~ ^[0-9]+$ && "${completed:-}" =~ ^[0-9]+$ ]] ¦¦ return 1
  [[ $(( total - completed )) -gt 0 ]] ¦¦ return 0
  kids="$(spec_sub_issues "$1")" ¦¦ return 1
  jq -r --argjson total "$total" --argjson open "$(( total - completed ))" --arg spec "$1" --arg ready "$READY_LABEL" --arg sco
pe "$SCOPE_LABEL" --arg rv "$IN_REVIEW_LABEL" '
      if length < $total then "the sub_issues lookup returned \(length) of \($total) sub-issues of spec #\($spec) (\($open) ope
n)"
      else (map(select(.state == "open" and (.labels ¦ index($ready)) != null and ($scope == "" or (.labels ¦ index($scope)) !=
null)
                       and (.labels ¦ index($rv)) == null)) ¦ length) as $n
        ¦ if $n > 0 then "spec #\($spec) has \($n) open \($ready) sub-issue\(if $n == 1 then "" else "s" end) the lookup did no
t return" else empty end
      end' <<<"$kids"
}

# 티켓 N의 열린 PR — head 브랜치가 `<BRANCH_PREFIX>N`과 같거나 `<BRANCH_PREFIX>N-`로 시작하는 것.
# 워커는 implement 스킬대로 issue-N-<slug>를 만들지만, 루프가 만든 워크트리 브랜치 issue-N에 그대로 커밋해 PR을 연 실측이 있다
# (rental-portal #102 → PR #120, 0.8.5). 접두 `N-`만 보면 그 PR을 못 찾아 "PR 없이 멈춤"으로 죽었다.
# `N`만 접두로 보면 issue-1020이 #102에 걸리므로 정확히 같거나 `N-`로 시작할 때만이다
open_pr_for() {
  local prs
  prs="$(gh pr list --state open --json number,headRefName)" ¦¦ return 1
  jq -r --arg b "${BRANCH_PREFIX}$1" 'map(select(.headRefName == $b or (.headRefName ¦ startswith($b + "-")))) ¦ .[0].number //
empty' <<<"$prs"
}

issue_title() { gh issue view "$1" --json title -q .title; }

# 번호 지정 티켓의 조회 하나 — "상태<TAB>하위 이슈 수<TAB>닫힌 하위 이슈 수<TAB>제목"(상태는 open ¦ closed). 조회 실패면 출력
없이 비0
issue_facts() {
  gh api "repos/${OWNER_REPO}/issues/$1" --jq '[.state, (.sub_issues_summary.total // 0), (.sub_issues_summary.completed // 0),
.title] ¦ @tsv'
}

# 번호 지정 티켓 $1을 한 번 보고(issue_facts) GIVEN_STATE(open ¦ closed ¦ spec, 조회 실패면 빈 값)·GIVEN_TITLE·GIVEN_WHY(spec일
때 멈춤 사유)를
# 채운다 — 커맨드 치환 없이 부른다. 자동 선택은 닫힌 이슈를 목록에서 거르고 하위 이슈가 달린 이슈를 "spec issue with sub-issue
s"로 건너뛰지만,
# 번호 지정은 그 길을 타지 않아 닫힌 이슈(#46 — 멈춤 기록의 rerun을 그대로 돌림)와 스펙 이슈(#53)에 워커를 띄웠다. 스펙 이슈는
/to-tickets로
# 쪼개 --parent로 돌린다 — 워커가 PR 하나로 구현할 일이 없다. 하위 이슈가 모두 닫힌 스펙도 할 구현이 없다(스펙을 닫을 일)
given_ticket() {
  local facts state total completed
  GIVEN_STATE=""; GIVEN_TITLE=""; GIVEN_WHY=""
  facts="$(issue_facts "$1")" ¦¦ return 0
  IFS=$'\t' read -r state total completed GIVEN_TITLE <<<"$facts"
  if [[ "$state" != open ]]; then GIVEN_STATE=closed
  elif [[ "${total:-0}" == 0 ]]; then GIVEN_STATE=open
  else
    GIVEN_STATE=spec
    if [[ "${completed:-0}" -ge "$total" ]]; then
      GIVEN_WHY="#$1 (${GIVEN_TITLE}) is a spec issue whose sub-issues are all closed — nothing to implement; close the spec in
stead"
    else
      GIVEN_WHY="#$1 (${GIVEN_TITLE}) is a spec issue with $(( total - completed )) open sub-issue$([[ $(( total - completed ))
-eq 1 ]] ¦¦ echo s) — run them with: bash scripts/ticket-loop.sh --parent $1"
    fi
  fi
}

# 번호 지정 실행의 시작 검사(#46·#53) — 사람이 앞에 있을 때 번호 전부를 한 번에 본다(--dry-run 포함). 닫힌 이슈는 skip 줄을 남
기고 목록에서
# 뺀다 — 처리 건수(--max)·마지막 티켓 판정·다시 실행할 명령이 남은 번호로 센다. 스펙 이슈가 있으면 모두 적어 워크트리를 만들기
전에 멈춘다.
# 조회가 실패하면 멈춘다(통과시키지 않는다). pick_targets가 낼 "번호<TAB>제목" 줄들을 GIVEN_TARGETS에 둔다(제목을 다시 조회하지
않게)
GIVEN_TARGETS=""
given_tickets_check() {
  local n whys="" open=()
```

```
  GIVEN_TARGETS=""
  for n in "${TICKETS[@]}"; do
    given_ticket "${n}"
    case "$GIVEN_STATE" in
      open) open+=("${n}"); GIVEN_TARGETS+="${n}"$'\t'"${GIVEN_TITLE}"$'\n' ;;
      closed) log "skip #${n} (${GIVEN_TITLE}): already closed" ;;
      spec) whys+="${whys:+; }${GIVEN_WHY}" ;;
      *) die "#${n}: could not look up the issue (GitHub)" ;;
    esac
  done
  [[ -z "$whys" ]] || die "$whys"
  TICKETS=(${open[@]+"${open[@]}"})
}

issue_has_label() { gh issue view "$1" --json labels -q '.labels[].name' 2>/dev/null | grep -Fxq -- "$2"; }
# GitHub 시각(ISO 8601, UTC) → epoch. macOS·Linux의 date 차이를 피해 jq로. 못 읽으면 빈 값
iso_epoch() { jq -rn --arg t "$1" '$t | sub("\\.[0-9]+Z$"; "Z") | fromdate' 2>/dev/null || true; }
# PR 생성 시각(GitHub) → epoch. 지금보다 늦으면 지금으로 자른다 — 미국 서머타임이 끝나는 날 태평양 01:00–02:00(PDT)에 생긴 사건을
# GitHub API가 1시간 뒤(PST)로 돌려준다(#44, rental-portal PR #587). 그대로 쓰면 구현 몫이 1시간 늘고, 단계 시각이 미래로 밀려
그 뒤
# CI·병합 뒤 검증이 0초가 되며, idle 판정은 PR이 idle 뒤에 열렸다고 본다. 몇 초는 두 시계의 오차라 조용히 자르고, 1분을 넘으면
알린다
pr_created_epoch() {
  local c now
  c="$(iso_epoch "$(gh pr view "$1" --json createdAt -q .createdAt 2>/dev/null || true)")"
  [[ -n "$c" ]] || return 0
  now="$(date +%s)"
  if [[ "$c" -gt "$now" ]]; then
    [[ $(( c - now )) -le 60 ]] || log warn "PR #$1: GitHub reports it created at $(fmt_epoch "$c"), $(( c - now ))s from now —
using now"
    c="$now"
  fi
  printf '%s\n' "$c"
}

candidates() {
  # READY_LABEL(+SCOPE_LABEL)(+PARENT의 하위 이슈), IN_REVIEW_LABEL 없음, 제목 순번 → 이슈 번호 순
  local list kids
  local args=(--label "$READY_LABEL" --state open --limit 200 --json number,title,labels,assignees)
  [[ -z "$SCOPE_LABEL" ]] || args+=(--label "$SCOPE_LABEL")
  list="$(gh issue list "${args[@]}")" || return 1
  if [[ -n "$PARENT" ]]; then
    kids="$(spec_sub_issues "$PARENT")" || return 1
    list="$(jq --argjson kids "$(jq -c 'map(.number)' <<<"$kids")" 'map(select(.number as $n | $kids | index($n)))' <<<"$lis
t")"
  fi
  jq -r --arg re "$ORDER_REGEX" --arg rv "$IN_REVIEW_LABEL" '
      map(select((.labels | map(.name) | index($rv)) == null))
      | map(. + {o: ([.title | capture($re) | .n] | first)})
      | sort_by([(if .o then 0 else 1 end), ((.o // .number) | tonumber)])
      | .[] | "\(.number)\t\(.assignees | length)\t\(.title)"' <<<"$list"
}

# 대상 목록("번호<TAB>제목" 줄들). 목록 조회(candidates)가 실패하면 1 — 부르는 쪽이 정한다(시작 전이면 die, 티켓 사이면 PICK_RE
TRY_SEC 동안
# 다시, pm_due는 검증을 미룬다). 조회가 성공했는데 후보가 0행이면 2(목록은 비어 있다) — 후보가 있었지만 모두 걸러진 빈 목록(0,
skip 줄이 남는다)과
# 구분한다. 성공한 빈 응답이 "남은 티켓 없음"으로 읽힌 실측이 있어(#22) 티켓 사이에는 pick_next_targets가 한 번 더 확인한다.
# 후보 하나의 검사(sub_issue_count·blocked_by·open_pr_for)가 실패하면 그 티켓만 고르지 않는다 — 막혔거나
# PR이 열렸을 수 있으니 안전한 쪽으로. 명령 치환 안에서 돌므로(set -e 없음) 실패는 종료 코드로만 알 수 있다
pick_targets() {
  if [[ ${TICKETS_GIVEN:-0} -eq 1 ]]; then printf '%s' "$GIVEN_TARGETS"; return; fi   # 번호 지정 — 시작 검사(given_tickets_che
ck)가 거른 목록
  local n assignees title list subs blocked pr
  list="$(candidates)" || return 1
  [[ -n "$list" ]] || { log "ticket lookup returned 0 candidates"; return 2; }
  while IFS=$'\t' read -r n assignees title; do
    [[ -n "${n}" ]] || continue
    if [[ "${assignees}" != "0" ]]; then log "skip #${n} (${title}): has an assignee (in progress or taken by a person)"; conti
nue; fi
    subs="$(sub_issue_count "${n}")" || { log "skip #${n} (${title}): could not check (GitHub)"; continue; }
```

```
    if [[ "$subs" != "0" ]]; then log "skip #${n} (${title}): spec issue with sub-issues"; continue; fi
    blocked="$(blocked_by "${n}")" || { log "skip #${n} (${title}): could not check (GitHub)"; continue; }
    if [[ "$blocked" != "0" ]]; then log "skip #${n} (${title}): blocked by an open issue"; continue; fi
    pr="$(open_pr_for "${n}")" || { log "skip #${n} (${title}): could not check (GitHub)"; continue; }
    if [[ -n "$pr" ]]; then log "skip #${n} (${title}): PR already open"; continue; fi
    printf '%s\t%s\n' "${n}" "${title}"
  done <<<"$list"
}

# 티켓 사이의 다음 대상 — pick_targets에 빈 목록 재확인을 더한 것(이슈 #22). 조회가 성공하면서 후보 0행이면(skip 줄 하나 없이 빈 목록) GitHub가
# 성공한 빈 응답을 돌려준 것일 수 있다 — 실측(0.9.12, --parent 313): 열린 하위 이슈 12건을 두고 issue list나 sub_issues가 빈 값을 줘 pm_due가
# "no more tickets"로 병합 뒤 검증을 앞당겼다. 같은 경로가 메인 고리에서 나면 티켓을 남기고 정상 종료한다. 그래서 0행이면 PICK_RECHECK_SEC
# 쉬고 한 번 더 조회하고, 두 번째도 0행이면 그때 빈 목록으로 확정한다. --parent면 스펙과 맞춰 봐서(spec_empty_list_check — 다른 끝점의 요약과
# 하위 이슈의 상태·라벨) 빈 목록이 말이 안 되면 믿지 않는다. 후보가 있었지만 모두 걸러진(막힘·담당자·PR) 빈 목록은 재확인 없이 그대로다 — 스펙
# 끝의 정상 경로. 시작 전 첫 계산은 사람이 앞에 있으니 pick_targets 그대로다.
# 반환 0 목록(비어 있을 수 있다), 1 조회 실패(어느 조회든), 2 두 번 다 0행인데 스펙 #PARENT에 열린 하위 이슈가 남음 — 부르는 쪽(pm_due는 미룸,
# 메인 고리는 PICK_RETRY_SEC 동안 다시 해 본 뒤 멈춤)이 정한다. 명령 치환 안에서 돌므로 로그는 남기되 변수는 돌려주지 못한다
pick_next_targets() {
  local list rc=0 why
  list="$(pick_targets)" || rc=$?
  [[ $rc -eq 2 ]] || { printf '%s' "$list"; return $rc; }   # 목록(0)이나 조회 실패(1, 출력 없음)는 그대로
  log "empty ticket list — re-checking in ${PICK_RECHECK_SEC}s"
  sleep "$PICK_RECHECK_SEC"
  rc=0; list="$(pick_targets)" || rc=$?
  [[ $rc -eq 2 ]] || { printf '%s' "$list"; return $rc; }
  [[ -n "$PARENT" ]] || return 0
  why="$(spec_empty_list_check "$PARENT")" || return 1
  [[ -n "$why" ]] || return 0
  log warn "ticket list empty but ${why} — not trusting it"
  return 2
}

# ---------- 백엔드 ----------
# 루프가 워크트리·에이전트 터미널에 대해 하는 일은 아래 아홉 가지뿐이다. 백엔드마다 같은 이름의 함수(<backend>_*)를 구현하고
# BACKEND로 고른다. 티켓 선택·PR·CI·수리·병합·병합 뒤 검증·기록·리포트는 백엔드와 무관하게 한 벌이다.
#   _check                      시작 때 한 번: CLI가 있고 앱에 닿는지, 저장소가 등록돼 있는지
#   _worktree_create N PROMPT [NAME]  기본 브랜치에서 워크트리를 만들고 에이전트를 띄운다. 전역 WT(백엔드의 워크트리/워크스페이스 id),
#                               WT_PATH, WT_BRANCH(백엔드가 만든 브랜치 — 워커가 바꾸기 전 이름), HANDLE(에이전트 터미널)을 채운다.
#                               NAME(워크트리·브랜치 이름)은 기본 <BRANCH_PREFIX>N — 무인 수습 세션은 loop-triage-N-<시각>을 준다
#   _worktree_remove WT WT_PATH  워크트리를 지운다(브랜치 정리는 공통부가 한다)
#   _send HANDLE TEXT           프롬프트 한 줄을 에이전트 입력에 넣고 제출한다. 수락되지 않으면 비0
#   _type HANDLE TEXT           셸 명령 한 줄을 입력하고 Enter(에이전트 재기동용 — 에이전트 입력 검사 없이)
#   _interrupt HANDLE           Ctrl-C
#   _wait_idle HANDLE MS        반환 0: idle 확인, 1: 타임아웃(아직 작업 중), 3: 터미널에 닿지 못함. 3을 돌려주기 전에 reach_lost로 원인을
#                               남기고(호출부가 멈춤 사유에 싣는다), 앱이 응답하면 reach_ok를 부른다(아래 "터미널에 닿지 못할 때" 절)
#   _tail HANDLE                화면 마지막 25줄
#   _set_status WT STATUS [PR] [COMMENT]   STATUS: in-progress ¦ in-review ¦ todo(확인 필요) ¦ completed. 실패해도 루프는 간다
backend_check()           { "${BACKEND}_check" "$@"; }
backend_worktree_create() { "${BACKEND}_worktree_create" "$@"; }
backend_worktree_remove() { "${BACKEND}_worktree_remove" "$@"; }
backend_send()            { "${BACKEND}_send" "$@"; }
backend_type()            { "${BACKEND}_type" "$@"; }
backend_interrupt()       { "${BACKEND}_interrupt" "$@"; }
backend_wait_idle()       { "${BACKEND}_wait_idle" "$@"; }
backend_tail()            { "${BACKEND}_tail" "$@"; }
backend_set_status()      { "${BACKEND}_set_status" "$@"; }
case "$BACKEND" in orca) BACKEND_NAME=Orca ;; cmux¦tmux) BACKEND_NAME="$BACKEND" ;; *) die "Unknown BACKEND=${BACKEND} (orca ¦ cmux ¦ tmux)." ;; esac

# 프롬프트를 셸 명령 인자로 넘길 때는 파일을 거친다(따옴표·백틱이 섞여도 안전). 파일은 티켓이 끝나도 지우지 않는다(디버그용, tmp)
```

```
prompt_file() { local f; f="$(mktemp "${TMPDIR:-/tmp}/ticket-loop-prompt-$1.XXXXXX")"; printf '%s' "$2" > "$f"; printf '%s' "$f"; }
agent_launch_cmd() { printf '%s "$(cat %q)"' "$AGENT_RESTART_CMD" "$1"; }   # 셸이 해석할 한 줄: claude¦codex … "$(cat /tmp/…)"

# ----- 터미널에 닿지 못할 때 (#61) -----
# 대기 함수(_wait_idle)가 터미널 화면을 읽지 못하면 원인을 하나로 가른다(cmux_reach·orca_reach — 아홉 가지 밖의 백엔드 내부 함수. tmux는
# 창이 없으면 워커도 없어 늘 terminal이다):
#   terminal      앱은 응답했지만 터미널이 없다(cmux not_found, Orca terminal_handle_stale)
#   app           앱에 닿지 못했고 앱 프로세스도 없다 — 기다려도 소용없다
#   unresponsive  앱에 닿지 못했지만 앱 프로세스는 있다 — 앱이 잠깐 멈췄다
# 앞의 둘은 전처럼 TERMINAL_RETRY번 연달아면 3이다. unresponsive는 TERMINAL_UNRESPONSIVE_SEC까지 기다리고 그 상한은 횟수가 아니라 경과
# 시간으로 잰다 — 멈춘 앱에는 CLI 호출 하나가 15초(cmux)에서 60초(Orca)씩 걸린다. 2026-11-12 실측(키트 0.9.26·0.9.28, cmux 0.65.2):
# cmux 앱이 1~2분 멈춘 동안 소켓이 Connection refused를 냈고, 같은 앱을 쓰던 루프 둘이 1분 만에 "app closed?"로 멈췄다. 앱은 그대로
# 살아 있었고 워커 둘은 혼자 PR까지 열었다.
# 응답 없음이 시작된 시각은 REACH_STATE_DIR의 터미널별 파일에 둔다 — 대기 함수는 PROGRESS_SEC마다 돌아오고 수리 대기는 매 바퀴 bg_wait
# 서브셸에서 새로 불러 셸 변수가 남지 않는다. 앱이 한 번이라도 응답하면 지운다. 3으로 끝날 때의 원인도 같은 자리에 남겨 호출부가
# 멈춤 사유에 싣는다(terminal_lost_why). 디렉터리는 루프가 잠금을 잡을 때 만들고 on_exit가 지운다
reach_file() { printf '%s/%s' "$REACH_STATE_DIR" "${1//[^A-Za-z0-9_.-]/_}"; }
# 대기 하나(PR·수리·수습 판정)를 시작할 때 — 앞 대기가 응답 없음 중에 끝났으면(그사이 PR이 열림) 그 시작 시각을 이어 쓰지 않는다
reach_reset() { rm -f "$(reach_file "$1")" "$(reach_file "$1").lost"; }
# 앱이 응답했다 — 응답 없음이 이어지던 중이었으면 회복 줄을 한 번 찍는다
reach_ok() {
  local f since=""
  f="$(reach_file "$1")"
  [[ -s "$f" ]] ¦¦ return 0
  read -r since _ < "$f" ¦¦ true
  rm -f "$f"
  log ok "terminal $1: the ${BACKEND_NAME} app is responding again after $(( $(date +%s) - ${since:-0} ))s"
}
# 앱에 닿지 못했지만 프로세스는 있다 — 상한 안이면 0, 넘었으면 원인을 남기고 1. 시작할 때 한 줄, 이어지는 동안 TERMINAL_UNRESPONSIVE_LOG_SEC마다 한 줄
reach_unresponsive() {
  local f now since logged_at
  f="$(reach_file "$1")"; now="$(date +%s)"
  if [[ -s "$f" ]]; then
    read -r since logged_at < "$f" ¦¦ true
  else
    since="$now"; logged_at="$now"; printf '%s %s\n' "$since" "$logged_at" > "$f"
    log warn "terminal $1: cannot reach the ${BACKEND_NAME} app, but its process is running — waiting up to ${TERMINAL_UNRESPONSIVE_SEC}s for it to respond"
  fi
  if [[ $(( now - since )) -ge $TERMINAL_UNRESPONSIVE_SEC ]]; then reach_lost "$1" unresponsive $(( now - since )); return 1; fi
  if [[ $(( now - logged_at )) -ge $TERMINAL_UNRESPONSIVE_LOG_SEC ]]; then
    printf '%s %s\n' "$since" "$now" > "$f"
    log warn "terminal $1: the ${BACKEND_NAME} app has not responded for $(( now - since ))s (waiting up to ${TERMINAL_UNRESPONSIVE_SEC}s)"
  fi
  return 0
}
reach_lost() { printf '%s %s\n' "$2" "${3:-}" > "$(reach_file "$1").lost"; }   # 3을 돌려주기 직전 — $2 원인, $3 응답 없음이 이어진 초
# 대기가 3으로 끝난 원인을 멈춤 사유의 구절로 — 앱이 살아 있으면 "app closed?" 같은 추측을 쓰지 않는다
terminal_lost_why() {
  local kind="" secs="" f
  f="$(reach_file "$1").lost"
  [[ ! -f "$f" ]] ¦¦ read -r kind secs < "$f" ¦¦ true
  case "$kind" in
    unresponsive) echo "the ${BACKEND_NAME} app has not responded for ${secs}s, though its process is still running" ;;
    app)          echo "the ${BACKEND_NAME} app is not running" ;;
    terminal)     echo "the terminal no longer exists" ;;
    *)            echo "the terminal could not be read" ;;
  esac
}
# 그 멈춤에 붙이는 안내(사용량 한도 멈춤과 같은 뜻) — $1 루프 밖에서 PR을 열 수 있는 쪽(worker ¦ triage session). 2026-11-12 실
```

```
측(#61)에서
# 루프가 멈춘 뒤에도 워커 둘이 혼자 PR을 열었다
terminal_lost_note() { echo "The $1 may keep running outside the loop and open or update a PR — check for one before you rerun
(the loop does not take over an open PR)."; }

# ----- Orca: 워크트리·터미널·idle 대기를 모두 Orca CLI가 한다 -----

orca_json() { "$ORCA_CLI" "$@" --json; }

orca_check() {
  command -v "$ORCA_CLI" >/dev/null 2>&1 || die "Orca CLI (${ORCA_CLI}) not found (Orca Settings → General → Orca CLI)."
  # canonicalKey는 보통 github.com/<owner>/<repo>지만, SSH 별칭 호스트(~/.ssh/config의 github.com-<계정>)를 쓰면
  # github.com-<계정>/<owner>/<repo>가 된다(2026-10-12 실측). 호스트는 보지 않고 /<owner>/<repo>로 끝나는지만 본다
  REPO_ID="$("$ORCA_CLI" repo list --json \
    | jq -r --arg suffix "/${OWNER_REPO}" \
        '.result.repos[] | select((.gitRemoteIdentity.canonicalKey // "") | endswith($suffix)) | .id' | head -n1)"
  [[ -n "$REPO_ID" ]] || die "Repository ${OWNER_REPO} is not registered in Orca. Register it with '${ORCA_CLI} repo add --path
${ROOT} --json'."
  # --surface가 없으면 에이전트가 Orca 앱의 Chat UI 설정을 따라 채팅으로 뜰 수 있다 — 루프는 터미널로만 본다(#41).
  # CLI가 그 플래그를 알고(도움말) 실행 중인 앱(호스트)도 받을 때만 넘긴다. 자동 업데이트로 번들 CLI만 새것이고 앱은 재시작 전
이면
  # 호스트가 --surface를 incompatible_runtime으로 거절한다. 못 넘긴 채 채팅으로 뜨면 orca_worktree_create의 outcome.kind 검사가
멈춘다
  ORCA_SURFACE_ARGS=()
  if "$ORCA_CLI" worktree create --help 2>/dev/null | grep -q -- '--surface' \
    && orca_json status 2>/dev/null | jq -e '.result.runtime.capabilities // [] | index("agent.launch.surface.v1")' >/dev/null
2>&1; then
    ORCA_SURFACE_ARGS=(--surface terminal)
  fi
}

orca_worktree_create() {
  local n="$1" prompt="$2" name="${3:-${BRANCH_PREFIX}${1}}" comment created kind delivered
  comment="티켓 루프: ${SKILL_PREFIX}implement ${n} 진행 중"
  [[ "$name" == "${BRANCH_PREFIX}${n}" ]] || comment="티켓 루프: 무인 수습 (#${n})"
  created="$(orca_json worktree create --repo "id:$REPO_ID" --name "$name" --no-parent \
    --base-branch "origin/${BASE_BRANCH}" --issue "${n}" --agent "$AGENT" ${ORCA_SURFACE_ARGS[@]+"${ORCA_SURFACE_ARGS[@]}"} \
    --prompt "$prompt" --comment "$comment")"
  jq -e '.ok == true' <<<"$created" >/dev/null || die "Worktree creation failed: $(jq -c '.error' <<<"$created")"
  # Orca 1.4.244부터 호스트가 agent.launch.v2를 알리면 CLI가 agent.launch로 띄우고 결과를 result.launch에 싣는다(#41) —
  # 핸들은 launch.outcome.handle, 워크트리 id는 launch.worktreeId에도 있다. result.worktree는 시작 뒤 worktree.show로 채우는데
  # 그게 실패하면 결과에서 빠진다. result.agentTerminalHandle은 그보다 낡은 호스트의 모양이다(2026-10-30 rsrehab/detection-tsli
te 실측)
  WT="$(jq -r '.result.worktree.id // .result.launch.worktreeId // empty' <<<"$created")"
  [[ -n "$WT" ]] || die "Worktree creation returned no worktree id: $(jq -c '.result | keys' <<<"$created")"
  WT_PATH="${WT#*::}"
  kind="$(jq -r '.result.launch.outcome.kind // empty' <<<"$created")"
  [[ "$kind" != structured ]] || die "The agent started as a chat, not a terminal (${WT}) — turn off Chat UI in Orca settings,
or update Orca so the loop can pass --surface terminal"
  HANDLE="$(jq -r '.result.launch.outcome.handle // .result.agentTerminalHandle // .result.startupTerminal.handle // empty' <<
<"$created")"
  [[ -n "${HANDLE}" ]] || die "No agent terminal handle returned (${WT})"
  # 프롬프트가 도착하지 않았어도 멈추지 않는다 — 워커가 PR 없이 idle이 되면 그 경로(무인 수습)가 잡는다. 사유는 여기 남긴다
  delivered="$(jq -r '.result.launch.prompt.outcome // empty' <<<"$created")"
  case "$delivered" in ""|journaled|handed-to-terminal) ;; *) log warn "#${n}: Orca reports the prompt as '${delivered}' — the
agent may not have received it" ;; esac
  # Orca가 워크트리용으로 만든 브랜치(설정에 따라 `issue-N` 또는 `<git 사용자>/issue-N`) — 워커가 브랜치를 바꾸기 전에 기록한다
  WT_BRANCH="$(git -C "$WT_PATH" rev-parse --abbrev-ref HEAD 2>/dev/null || true)"
}

orca_worktree_remove() { orca_json worktree rm --worktree "id:$1" --force >/dev/null; }

orca_send() {
  orca_json terminal send --terminal "$1" --text "$2" --enter --wait-submit 15 \
    | jq -e '.result.send.accepted == true' >/dev/null
}
orca_type()      { orca_json terminal send --terminal "$1" --text "$2" --enter >/dev/null; }
orca_interrupt() { orca_json terminal send --terminal "$1" --text $'\x03' >/dev/null; }

orca_wait_idle() {
  # `orca terminal wait`는 타임아웃이면 {ok:false, error.code:"timeout"}과 함께 exit 1을 돌려준다(Orca 1.4.223 실측).
  # exit 코드만 보면 "닿지 못함"으로 오판해 일하는 워커를 두고 루프가 죽는다 — JSON으로 가른다.
```

```
  # 그 밖의 응답(빈 출력·다른 error.code)은 한 번으로 "닿지 못함"이라 판정하지 않는다. 사람이 Orca 앱에서 워커
  # 워크트리를 열면 세션이 다시 붙으며(데몬 로그 session-attached) 기다리던 wait가 그렇게 끝난다 — 2026-10-12 #72
  # (Orca 1.4.224)에서 일하던 워커를 두고 루프가 죽었다. 터미널을 읽을 수 있으면 아직 작업 중(1)으로 보고, 못 읽으면 원인(orca_
reach)대로
  # cmux와 같은 규칙이다 — 터미널·앱이 없으면 TERMINAL_RETRY번 연달아일 때 3, 앱이 멈췄으면 TERMINAL_UNRESPONSIVE_SEC까지 기다
리며
  # 이번 호출의 상한($2)에서 1로 돌아온다. 터미널이 사라지면 read가 ok:false(terminal_handle_stale)다.
  local handle="$1" deadline=$(( $(date +%s) + $2 / 1000 )) out code fails=0 why
  out="$(orca_json terminal wait --terminal "$handle" --for tui-idle --timeout-ms "$2" 2>/dev/null)" || true
  if jq -e '.result.wait.satisfied == true' <<<"$out" >/dev/null 2>&1; then reach_ok "$handle"; return 0; fi
  code="$(jq -r '.error.code // empty' <<<"$out" 2>/dev/null)"
  if [[ "$code" == "timeout" ]]; then reach_ok "$handle"; return 1; fi
  while :; do
    sleep "$TERMINAL_RETRY_SEC"
    why="$(orca_reach "$handle")"
    case "$why" in
      ok)
        reach_ok "$handle"
        log warn "terminal ${handle}: unexpected wait response (${code:-empty output}) — terminal is alive, keep waiting"
        return 1 ;;
      unresponsive)
        fails=0; reach_unresponsive "$handle" || return 3
        [[ "$(date +%s)" -lt $deadline ]] || return 1 ;;
      *)
        [[ "$why" != terminal ]] || reach_ok "$handle"
        fails=$((fails + 1)); [[ $fails -lt $TERMINAL_RETRY ]] || { reach_lost "$handle" "$why"; return 3; } ;;
    esac
  done
}

# 터미널을 읽어 닿는지 가른다 — ok | terminal | app | unresponsive(위 "터미널에 닿지 못할 때" 절). 앱에 닿지 못했다는 응답은 CL
I의 전송 계층만
# 낸다: runtime_unavailable(연결 거절·앱 꺼짐), runtime_timeout(요청 상한 60초 안에 답이 없음), invalid_runtime_response, 그리
고 빈 출력
# (Orca 1.4.224 out/cli/runtime/transport.js). 그 밖의 error.code(terminal_handle_stale·terminal_not_found 등)는 앱이 답한 것이
다.
# 멈춘 앱의 응답 모양은 실측하지 못했다(#61)
orca_reach() {
  local out code
  out="$(orca_json terminal read --terminal "$1" 2>/dev/null)" || true
  if jq -e '.ok == true' <<<"$out" >/dev/null 2>&1; then echo ok; return 0; fi
  code="$(jq -r '.error.code // empty' <<<"$out" 2>/dev/null || true)"
  case "$code" in
    ""|runtime_unavailable|runtime_timeout|invalid_runtime_response) ;;
    *) echo terminal; return 0 ;;
  esac
  if orca_app_running; then echo unresponsive; else echo app; fi
}
# Orca 앱 프로세스가 있는가. orca status는 앱에 1초 상한으로 묻고, 답이 없으면 런타임 메타데이터의 pid가 살아 있는지 본다(out/c
li/runtime/status.js)
# — 멈춘 앱에도 오래 막히지 않는다. 앱이 꺼져 있으면 app.running false·app.pid null이다(1.4.224 실측)
orca_app_running() { orca_json status 2>/dev/null | jq -e '.result.app.running == true or .result.app.pid != null' >/dev/null 2
>&1; }

orca_tail() { orca_json terminal read --terminal "$1" | jq -r '.result.terminal.tail[]?' | tail -n 25; }

orca_set_status() {
  # local 한 문장 안에서 선언한 변수를 같은 문장에서 쓰면 set -u 에 걸린다(2026-10-12 boiler-webapp #34 실측) — 줄을 나눈다
  local wt="$1" status="$2" pr="${3:-}" comment="${4:-}"
  local args=(--worktree "id:${wt}" --workspace-status "$status")
  [[ -z "$pr" ]] || args+=(--pr "$pr")
  [[ -z "$comment" ]] || args+=(--comment "$comment")
  [[ "$status" != "todo" ]] || args+=(--unread)
  orca_json worktree set "${args[@]}" >/dev/null
}

# ----- cmux·tmux 공통 -----
# Claude Code는 새 워크트리에서 "이 폴더를 신뢰하나" 대화상자를 띄우는데, 그 판정은 메인 체크아웃 경로로 한다(2026-10-13 실측:
# 메인 체크아웃이 ~/.claude.json projects에 없던 더미 저장소에서 대화상자가 떴고, 수락하면 메인 체크아웃 경로가 trust=true로 기
록됐다.
# boiler-webapp cmux 실측에서 안 뜬 것은 메인 체크아웃을 이미 신뢰했기 때문). 대화상자가 떠 있으면 훅도 프롬프트도 돌지 않으니
# 시작 때 막는다. Orca는 워크스페이스마다 신뢰를 기록해 주므로 여기서 보지 않는다
```

```
claude_trust_check() {
  [[ "$AGENT" == claude ]] || return 0
  local cfg="${CLAUDE_CONFIG_DIR:+$CLAUDE_CONFIG_DIR/.claude.json}" trusted
  cfg="${cfg:-$HOME/.claude.json}"
  [[ -f "$cfg" ]] || return 0
  trusted="$(jq -r --arg p "$MAIN_WT" '.projects[$p].hasTrustDialogAccepted // false' "$cfg" 2>/dev/null)" || return 0
  [[ "$trusted" == true ]] \
    || die "Claude Code does not trust ${MAIN_WT} yet — workers would stall at the trust dialog in a new worktree. Open claude
once in the main checkout and choose 'Yes, I trust this folder'."
}

# 워크트리는 루프가 git으로 만든다 (Orca는 orca worktree create가 한다)
# WT_PATH·WT_BRANCH를 채운다. 백엔드는 그 위에 에이전트 터미널을 연다. $2 이름(기본 <BRANCH_PREFIX>N)
# 워커 워크트리의 준비(WORKTREE_SETUP_CMD)가 실패하면 수습 세션을 띄워 본다 — 고쳐지면 워크트리를 기본 브랜치로 당겨 준비를 다
시 돌리고,
# 아니면 멈춘다. 수습 세션 자신의 워크트리는 준비가 실패해도 띄운다(그 실패를 수습 세션이 직접 본다)
loop_worktree_add() {
  local n="$1" name="${2:-${BRANCH_PREFIX}${1}}" path reason
  path="${WORKTREE_DIR}/${name}"
  [[ ! -e "$path" ]] || die "Worktree path already exists: ${path} (if left over from a previous run, 'git worktree remove --fo
rce' it and retry)"
  mkdir -p "$WORKTREE_DIR"
  git -C "$ROOT" branch -D "$name" >/dev/null 2>&1 || true
  git -C "$ROOT" worktree add -q -b "$name" "$path" "origin/${BASE_BRANCH}" || die "git worktree add failed (${path})"
  if [[ -n "$WORKTREE_SETUP_CMD" ]]; then
    log step "#${n}: worktree setup — ${WORKTREE_SETUP_CMD}"
    while ! bg_wait env LOOP_ROOT_PATH="$MAIN_WT" LOOP_WORKTREE_PATH="$path" bash -c 'cd "$1" && exec bash -c "$2"' _ "$path"
"$WORKTREE_SETUP_CMD"; do
      reason="#${n}: WORKTREE_SETUP_CMD failed (${path})"
      if [[ "$name" != "${BRANCH_PREFIX}${n}" ]]; then log warn "${reason} — starting the session anyway"; break; fi
      STOP_KIND=setup_failed
      triage setup_failed "$reason" || die "${reason}${TRIAGE_NOTE}"
      git -C "$path" merge --ff-only -q "origin/${BASE_BRANCH}" \
        || die "#${n}: could not fast-forward the worktree to origin/${BASE_BRANCH} after the triage fix (${path})"
      log step "#${n}: worktree setup again after triage — ${WORKTREE_SETUP_CMD}"
    done
  fi
  WT_PATH="$path"
  WT_BRANCH="$name"
}

# ----- cmux: 워크트리는 루프가 git으로 만들고, 터미널은 cmux 워크스페이스다 (cmux 0.65 실측) -----
# cmux에는 워크트리 생성 명령도, idle까지 블로킹하는 명령도 없다. 그래서
#   - `git worktree add` 뒤 WORKTREE_SETUP_CMD(orca.yaml setup에 해당)를 돌리고 `cmux workspace create --cwd … --command "<AGEN
T_RESTART_CMD> …"`
#     --command는 워크스페이스의 로그인 셸(zsh)이 실행하고, 그 셸의 PATH 앞에 cmux의 에이전트 shim(claude·codex)이 있어 lifecyc
le 훅이 자동으로 붙는다
#   - idle은 `cmux rpc surface.input_state`가 돌려주는 lifecycle(running ¦ idle ¦ needsInput ¦ unknown — 에이전트 훅이 보고. Cl
aude Code 실측,
#     Codex는 shim이 같은 훅을 주입하지만 아직 실측 전)을
#     IDLE_POLL_SEC마다 읽어 판정한다. 이벤트 스트림(`cmux events`)의 이름에 기대지 않는다
#   - 소켓 모드가 cmuxOnly(기본)면 cmux 터미널 밖에서는 "액세스 거부"다. 루프를 cmux 터미널 안에서 돌린다
#   - 백그라운드 idle 에이전트를 하이버네이션할 수 있어, 프롬프트를 보내기 전에 깨운다
# 실측한 것: `--json`은 명령 뒤에 둬야 JSON이 나온다(앞에 두면 "OK workspace:N" 한 줄). `workspace create --json --id-format uu
ids`는
# {workspace_id, surface_id, window_id, group_id}. `workspace status set`의 lane은 todo¦working¦needs-attention¦review¦done¦aut
o.
# `workspace close`는 프로세스가 살아 있으면 --force 없이 거부한다. 옛 이름(new-workspace 등)은 deprecation 한 줄을 찍으니 쓰지
않는다.

cmux_j() { CMUX_QUIET=1 "$CMUX_CLI" "$@" --json; }

cmux_check() {
  [[ -x "$CMUX_CLI" ]] || command -v "$CMUX_CLI" >/dev/null 2>&1 || die "cmux CLI (${CMUX_CLI}) not found (set CMUX_CLI=<path
>)."
  "$CMUX_CLI" identify --json >/dev/null 2>&1 \
    || die "Cannot reach the cmux socket. Run inside a cmux terminal or change the socket mode ('${CMUX_CLI} socket-status --js
on')."
  [[ -n "$WORKTREE_DIR" ]] || WORKTREE_DIR="$HOME/.cache/ticket-loop/${OWNER_REPO//\//-}/worktrees"
  claude_trust_check
}
```

```
cmux_worktree_create() {
  local n="$1" prompt="$2" name="${3:-${BRANCH_PREFIX}${1}}" out pf
  loop_worktree_add "$n" "$name"
  pf="$(prompt_file "$n" "$prompt")"
  out="$(cmux_j workspace create --name "$name" --cwd "$WT_PATH" --focus false --id-format uuids \
          --command "$(agent_launch_cmd "$pf")" 2>/dev/null)" || die "cmux workspace create failed"
  WT="$(jq -r '.workspace_id // empty' <<<"$out" 2>/dev/null || true)"
  HANDLE="$(jq -r '.surface_id // empty' <<<"$out" 2>/dev/null || true)"
  [[ -n "$WT" && -n "$HANDLE" ]] || die "workspace_id/surface_id missing from cmux workspace create response: ${out}"
}

cmux_worktree_remove() {
  # 에이전트 프로세스가 살아 있는 워크스페이스라 --force가 필요하다(없으면 confirmation_required로 거부)
  CMUX_QUIET=1 "$CMUX_CLI" workspace close "$1" --force >/dev/null 2>&1 || true
  git -C "$ROOT" worktree remove --force "$2"
}

# 붙여 넣은 뒤 lifecycle이 직전 턴의 idle에서 벗어나는지 SEND_ACCEPT_SEC 동안 본다(tmux의 sent→running 확인과 같은 뜻). 붙여 넣은
# 직후에는 cmux가 아직 새 턴을 몰라 직전 턴의 idle을 그대로 돌려준다 — 0.9.2까지는 확인 없이 돌아와, 수리 프롬프트 직후의 묵은 idle을
# 수리 끝으로 읽었다(2026-10-20 rental-portal #251: 수리 요청 5초 뒤 "no new commit"으로 멈췄고, 워커는 2분 뒤 수리 커밋을 push했다).
# running이면 수락이다. 빈 값·unknown은 판정할 수 없지만 cmux_wait_idle이 그 값을 idle로 읽지 않으니 넘어간다. 끝까지 idle이면 화면으로
# 가른다 — 붙여 넣은 프롬프트가 화면에 찍혔으면 들어간 것으로 보고(훅 보고가 늦은 경우, 남은 묵은 idle은 repair가 거른다), 화면도
# 그대로면 비0이다
cmux_send() {
  local handle="$1" i before
  "$CMUX_CLI" agent-hibernation wake "$handle" >/dev/null 2>&1 || true
  before="$(cmux_tail "$handle" 2>/dev/null | cksum)" || before=""
  # paste --submit: 한 줄을 붙여 넣고 제출. 에이전트 입력에 초안이나 대화상자가 있으면 거부(비0)한다 — 사람이 끼어든 것이니 보내지 않는다
  "$CMUX_CLI" paste --surface "$handle" --submit "$2" >/dev/null || return 1
  for (( i = 0; i < SEND_ACCEPT_SEC * 2; i++ )); do
    case "$(cmux_lifecycle "$handle")" in idle|needsInput) ;; *) return 0 ;; esac
    sleep 0.5
  done
  [[ -z "$before" || "$(cmux_tail "$handle" 2>/dev/null | cksum)" != "$before" ]]
}
cmux_type()      { "$CMUX_CLI" send --surface "$1" --force "$2" >/dev/null && "$CMUX_CLI" send-key --surface "$1" enter >/dev/null; }
cmux_interrupt() { "$CMUX_CLI" send-key --surface "$1" ctrl+c >/dev/null; }

cmux_lifecycle() {   # running | idle | needsInput | unknown, 소켓에 닿지 못하면 빈 값
  "$CMUX_CLI" rpc surface.input_state "{\"surface_id\":\"$1\"}" 2>/dev/null | jq -r '.lifecycle // empty' 2>/dev/null || true
}

# 화면 한 줄을 읽어 터미널에 닿는지 가른다 — ok | terminal | app | unresponsive(위 "터미널에 닿지 못할 때" 절). cmux 0.65.2 실측(#61): 앱이 멈추면
# 연결 대기열에 자리가 있을 때 15초 뒤 "Command timed out", 대기열이 차면 바로 "Connection refused"다. 앱이 죽고 소켓 파일만 남아도 같은
# "Connection refused", 정상 종료면 "Socket not found"다 — 문구로는 멈춤과 죽음을 가를 수 없어 소켓을 쥔 프로세스를 따로 본다.
# "앱이 답했다"는 오류 코드 not_found로만 가린다(Orca는 거꾸로 "닿지 못함"을 코드로 가린다 — orca_reach). cmux의 연결 오류는 자유 문구라
# 그 목록에 기대면 CLI 문구가 바뀔 때 멈춘 앱을 터미널 없음으로 읽어 1분 만에 멈추는 쪽으로 틀린다. 지금은 모르는 오류에 앱이 살아 있으면
# 기다리는 쪽(상한까지)으로 틀린다
cmux_reach() {
  local err
  err="$("$CMUX_CLI" read-screen --surface "$1" --lines 1 2>&1 >/dev/null)" && { echo ok; return 0; }
  if grep -q 'not_found' <<<"$err"; then echo terminal
  elif cmux_app_running; then echo unresponsive
  else echo app
  fi
}
# 소켓을 쥔 프로세스(cmux 앱)가 있는가 — 앱에 연결하지 않고 본다. socket-status는 소켓에 연결하지 않는다(0.65.2: 유닉스 소켓 연결을 막아도
# 답했다). 다만 CMUX_SOCKET_PATH를 따르지 않으므로 그 값이 있으면 그쪽이다(CLI는 그 소켓으로 간다). lsof -t는 소켓을 listen하는 프로세스만
# 돌려주고 대기열의 클라이언트나 주인 없이 남은 소켓 파일에는 아무것도 돌려주지 않는다(실측)
```

```
cmux_app_running() {
  local sock="${CMUX_SOCKET_PATH:-}"
  [[ -n "$sock" ]] || sock="$("$CMUX_CLI" socket-status --json 2>/dev/null | jq -r '.socket_path // empty' 2>/dev/null || tru
e)"
  [[ -n "$sock" && -n "$(lsof -t "$sock" 2>/dev/null)" ]]
}

cmux_wait_idle() {
  # lifecycle이 빈 값으로 오는 것(rpc 실패·훅 보고 공백)을 연달아 몇 번 봤다고 "닿지 못함"이라 판정하지 않는다 — 2026-10-14
  # rental-portal #100(cmux 0.65)에서 워커가 ticket-review 서브에이전트 2개와 헤드리스 브라우저를 돌리는 동안 빈 값이 5번(25초)
  # 이어져 일하던 워커를 두고 루프가 죽었다. 앱은 켜져 있었고 서피스도 살아 있었으며 워커는 혼자 PR까지 마쳤다. Orca의 #72와
  # 같은 규칙: 화면을 읽을 수 있으면 아직 작업 중으로 보고, 화면도 못 읽으면 원인(cmux_reach)대로 — 터미널·앱이 없으면 TERMINAL
_RETRY번
  # 연달아일 때 3, 앱이 멈췄으면 TERMINAL_UNRESPONSIVE_SEC까지 기다린다(그 사이 TERMINAL_RETRY_SEC씩 쉰다)
  local handle="$1" deadline_ms=$(( $(date +%s) * 1000 + $2 )) lc fails=0 warned=0 why
  while :; do
    lc="$(cmux_lifecycle "$handle")"
    case "$lc" in
      idle¦needsInput) reach_ok "$handle"; return 0 ;;    # needsInput: 질문·권한 대화상자 — 공통부가 PR 유무와 화면으로 "멈
춤"을 가른다
      running¦unknown) reach_ok "$handle"; fails=0; warned=0 ;;
      *)
        why="$(cmux_reach "$handle")"
        case "$why" in
          ok)
            reach_ok "$handle"; fails=0
            [[ $warned -eq 1 ]] || { log warn "terminal ${handle}: empty lifecycle — screen is readable, keep waiting"; warned=
1; } ;;
          unresponsive)
            fails=0; reach_unresponsive "$handle" || return 3
            sleep "$TERMINAL_RETRY_SEC" ;;
          *)
            [[ "$why" != terminal ]] || reach_ok "$handle"
            fails=$((fails + 1)); [[ $fails -lt $TERMINAL_RETRY ]] || { reach_lost "$handle" "$why"; return 3; }
            sleep "$TERMINAL_RETRY_SEC" ;;
        esac ;;
    esac
    # 연달아 실패하는 중에는 상한에서 돌아오지 않는다 — 횟수는 호출마다 0부터라, TERMINAL_RETRY를 늘리면 PROGRESS_SEC마다 돌아
오는 대기가
    # 3에 끝내 닿지 못한다(Orca도 다시 읽기를 다 해 본 뒤에 돌아온다)
    [[ $fails -gt 0 ]] || $(( $(date +%s) * 1000 )) -lt $deadline_ms ]] || return 1
    sleep "$IDLE_POLL_SEC"
  done
}

cmux_tail() { "$CMUX_CLI" read-screen --surface "$1" --lines 25; }

cmux_set_status() {
  local wt="$1" status="$2" pr="${3:-}" comment="${4:-}" lane
  case "$status" in in-progress) lane=working ;; in-review) lane=review ;; completed) lane=done ;; *) lane=todo ;; esac
  "$CMUX_CLI" workspace status set "$lane" --workspace "$wt" >/dev/null 2>&1 || true
  [[ -z "$pr" ]] || "$CMUX_CLI" pr "$pr" --workspace "$wt" >/dev/null 2>&1 || true
  [[ -z "$comment" ]] || "$CMUX_CLI" log --workspace "$wt" "$comment" >/dev/null 2>&1 || true
}

# ----- tmux: 워크트리는 루프가 git으로 만들고, 터미널은 tmux 창이다. idle은 에이전트 훅이 파일에 쓴다 -----
# tmux는 에이전트를 모른다(Orca의 tui-idle, cmux의 lifecycle 같은 신호가 없다). 그래서
#   - 워크트리는 cmux와 같이 `git worktree add` + WORKTREE_SETUP_CMD. 터미널은 TMUX_SESSION의 새 창 — 셸을 띄우고 거기에 기동
명령을 친다
#     (Orca와 같은 방식이라 에이전트가 죽어도 셸이 남아 재기동 명령을 받을 수 있다)
#   - idle은 에이전트의 훅이 보고한다. 시작 점검 때 ~/.cache/ticket-loop/<owner-repo>/tmux/ 에 hook.sh와 Claude Code 훅 설정
#     claude-hooks.json을 쓰고, 기동 명령에 `--settings <그 파일>`(claude) 또는 `-c notify=[…]`(codex)를 붙인다. 훅은
#     UserPromptSubmit → running, Stop → idle, Notification(permission_prompt·elicitation_dialog) → needsInput, SessionEnd → ex
ited 를
#     state/<워크트리 디렉터리 이름> 파일에 한 단어로 쓰고, 루프가 IDLE_POLL_SEC마다 읽는다(cmux의 lifecycle과 같은 뜻)
#   - 프롬프트가 수락됐는지도 그 파일로 안다: 보내기 전에 sent를 쓰고 SEND_ACCEPT_SEC(15초) 안에 running(UserPromptSubmit)이 되
는지 본다.
#     Codex의 notify는 턴 종료(agent-turn-complete)만 알리므로 보낸 것으로 치고 루프가 직접 running을 쓴다
#   - Stop은 메인 에이전트의 턴이 끝날 때마다 온다 — 백그라운드 서브에이전트(ticket-review)를 기다리는 동안도 idle로 보인다(실
측 1회).
#     서브에이전트가 돌아오면 Claude Code가 스스로 턴을 열어 UserPromptSubmit이 running을 다시 쓰고, 공통부는 화면이 바뀌면 연
속
```

```
#     횟수를 0부터 세므로 멈추지 않는다. cmux의 lifecycle도 같은 훅에 기대므로 같은 성질이다
#   - 상태 표시는 창 이름 뒤에 붙인다(issue-12 [review]). 사람은 `tmux attach -t <세션>`으로 어느 터미널 앱에서나(또는 SSH로) 본다
#   - 루프를 tmux 안에서 돌리면 그 세션에 창을 열고, 밖에서 돌리면 ticket-loop 세션을 만든다(TMUX_SESSION으로 바꾼다)

tmux_check() {
  command -v "$TMUX_CLI" >/dev/null 2>&1 || die "tmux (${TMUX_CLI}) not found (brew install tmux, or set TMUX_CLI=<path>)."
  case "$AGENT" in claude|codex) ;; *) die "The tmux backend supports only AGENT=claude|codex (idle is reported by agent hooks)." ;; esac
  [[ -n "$WORKTREE_DIR" ]] || WORKTREE_DIR="$HOME/.cache/ticket-loop/${OWNER_REPO//\//-}/worktrees"
  claude_trust_check
  if [[ -z "$TMUX_SESSION" ]]; then
    if [[ -n "${TMUX:-}" ]]; then TMUX_SESSION="$("$TMUX_CLI" display -p '#S')"; else TMUX_SESSION="ticket-loop"; fi
  fi
  TMUX_DIR="$HOME/.cache/ticket-loop/${OWNER_REPO//\//-}/tmux"
  TMUX_STATE_FILE=""
  mkdir -p "$TMUX_DIR/state"
  tmux_write_hook
  case "$AGENT" in
    claude) AGENT_RESTART_CMD="$AGENT_RESTART_CMD --settings $(printf %q "$TMUX_DIR/claude-hooks.json")" ;;
    codex)  AGENT_RESTART_CMD="$AGENT_RESTART_CMD -c $(printf %q "notify=[\"bash\",\"$TMUX_DIR/hook.sh\",\"$TMUX_DIR/state\"]")" ;;
  esac
  log "tmux session ${TMUX_SESSION} — view worker windows with '${TMUX_CLI} attach -t ${TMUX_SESSION}'"
}

# 훅 스크립트와 Claude Code 훅 설정. 루프가 시작할 때마다 다시 쓴다(키트 버전을 따라간다)
tmux_write_hook() {
  cat > "$TMUX_DIR/hook.sh" <<'SH'
#!/usr/bin/env bash
# ticket-loop.sh tmux 백엔드의 에이전트 훅 — 루프가 시작할 때마다 다시 쓴다. 손으로 고치지 않는다.
# 에이전트가 보고한 상태를 <state 디렉터리>/<워크트리 디렉터리 이름> 에 한 단어로 쓴다: running ¦ idle ¦ needsInput ¦ exited
# Claude Code: 훅 입력 JSON을 stdin으로 준다(hook_event_name, notification_type, cwd). Codex: notify가 JSON을 둘째 인자로 준다(type, cwd)
dir="$1"; payload="${2:-}"
[[ -n "$payload" ]] || payload="$(cat)"
{ read -r ev; read -r nt; read -r cwd; } < <(jq -r '(.hook_event_name // .type // "-"), (.notification_type // "-"), (.cwd // "-")' <<<"$payload" 2>/dev/null)
case "${ev:-}" in
  UserPromptSubmit) s=running ;;
  Stop|agent-turn-complete) s=idle ;;
  Notification) case "${nt:-}" in permission_prompt|elicitation_dialog) s=needsInput ;; idle_prompt) s=idle ;; *) exit 0 ;; esac ;;
  SessionEnd) s=exited ;;
  *) exit 0 ;;
esac
[[ -n "${cwd:-}" && "$cwd" != "-" ]] || exit 0
printf '%s\n' "$s" > "$dir/$(basename "$cwd")"
exit 0
SH
  chmod +x "$TMUX_DIR/hook.sh"
  jq -n --arg cmd "bash $(printf %q "$TMUX_DIR/hook.sh") $(printf %q "$TMUX_DIR/state")" '
    def h: [{hooks: [{type: "command", command: $cmd}]}];
    {hooks: {UserPromptSubmit: h, Stop: h, Notification: h, SessionEnd: h}}' > "$TMUX_DIR/claude-hooks.json"
}

tmux_state() { cat "$TMUX_STATE_FILE" 2>/dev/null || true; }
tmux_alive() { "$TMUX_CLI" display -p -t "$1" '#{pane_id}' >/dev/null 2>&1; }

tmux_worktree_create() {
  local n="$1" prompt="$2" name="${3:-${BRANCH_PREFIX}${1}}" out pf
  loop_worktree_add "$n" "$name"
  TMUX_STATE_FILE="$TMUX_DIR/state/$(basename "$WT_PATH")"
  printf 'running\n' > "$TMUX_STATE_FILE"   # 지난 실행의 idle이 남아 있으면 기동 직후를 idle로 오판한다
  if "$TMUX_CLI" has-session -t "=${TMUX_SESSION}" 2>/dev/null; then
    out="$("$TMUX_CLI" new-window -d -t "=${TMUX_SESSION}:" -n "$name" -c "$WT_PATH" -P -F '#{window_id} #{pane_id}')" \
      || die "tmux new-window failed"
  else
    out="$("$TMUX_CLI" new-session -d -s "$TMUX_SESSION" -n "$name" -c "$WT_PATH" -x 200 -y 50 -P -F '#{window_id} #{pane_id}')" \
      || die "tmux new-session failed"
  fi
```

```
  WT="${out%% *}"; HANDLE="${out##* }"
  [[ "$WT" == @* && "$HANDLE" == %* ]] || die "No tmux window id returned: ${out}"
  pf="$(prompt_file "$n" "$prompt")"
  sleep 1   # 셸이 뜨기를 잠깐 기다린다
  tmux_type "$HANDLE" "$(agent_launch_cmd "$pf")" || die "tmux send-keys failed"
}

tmux_worktree_remove() {
  "$TMUX_CLI" kill-window -t "$1" >/dev/null 2>&1 || true
  git -C "$ROOT" worktree remove --force "$2"
  # 창을 닫으면 에이전트의 SessionEnd 훅이 뒤늦게 exited를 쓸 수 있어(실측) 파일이 남기도 한다. 다음 티켓은 만들 때 running으로 덮으니 무해하다
  rm -f "$TMUX_DIR/state/$(basename "$2")"
}

tmux_type()      { "$TMUX_CLI" send-keys -t "$1" -l -- "$2" && sleep 0.3 && "$TMUX_CLI" send-keys -t "$1" Enter; }
tmux_interrupt() { "$TMUX_CLI" send-keys -t "$1" C-c; }

tmux_send() {
  local handle="$1" i
  [[ -n "$TMUX_STATE_FILE" ]] || return 1
  printf 'sent\n' > "$TMUX_STATE_FILE"
  tmux_type "$handle" "$2" || return 1
  if [[ "$AGENT" != claude ]]; then printf 'running\n' > "$TMUX_STATE_FILE"; return 0; fi   # 제출 훅이 없는 에이전트
  for (( i = 0; i < SEND_ACCEPT_SEC * 2; i++ )); do   # UserPromptSubmit 훅이 running을 쓰면 수락된 것
    [[ "$(tmux_state)" != running ]] || return 0
    sleep 0.5
  done
  return 1
}

tmux_wait_idle() {
  local handle="$1" deadline_ms=$(( $(date +%s) * 1000 + $2 )) fails=0
  while :; do
    if tmux_alive "$handle"; then
      fails=0
      case "$(tmux_state)" in idle|needsInput|exited) return 0 ;; esac   # 그 밖(running·sent·없음)은 작업 중
    else   # 창이 닫혔거나 tmux 서버가 끝났다 — 워커도 함께 끝나므로 응답 없음을 기다리지 않는다(#61)
      fails=$((fails + 1)); [[ $fails -lt $TERMINAL_RETRY ]] || { reach_lost "$handle" terminal; return 3; }
    fi
    [[ $fails -gt 0 ]] || $(( $(date +%s) * 1000 )) -lt $deadline_ms ]] || return 1   # 연달아 실패하는 중에는 돌아오지 않는다(cmux_wait_idle과 같다)
    sleep "$IDLE_POLL_SEC"
  done
}

# 화면의 빈 줄을 빼고 마지막 25줄 — TUI는 창 높이를 다 쓰므로 빈 줄이 많다
tmux_tail() { "$TMUX_CLI" capture-pane -p -t "$1" | sed -e 's/[[:space:]]*$//' | awk 'NF' | tail -n 25; }

tmux_set_status() {
  local wt="$1" status="$2" name lane
  case "$status" in in-progress) lane=working ;; in-review) lane=review ;; completed) lane=done ;; *) lane=needs-attention ;; esac
  name="$("$TMUX_CLI" display -p -t "$wt" '#{window_name}' 2>/dev/null)" || return 0
  "$TMUX_CLI" rename-window -t "$wt" "${name%% *} [${lane}]" >/dev/null 2>&1 || true
}

# ----- 공통 -----

terminal_idle() { backend_wait_idle "$@"; }
terminal_tail() { backend_tail "$@"; }
send_prompt() { backend_send "$1" "$2" || die "Prompt was not accepted (terminal $1)"; }

worker_prompt() {
  # 한 줄로 보낸다 — TUI에서는 줄바꿈이 곧 제출이다.
  # 절차(선행 확인·브랜치·seam·리뷰·PR 본문·라벨·병합 금지)는 저장소의 implement 스킬이 단일 진실이다.
  # 여기에는 무인 실행에서만 참인 것만 둔다 — 질문 금지, 막히면 코멘트 후 멈춤, 규칙 파일. 특정 에이전트의 도구·명령 이름은 쓰지 않는다(SKILL_PREFIX만 다르다).
  local n="$1" rules=""
  [[ ! -f "$ROOT/$WORKER_RULES_FILE" ]] || rules=" 저장소의 ${WORKER_RULES_FILE} 규칙도 따라라."
  printf '%s' "${SKILL_PREFIX}implement ${n} — 무인 루프(ticket-loop.sh)에서 실행 중이다. 사람이 답할 수 없으니 사람에게 묻거나 확인을 구하지 말고(질문 도구도 쓰지 말고) 스스로 결정해 끝까지 진행해라. PR은 병합하지 마라 — 루프가 CI 뒤에 병합한다. 진행이
```

```
불가능하면(열린 선행 이슈 포함) 이슈 #${n}에 이유를 코멘트로 남기고 PR 없이 멈춰라.${rules}"
}

# 에이전트가 실제로 떴는지 확인한다. Orca가 셸에 기동 명령을 넣을 때 앞 글자가 빠져
# (`laude …`) 셸 프롬프트에 멈춘 실측이 있다(4연속). 배너가 안 보이면 입력 줄을 지우고 한 번 다시 띄운다.
agent_banner_seen() { terminal_tail "$1" 2>/dev/null ¦ grep -Eq "$AGENT_BANNER_REGEX"; }

# $3 다시 띄울 때 줄 프롬프트(기본 워커 프롬프트 — 무인 수습 세션은 자기 프롬프트를 준다)
ensure_agent_started() {
  local n="$1" handle="$2" prompt="${3:-}" i attempt promptfile
  [[ -n "$prompt" ]] ¦¦ prompt="$(worker_prompt "${n}")"
  for attempt in 1 2; do
    for i in $(seq 1 $(( AGENT_START_WAIT_SEC / 3 ))); do
      agent_banner_seen "${handle}" && return 0
      sleep 3
    done
    [[ $attempt -eq 1 ]] ¦¦ break
    log warn "#${n}: agent did not start — relaunching in the terminal"
    promptfile="$(prompt_file "${n}" "$prompt")"
    backend_interrupt "${handle}" ¦¦ true   # Ctrl-C로 입력 줄 비움
    sleep 1
    backend_type "${handle}" "$(agent_launch_cmd "${promptfile}")" ¦¦ die "#${n}: failed to send the relaunch command"
  done
  die "#${n}: agent did not start in terminal ${handle}"
}

# ----- 진행 로그 -----
# 워크트리를 만든 뒤 PR이 열리기까지(보통 수십 분) 루프 터미널에는 아무 줄도 없어 멈춘 것처럼 보였다 — 워커는 다른 창에서 일한다.
# 그 사이 PROGRESS_SEC마다 워크트리를 git으로만 들여다본다(백엔드·에이전트와 무관하고 네트워크도 안 쓴다):
#   브랜치가 바뀌면(워커가 issue-N → issue-N-<slug>로 옮김. 옮기지 않고 issue-N에 그대로 커밋해도 PR은 open_pr_for가 잡는다), 처음 파일이 수정되면, 새 커밋이 들어오면(제목 그대로, 오래된 것부터) 바로 한 줄.
#   커밋 줄은 워커가 만든 것만 — origin/<BASE_BRANCH>에서 닿는 커밋은 뺀다. 워커가 더 새로운 기본 브랜치에서 브랜치를 따 HEAD가 기본 브랜치 안에서만
#   옮겨 갔으면 "base moved to <sha>" 한 줄(그 사이 병합된 남의 커밋이 워커 커밋처럼 보이지 않게). 원격 추적 ref가 없으면 전처럼 범위의 커밋을 다 찍는다
#   그 밖에는 커밋 안 된 변경 파일 목록이 지난 줄과 달라졌을 때만, HEARTBEAT_SEC에 한 번까지 "작업 중" 한 줄에 경과 시간과 파일(이름 셋까지)을
#   적는다 — 이슈를 읽고 TDD로 도는 첫 수십 분은 커밋이 없어도 무엇을 만지는지 보인다. 같은 내용은 되풀이하지 않는다(변화가 없으면 조용하다).
#   화면을 읽어 도구 호출을 뽑는 쪽은 에이전트마다 형식이 달라 쓰지 않는다
# wait_for_pr는 백그라운드 서브셸(bg_wait)로 불려 그 안에서 바뀐 PROG_*가 부모에 남지 않는다 — run_ticket이 전에 progress_reset을,
# PR이 열린 뒤 progress_sync를 불러 수리 단계의 progress_check가 이미 찍은 커밋을 되풀이하지 않게 한다
progress_reset() {   # $1 워크트리 경로. 티켓마다 한 번
  PROG_WT="$1"; PROG_START="$(date +%s)"; PROG_LAST_LOG="$PROG_START"; PROG_DIRTY_SEEN=0
  progress_sync
}

progress_sync() {   # 지금 HEAD·브랜치를 기준점으로(경과 시간은 그대로)
  PROG_HEAD="$(git -C "$PROG_WT" rev-parse HEAD 2>/dev/null ¦¦ true)"
  PROG_BRANCH="$(git -C "$PROG_WT" rev-parse --abbrev-ref HEAD 2>/dev/null ¦¦ true)"
  PROG_LAST_LOG="$(date +%s)"; PROG_LAST_FILES="$(progress_dirty)"
  [[ -z "$PROG_LAST_FILES" ]] ¦¦ PROG_DIRTY_SEEN=1
}

progress_dirty() {   # 커밋 안 된 변경 파일 경로(한 줄에 하나, 이름 바꿈은 새 이름). 무시 파일은 빠진다
  git -C "$PROG_WT" status --porcelain --untracked-files=all 2>/dev/null ¦ sed -e 's/^...//' -e 's/.* -> //'
}

progress_check() {   # $1 티켓 번호. 바뀐 것이 있으면 한 줄씩, 없으면 HEARTBEAT_SEC마다 한 줄
  local n="$1" now head base branch commits line files cnt names noted=0
  [[ -n "${PROG_WT:-}" && -d "$PROG_WT" ]] ¦¦ return 0
  now="$(date +%s)"
  branch="$(git -C "$PROG_WT" rev-parse --abbrev-ref HEAD 2>/dev/null ¦¦ true)"
  if [[ -n "$branch" && "$branch" != "$PROG_BRANCH" ]]; then
    log step "#${n}: branch ${branch}"; PROG_BRANCH="$branch"; noted=1
  fi
  head="$(git -C "$PROG_WT" rev-parse HEAD 2>/dev/null ¦¦ true)"
  if [[ -n "$head" && "$head" != "$PROG_HEAD" ]]; then
    # 기본 브랜치의 원격 추적 ref. 워크트리는 메인 체크아웃과 ref를 공유하므로 워커의 git fetch가 갱신한 값을 그대로 쓴다(루프는 fetch하지 않는다).
```

```
    # 없으면(로컬 전용 저장소 등) 빈 값 — 아래는 전과 같이 동작한다
    base=""; [[ -z "${BASE_BRANCH:-}" ]] || base="$(git -C "$PROG_WT" rev-parse --verify -q "refs/remotes/origin/${BASE_BRANCH}" 2>/dev/null || true)"
    if [[ -n "$base" ]] && git -C "$PROG_WT" merge-base --is-ancestor "$head" "$base" 2>/dev/null; then
      # HEAD가 기본 브랜치 안에서만 옮겨 갔다(워커가 더 새로운 origin/<base>에서 브랜치를 땄다) — 그 사이 병합된 남의 커밋을 워커 커밋처럼 찍지 않는다
      log step "#${n}: base moved to $(git -C "$PROG_WT" rev-parse --short "$head" 2>/dev/null || echo "${head:0:7}")"
    else
      # 지난 HEAD 이후의 커밋들 — 기본 브랜치에서 닿는 커밋은 뺀다. HEAD가 뒤로 갔거나(amend·reset) 지난 값이 없으면 지금 커밋 하나만
      commits="$(git -C "$PROG_WT" log --reverse --format='%h %s' "${PROG_HEAD:-$head}..${head}" ${base:+--not "$base"} 2>/dev/null || true)"
      [[ -n "$commits" ]] || commits="$(git -C "$PROG_WT" log -1 --format='%h %s' "$head" 2>/dev/null || true)"
      while IFS= read -r line; do [[ -z "$line" ]] || log step "#${n}: commit ${line}"; done <<<"$commits"
    fi
    PROG_HEAD="$head"; noted=1
  fi
  files="$(progress_dirty)"
  cnt=0; [[ -z "$files" ]] || cnt="$(printf '%s\n' "$files" | grep -c .)"
  names="$(printf '%s\n' "$files" | head -n 3 | paste -sd ',' - | sed 's/,/, /g')"
  [[ $cnt -le 3 ]] || names="${names} +$((cnt - 3)) more"
  if [[ $PROG_DIRTY_SEEN -eq 0 && $cnt -gt 0 ]]; then
    log step "#${n}: first file changes — ${names}"; PROG_DIRTY_SEEN=1; noted=1
  fi
  # 심장박동: 변경 파일 목록이 지난 줄과 다를 때만. 같은 목록(또는 빈 목록)이면 시간이 아무리 지나도 되풀이하지 않는다
  if [[ $noted -eq 0 && $cnt -gt 0 && "$files" != "$PROG_LAST_FILES" && $(( now - PROG_LAST_LOG )) -ge $HEARTBEAT_SEC ]]; then
    log "#${n}: working — $(( (now - PROG_START) / 60 ))m elapsed, ${cnt} uncommitted file$([[ $cnt -eq 1 ]] || echo s): ${names}"; noted=1
  fi
  [[ $noted -eq 0 ]] || { PROG_LAST_LOG="$now"; PROG_LAST_FILES="$files"; }
}

# idle을 한 번 기다리는 길이. 진행 로그를 켜 두면 PROGRESS_SEC마다 돌아와 워크트리를 본다
idle_wait_ms() { if [[ "$PROGRESS_SEC" -gt 0 ]]; then echo $(( PROGRESS_SEC * 1000 )); else echo "$IDLE_WAIT_MS"; fi; }
# ----- 진행 로그 끝 -----

# idle 오판 $1번을 센다(idle_fp_count). 오판은 일하던 워커를 idle로 본 것이다 — 그 뒤 워커가 다시 일했거나(화면이 바뀜·idle이 풀림)
# PR을 열었다
idle_fp_add() { local i; for (( i = 0; i < $1; i++ )); do echo >> "${IDLE_FP_FILE:-/dev/null}"; done; }

# idle로 보이는 워커가 실제로는 일하는 중인지를 Claude Code 세션 기록으로 본다 — $1 작업 폴더(워커의 WT_PATH, 수습 세션의 워크트리)의
# 세션 기록 폴더(claude_project_dir — 토큰 집계와 같은 변환이라 Orca처럼 WORKTREE_DIR 밖의 워크트리도 맞는다) 아래 *.jsonl(하위
# subagents/ 포함) 중 가장 최근 것이 SESSION_LOG_FRESH_SEC 안에 갱신됐으면 0. AGENT가 claude가 아니거나 폴더·파일이 없으면 1 — 호출부는
# idle 판정을 그대로 쓴다. 갱신 뒤 지난 초를 SESSION_LOG_AGE에 남긴다(로그용). 세 대기(wait_for_pr·repair·wait_for_verdict)가 strike를
# 세기 직전에 부른다. cmux에서 백그라운드 작업 알림으로 시작되는 턴 전후에 lifecycle이 잠깐 idle로 보이는 것으로 추정된다 — 2026-10-22
# rental-portal #266에서 strike 3회 중 앞의 2회는 워커가 도구를 7번 호출하던 중이었다. 진짜 idle(백그라운드 verify를 기다리며 턴이 끝난
# 상태 등)은 기록이 갱신되지 않아 전처럼 센다 — 이 가드는 strike를 줄이기만 하고 늘리지 않는다. mtime은 stat으로 읽는다(macOS -f %m,
# Linux -c %Y). Codex는 세션 기록 형식이 달라 아직 보지 않는다
session_active_recently() {
  local dir newest fmt
  SESSION_LOG_AGE=""
  [[ "${AGENT:-}" == claude && -n "${1:-}" ]] || return 1
  dir="$(claude_project_dir "$1")"
  [[ -d "$dir" ]] || return 1
  if stat -f %m "$dir" >/dev/null 2>&1; then fmt=(-f %m); else fmt=(-c %Y); fi
  newest="$(find "$dir" -type f -name '*.jsonl' -exec stat "${fmt[@]}" {} + 2>/dev/null | sort -n | tail -n 1)"
  [[ -n "$newest" ]] || return 1
  SESSION_LOG_AGE=$(( $(date +%s) - newest ))
  [[ "$SESSION_LOG_AGE" -le "$SESSION_LOG_FRESH_SEC" ]]
}

# ----- 화면 분류 -----
# 세 대기(wait_for_pr·repair·wait_for_verdict)가 strike를 세기 직전 — session_active_recently 가드 뒤(일하는 중이면 분류할 필요가 없다) — 와
# 백엔드 idle 대기가 타임아웃으로 돌아온 바퀴마다 워커 화면 끝 25줄(terminal_tail)을 본다. 두 가지를 가른다:
```

```
#   usage_limit       사용량 한도 문구(USAGE_LIMIT_REGEX, 대소문자 무시). idle이면 strike 없이 바로 멈춘다. 무인 수습은 띄우지 않는다 — 수습
#                     세션도 같은 한도에 걸린다(2026-10 실측 rental-portal #262: 한도 문구를 일반 idle로 보고 strike 셋 뒤 worker_no_pr로 멈춰 수습
#                     세션까지 헛돌았다. 2026-11에 Claude Code 문구가 바뀌어 같은 일이 다시 났다 — #52).
#                     사람이 터미널에서 잇거나, 한도가 풀린 뒤 루프를 다시 돌려 새 워커로 시작한다. Claude Code는 재설정 시각에 워커를 스스로 이어 가
#                     루프 밖에서 PR을 열 수 있다(#52 실측) — 루프는 열린 PR을 이어받지 않으므로 멈춤 사유가 다시 돌리기 전에 그 PR을 보라고 적는다
#   permission_prompt 권한 확인 창 문구(PERMISSION_PROMPT_REGEX). 그 문구가 든 화면이 PROMPT_STALL_SEC 동안 글자 그대로(체크섬)면 idle 신호와 무관하게
#                     확인 창에 멈춘 워커로 보고 worker_no_pr(진단만 하는 수습)로 멈춘다. idle 신호가 살아 있어 strike가 먼저 차면 전과 같은 경로다.
#                     idle 신호가 빈 값으로 이어지는 백엔드 상태(#2)에서도 닿는 경로다 — 2026-10 실측(#3, tmux)에서 bypass 모드의 워커(서브에이전트의
#                     Bash)가 "Do you want to proceed?"에 27분 멈춰 있는 동안 루프 로그는 한 줄도 없었다. 루프는 확인 창에 답하지 않는다(승인·거부
#                     둘 다) — bypass 모드에서도 물었다는 것은 deny 규칙 때문일 수 있다
# 빈 정규식은 그 종류를 감지하지 않는다(codex 프리셋 — 실측 전). 분류 결과는 멈춤 기록의 kind와 사유에만 남는다(새 필드 없음).
#
# 대기 함수가 "멈춤"의 종류·사유를 호출부에 넘기는 길: wait_for_pr·wait_for_verdict는 bg_wait 서브셸에서 돌아 변수가 부모에 남지 않으므로
# WAIT_STOP_FILE("종류<탭>사유" 한 줄)을 거친다. 호출부는 대기 전에 wait_stop_reset(파일은 부모가 만든다), 대기 뒤 wait_stop_read로
# WAIT_STOP_KIND·WAIT_STOP_REASON을 채운다. 비어 있으면 전처럼 idle strike로 멈춘 것이다
WAIT_STOP_KIND=""; WAIT_STOP_REASON=""; WAIT_STOP_FILE=""
wait_stop_reset() {
  WAIT_STOP_KIND=""; WAIT_STOP_REASON=""
  [[ -n "$WAIT_STOP_FILE" ]] || WAIT_STOP_FILE="$(mktemp "${TMPDIR:-/tmp}/ticket-loop-stop.XXXXXX")"
  : > "$WAIT_STOP_FILE"
}
wait_stop() { WAIT_STOP_KIND="$1"; WAIT_STOP_REASON="$2"; [[ -z "$WAIT_STOP_FILE" ]] || printf '%s\t%s\n' "$1" "$2" > "$WAIT_STOP_FILE"; }
wait_stop_read() {
  [[ -n "$WAIT_STOP_FILE" && -s "$WAIT_STOP_FILE" ]] || return 0
  IFS=$'\t' read -r WAIT_STOP_KIND WAIT_STOP_REASON < "$WAIT_STOP_FILE" || true
  : > "$WAIT_STOP_FILE"
}

# screen_kind 터미널 → usage_limit | permission_prompt | 빈 값을 stdout에 쓰고 SCREEN_KIND에도 둔다(커맨드 치환 없이 불러야 아래 값이 남는다).
# 맞은 줄은 SCREEN_LINE(앞뒤 공백을 지운 것 — 멈춤 사유에 싣는다), 화면 체크섬은 SCREEN_SUM(확인 창 체류 시간용), 사용량 한도면 화면의
# 재설정 시각 문구("resets at …", "try again at …")를 SCREEN_RESET에 둔다(맞은 줄에 재설정 문구가 이미 있으면 비운다 — 화면의 다른 줄, 예컨대
# 서브에이전트의 API 오류 줄에서 뽑으면 같은 시각을 오류 꼬리와 함께 되풀이한다, #52). 두 문구가 다 있으면 한도가 먼저다
SCREEN_KIND=""; SCREEN_LINE=""; SCREEN_SUM=""; SCREEN_RESET=""
screen_trim() { sed -e 's/^[[:space:]]*\(|\)\{0,1\}[[:space:]]*//' -e 's/[[:space:]]*\(|\)\{0,1\}[[:space:]]*$//'; }   # 앞뒤 공백과 TUI 상자 테두리(|)
screen_kind() {
  local tail line reset_re='(resets?|try again)( at| in| on)?[[:space:]]+'
  SCREEN_KIND=""; SCREEN_LINE=""; SCREEN_SUM=""; SCREEN_RESET=""
  [[ -n "${USAGE_LIMIT_REGEX:-}" || -n "${PERMISSION_PROMPT_REGEX:-}" ]] || return 0
  tail="$(terminal_tail "$1" 2>/dev/null)" || true
  [[ -n "$tail" ]] || return 0
  SCREEN_SUM="$(cksum <<<"$tail" | cut -d' ' -f1)"
  if [[ -n "${USAGE_LIMIT_REGEX:-}" ]]; then
    # 대소문자를 가리지 않는다 — 한도 문구는 문장 첫머리("Usage limit reached")로도 나온다(#52). 아래 재설정 시각 추출과 같은 기준
    line="$(grep -iE "$USAGE_LIMIT_REGEX" <<<"$tail" | tail -n 1 || true)"
    if [[ -n "$line" ]]; then
      SCREEN_KIND=usage_limit; SCREEN_LINE="$(screen_trim <<<"$line")"
      grep -iqE "$reset_re" <<<"$SCREEN_LINE" \
        || SCREEN_RESET="$(grep -iEo "${reset_re}[^|]*" <<<"$tail" | head -n 1 | screen_trim || true)"
    fi
  fi
  if [[ -z "$SCREEN_KIND" && -n "${PERMISSION_PROMPT_REGEX:-}" ]]; then
    line="$(grep -E "$PERMISSION_PROMPT_REGEX" <<<"$tail" | tail -n 1 || true)"
    [[ -z "$line" ]] || { SCREEN_KIND=permission_prompt; SCREEN_LINE="$(screen_trim <<<"$line")"; }
  fi
  printf '%s' "$SCREEN_KIND"
}
```

```
# 확인 창 체류 시간 — $1 지금 화면의 체크섬. 처음 본 시각(PROMPT_AT)과 그때의 화면(PROMPT_SUM)을 두고, 화면이 바뀌면 다시 센다.
# 같은 화면이 PROMPT_STALL_SEC 이상 머물렀으면 0. 대기 함수가 시작할 때 PROMPT_SUM=""로 비운다
PROMPT_SUM=""; PROMPT_AT=0
prompt_stalled() {
  local now; now="$(date +%s)"
  if [[ "$1" != "$PROMPT_SUM" ]]; then PROMPT_SUM="$1"; PROMPT_AT="$now"; return 1; fi
  [[ $(( now - PROMPT_AT )) -ge $PROMPT_STALL_SEC ]]
}

# 대기 한 바퀴의 화면 확인 — $1 터미널, $2 사유의 주어(worker ¦ triage session), $3 idle 판정이면 1(세션 기록 가드 뒤에서만 부른다), 타임아웃으로
# 돌아온 바퀴면 0. 멈출 일이면 wait_stop을 채우고 0을 돌려준다(대기 함수는 "멈춤"으로 돌아간다). 아니면 1 — 대기 함수는 전처럼 간다
screen_check() {
  local handle="$1" who="$2" idle="${3:-0}"
  screen_kind "$handle" >/dev/null
  case "$SCREEN_KIND" in
    usage_limit)
      [[ $idle -eq 1 ]] ¦¦ return 1   # 한도는 idle로 확인된 화면에서만 — 일하는 워커의 화면에 그 낱말이 지나갈 수 있다
      wait_stop usage_limit "${who} hit the usage limit: ${SCREEN_LINE}${SCREEN_RESET:+ — ${SCREEN_RESET}}"; return 0 ;;
    permission_prompt)
      prompt_stalled "$SCREEN_SUM" ¦¦ return 1
      wait_stop worker_no_pr "${who} is waiting at a confirmation prompt: ${SCREEN_LINE} (screen unchanged for $(( $(date +%s) - PROMPT_AT ))s)"; return 0 ;;
    *) PROMPT_SUM=""; return 1 ;;   # 확인 창이 사라졌다 — 다시 보이면 처음부터 센다
  esac
}

# 대기가 화면 분류로 멈췄으면 여기서 멈춘다(die) — $1 티켓, $2 사유에 덧붙일 맥락(선택, 예 " while repairing PR #34"). 사용량 한도는 수습 없이
# usage_limit로(수습 세션도 같은 한도에 걸린다 — 멈춤 기록은 stop_report가 manual로 쓴다), 확인 창은 전의 worker_no_pr처럼 진단만 하는 수습 뒤
# 멈춘다. 분류가 없으면 0 — 호출부가 전처럼 간다(idle strike → worker_no_pr, 수리 끝 → head 비교)
screen_stop_check() {
  local n="$1" ctx="${2:-}" reason
  wait_stop_read
  case "$WAIT_STOP_KIND" in
    usage_limit)
      STOP_KIND=usage_limit
      die "#${n}: ${WAIT_STOP_REASON}${ctx}. No triage (it would hit the same limit) — rerun the loop after the limit resets. The ${BRANCH_PREFIX}${n} worktree is kept in ${BACKEND_NAME}. The worker may resume on its own when the limit resets and open a PR outside the loop — check for one before you rerun (the loop does not take over an open PR)." ;;
    worker_no_pr)
      reason="#${n}: ${WAIT_STOP_REASON}${ctx}. The loop does not answer prompts — check the ${BRANCH_PREFIX}${n} worktree in ${BACKEND_NAME}."
      STOP_KIND=worker_no_pr
      triage worker_no_pr "$reason" ¦¦ true   # 진단만 — 요약을 사유에 붙여 멈춘다
      die "${reason}${TRIAGE_NOTE}" ;;
  esac
  return 0
}
# ----- 화면 분류 끝 -----

# PR을 찾은 뒤 워커가 이슈 정리(implement 스킬의 마지막 단계 — 체크박스, 맨 끝에 IN_REVIEW_LABEL)를 마치기를 기다린다. 이슈에 그 라벨이
# 붙었거나 워커가 idle이면 끝, 아니면 PR_SETTLE_SEC에서 넘어간다. 기다림은 남긴다 — CI 경로 필터로 체크 없이 바로 병합하는 PR은 워커보다
# 먼저 병합·이슈 닫힘·라벨 제거가 일어난다. 0.9.1까지는 idle 하나만 IDLE_WAIT_MS(10분)까지 기다렸다 — rental-portal 스펙 #141(cmux)에서
# lifecycle이 빈 값으로 이어져 13건 중 3건이 10분을 다 썼고, CI는 그사이 끝나 2~3초로 찍혔다. 세 건 모두 라벨은 PR 생성 8~12초 뒤에 붙어 있었다
pr_settle() {
  local n="$1" handle="$2" until=$(( $(date +%s) + PR_SETTLE_SEC )) rc
  while :; do
    ! issue_has_label "$n" "$IN_REVIEW_LABEL" ¦¦ return 0
    rc=0; terminal_idle "$handle" 10000 ¦¦ rc=$?
    [[ $rc -ne 0 && $rc -ne 3 ]] ¦¦ return 0   # idle, 또는 터미널에 닿지 못함(기다려도 소용없다)
    if [[ "$(date +%s)" -ge "$until" ]]; then
      log warn "#${n}: no ${IN_REVIEW_LABEL} label on the issue and the worker not idle ${PR_SETTLE_SEC}s after finding the PR — moving on"
      return 0
```

```
    fi
  done
}

# PR이 생길 때까지(또는 워커가 멈출 때까지) 기다린다. 성공 시 PR 번호를 stdout에 쓴다.
# 반환: 0 PR 열림, 1 PR 없이 멈춤, 2 타임아웃, 3 백엔드 터미널 끊김
# idle 오판(idle_fp_add)은 PR 없이 센 idle 뒤에 워커가 다시 일한 것, 그리고 idle을 잡은 뒤 열린 PR이다. 0.9.1까지는 "idle 뒤 다시 보니
# PR이 열려 있음"을 다 셌는데, 대개 일을 마친 워커였다(PR을 연 뒤 idle) — 폴링 순서의 부산물이지 오판이 아니다
wait_for_pr() {
  local n="$1" handle="$2" deadline="$3" strikes=0 pending=0 pr started idle idle_at created last_screen="" screen wait_ms acti
ve_said=0
  reach_reset "$handle"
  started="$(date +%s)"; wait_ms="$(idle_wait_ms)"; PROMPT_SUM=""; PROMPT_AT=0
  while :; do
    pr="$(open_pr_for "${n}")"
    if [[ -n "${pr}" ]]; then
      idle_fp_add "$pending"   # 앞서 센 idle 뒤에 워커가 PR까지 갔다
      pr_settle "${n}" "${handle}"
      [[ "$PROGRESS_SEC" -le 0 ]] || progress_check "${n}"
      echo "${pr}"; return 0
    fi
    [[ "$(date +%s)" -lt "$deadline" ]] || return 2
    idle=0; terminal_idle "${handle}" "$wait_ms" || idle=$?
    [[ $idle -ne 3 ]] || return 3
    [[ "$PROGRESS_SEC" -le 0 ]] || progress_check "${n}"
    if [[ $idle -eq 0 ]]; then
      idle_at="$(date +%s)"
      if [[ $(( idle_at - started )) -lt $STARTUP_GRACE_SEC ]]; then sleep "$IDLE_RECHECK_SEC"; continue; fi
      # PR을 연 직후에는 idle이 PR 조회보다 먼저 잡힌다(매 티켓 실측) — 잠깐 뒤 PR부터 다시 본다. PR이 idle보다 먼저 열렸으면
일을 마친
      # 워커다. idle을 잡은 뒤에 열렸으면 그 idle은 일하던 워커를 본 오판이다
      sleep "$IDLE_RECHECK_SEC"
      pr="$(open_pr_for "${n}")"
      if [[ -n "$pr" ]]; then
        created="$(pr_created_epoch "$pr")"
        [[ -z "$created" || "$created" -le "$idle_at" ]] || idle_fp_add 1
        continue
      fi
      # 지난 idle 이후 화면이 바뀌었으면 그사이 일했다는 뜻 — 앞서 센 idle은 오판이고, 연속 횟수를 새로 센다
      screen="$(terminal_tail "${handle}" | cksum | cut -d' ' -f1)"
      if [[ "$screen" != "$last_screen" ]]; then idle_fp_add "$pending"; pending=0; strikes=0; last_screen="$screen"; fi
      # 세션 기록이 방금 갱신됐으면 일하던 워커를 idle로 본 것 — 이번 idle과 앞서 센 idle은 오판이고 strike를 세지 않는다(화면
이 바뀐
      # 경우와 같은 처리). 로그는 연속된 idle 판정에서 한 번만
      if session_active_recently "${WT_PATH:-}"; then
        [[ $active_said -eq 1 ]] || log "#${n}: idle reported, but the session log was written ${SESSION_LOG_AGE}s ago — still
working"
        idle_fp_add $((pending + 1)); pending=0; strikes=0; active_said=1
        sleep "$IDLE_RECHECK_SEC"; continue
      fi
      active_said=0
      # 화면 분류(screen_check): 사용량 한도면 strike 없이 바로, 확인 창에 PROMPT_STALL_SEC 머물렀으면 worker_no_pr로 — 둘 다 P
R 없이 멈춤(1)
      ! screen_check "${handle}" worker 1 || return 1
      strikes=$((strikes + 1)); pending=$((pending + 1))
      log warn "#${n}: worker idle, no PR (${strikes}/${IDLE_STRIKES})"
      [[ ${strikes} -lt $IDLE_STRIKES ]] || return 1   # 정말 멈췄다 — 이 연속 idle은 오판이 아니다
      sleep "$IDLE_RECHECK_SEC"
    else
      idle_fp_add "$pending"; pending=0; strikes=0; active_said=0   # 워커가 계속 일한다 — 앞서 센 idle은 오판
      ! screen_check "${handle}" worker 0 || return 1   # idle 신호가 없어도 확인 창에 멈춘 워커는 여기서 잡는다(타임아웃으로
돌아온 바퀴)
    fi
  done
}

# 수리 프롬프트를 보내고 워커가 다시 idle이 될 때까지 기다린다(그동안의 커밋도 진행 로그에 찍힌다). $4 PR 번호를 주면 두 가지를
더 본다.
#   - idle인데 PR에 새 커밋이 아직 없으면 바로 끝내지 않는다. 화면이 그대로인 idle을 IDLE_STRIKES번 연달아(그 사이 IDLE_RECHECK
_SEC) 볼
#     때만 끝이고, 호출부가 head를 비교해 "새 커밋 없음"으로 멈춘다. 그 전에 커밋이 들어오면 끝, 화면이 바뀌거나 워커가 다시 일
```

```
하면 처음부터
#     센다(wait_for_pr와 같은 규칙). 프롬프트 직후의 묵은 idle(2026-10-20 rental-portal #251, cmux — cmux_send 참고)과 백그라운
드
#     서브에이전트를 기다리는 동안의 idle을 수리 끝으로 읽지 않기 위해서다. 0.9.2까지는 첫 idle에서 끝냈다
#   - idle이 끝내 안 잡히는 백엔드(cmux lifecycle이 빈 값으로 이어질 때 — rental-portal 스펙 #141에서 하루 126번)에서도 끝을 알
아챈다:
#     수리 프롬프트 뒤 새 커밋이 push됐고 head가 REPAIR_SETTLE_SEC 동안 그대로이며 워크트리에 커밋 안 된 변경이 없으면 끝난 것
으로 본다.
#     0.9.1까지는 티켓 제한 시간까지 기다렸다
repair() {
  local handle="$1" text="$2" deadline="$3" pr="${4:-}" idle wait_ms before="" head="" seen="" seen_at=0 strikes=0 last_screen
="" screen active_said=0
  reach_reset "$handle"
  wait_ms="$(idle_wait_ms)"; PROMPT_SUM=""; PROMPT_AT=0
  [[ -z "$pr" ]] || before="$(pr_head_sha "$pr" 2>/dev/null || true)"
  send_prompt "${handle}" "$text"
  while :; do
    idle=0; bg_wait terminal_idle "${handle}" "$wait_ms" || idle=$?
    [[ $idle -eq 3 ]] || [[ "$PROGRESS_SEC" -le 0 ]] || progress_check "${CUR_N}"
    [[ $idle -ne 3 ]] || die "#${CUR_N}: cannot reach ${BACKEND_NAME} terminal ${handle}${pr:+ while repairing PR #${pr}}: $(te
rminal_lost_why "${handle}"). $(terminal_lost_note worker)"
    [[ -z "$before" ]] || head="$(pr_head_sha "$pr" 2>/dev/null || true)"
    if [[ $idle -eq 0 ]]; then
      # 새 커밋이 있으면 끝이다. PR 번호가 없거나 head를 못 읽으면 전처럼 idle 하나로 끝낸다
      [[ -n "$before" && -n "$head" && "$head" == "$before" ]] || return 0
      screen="$(terminal_tail "${handle}" | cksum | cut -d' ' -f1)"
      if [[ "$screen" != "$last_screen" ]]; then strikes=0; last_screen="$screen"; fi
      if session_active_recently "${WT_PATH:-}"; then   # 세션 기록이 방금 갱신됐으면 아직 일하는 중 — strike를 세지 않는다(wai
t_for_pr와 같은 규칙)
        [[ $active_said -eq 1 ]] || log "#${CUR_N}: idle reported, but the session log was written ${SESSION_LOG_AGE}s ago — st
ill working"
        strikes=0; active_said=1
      else
        active_said=0
        ! screen_check "${handle}" worker 1 || return 0   # 사용량 한도·확인 창 — 호출부(screen_stop_check)가 종류대로 멈춘다
        strikes=$((strikes + 1))
        log warn "#${CUR_N}: worker idle after the repair prompt, no new commit on PR #${pr} yet (${strikes}/${IDLE_STRIKES})"
        [[ ${strikes} -lt $IDLE_STRIKES ]] || return 0
      fi
      sleep "$IDLE_RECHECK_SEC"
    else
      strikes=0; active_said=0   # 워커가 다시 일한다
      ! screen_check "${handle}" worker 0 || return 0   # idle 신호가 없어도 확인 창에 멈춘 워커는 여기서 잡는다
      if [[ -n "$head" && "$head" != "$before" ]]; then
        [[ "$head" == "$seen" ]] || { seen="$head"; seen_at="$(date +%s)"; }
        if [[ $(( $(date +%s) - seen_at )) -ge $REPAIR_SETTLE_SEC && -z "$(progress_dirty)" ]]; then
          log warn "#${CUR_N}: the worker never went idle, but PR #${pr} has had a new commit (${head:0:7}) for ${REPAIR_SETTLE
_SEC}s+ and the worktree is clean — treating the repair as done"
          return 0
        fi
      fi
    fi
    [[ "$(date +%s)" -lt "$deadline" ]] || return 1
  done
}

# ---------- CI · 병합 ----------

wait_checks_appear() {
  local pr="$1" until=$(( $(date +%s) + CI_APPEAR_TIMEOUT_SEC ))
  while [[ "$(gh pr checks "${pr}" --json name --jq 'length' 2>/dev/null || echo 0)" == "0" ]]; do
    [[ "$(date +%s)" -lt "$until" ]] || return 1
    sleep 30
  done
}

# --watch는 GitHub API 오류 한 번에도 1로 끝난다 — 체크 실패와 같은 코드다. 그대로 실패로 읽으면 워커에게 할 일 없는 수리를 보
낸다.
# 1이면 체크 상태(bucket)를 다시 읽어 실패·취소가 있을 때만 실패, 다 통과했으면 통과, 아직 도는 체크가 있으면 다시 본다(GH_TRIE
S번까지).
# 체크 목록이 비었거나 못 읽었으면 실패가 아니라 재등록 중으로 본다 — GitHub가 job을 다시 큐에 넣는 순간(재시도·장애 복구) PR의
체크
```

```
# 목록이 잠깐 비고 --watch는 "no checks reported"로 끝난다(이슈 #24: 2026-10-23 rental-portal #316, run은 success인데 check-run 하나가
# 장애로 in_progress에 굳었다가 27분 뒤 attempt 2로 다시 돌았다 — 0.9.12는 그 빈 목록을 실패로 읽어 할 일 없는 수리를 보냈다).
# 그래서 빈 목록이면 체크가 다시 나타나기를 기다린 뒤(wait_checks_appear, CI_APPEAR_TIMEOUT_SEC) 다시 본다. 그 안에 나타나지 않거나
# GH_TRIES번 뒤에도 비어 있으면 전처럼 실패다. --watch의 stderr는 버린다 — 타임스탬프 없는 줄이 루프 로그에 그대로 섞이기 때문이고,
# 결과는 어차피 bucket으로 다시 읽는다
ci_passed() {
  local try buckets
  CI_NO_CHECKS=0
  wait_checks_appear "$1" || { log warn "PR #$1: no checks were registered"; CI_NO_CHECKS=1; return 1; }
  for (( try = 1; try <= GH_TRIES; try++ )); do
    bg_wait gh pr checks "$1" --watch --fail-fast >/dev/null 2>&1 && return 0   # 끝날 때까지 블록, 실패 시 비0
    buckets="$(gh pr checks "$1" --json bucket --jq '[.[].bucket] | unique | join(" ")' 2>/dev/null || true)"
    if [[ -z "$buckets" ]]; then
      [[ $try -lt $GH_TRIES ]] || { log warn "PR #$1: the check list was empty after ${GH_TRIES} CI watches — treating it as a failure"; break; }
      log warn "PR #$1: CI watch ended with an empty check list (checks being re-registered?) — waiting for them to reappear (${try}/${GH_TRIES})"
      wait_checks_appear "$1" || { log warn "PR #$1: the checks did not reappear within ${CI_APPEAR_TIMEOUT_SEC}s"; CI_NO_CHECKS=1; return 1; }
      continue
    fi
    [[ " $buckets " != *" fail "* && " $buckets " != *" cancel "* ]] || return 1
    [[ " $buckets " == *" pending "* ]] || { log warn "PR #$1: CI watch failed but every check passed (GitHub error?) — going on"; return 0; }
    [[ $try -lt $GH_TRIES ]] || break
    log warn "PR #$1: CI watch failed with no failed check (GitHub error?) — watching again (${try}/${GH_TRIES})"
  done
  return 1
}

pr_head_sha() { gh pr view "$1" --json headRefOid -q .headRefOid; }

# CI 수리를 보내고 기다린다. 0: 충돌·CI 확인부터 다시 볼 것, 1: 새 커밋도 없고 CI도 그대로 실패. 제한 시간이면 멈춘다.
# 코드를 바꾸지 않는 수리도 있다 — 러너·네트워크 장애로 실패한 job은 `gh run rerun --failed`로 다시 돌리면 녹색이 되고, push할 커밋이
# 없다(2026-10-21 rental-portal #258: 같은 HOME을 나눠 쓰는 self-hosted 러너 둘의 설치 경합. 워커는 재실행만 했고 1분 뒤 통과했는데,
# 0.9.4까지는 head가 그대로라 멈췄다). 그래서 head가 그대로면 CI를 한 번 더 본다 — 재실행이 도는 중이면 끝까지 보고, 통과면 0이다.
# 체크가 끝내 등록되지 않은 실패(CI_NO_CHECKS=1)는 고칠 코드도 다시 돌릴 job도 없다 — 진단 프롬프트를 보낸다. 2026-11-11 rental-portal #833:
# GitHub Actions 장애 중 PR 실행이 job 0개로 queued에 굳었고, 장애가 풀린 뒤에도 그 실행은 취소("already completed")·재실행("already running")이
# 모두 거절됐다. 같은 커밋의 workflow_dispatch 실행은 통과했지만 그 체크는 gh pr checks에 잡히지 않았고, 빈 커밋 push가 새 pull_request 실행을
# 만들어 통과했다. 워커는 '실패' 프롬프트를 받고 22분 걸려 이 길을 찾았다
repair_ci() {
  local n="$1" pr="$2" handle="$3" deadline="$4" before rc=0 prompt
  before="$(pr_head_sha "${pr}")"
  phase repair
  if [[ "${CI_NO_CHECKS:-0}" -eq 1 ]]; then
    prompt="PR #${pr} 의 CI 체크가 ${CI_APPEAR_TIMEOUT_SEC}초 안에 등록되지 않았다 — 실패한 job은 없다. \`gh run list --branch <이 PR 브랜치>\` 로 실행을 찾고 \`gh api repos/{owner}/{repo}/actions/runs/<run-id>/jobs --jq .total_count\` 로 job 수를 본다. job이 0개인 채 queued면 GitHub 쪽 정체다: https://www.githubstatus.com 의 Actions가 회복되기를 기다린 뒤 코드를 바꾸지 말고 빈 커밋(\`git commit --allow-empty\`)을 push해 새 pull_request 실행을 만든다 — 굳은 실행은 취소·재실행이 거절될 수 있고, workflow_dispatch 실행의 체크는 \`gh pr checks\` 에 잡히지 않는다. job이 있는데 queued면 러너가 받지 못한 것이니 러너 상태를 확인하고, 사람이 할 일이면 그 사실을 PR 코멘트로 남긴다. 질문하지 말고 끝까지 해라."
  else
    prompt="PR #${pr} 의 CI가 실패했다. \`gh pr checks ${pr}\` 와 \`gh run view --log-failed\` 로 원인을 보고 고친 뒤 테스트·lint를 다시 돌리고 같은 브랜치에 push해라. 테스트를 건너뛰거나 지우지 마라. 같은 커밋의 다른 job은 통과했고 실패가 러너·네트워크 오류처럼 코드와 무관하면 코드를 바꾸지 말고 \`gh run rerun <run-id> --failed\` 로 실패한 job만 다시 돌려라(push할 커밋이 없어도 된다). 질문하지 말고 끝까지 해라."
  fi
  repair "${handle}" "$prompt" "$deadline" "${pr}" \
    || die "#${n}: timed out while repairing CI"
  phase ""
  screen_stop_check "${n}" " while repairing PR #${pr}"   # 수리 중 사용량 한도·확인 창이면 여기서 멈춘다
  [[ "$(pr_head_sha "${pr}")" == "$before" ]] || return 0
  log warn "#${n}: no new commit after the repair — checking CI again (a re-run needs no commit)"
```

```
  phase ci   # 재실행을 기다리는 동안은 CI다
  ci_passed "${pr}" || rc=$?
  phase ""
  return "$rc"
}

# CI가 실제로 돈 시간(초) — PR head 체크들의 가장 이른 시작부터 가장 늦은 끝까지(GitHub 시각). 루프가 CI 대기를 시작한 때부터 재면 그 전에
# 끝난 CI가 2~3초로 찍힌다(0.9.1까지 — PR을 알아채는 게 늦으면). 못 읽으면 빈 값이고, 부르는 쪽이 루프가 잰 값을 쓴다
ci_run_sec() {
  { gh pr checks "$1" --json startedAt,completedAt 2>/dev/null || true; } | jq -r '
    def t: if . == null or . == "" or startswith("0001-") then empty else sub("\\.[0-9]+Z$"; "Z") | fromdate end;
    ([.[].startedAt | t] | min) as $s | ([.[].completedAt | t] | max) as $e
    | if $s == null or $e == null or $e < $s then empty else ($e - $s) end' 2>/dev/null || true
}

# 반환 0: CI가 도는 PR, 1: 저장소 CI의 경로 필터에 걸려 체크가 안 붙는 게 정상인 PR(CI_SKIP_CMD가 "skip", 또는 바뀐 파일이 전부 CI_SKIP_REGEX에 맞음). 둘 다 없거나 판정이 안 되면 0(기다리는 쪽).
ci_expected() {
  local files verdict
  [[ -n "$CI_SKIP_CMD" || -n "$CI_SKIP_REGEX" ]] || return 0
  files="$(gh pr diff "$1" --name-only 2>/dev/null)" || return 0
  [[ -n "$files" ]] || return 0
  if [[ -n "$CI_SKIP_CMD" ]]; then
    verdict="$(cd "$ROOT" && printf '%s\n' "$files" | bash -c "$CI_SKIP_CMD" 2>/dev/null)" || return 0
    [[ "$verdict" != "skip" ]]
    return
  fi
  grep -Evq "$CI_SKIP_REGEX" <<<"$files"
}

mergeable_state() {
  # MERGEABLE | CONFLICTING | UNKNOWN(계산 중)
  local pr="$1" s i
  for i in 1 2 3 4 5 6; do
    s="$(gh pr view "${pr}" --json mergeable -q .mergeable)"
    [[ "$s" == "UNKNOWN" ]] || { echo "$s"; return; }
    sleep 10
  done
  echo "UNKNOWN"
}

# PR을 병합한다. GitHub 일시 장애(GH_TRANSIENT_RE, 그리고 "Base branch was modified")면 GH_TRIES번까지 다시 해 본다(GH_RETRY_SEC×회차씩 쉬고).
# 실패할 때마다 먼저 이미 병합됐는지 본다 — 서버는 병합을 끝냈는데 응답에서만 502가 났을 수 있고, 그대로 다시 부르면 "already merged"로
# 실패한다. 그 밖의 실패(권한, 필수 리뷰 같은 브랜치 보호, 충돌)는 바로 1이다. 마지막 gh 오류의 첫 줄을 MERGE_ERR에 남긴다.
# 실측(0.9.2, cmux): CI 통과 직후 병합 한 번이 "non-200 OK status code: 502 Bad Gateway"를 받아 루프가 멈췄다(이슈 #7)
MERGE_ERR=""
merge_pr() {
  local pr="$1" out try pause
  local transient="${GH_TRANSIENT_RE}|base branch was modified"
  MERGE_ERR=""
  for (( try = 1; try <= GH_TRIES; try++ )); do
    out="$(gh pr merge "$pr" "--$MERGE_METHOD" 2>&1)" && return 0
    if [[ "$(gh pr view "$pr" --json state -q .state 2>/dev/null || true)" == MERGED ]]; then
      log warn "PR #${pr}: the merge call failed but the PR is merged — going on"
      return 0
    fi
    [[ -z "$out" ]] || printf '%s\n' "$out" | dim_lines
    MERGE_ERR="$(grep -m1 . <<<"$out" || true)"
    grep -Eqi "$transient" <<<"$out" || return 1
    [[ $try -lt $GH_TRIES ]] || break
    pause=$(( try * GH_RETRY_SEC ))
    log warn "PR #${pr}: merge hit a transient GitHub error (${try}/${GH_TRIES}) — retrying in ${pause}s"
    sleep "$pause"
  done
  return 1
}

# ---------- 병합 뒤 검증 ----------
# 워커가 돌리기 어려운 검증(실제 서버·공유 DB·고정 포트를 쓰는 통합·E2E 테스트)을 병합된 기본 브랜치로
```

```
# 한 번 돌린다. 루프는 직렬이라 여기서 돌리면 워커끼리 부딪히지 않는다. 저장소 밖 전용 워크트리를 쓴다.

port_in_use() { lsof -nP -iTCP:"$1" -sTCP:LISTEN >/dev/null 2>&1; }
port_holder() { lsof -nP -iTCP:"$1" -sTCP:LISTEN 2>/dev/null ¦ awk 'NR==2 {print $1" pid "$2}'; }   # 예: "node pid 48213"

post_merge_worktree_path() {
  if [[ -n "$POST_MERGE_WORKTREE" ]]; then printf '%s' "$POST_MERGE_WORKTREE"; return; fi
  printf '%s/.cache/ticket-loop/%s/post-merge' "$HOME" "${OWNER_REPO//\//-}"
}

# ----- 묶어 돌리는 검증(POST_MERGE_EVERY) -----
# 병합 뒤 검증은 누적이다 — 기본 브랜치 끝에서 도는 검증은 그 전의 병합을 모두 덮는다. 그래서 병합 몇 개마다 한 번 돌려도 빠지는 병합은 없고,
# 잃는 것은 "어느 병합이 깨뜨렸나"뿐이다(그 답은 실패했을 때 pm_bisect가 찾는다). 루프는 한 번에 한 티켓이라 병합 직전 기본 브랜치는 워커가
# 테스트한 그 상태다 — 워커의 관련 테스트가 이미 돈 코드에 전체 스위트를 다시 도는 것이다. 실측(rental-portal, 0.9.5): 통합 테스트가
# 티켓마다 47~52분, 스펙 #248에서 12회 9.8시간이었고, 워커가 PR 전에 관련 통합 테스트를 돌리기 시작한 뒤 34회 동안 잡은 회귀는 없었다.
#   PM_BASE     창의 시작 — 마지막으로 검증을 통과한 커밋(없으면 창 첫 병합의 부모)
#   PM_PENDING  아직 검증을 거치지 않은 루프의 병합, 한 줄에 "이슈<탭>PR<탭>병합 커밋"(병합 순서)
#   PM_SPEC_COMPLETE  창의 병합이 끝낸 스펙 번호들(공백 구분, 보통 하나) — 검증 명령이 TICKET_LOOP_POST_MERGE_SPEC_COMPLETE로 받는다.
#               병합이 모두 검증을 거친 뒤에 닫힌 스펙이면 창이 비어도 남는다(루프 끝 검증이 받는다 — pm_flush_end)
# 셋 다 PM_STATE_FILE에 남는다. 이분 탐색은 PM_PENDING이 아니라 git의 PM_BASE..끝을 훑는다 — 사람이 병합한 커밋도 범위에 든다
#   PM_SPEC_COVERED  이번 실행에서 TICKET_LOOP_POST_MERGE_SPEC_COMPLETE에 실려 통과한 스펙 번호들 — 창에 다시 적지 않는다(저장하지 않는다)
PM_BASE=""; PM_PENDING=""; PM_SPEC_COMPLETE=""; PM_SPEC_COVERED=""; PM_WHY=""; PM_WHY_KEY=""; PM_BISECT=""; PM_FIRST_FAILING
=""; PM_REASON=""
NEXT_TARGETS=""; NEXT_TARGETS_SET=0   # pm_due가 미리 계산한 다음 대상 — 메인 고리가 그대로 쓴다

pm_count() { local c; c="$(grep -c . <<<"$PM_PENDING" ¦¦ true)"; printf '%s' "${c:-0}"; }
pm_list() { awk -F'\t' 'NF && $1 != "" { printf "%s#%s", (k++ ? " " : ""), $1 }' <<<"$PM_PENDING"; }
# 창에 적힌 스펙 완료 — "#40" 또는 드물게 "#40 #41"
pm_spec_list() { [[ -z "$PM_SPEC_COMPLETE" ]] ¦¦ printf '#%s' "${PM_SPEC_COMPLETE// / #}"; }
# 검증 결과 문자열에 붙일 꼬리 — 창에 이 티켓 말고 다른 병합이 있을 때만 ", covering #a #b …"
pm_cover() { [[ -z "$PM_PENDING" ¦¦ "$(pm_list)" == "#$1" ]] ¦¦ printf ', covering %s' "$(pm_list)"; }
# 미룬 병합 수 — "2/4"(창이 차면 돈다) 또는 POST_MERGE_EVERY=0이면 "2"(스펙·실행이 끝날 때만 돈다)
pm_of() { printf '%s' "$(pm_count)"; [[ "$POST_MERGE_EVERY" -eq 0 ]] ¦¦ printf '/%s' "$POST_MERGE_EVERY"; }

pm_save() {
  [[ -n "${PM_STATE_FILE:-}" ]] ¦¦ return 0
  if [[ -z "$PM_BASE" && -z "$PM_PENDING" && -z "$PM_SPEC_COMPLETE" ]]; then rm -f "$PM_STATE_FILE"; return 0; fi
  jq -Rn --arg base "$PM_BASE" --arg specs "$PM_SPEC_COMPLETE" '
    {base: (if $base == "" then null else $base end),
     pending: [inputs ¦ split("\t") ¦ select(length == 3 and .[0] != "")
               ¦ {issue: (.[0] ¦ tonumber), pr: (if .[1] == "" then null else (.[1] ¦ tonumber) end), commit: (if .[2] == "" th
en null else .[2] end)}],
     spec_complete: ($specs ¦ split(" ") ¦ map(select(. != "") ¦ tonumber))}' \
    <<<"$PM_PENDING" > "$PM_STATE_FILE" 2>/dev/null ¦¦ log warn "could not save the post-merge check state (${PM_STATE_FILE})"
}

# 시작 때 지난 실행이 남긴 창을 읽는다. 창의 시작이 기본 브랜치에 없으면(강제 push 등) 시작만 버린다 — 병합들은 여전히 검증해야 한다
pm_load() {
  [[ -n "${PM_STATE_FILE:-}" && -f "$PM_STATE_FILE" ]] ¦¦ return 0
  local base pending specs
  if ! base="$(jq -r '.base // ""' "$PM_STATE_FILE" 2>/dev/null)" \
     ¦¦ ! pending="$(jq -r '.pending[]? ¦ [(.issue ¦ tostring), (.pr // "" ¦ tostring), (.commit // "")] ¦ join("\t")' "$PM_STA
TE_FILE" 2>/dev/null)" \
     ¦¦ ! specs="$(jq -r '[.spec_complete[]? ¦ tostring] ¦ join(" ")' "$PM_STATE_FILE" 2>/dev/null)"; then
    log warn "post-merge check: unreadable state ${PM_STATE_FILE} — ignored"
    return 0
  fi
  if [[ -n "$base" ]]; then
    git -C "$ROOT" fetch -q origin "$BASE_BRANCH" 2>/dev/null ¦¦ true
    if ! git -C "$ROOT" merge-base --is-ancestor "$base" "origin/${BASE_BRANCH}" 2>/dev/null; then
      log warn "post-merge check: the last passed commit ${base:0:7} is not on origin/${BASE_BRANCH} any more — bisecting a fai
lure will not be possible until the next pass"
      base=""
    fi
  fi
```

```
  PM_BASE="$base"; PM_PENDING="$pending"; PM_SPEC_COMPLETE="$specs"
}

# 루프가 병합한 티켓을 창에 넣는다. $1 이슈 $2 PR $3 병합 커밋(모르면 빈 값). 창이 처음 열리면 시작은 그 병합의 부모다
pm_add() {
  local commit="${3:-}"
  if [[ -z "$PM_BASE" && -z "$PM_PENDING" && -n "$commit" ]]; then
    PM_BASE="$(git -C "$ROOT" rev-parse -q --verify "${commit}^1" 2>/dev/null || true)"
  fi
  PM_PENDING+="${PM_PENDING:+$'\n'}${1}"$'\t'"${2:-}"$'\t'"${commit}"
  pm_save
}

# 스펙 $1(없으면 0)이 끝났나 — 이번 실행이 그 스펙 이슈를 닫았다(close_spec_if_done). pm_due의 spec_end와 창에 적는 스펙 완료가 같은 조건을 쓴다
spec_closed_this_run() { [[ "${1:-0}" != 0 && " ${CLOSED_SPECS:-} " == *" $1 "* ]]; }

# 스펙 $1이 끝났으면 창에 적는다 — 창을 덮는 검증이 받는다(pm_exec). POST_MERGE_EVERY=1이면 스펙 끝의 검증도 이유가 every라 이 유만으로는
# 모른다. 창이 비었어도 적는다 — 병합이 모두 검증을 거친 뒤에 닫힌 스펙은 루프 끝 검증(pm_flush_end)이 그 번호로 한 번 돈다(#3 4).
# 이번 실행에서 그 번호를 싣고 통과한 검증이 있었으면 적지 않는다. 검증이 꺼진 실행은 창을 건드리지 않는다
pm_spec_closed() {
  local spec="${1:-0}"
  [[ ${POST_MERGE:-0} -eq 1 && " ${PM_SPEC_COMPLETE} ${PM_SPEC_COVERED} " != *" ${spec} "* ]] && spec_closed_this_run "$spec" || return 0
  PM_SPEC_COMPLETE="${PM_SPEC_COMPLETE:+${PM_SPEC_COMPLETE} }${spec}"
  pm_save
}

# 검증이 $1 커밋에서 통과했다 — 창을 닫고 그 커밋이 다음 창의 시작이 된다. 그 검증이 받은 스펙은 덮였다(PM_SPEC_COVERED)
pm_passed() { PM_BASE="${1:-$PM_BASE}"; PM_SPEC_COVERED+="${PM_SPEC_COMPLETE:+ ${PM_SPEC_COMPLETE}}"; PM_PENDING=""; PM_SPEC_COMPLETE=""; pm_save; }

# 이번 병합 뒤에 검증을 돌릴 때인가. 창이 POST_MERGE_EVERY개 찼거나(0이면 이 조건은 없다), 이 티켓의 스펙이 끝났거나(이번 실행이 스펙 이슈를 닫음), 이번 실행의
# 마지막 티켓이면 0. 마지막인지는 다음 대상을 미리 계산해 본다 — 미룰 때는 그 목록을 메인 고리가 그대로 쓴다(NEXT_TARGETS, 같은 조회를 두 번
# 하지 않게). 검증을 돌린 뒤에는 메인 고리가 다시 계산한다(검증이 도는 동안 바뀌었을 수 있다). PM_WHY에 이유를, PM_WHY_KEY에 검증 명령이
# 받는 고정 값(every·spec_end·run_end — pm_exec)을 남긴다. $1 이슈 $2 스펙(없으면 0)
pm_due() {
  local n="$1" spec="${2:-0}" cand ctitle next=""
  PM_WHY=""; PM_WHY_KEY=""; NEXT_TARGETS_SET=0
  [[ -n "$PM_PENDING" ]] || return 1
  if [[ "$POST_MERGE_EVERY" -gt 0 && "$(pm_count)" -ge "$POST_MERGE_EVERY" ]]; then
    PM_WHY="every ${POST_MERGE_EVERY} merged ticket$([[ $POST_MERGE_EVERY -eq 1 ]] || echo s)"; PM_WHY_KEY=every; return 0
  fi
  if spec_closed_this_run "$spec"; then PM_WHY="spec #${spec} complete"; PM_WHY_KEY=spec_end; return 0; fi
  # 번호 지정은 목록의 자리(TICKET_IDX)로 본다 — 도중에 닫혀 건너뛴 번호는 처리 건수(done_count)에 세지 않는다(#46)
  if [[ ${MAX:-0} -gt 0 && $(( ${done_count:-0} + 1 )) -ge ${MAX:-0} ]] \
     || [[ ${#TICKETS[@]} -gt 0 && $(( ${TICKET_IDX:-0} + 1 )) -ge ${#TICKETS[@]} ]]; then
    PM_WHY="last ticket of this run"; PM_WHY_KEY=run_end; return 0
  fi
  if [[ ${#TICKETS[@]} -eq 0 ]]; then
    # 조회가 실패하거나, 두 번 다 빈 목록인데 스펙에 열린 하위 이슈가 남으면(pick_next_targets 2) 미룬다 — 빈 목록을 "남은 티켓 없음"으로
    # 읽어 검증을 앞당기지 않게(#8·#22). 메인 고리가 다시 계산하고, 대상 없이 끝나면 루프 끝 검증(pm_flush_end)이 덮는다
    if ! NEXT_TARGETS="$(pick_next_targets </dev/null)"; then NEXT_TARGETS=""; return 1; fi
    while IFS=$'\t' read -r cand ctitle; do
      [[ -n "$cand" && "$cand" != "$n" && " ${PROCESSED:-} " != *" ${cand} "* ]] || continue
      next="$cand"; break
    done <<<"$NEXT_TARGETS"
    if [[ -z "$next" ]]; then PM_WHY="no more tickets"; PM_WHY_KEY=run_end; return 0; fi
    NEXT_TARGETS_SET=1
  fi
  return 1
}

# 검증 명령 하나를 검증 워크트리 $1에서 돌린다 — bg_wait로 부르고 출력은 검증 로그로 보낸다. $2 명령, $3 첫 실행 로그(재실행·이분 탐색이
# POST_MERGE_FAILED_LOG로 받는다). 왜 도는지와 창이 끝낸 스펙을 env로 넘기고(머리 주석 POST_MERGE_CMD, #32) 로그 첫 줄에 남긴
```

```
다.
# 재실행·이분 탐색·수습 뒤 재검증도 같은 창이라 같은 값이다 — 실패한 검증과 같은 조건으로 다시 돈다
pm_exec() {
  local vars=("TICKET_LOOP_POST_MERGE_WHY=${PM_WHY_KEY}" "TICKET_LOOP_POST_MERGE_SPEC_COMPLETE=${PM_SPEC_COMPLETE}")
  [[ -z "${3:-}" ]] || vars+=("POST_MERGE_FAILED_LOG=$3")
  printf '# ticket-loop: TICKET_LOOP_POST_MERGE_WHY="%s" TICKET_LOOP_POST_MERGE_SPEC_COMPLETE="%s"\n' "$PM_WHY_KEY" "$PM_SPEC_C
OMPLETE"
  env "${vars[@]}" bash -c 'cd "$1" && exec bash -c "$2"' _ "$1" "$2"
}

# 이분 탐색의 한 번 — 검증 워크트리를 $3 커밋으로 옮겨 POST_MERGE_RETRY_CMD를 첫 실행 로그($4)와 함께 돌린다.
# 반환 0 통과, 1 실패, 2 포트가 끝내 안 빔, 3 그 커밋으로 옮기지 못함
pm_probe() {
  local n="$1" wt="$2" c="$3" failed_log="$4" plog rc=0
  git -C "$wt" checkout -q -f --detach "$c" 2>/dev/null || return 3   # 루프의 검증 워크트리다 — 지난 실행이 남긴 변경은 버린다
  wait_post_merge_port "$n" || return 2
  plog="${failed_log%.log}-bisect-${c:0:7}.log"
  phase post
  bg_wait pm_exec "$wt" "$POST_MERGE_RETRY_CMD" "$failed_log" >"$plog" 2>&1 || rc=$?
  phase ""
  log "#${n}: failed tests at ${c:0:7} $(git -C "$wt" log -1 --format=%s "$c" 2>/dev/null | cut -c1-60) — $([[ $rc -eq 0 ]] &&
echo pass || echo fail) (log ${plog})"
  [[ $rc -eq 0 ]] || return 1
}

# 이분 탐색이 찾은 커밋을 사람이 읽는 꼴로 — 창의 병합이면 "#이슈 (PR #P, 커밋)", 아니면(사람이 병합한 것 등) "커밋 제목"
pm_describe() {
  local line issue pr pc
  while IFS= read -r line; do   # 탭 IFS로 read 하면 빈 칸(PR을 모름)이 붙어 버린다 — 직접 자른다
    issue="${line%%$'\t'*}"; pc="${line#*$'\t'}"; pr="${pc%%$'\t'*}"; pc="${pc#*$'\t'}"
    [[ -z "$pc" || "$pc" != "$1" ]] || { printf '#%s (%scommit %s)' "$issue" "${pr:+PR #${pr}, }" "${1:0:7}"; return 0; }
  done <<<"$PM_PENDING"
  printf 'commit %s %s' "${1:0:7}" "$(git -C "$2" log -1 --format=%s "$1" 2>/dev/null)"
}

# 묶어 돌린 검증이 재실행까지 실패했을 때 처음 실패하는 병합을 찾는다. 실패한 테스트만 다시 돌리는 명령(POST_MERGE_RETRY_CMD — 1~2분)을
# 창의 시작에서 한 번(통과해야 탐색이 성립한다 — 거기서도 실패하면 플레이크·환경, 또는 그 명령이 테스트를 못 고른 것), 그다음 PM_BASE..끝의
# 첫 부모 커밋들을 이분 탐색한다(끝은 방금 재실행이 실패했다). 전체 스위트(한 번에 수십 분)로는 탐색하지 않는다 — 그때는 묶음만 적고
# 원인 찾기는 무인 수습에 맡긴다. 결과는 PM_BISECT(사유에 붙일 문장)와 PM_FIRST_FAILING(커밋). 검증 워크트리는 끝 커밋으로 되돌린다
pm_bisect() {
  local n="$1" wt="$2" failed_log="$3" head c lo hi mid rc=0 base_rc=0 commits=()
  PM_BISECT=""; PM_FIRST_FAILING=""
  [[ -n "$POST_MERGE_RETRY_CMD" && -n "$PM_BASE" ]] || return 0
  head="$(git -C "$wt" rev-parse HEAD 2>/dev/null)" || return 0
  git -C "$wt" merge-base --is-ancestor "$PM_BASE" "$head" 2>/dev/null || return 0
  while IFS= read -r c; do [[ -z "$c" ]] || commits+=("$c"); done < <(git -C "$wt" rev-list --first-parent --reverse "${PM_BAS
E}..${head}" 2>/dev/null)
  [[ ${#commits[@]} -ge 2 ]] || return 0
  log step "#${n}: finding the first failing merge — rerunning the failed tests at ${#commits[@]} merges since ${PM_BASE:0:7}
(bisect, POST_MERGE_RETRY_CMD)"
  pm_probe "$n" "$wt" "$PM_BASE" "$failed_log" || base_rc=$?
  if [[ $base_rc -eq 0 ]]; then
    lo=0; hi=$(( ${#commits[@]} - 1 ))
    while [[ $lo -lt $hi ]]; do
      mid=$(( (lo + hi) / 2 )); rc=0
      pm_probe "$n" "$wt" "${commits[$mid]}" "$failed_log" || rc=$?
      [[ $rc -le 1 ]] || break
      if [[ $rc -eq 0 ]]; then lo=$(( mid + 1 )); else hi=$mid; fi
    done
  fi
  git -C "$wt" checkout -q -f --detach "$head" 2>/dev/null || log warn "#${n}: could not move the check worktree back to ${hea
d:0:7}"
  if [[ $base_rc -eq 2 || $rc -eq 2 ]]; then
    PM_BISECT="first failing merge not found: port ${POST_MERGE_PORT} did not free up"
  elif [[ $base_rc -eq 3 || $rc -eq 3 ]]; then
    PM_BISECT="first failing merge not found: could not check out a commit in the check worktree"
  elif [[ $base_rc -ne 0 ]]; then
    PM_BISECT="first failing merge not found: the failed tests fail at ${PM_BASE:0:7} (last passed) too — flaky, environment, o
```

```
r the retry command could not pick them"
  else
    PM_FIRST_FAILING="${commits[$lo]}"
    PM_BISECT="first failing merge: $(pm_describe "$PM_FIRST_FAILING" "$wt")"
  fi
  log warn "#${n}: ${PM_BISECT}"
}

# 스펙 리포트·멈춤 기록에 싣는 창 — 병합도 스펙 완료도 없으면 null
pm_window_json() {
  [[ -n "$PM_PENDING" ]] || -n "$PM_SPEC_COMPLETE" ]] || { echo null; return 0; }
  jq -Rnc --arg base "$PM_BASE" --arg bisect "$PM_BISECT" --arg first "$PM_FIRST_FAILING" --arg specs "$PM_SPEC_COMPLETE" '
    def str: if . == "" then null else . end;
    {base: ($base | str), first_failing_commit: ($first | str), bisect: ($bisect | str),
     pending: [inputs | split("\t") | select(length == 3 and .[0] != "")
               | {issue: (.[0] | tonumber), pr: (if .[1] == "" then null else (.[1] | tonumber) end), commit: (.[2] | str)}],
     spec_complete: ($specs | split(" ") | map(select(. != "") | tonumber))}' <<<"$PM_PENDING"
}

# POST_MERGE_PORT가 비기를 기다린다. 반환 0: 비었음(포트 설정이 없으면 바로), 1: PORT_WAIT_SEC 안에 비지 않음.
# 첫 실행과 재실행이 같이 쓴다 — 첫 실행이 띄운 서버가 명령이 끝난 뒤에도 포트를 늦게 놓을 수 있다
wait_post_merge_port() {
  local n="$1" port_deadline holder
  [[ -n "$POST_MERGE_PORT" ]] || return 0
  port_deadline=$(( $(date +%s) + PORT_WAIT_SEC ))
  # 기다리는 동안 아무것도 찍지 않으면 루프가 멈춘 것처럼 보인다(실측: 사람이 띄운 개발 서버가 포트를 쥔 채 4분을
  # 조용히 기다렸다). 처음 한 번 점유 프로세스와 기한을 적는다 — 그 서버를 끄면 30초 안에 이어 간다
  if port_in_use "$POST_MERGE_PORT"; then
    holder="$(port_holder "$POST_MERGE_PORT")"
    log warn "#${n}: port ${POST_MERGE_PORT} in use (${holder:-?}) — waiting for it to free up until $(fmt_epoch "$port_deadlin
e"). Stop that server and the loop continues right away"
  fi
  while port_in_use "$POST_MERGE_PORT"; do
    [[ "$(date +%s)" -lt "$port_deadline" ]] || { log warn "#${n}: port ${POST_MERGE_PORT} did not free up — keep it free while
the loop runs, or use --skip-post-merge"; return 1; }
    sleep 30
  done
}

# $1이 이 저장소의 연결 워크트리인가 — git 공통 디렉터리가 이 저장소의 것과 같고(다른 저장소·클론이 아니다), 자기 git 디렉터리
는 공통
# 디렉터리와 다르다(메인 체크아웃이 아니다). 정리(pm_clean)는 지우는 일이라, 이미 있는 검증 워크트리 자리가 이것이 아니면 손대
지 않는다
pm_linked_worktree() {
  local common wt_common wt_gitdir
  common="$(git -C "$ROOT" rev-parse --path-format=absolute --git-common-dir 2>/dev/null)" || return 1
  wt_common="$(git -C "$1" rev-parse --path-format=absolute --git-common-dir 2>/dev/null)" || return 1
  wt_gitdir="$(git -C "$1" rev-parse --path-format=absolute --git-dir 2>/dev/null)" || return 1
  common="$(cd "$common" && pwd -P)"; wt_common="$(cd "$wt_common" && pwd -P)"; wt_gitdir="$(cd "$wt_gitdir" && pwd -P)"
  [[ -n "$wt_common" && "$wt_common" == "$common" && "$wt_gitdir" != "$wt_common" ]]
}

# 검증 워크트리 $2의 추적 안 된 파일과 무시된 파일을 지운다 — 중첩 git 저장소(-ff)와 빈 디렉터리(-d)도. 검증 하나가 시작할 때
강제 체크아웃
# 뒤에 한 번만 부른다(run_post_merge) — 재실행·이분 탐색 사이와 검증이 끝날 때는 부르지 않는다. 그래서 검증 워크트리는 "남긴 것
+ 검증 한 번이
# 만드는 것"보다 커지지 않고, 실패한 검증의 산출물은 다음 검증이 시작할 때까지 남는다. 실측(rsrehab/rental-portal, 0.9.22, #4
7): 정리 없이
# 재사용한 검증 워크트리가 57GB였다(.next/dev/cache 51GB).
# POST_MERGE_KEEP의 패턴(.gitignore 꼴, 검증 워크트리 루트 기준)에 맞는 것은 남긴다 — -x가 저장소의 무시 규칙을 꺼도 -e로 준 규
칙은 따른다.
# 패턴은 read -a로 나눈다(for의 단어 나누기는 *.log 같은 글롭을 지금 디렉터리의 파일 이름으로 펼친다). 지운 최상위 경로 수만 센
다 — git이 지운
# 것마다 한 줄씩 적는다(LC_ALL=C: 문구가 번역되지 않게). 크기는 재지 않는다(수십 GB를 du로 재는 일이 느리다)
pm_clean() {
  local n="$1" wt="$2" out errf removed p rc=0 keep=() excludes=()
  read -r -a keep <<<"$POST_MERGE_KEEP"
  for p in ${keep[@]+"${keep[@]}"}; do excludes+=(-e "$p"); done
  errf="$(mktemp "${TMPDIR:-/tmp}/ticket-loop-clean.XXXXXX")"
  out="$(LC_ALL=C git -C "$wt" clean -ffdx ${excludes[@]+"${excludes[@]}"} 2>"$errf")" || rc=$?
  removed="$(grep -c '^Removing ' <<<"$out" || true)"
  [[ "$removed" -eq 0 ]] || log step "#${n}: cleaned the check worktree — removed ${removed} untracked or ignored path$([[ "$re
```

```
moved" -eq 1 ]] || echo s)${keep[*]:+ (kept POST_MERGE_KEEP: ${keep[*]})}"
  # 지우지 못한 것이 있어도(권한 등) 검증은 돈다 — 정리 전에는 늘 이 상태로 돌았으므로 지금보다 나빠지지 않는다
  [[ $rc -eq 0 ]] || log warn "#${n}: could not clean the check worktree (${wt}): $(grep -m1 . "$errf" || echo "git clean exite
d ${rc}") — running the check anyway"
  rm -f "$errf"
}

# 반환 0: 통과(재실행에서 통과한 것 포함), 1: 실패(로그 꼬리를 남긴다). $2 로그 이름에 붙일 꼬리표 — 무인 수습 뒤 다시 돌릴 때
# (post-merge-N-after-triage.log) 앞 실행의 로그를 덮지 않게
run_post_merge() {
  local n="$1" tag="${2:-}" wt logfile retry_log started first_sec retry_sec f head cover branch
  wt="$(post_merge_worktree_path)"
  POST_MERGE_WT="$wt"
  POST_MERGE_RESULT="did not start"   # 검증 명령을 돌리기 전에 끝나면(워크트리·포트) 이 값이 남는다
  POST_MERGE_WHY="$PM_WHY_KEY"; POST_MERGE_SPEC_COMPLETE="$PM_SPEC_COMPLETE"   # 기록용(record_ticket) — 통과하면 창과 함께 비
므로 떠 둔다
  PM_BISECT=""; PM_FIRST_FAILING=""   # 앞 검증의 이분 탐색 결과가 이번 멈춤 사유에 섞이지 않게
  if [[ ! -d "$wt/.git" && ! -f "$wt/.git" ]]; then
    mkdir -p "$(dirname "$wt")"
    git -C "$ROOT" worktree add -q --detach "$wt" "origin/${BASE_BRANCH}" || { log warn "#${n}: failed to create the post-merge
check worktree (${wt})"; return 1; }
  elif ! pm_linked_worktree "$wt"; then
    log warn "#${n}: ${wt} is not a linked worktree of this repository — not cleaning it, the post-merge check did not start. S
et POST_MERGE_WORKTREE to a directory of its own (or leave it empty for the default)"
    return 1
  elif branch="$(git -C "$wt" symbolic-ref -q --short HEAD)"; then
    # 루프는 검증 워크트리를 늘 브랜치 없이 만들고 옮긴다(worktree add·checkout·이분 탐색 모두 --detach). 브랜치에 올라 있으면
사람의 작업
    # 워크트리(또는 워커 워크트리)다 — 강제 체크아웃과 정리가 그 작업을 지우므로 손대지 않는다
    log warn "#${n}: ${wt} has branch ${branch} checked out — the loop keeps its check worktree detached, so this is someone's
working tree. Not cleaning it, the post-merge check did not start. If it is the check worktree, detach it (git -C ${wt} switch
--detach); otherwise set POST_MERGE_WORKTREE to a directory of its own"
    return 1
  fi
  git -C "$ROOT" fetch -q origin "$BASE_BRANCH"
  git -C "$wt" checkout -q -f --detach "origin/${BASE_BRANCH}" || { log warn "#${n}: failed to move the check worktree to origi
n/${BASE_BRANCH}"; return 1; }
  pm_clean "$n" "$wt"
  head="$(git -C "$wt" rev-parse HEAD)"
  cover="$(pm_cover "$n")"   # 묶어 돌리면(POST_MERGE_EVERY) 이 검증이 덮는 병합들
  for f in $POST_MERGE_COPY; do
    [[ ! -f "$MAIN_WT/$f" ]] || cp "$MAIN_WT/$f" "$wt/$f"
  done
  wait_post_merge_port "$n" || return 1
  # 검증 로그는 LOG_FILE과 같은 디렉터리(절대 경로 — 아래 리다이렉트는 cd 한 뒤의 서브셸에서 열리므로 상대 경로면 워크트리 안에
생긴다)
  logfile="${TMPDIR:-/tmp}/ticket-loop-post-merge-${n}${tag}.log"
  [[ "$LOG_FILE" == "/dev/null" ]] || logfile="$(dirname "$LOG_FILE")/post-merge-${n}${tag}.log"
  POST_MERGE_LOG="$logfile"; POST_MERGE_RETRY_LOG=""
  mkdir -p "$(dirname "$logfile")"
  started="$(date +%s)"
  log step "#${n}: post-merge check started (${wt}, log ${logfile})"
  [[ -z "$cover" ]] || log step "#${n}: this check covers the merged tickets ${cover#, covering }${PM_WHY:+ (${PM_WHY})}"
  phase post   # 명령이 도는 동안만(포트 대기는 빼고) — 리포트의 병합 뒤 검증 시간
  if bg_wait pm_exec "$wt" "$POST_MERGE_CMD" >"$logfile" 2>&1; then
    phase ""
    POST_MERGE_RESULT="passed in $(( $(date +%s) - started ))s${cover}"
    log ok "#${n}: post-merge check ${POST_MERGE_RESULT}"
    pm_passed "$head"
    return 0
  fi
  phase ""
  first_sec=$(( $(date +%s) - started ))
  POST_MERGE_RESULT="failed in ${first_sec}s${cover}"
  log warn "#${n}: post-merge check failed — tail ---"; tail -n 40 "$logfile" | dim_lines || true
  [[ -n "$POST_MERGE_RETRY_CMD" ]] || return 1
  # 재실행은 딱 한 번, 같은 워크트리에서. 무거운 스위트는 그 병합과 무관하게 아무 테스트나 가끔 시간 초과로 깨진다(실측: rental
-portal
  # 44분 통합 테스트 7회 중 2회, 매번 다른 테스트 하나 — 그것만 다시 돌리면 통과할 성격이었다). 무엇을 다시 돌릴지는 재실행 명
령이
  # 첫 실행 로그(POST_MERGE_FAILED_LOG)를 보고 정한다. 키트는 테스트 도구를 모르므로 그 판단을 저장소 쪽 명령에 맡긴다
  wait_post_merge_port "$n" || return 1
```

```
  retry_log="${logfile%.log}-retry.log"
  POST_MERGE_RETRY_LOG="$retry_log"
  started="$(date +%s)"
  log step "#${n}: retrying the post-merge check once (log ${retry_log})"
  phase post
  if bg_wait pm_exec "$wt" "$POST_MERGE_RETRY_CMD" "$logfile" >"$retry_log" 2>&1; then
    phase ""
    retry_sec=$(( $(date +%s) - started ))
    POST_MERGE_RESULT="passed on retry in ${retry_sec}s (first run failed in ${first_sec}s)${cover}"
    log warn "#${n}: post-merge check ${POST_MERGE_RESULT} — likely flaky, continuing"
    post_merge_flaky_record "$n" "$logfile" "$retry_log" "$first_sec" "$retry_sec"
    pm_passed "$head"
    return 0
  fi
  phase ""
  POST_MERGE_RESULT="failed on retry in $(( $(date +%s) - started ))s (first run failed in ${first_sec}s)${cover}"
  log warn "#${n}: post-merge retry failed too — tail ---"; tail -n 40 "$retry_log" | dim_lines || true
  # 여러 병합을 묶어 돌렸으면 처음 실패하는 병합을 찾는다(재실행이 실패한 테스트를 골라 돌릴 수 있을 때만 — pm_bisect)
  if [[ -n "$cover" ]]; then
    pm_bisect "$n" "$wt" "$logfile"
    [[ -z "$PM_BISECT" ]] || POST_MERGE_RESULT+="; ${PM_BISECT}"
  fi
  return 1
}

# 병합 뒤 검증 실패 뒤처리 — 멈추기 전에 무인 수습을 띄운다(post_merge_failed). 수정 PR이 병합되면(또는 플레이크로 판정하면) 병합된 기본
# 브랜치로 검증을 다시 돌리고(로그 post-merge-N-after-triage.log) 통과하면 0(이어 간다). 아니면 1, 멈출 사유는 PM_REASON.
# 재실행까지 실패했으면 두 로그를 다 적는다. 재실행이 실패한 테스트를 못 찾아 끝냈을 수도, 같은 플레이크가 또 났을 수도 있어 판단은
# 수습 세션(또는 사람)이 한다. 여러 병합을 묶어 돌렸으면 묶음과 이분 탐색 결과를, 창이 빈 스펙 끝 검증(pm_flush_end)이면 그렇다고
# 사유에 적는다 — 수습 세션은 멈춤 기록의 post_merge_window에서 같은 것을 읽는다
post_merge_recover() {
  local n="$1" reason scope="" first_result post_rc=0
  [[ -z "$(pm_cover "$n")" ]] || scope=" (merges checked together: $(pm_list)${PM_BISECT:+ — ${PM_BISECT}})"
  [[ -n "$PM_PENDING" || -z "$PM_SPEC_COMPLETE" ]] || scope=" (spec-end check for spec $(pm_spec_list) — no merge was left unchecked)"
  if [[ -z "$POST_MERGE_RETRY_LOG" ]]; then
    reason="#${n}: post-merge check failed on the merged ${BASE_BRANCH}${scope}. Check the log (${POST_MERGE_LOG}) and the check worktree ($(post_merge_worktree_path)), then rerun after a fix PR."
  else
    reason="#${n}: post-merge check failed on the merged ${BASE_BRANCH}${scope}, and again on retry. Check the logs (first run ${POST_MERGE_LOG}, retry ${POST_MERGE_RETRY_LOG}) and the check worktree ($(post_merge_worktree_path)). If it looks like a flake, just rerun; otherwise rerun after a fix PR."
  fi
  STOP_KIND=post_merge_failed
  if triage post_merge_failed "$reason"; then
    first_result="$POST_MERGE_RESULT"
    run_post_merge "${n}" -after-triage || post_rc=$?
    POST_MERGE_RESULT="${first_result}; after triage${TRIAGE_FIX_PR:+ (fix PR #${TRIAGE_FIX_PR})}: ${POST_MERGE_RESULT}"
    if [[ $post_rc -ne 0 ]]; then
      STOP_KIND=post_merge_failed
      reason="#${n}: post-merge check failed again after triage${TRIAGE_FIX_PR:+ (fix PR #${TRIAGE_FIX_PR} merged)}. Check the logs (${POST_MERGE_LOG}${POST_MERGE_RETRY_LOG:+, retry ${POST_MERGE_RETRY_LOG}}) and the check worktree ($(post_merge_worktree_path))."
    fi
  else
    reason+="$TRIAGE_NOTE"; post_rc=1
  fi
  PM_REASON="$reason"
  return $post_rc
}

# 루프가 끝날 때 창에 병합이 남았으면 검증한다 — 마지막 티켓 뒤에 돌리지 못한 경우(다음 대상이 있다고 봤는데 고리가 대상 없이 끝남,
# 지난 실행이 멈추며 남긴 창에 이번 실행이 처리할 티켓이 없음). 창이 비었어도 스펙 완료가 적혀 있으면 기본 브랜치 끝에서 한 번 돈다(#34) —
# 병합이 모두 검증을 거친 뒤에 닫힌 스펙(POST_MERGE_EVERY=1에서 사람이 마지막 티켓을 닫음·하위 이슈 반영 지연). 그때 이유는 spec_end다.
# 어느 티켓의 기록에도 들지 않는다(로그·Slack에만 남는다). 실패하면 창의 마지막 티켓(창이 비었으면 마지막 스펙 이슈)으로 멈춘다
pm_flush_end() {
```

```
  [[ $POST_MERGE -eq 1 && ( -n "$PM_PENDING" ¦¦ -n "$PM_SPEC_COMPLETE" ) ]] ¦¦ return 0
  local n pr=""
  if [[ -n "$PM_PENDING" ]]; then
    n="$(tail -n 1 <<<"$PM_PENDING" ¦ cut -f1)"; pr="$(tail -n 1 <<<"$PM_PENDING" ¦ cut -f2)"
    PM_WHY="end of run"; PM_WHY_KEY=run_end
    log step "post-merge check for the merged tickets not checked yet ($(pm_list))"
  else
    n="${PM_SPEC_COMPLETE##* }"
    PM_WHY="spec $(pm_spec_list) complete"; PM_WHY_KEY=spec_end
    log step "post-merge check for the completed spec $(pm_spec_list) — no passed check has covered it yet"
  fi
  STOP_ISSUE="$n"; STOP_KIND=""; STOP_RECORD=""; TRIAGE_RESULT=""
  # 멈춤 기록·GitHub 기록이 앞 티켓의 PR·시작 시각·워크트리를 싣지 않게 — 창의 마지막 병합(창이 비었으면 마지막 스펙 이슈)으로 채운다
  CUR_PR="$pr"; CUR_TITLE="$(issue_title "$n" 2>/dev/null ¦¦ true)"; CUR_STARTED=""; WT_PATH=""
  POST_MERGE_RESULT=""; POST_MERGE_LOG=""; POST_MERGE_RETRY_LOG=""; POST_MERGE_WHY=""; POST_MERGE_SPEC_COMPLETE=""
  if ! run_post_merge "$n" -end; then
    post_merge_recover "$n" ¦¦ die "$PM_REASON"
  fi
  log ok "post-merge check at the end of the run: ${POST_MERGE_RESULT}"
  STOP_ISSUE=""
}

# ---------- 무인 수습 ----------
# 멈출 일이 생긴 자리 네 곳(머리 주석 "무인 수습" 절)에서 바로 die하지 않고 triage를 부른다. 수습 세션은 워커와 같은 길
# (backend_worktree_create)로 새 워크트리에 띄운 에이전트이고, 할 일은 기본 브랜치에 커밋된 loop-triage 스킬이 정한다(멈춤 기록을 읽고
# 분류 → 고치거나 진단 → 판정 파일). 루프는 판정 파일만 보고, 수정 PR은 티켓 PR처럼 CI 뒤에 병합한다 — 수습 세션은 병합하지 않는다.
# 이어 가는 곳은 멈춘 그 자리다. 처음부터 다시 실행하면 진행 중인 티켓(PR이 열렸거나 담당자가 붙은)을 자동 선택이 건너뛰어 버린다.
# 루프 스크립트 자체(키트 관리 파일)는 수습 세션이 고치지 않는다 — 루프 버그는 진단과 키트 버그 보고서까지만 하고 멈춘다

TRIAGE_USED=0; TRIAGE_KEYS=""; TRIAGE_NOTE=""; TRIAGE_RESULT=""; TRIAGE_FIX_PR=""

triage_skill_on_base() { git -C "$ROOT" cat-file -e "origin/${BASE_BRANCH}:.agents/skills/loop-triage/SKILL.md" 2>/dev/null; }

# 한 줄로 보낸다(워커 프롬프트와 같은 이유). 무인 실행에서만 참인 것 — 질문 금지, 병합 금지, 판정 파일 — 만 둔다. 절차는 스킬이 단일 진실이다
triage_prompt() {
  printf '%s' "${SKILL_PREFIX}loop-triage $1 — 무인 루프(ticket-loop.sh)가 멈춘 지점에서 띄운 수습 세션이다. 사람이 답할 수 없으니 사람에게 묻거나 확인을 구하지 말고(질문 도구도 쓰지 말고) 스스로 판단해 끝까지 진행해라. PR은 병합하지 마라 — 루프가 CI 뒤에 병합한다. 마지막에 판정을 $2 에 써라."
}

# 수습 세션의 워크트리와 에이전트를 띄운다. bg_wait로(백그라운드 서브셸에서) 부른다 — 백엔드 함수는 실패하면 die하는데, 여기서의 실패가
# 원래 멈춤 사유를 덮으면 안 되므로 die를 "이 서브셸만 끝냄"으로 바꾼다(EXIT trap은 서브셸에 이어지지 않는다).
# 결과(WT, WT_PATH, WT_BRANCH, HANDLE)를 $4 파일에 탭으로 쓴다
triage_spawn() {
  local n="$1" prompt="$2" name="$3" out="$4"
  set -e
  die() { log warn "#${n}: triage session did not start — $*"; exit 1; }
  WT=""; WT_PATH=""; WT_BRANCH=""; HANDLE=""
  backend_worktree_create "$n" "$prompt" "$name"
  ensure_agent_started "$n" "$HANDLE" "$prompt"
  printf '%s\t%s\t%s\t%s\n' "$WT" "$WT_PATH" "$WT_BRANCH" "$HANDLE" > "$out"
}

verdict_written() { jq -e '.verdict == "resume" or .verdict == "halt"' "$1" >/dev/null 2>&1; }

# 수습 세션이 판정 파일을 쓸 때까지 기다린다 — wait_for_pr와 같은 틀(idle 연속 IDLE_STRIKES번이면 판정 없이 멈춘 것, 그사이 화면이 바뀌면
# 새로 센다). bg_wait로 부른다. 진행 로그는 수습 워크트리를 본다("#N triage: commit …").
# 반환: 0 판정 있음, 1 판정 없이 멈춤, 2 시간 초과, 3 터미널 끊김
wait_for_verdict() {
  local n="$1" handle="$2" vfile="$3" deadline="$4" wt_path="$5" strikes=0 started idle last_screen="" screen wait_ms active_said=0
  reach_reset "$handle"
  started="$(date +%s)"; wait_ms="$(idle_wait_ms)"; PROMPT_SUM=""; PROMPT_AT=0
  [[ "$PROGRESS_SEC" -le 0 ]] ¦¦ progress_reset "$wt_path"
  while :; do
```

```
    if verdict_written "$vfile"; then
      terminal_idle "$handle" 120000 ¦¦ true   # 판정을 쓴 뒤 마무리(코멘트 등)가 끝나도록 잠깐
      return 0
    fi
    [[ "$(date +%s)" -lt "$deadline" ]] ¦¦ return 2
    idle=0; terminal_idle "$handle" "$wait_ms" ¦¦ idle=$?
    [[ $idle -ne 3 ]] ¦¦ return 3
    [[ "$PROGRESS_SEC" -le 0 ]] ¦¦ progress_check "${n} triage"
    if [[ $idle -eq 0 ]]; then
      if [[ $(( $(date +%s) - started )) -lt $STARTUP_GRACE_SEC ]]; then sleep "$IDLE_RECHECK_SEC"; continue; fi
      sleep "$IDLE_RECHECK_SEC"
      ! verdict_written "$vfile" ¦¦ continue
      screen="$(terminal_tail "$handle" ¦ cksum ¦ cut -d' ' -f1)"
      if [[ "$screen" != "$last_screen" ]]; then strikes=0; last_screen="$screen"; fi
      if session_active_recently "$wt_path"; then   # 수습 세션의 기록이 방금 갱신됐으면 아직 일하는 중 — strike를 세지 않는다
        [[ $active_said -eq 1 ]] ¦¦ log "#${n}: idle reported, but the session log was written ${SESSION_LOG_AGE}s ago — still
working"
        strikes=0; active_said=1
      else
        active_said=0
        ! screen_check "$handle" "triage session" 1 ¦¦ return 1   # 수습 세션도 같은 한도에 걸린다 — "판정 없이 멈춤"이 아니라
usage_limit로
        strikes=$((strikes + 1))
        log warn "#${n}: triage session idle, no verdict (${strikes}/${IDLE_STRIKES})"
        [[ ${strikes} -lt $IDLE_STRIKES ]] ¦¦ return 1
      fi
      sleep "$IDLE_RECHECK_SEC"
    else
      strikes=0; active_said=0
      ! screen_check "$handle" "triage session" 0 ¦¦ return 1
    fi
  done
}

# 수습 세션이 낸 수정 PR을 티켓 PR처럼 병합한다 — 기본 브랜치로 가는 열린 PR인지 보고, TRIAGE_LABEL을 붙이고, CI를 기다려 병합
한다.
# 반환 1이면 TRIAGE_NOTE에 이유를 남긴다
triage_merge_fix_pr() {
  local pr="$1" info state base head
  info="$(gh pr view "$pr" --json state,baseRefName,headRefName 2>/dev/null)" ¦¦ { TRIAGE_NOTE=" (fix PR #${pr} not found)"; re
turn 1; }
  state="$(jq -r .state <<<"$info")"; base="$(jq -r .baseRefName <<<"$info")"; head="$(jq -r .headRefName <<<"$info")"
  [[ "$state" == OPEN && "$base" == "$BASE_BRANCH" ]] \
    ¦¦ { TRIAGE_NOTE=" (fix PR #${pr} is not an open PR into ${BASE_BRANCH}: ${state}, base ${base})"; return 1; }
  gh label create "$TRIAGE_LABEL" --color 1D76DB --description "ticket-loop 무인 수습이 낸 수정 PR" </dev/null >/dev/null 2>&1
¦¦ true
  gh pr edit "$pr" --add-label "$TRIAGE_LABEL" </dev/null >/dev/null 2>&1 ¦¦ log warn "fix PR #${pr}: failed to add the ${TRIAG
E_LABEL} label"
  if ci_expected "$pr"; then
    log step "triage: waiting for CI (fix PR #${pr})"
    ci_passed "$pr" ¦¦ { TRIAGE_NOTE=" (fix PR #${pr}: CI failed — gh pr checks ${pr})"; return 1; }
    log ok "triage: fix PR #${pr} CI passed"
  else
    log step "triage: fix PR #${pr} excluded by the repository CI path filter — merging without checks"
  fi
  merge_pr "$pr" ¦¦ { TRIAGE_NOTE=" (fix PR #${pr}: merge failed${MERGE_ERR:+ — ${MERGE_ERR}})"; return 1; }
  log ok "triage: fix PR #${pr} merged"
  git -C "$ROOT" push -q origin --delete "$head" >/dev/null 2>&1 ¦¦ true
  git -C "$ROOT" fetch -q origin "$BASE_BRANCH" ¦¦ true
}

# 수정 PR의 병합 커밋이 처리 중인 티켓 이슈를 닫았으면 다시 연다 — 병합 뒤 검증 실패가 아니면 그 티켓은 아직 진행 중이다. GitHu
b는
# 기본 브랜치에 들어온 커밋 메시지의 닫기 키워드(병합 커밋에는 PR 제목이 들어간다 — `fix: #17 …`)로 이슈를 닫는다. 실측(0.9.0,
더미
# 저장소): 준비 실패의 수정 PR #18 병합 커밋이 #17을 닫았고, 워커가 알아채 다시 열었다. 스킬이 그런 제목을 막지만 커밋 메시지까
지
# 다 막는다는 보장은 없다. 닫힘은 병합 몇 초 뒤에 일어나므로 REOPEN_WATCH_SEC 간격으로 다섯 번 본다
triage_keep_issue_open() {
  local n="$1" pr="$2" i state=""
  for i in 1 2 3 4 5; do
    state="$(gh issue view "$n" --json state -q .state </dev/null 2>/dev/null ¦¦ true)"
```

```
    [[ "$state" != CLOSED && $i -lt 5 ]] || break
    sleep "$REOPEN_WATCH_SEC"
  done
  [[ "$state" == CLOSED ]] || return 0
  log warn "#${n}: merging fix PR #${pr} closed the issue — reopening it (the ticket is still in progress)"
  gh issue reopen "$n" --comment "ticket-loop 무인 수습: 수정 PR #${pr}의 병합 커밋이 이 이슈를 닫아 다시 연다. 티켓은 아직 진행 중이다 — 루프가 이어 간다." \
    </dev/null >/dev/null 2>&1 || log warn "#${n}: failed to reopen the issue — reopen it by hand (gh issue reopen ${n})"
}

# 메인 체크아웃이 기본 브랜치에 있으면 fast-forward한다 — WORKTREE_SETUP_CMD가 메인 체크아웃의 스크립트를 부르는 저장소에서도 수정 PR이
# 반영되게(예: "$LOOP_ROOT_PATH/scripts/worktree-setup.sh"). git은 로컬 변경을 덮는 merge를 거부하므로 사람의 미커밋 변경은 그대로다
sync_main_checkout() {
  local cur
  cur="$(git -C "$MAIN_WT" rev-parse --abbrev-ref HEAD 2>/dev/null || true)"
  if [[ "$cur" != "$BASE_BRANCH" ]]; then
    log warn "main checkout is on ${cur:-?}, not ${BASE_BRANCH} — left as is (files the setup reads from it are not updated)"
  elif git -C "$MAIN_WT" merge --ff-only -q "origin/${BASE_BRANCH}" 2>/dev/null; then
    log step "main checkout fast-forwarded to origin/${BASE_BRANCH}"
  else
    log warn "main checkout could not fast-forward to origin/${BASE_BRANCH} (local changes in the way, or diverged) — left as is"
  fi
}

# triage 종류 사유 — 멈출 자리에서 수습 세션을 띄우고 판정을 기다린다. 반환 0: 고쳐졌으니 멈춘 자리에서 이어 간다(이어 갈 수 있는 종류의
# resume만), 1: 멈춘다 — 부른 쪽이 사유 뒤에 TRIAGE_NOTE(판정 요약, 또는 띄우지 않은 이유)를 붙여 die한다.
# 띄우지 않는 경우: LOOP_TRIAGE=0, 잠금을 안 잡은 실행, 이 실행에서 TRIAGE_MAX번을 다 씀, 같은 멈춤(종류·이슈)에 이미 띄움, 기본 브랜치에
# loop-triage 스킬이 아직 없음. TRIAGE_RESULT(티켓 기록·스펙 리포트), TRIAGE_FIX_PR(병합한 수정 PR)을 남긴다
triage() {
  local kind="$1" reason="$2" n="${STOP_ISSUE:-0}" resumable=0 key stopfile vfile prompt name tmp deadline rc=0 why
  local t_wt="" t_path="" t_branch="" t_handle="" saved_state="${TMUX_STATE_FILE:-}"
  local verdict category summary fix_pr pushed kit_bug action action_ko
  TRIAGE_NOTE=""; TRIAGE_FIX_PR=""
  # usage_limit는 띄우지 않는 쪽(*) — 수습 세션도 같은 한도에 걸린다. 사람이 터미널에서 잇거나 한도가 풀린 뒤 루프를 다시 돌린다
  case "$kind" in post_merge_failed|ci_failed|setup_failed) resumable=1 ;; worker_no_pr) ;; *) return 1 ;; esac
  [[ "$LOOP_TRIAGE" == 1 && $LOCK_HELD -eq 1 ]] || return 1
  key="${kind}#${n}"
  if [[ " ${TRIAGE_KEYS} " == *" ${key} "* ]]; then TRIAGE_NOTE=" (triage already ran for this stop)"; return 1; fi
  if [[ $TRIAGE_USED -ge $TRIAGE_MAX ]]; then TRIAGE_NOTE=" (no triage: ${TRIAGE_USED}/${TRIAGE_MAX} used in this run)"; return 1; fi
  git -C "$ROOT" fetch -q origin "$BASE_BRANCH" 2>/dev/null || true
  if ! triage_skill_on_base; then
    log warn "#${n}: no triage — the loop-triage skill is not on origin/${BASE_BRANCH} yet (merge the kit install PR)"
    TRIAGE_NOTE=" (no triage: loop-triage skill not on origin/${BASE_BRANCH})"; return 1
  fi
  TRIAGE_USED=$((TRIAGE_USED + 1)); TRIAGE_KEYS="${TRIAGE_KEYS} ${key}"
  STOP_KIND="$kind"
  stopfile="$(write_stop_record "$kind" "$reason" unattended)" \
    || { log warn "#${n}: no triage — could not write the stop record"; TRIAGE_NOTE=" (no triage: stop record not written)"; return 1; }
  STOP_RECORD="$stopfile"; vfile="${stopfile%.json}.verdict.json"
  log warn "#${n}: stopping — ${reason}"
  log step "#${n}: triage — starting a session (stop record ${stopfile#"$ROOT"/})"
  sleep 1   # 로그 파일은 tee가 비동기로 쓴다 — 위 줄이 Slack 꼬리에 들어가게
  notify "Ticket loop stopped — triage started" "$reason" triage "$(run_log | tail -n 12)"

  name="loop-triage-${n}-$(date +%H%M%S)"
  prompt="$(triage_prompt "$stopfile" "$vfile")"
  tmp="$(mktemp "${TMPDIR:-/tmp}/ticket-loop-triage-${n}.XXXXXX")"
  if ! bg_wait triage_spawn "$n" "$prompt" "$name" "$tmp" || [[ ! -s "$tmp" ]]; then
    rm -f "$tmp"
    TRIAGE_NOTE=" (triage session did not start — see the loop log)"; TRIAGE_RESULT="no session"
    return 1
  fi
  IFS=$'\t' read -r t_wt t_path t_branch t_handle < "$tmp"; rm -f "$tmp"
  CUR_AGENT_PATHS+="${t_path}"$'\n'   # 수습 세션의 토큰도 그 티켓에 센다
```

```
  [[ "$BACKEND" != tmux ]] ¦¦ TMUX_STATE_FILE="$TMUX_DIR/state/$(basename "$t_path")"   # tmux_* 가 보는 훅 상태 파일 — 끝나면 되돌린다
  backend_set_status "$t_wt" in-progress ¦¦ true
  log step "#${n}: triage session ${t_wt} (${t_path}) / terminal ${t_handle} — waiting for a verdict (up to $((TRIAGE_TIMEOUT_SEC / 60))m)"
  deadline=$(( $(date +%s) + TRIAGE_TIMEOUT_SEC ))
  wait_stop_reset
  bg_wait wait_for_verdict "$n" "$t_handle" "$vfile" "$deadline" "$t_path" ¦¦ rc=$?
  if [[ $rc -ne 0 ]]; then
    wait_stop_read
    case $rc in
      2) why="timed out after $((TRIAGE_TIMEOUT_SEC / 60))m" ;;
      3) why="lost the session terminal: $(terminal_lost_why "$t_handle"). $(terminal_lost_note "triage session")" ;;
      *) why="the session stopped without a verdict" ;;
    esac
    [[ -z "$WAIT_STOP_REASON" ]] ¦¦ why="$WAIT_STOP_REASON"   # 화면 분류로 멈춤 — 사용량 한도(usage_limit로 끝낸다) 또는 확인 창에 멈춘 세션
    [[ "$WAIT_STOP_KIND" != usage_limit ]] ¦¦ STOP_KIND=usage_limit
    log warn "#${n}: triage: ${why} — session output tail ---"; terminal_tail "$t_handle" ¦ dim_lines ¦¦ true
    TMUX_STATE_FILE="$saved_state"
    backend_set_status "$t_wt" todo "" "무인 수습: 판정 없이 끝남 — 확인 필요" ¦¦ true
    TRIAGE_NOTE=" — triage: ${why} (session worktree ${t_path})"; TRIAGE_RESULT="no verdict: ${why}"
    return 1
  fi
  TMUX_STATE_FILE="$saved_state"

  verdict="$(jq -r '.verdict' "$vfile")"
  category="$(jq -r '.category // "unknown"' "$vfile")"
  summary="$(jq -r '.summary // "" ¦ gsub("[\r\n]+"; " ")' "$vfile")"
  fix_pr="$(jq -r '.fix_pr // empty ¦ tostring ¦ select(test("^[0-9]+$"))' "$vfile" 2>/dev/null ¦¦ true)"
  pushed="$(jq -r 'if .pushed_to_pr == true then 1 else 0 end' "$vfile")"
  kit_bug="$(jq -r '.kit_bug_report // empty' "$vfile")"
  log "#${n}: triage verdict: ${verdict} (${category}) — ${summary}"
  if [[ "$verdict" == resume && $resumable -eq 0 ]]; then log warn "#${n}: triage: ${kind} is diagnosis only — stopping"; verdict=halt; fi
  if [[ "$verdict" == resume && -n "$fix_pr" && "$fix_pr" == "${CUR_PR:-}" ]]; then pushed=1; fix_pr=""; fi   # 티켓 PR 번호를 적었다 = 그 브랜치에 push
  if [[ "$verdict" == resume && -n "$fix_pr" ]]; then
    if triage_merge_fix_pr "$fix_pr"; then
      TRIAGE_FIX_PR="$fix_pr"
      [[ "$kind" == post_merge_failed ]] ¦¦ triage_keep_issue_open "$n" "$fix_pr"
      [[ "$kind" != setup_failed ]] ¦¦ sync_main_checkout
    else
      verdict=halt
    fi
  fi
  if [[ "$verdict" != resume ]]; then
    backend_set_status "$t_wt" todo "" "무인 수습: 멈춤 — 확인 필요" ¦¦ true
    TRIAGE_NOTE=" — triage (${category}): ${summary}${TRIAGE_NOTE}${kit_bug:+ · kit bug report ${kit_bug}}"
    TRIAGE_RESULT="halt (${category}): ${summary}"
    return 1
  fi

  if [[ -n "$TRIAGE_FIX_PR" ]]; then action="fix PR #${TRIAGE_FIX_PR} merged"; action_ko="수정 PR #${TRIAGE_FIX_PR} 병합"
  elif [[ $pushed -eq 1 ]]; then action="pushed to PR #${CUR_PR:-?}"; action_ko="PR #${CUR_PR:-?} 브랜치에 push"
  else action="no code change"; action_ko="코드 변경 없음"; fi
  backend_set_status "$t_wt" completed "" "무인 수습: 이어 감 (${action_ko})" ¦¦ true
  backend_worktree_remove "$t_wt" "$t_path" ¦¦ log warn "#${n}: failed to remove the triage worktree — clean up manually (${t_wt}, ${t_path})"
  git -C "$ROOT" worktree prune ¦¦ true
  [[ -z "$t_branch" ¦¦ "$t_branch" == HEAD ¦¦ "$t_branch" == "$BASE_BRANCH" ]] ¦¦ git -C "$ROOT" branch -D "$t_branch" >/dev/null 2>&1 ¦¦ true
  log ok "#${n}: triage: resuming (${action}) — ${summary}"
  TRIAGE_RESULT="resumed (${category}, ${action}): ${summary}"
  triage_resume_record "$n" "$reason" "$category" "$summary" "$action" "$action_ko"
  STOP_KIND=""; STOP_RECORD=""
  return 0
}

# ---------- 티켓 기록 · 스펙 리포트 ----------
# 루프의 상태는 GitHub에만 두지만(기억 없음), 벽시계·CI 소요·수리 횟수·병합 뒤 검증 결과·모델은 GitHub에 없다. 티켓
# 하나가 끝날 때(병합·--no-merge·중단) RECORDS_FILE에 한 줄을 덧붙이고, --parent 실행이 정상 종료하면 그 기록과 GitHub
```

```
# (하위 이슈·PR 본문)을 합쳐 REPORT_DIR/spec-<N>.html을 만든다. 기록이 없는 티켓(다른 PC·사람 처리)은 PR 시각으로 대신한다.

CUR_N=""; CUR_TITLE=""; CUR_STARTED=0; CUR_STARTED_ISO=""; CUR_PR=""; CUR_MODEL=""; CUR_CI_SEC=""; CUR_PICKUP_SEC=""; CUR_ROUND
=0
CUR_AGENT_PATHS=""   # 이 티켓의 에이전트 작업 폴더(워커·무인 수습), 한 줄에 하나 — 토큰을 셀 세션 기록을 찾는다(ticket_session
_dirs)

# 티켓의 경과 시간을 단계별로 센다 — 구현(에이전트가 뜬 때부터 PR이 열린 때까지), 수리(수리 프롬프트부터 수리 끝까지, 충돌·CI
모두),
# CI(첫 회는 PR이 열린 때부터 — CI는 그때 돈다 — 수리 뒤에는 루프가 다시 보기 시작한 때부터 결과까지), 병합 뒤 검증(명령이 도는
동안,
# 재실행 포함). 나머지(워크트리 준비, PR을 알아채기, 병합·정리, 포트 대기, 무인 수습)는 세지 않는다 — 리포트가 경과 시간에서 뺀
나머지로
# 보인다. 0.9.7까지는 경과 시간 하나와 마지막 CI 회차만 남아, 몇 분이 구현이고 몇 분이 CI·통합 테스트인지 리포트에서 알 수 없었
다.
# 단계는 메인 셸에서만 바꾼다(bg_wait 서브셸 안에서 바꾸면 부모에 남지 않는다). 멈추면 record_ticket이 열린 단계를 그 시각에 닫
는다
PH_NAME=""; PH_AT=0; PH_IMPL=0; PH_REPAIR=0; PH_CI=0; PH_POST=0
phase() {   # $1 새 단계(impl¦repair¦ci¦post, 빈 값이면 세지 않음), $2 바뀐 시각(생략하면 지금 — 지금 단계가 시작된 때보다 이르
면 그때)
  local at="${2:-}" d
  [[ -n "$at" ]] ¦¦ at="$(date +%s)"
  [[ "$at" -ge "${PH_AT:-0}" ]] ¦¦ at="$PH_AT"
  d=$(( at - ${PH_AT:-0} ))
  case "${PH_NAME:-}" in
    impl) PH_IMPL=$(( ${PH_IMPL:-0} + d )) ;;
    repair) PH_REPAIR=$(( ${PH_REPAIR:-0} + d )) ;;
    ci) PH_CI=$(( ${PH_CI:-0} + d )) ;;
    post) PH_POST=$(( ${PH_POST:-0} + d )) ;;
  esac
  PH_NAME="$1"; PH_AT="$at"
}

# 워커 터미널의 에이전트 상태줄·배너에서 모델 이름을 뽑는다(Claude Code: "[Opus 5.5 ¦ Max]", Codex: 기동 블록의 "model: gpt-5.1
-codex"). 못 찾으면 빈 값.
detect_model() { [[ -n "$MODEL_REGEX" ]] ¦¦ return 0; terminal_tail "$1" 2>/dev/null ¦ grep -oE "$MODEL_REGEX" ¦ tail -n1 ¦¦ tr
ue; }

# 티켓이 속한 스펙 이슈 번호 — --parent가 없으면(이슈 번호 직접 지정) GitHub의 부모 이슈를 묻는다. 없으면 0.
spec_of_ticket() {
  [[ -z "$PARENT" ]] ¦¦ { printf '%s' "$PARENT"; return; }
  gh api graphql -f query="{ repository(owner: \"${OWNER_REPO%%/*}\", name: \"${OWNER_REPO#*/}\") { issue(number: $1) { parent
{ number } } } }" \
    --jq '.data.repository.issue.parent.number // 0' 2>/dev/null ¦¦ echo 0
}

# 스펙 이슈의 하위 이슈가 모두 닫혔으면 스펙 이슈를 닫는다. 요약 값 sub_issues_summary.completed 대신 목록을 센다
# (blocked_by 요약처럼 늦게 바뀔 수 있다). 하위 이슈가 없거나 하나라도 열려 있거나 스펙이 이미 닫혔으면 아무것도 하지 않는다.
close_spec_if_done() {
  local spec="${1:-}" counts total open
  [[ -n "$spec" && "$spec" != "0" ]] ¦¦ return 0
  [[ "$(gh issue view "$spec" --json state -q .state 2>/dev/null)" == "OPEN" ]] ¦¦ return 0
  counts="$(gh api "repos/${OWNER_REPO}/issues/${spec}/sub_issues?per_page=100" --paginate 2>/dev/null \
    ¦ jq -rs 'add // [] ¦ "\(length) \(map(select(.state == "open")) ¦ length)"')" ¦¦ return 0
  read -r total open <<<"$counts"
  [[ "${total:-0}" -gt 0 && "${open:-1}" -eq 0 ]] ¦¦ return 0
  if gh issue close "$spec" --comment "하위 이슈 ${total}건이 모두 닫혀 완료 (ticket-loop)" >/dev/null 2>&1; then
    log ok "spec #${spec}: all ${total} sub-issues closed → closed the spec issue"
    CLOSED_SPECS="${CLOSED_SPECS:-} ${spec}"
  else
    log warn "spec #${spec}: all ${total} sub-issues are closed but the spec issue could not be closed — gh issue close ${spe
c}"
  fi
}

# 스펙의 티켓이 지나간 루프 로그들 — 루프가 멈췄다가 다른 날 다시 시작하면 로그 파일(loop-MMDD.log)이 여럿이 된다. 0.9.6까지 회
고
# 안내는 이번 실행의 로그만 적어, 이어 간 스펙의 회고가 끊긴 지점부터만 읽었다. 기록의 log(0.9.7부터)를 모으고, 그 필드가 없는
지난
# 기록은 로그 디렉터리의 loop-*.log에서 그 티켓의 머리줄(=== #N)을 찾는다. 이번 실행의 로그는 늘 넣는다. 지워진 로그는 뺀다.
# 저장소 루트 기준 경로를 기록 순서대로 한 줄에 하나씩 쓴다
spec_logs() {
```

```
  local spec="$1" cur n f p
  [[ "$LOG_FILE" != "/dev/null" ]] || return 0
  cur="$LOG_FILE"; [[ -z "${ROOT:-}" ]] || cur="${cur#"$ROOT"/}"
  {
    if [[ -f "${RECORDS_FILE:-}" ]]; then
      while IFS=$'\t' read -r n f; do
        if [[ -n "$f" ]]; then printf '%s\n' "$f"
        else grep -lF "] === #${n} " "$(dirname "$LOG_FILE")"/loop-*.log 2>/dev/null || true; fi
      done < <(jq -rR --argjson spec "$spec" 'fromjson? | select(.spec == $spec) | "\(.issue)\t\(.log // "")"' "$RECORDS_FILE"
2>/dev/null || true)
    fi
    printf '%s\n' "$LOG_FILE"
  } | while IFS= read -r p; do
    [[ -z "${ROOT:-}" ]] || p="${p#"$ROOT"/}"
    if [[ "$p" == "$cur" || -f "$p" || -f "${ROOT:-.}/$p" ]]; then printf '%s\n' "$p"; fi
  done | awk '!seen[$0]++'
}

# 끝난 스펙이 있으면 회고를 권한다 — 안내일 뿐, 아무것도 실행하지 않는다. /retro 는 사람이 후보를 보고 고르는 스킬이라 무인 단
계로 두지 않는다
suggest_retro() {
  local spec report skill="${SKILL_PREFIX:-/}retro" logs decs
  [[ -n "${CLOSED_SPECS// /}" ]] || return 0
  [[ -d "${ROOT:-.}/.agents/skills/retro" || -d "${ROOT:-.}/.claude/skills/retro" ]] || skill="${skill} (not in this repository
yet — install it with the kit's --workflow-skills)"
  for spec in $CLOSED_SPECS; do
    report="$REPORT_DIR/spec-${spec}.html"
    if [[ -f "$report" || -f "${ROOT:-.}/$report" ]]; then report="report ${report}"
    else report="the report can be generated with ticket-loop.sh --parent ${spec}"; fi
    logs="$(spec_logs "$spec" | awk 'NR > 1 { printf ", " } { printf "%s", $0 }')"
    if [[ -z "$logs" ]]; then logs="log (none — logging is off)"
    elif [[ "$logs" == *", "* ]]; then logs="logs ${logs}"
    else logs="log ${logs}"; fi
    decs=""
    [[ -z "${REPORT_DECISIONS_HEADING:-}" || "$spec" != "${PARENT:-}" || -z "${REPORT_DECISIONS:-}" ]] \
      || decs=" (its '${REPORT_DECISIONS_HEADING}' list collects ${REPORT_DECISIONS} PR$([[ "$REPORT_DECISIONS" == 1 ]] || echo
s) for a human)"
    log "spec #${spec} complete. If you like, review this batch with ${skill} — material to read: ${logs}, ${report}${decs}, an
d the 'remaining decisions' / 'not verified' sections of the merged PR bodies. If you end up changing the environment (skills,
worker-rules.md, automated checks), a PR is recommended"
  done
}

# idle 오판 수 — 일하던 워커를 idle로 본 횟수(wait_for_pr 머리 주석). 0.9.1까지는 "idle 뒤 다시 보니 PR이 열려 있음"을 셌다.
# wait_for_pr는 서브셸(bg_wait, 0.8.7까지는 커맨드 치환)에서 돌아 변수로 세면 부모에 남지 않는다 — 0.8.9까지 요약 줄과 기록의
# idle_false_positives가 늘 0이었다. 티켓마다 IDLE_FP_FILE에 한 번에 한 줄씩 쓰고 줄 수를 센다
idle_fp_count() { if [[ -n "${IDLE_FP_FILE:-}" && -f "$IDLE_FP_FILE" ]]; then echo $(( $(wc -l < "$IDLE_FP_FILE") )); else echo
0; fi; }

# 티켓에 든 에이전트 토큰 — Claude Code 세션 기록에서 더한다. 기록은 CLAUDE_CONFIG_DIR(기본 ~/.claude)의 projects/<작업 폴더 경
로에서
# 영숫자 밖 글자를 -로 바꾼 이름>/ 아래 *.jsonl이고, 서브에이전트 기록은 그 아래 subagents/에 있다. 응답마다 message.usage가 남
는데 한 응답이
# 내용 블록마다 줄로 나뉘어 같은 usage가 되풀이되므로 message.id로 하나만 센다(rental-portal #250: 394줄 → 응답 210개). 워커 세
션과 그 티켓의
# 무인 수습 세션(loop-triage-N-HHMMSS)을 더한다. Codex는 세션 기록 형식이 달라 아직 세지 않는다
claude_project_dir() { printf '%s/projects/%s' "${CLAUDE_CONFIG_DIR:-$HOME/.claude}" "$(printf '%s' "$1" | sed 's/[^A-Za-z0-9]/
-/g')"; }

# $1 이슈 번호, 나머지 그 밖의 작업 폴더(Orca 워크트리 등 — WORKTREE_DIR 밖) → 있는 세션 기록 폴더, 한 줄에 하나
ticket_session_dirs() {
  local n="$1" d
  shift
  {
    if [[ -n "${WORKTREE_DIR:-}" ]]; then
      claude_project_dir "${WORKTREE_DIR}/${BRANCH_PREFIX}${n}"; echo
      for d in "$(claude_project_dir "${WORKTREE_DIR}/loop-triage-${n}-")"*; do printf '%s\n' "$d"; done
    fi
    for d in "$@"; do [[ -z "$d" ]] || { claude_project_dir "$d"; echo; }; done
  } | while IFS= read -r d; do [[ ! -d "$d" ]] || printf '%s\n' "$d"; done | awk '!seen[$0]++'
}
```

```
# stdin 세션 기록 폴더들, $1 이 시각부터, $2 이 시각까지(빈 값이면 끝 없음 — 둘 다 ISO, 같은 경로를 쓴 앞선 시도를 빼려고) →
# {"input","output","cache_write","cache_read"} 한 줄. 응답이 하나도 없으면 빈 값
session_tokens() {
  local since="$1" upto="${2:-}" d f files=()
  while IFS= read -r d; do
    [[ -d "$d" ]] || continue
    while IFS= read -r -d '' f; do files+=("$f"); done < <(find "$d" -type f -name '*.jsonl' -print0 2>/dev/null)
  done
  [[ ${#files[@]} -gt 0 ]] || return 0
  cat "${files[@]}" | jq -nRc --arg since "$since" --arg upto "$upto" '
    reduce (inputs | fromjson? | select(type == "object" and .type == "assistant" and (.message.usage | type) == "object")
            | select((.timestamp // "") >= $since and ($upto == "" or (.timestamp // "") <= $upto))) as $m
      ({}; .[$m.message.id // $m.uuid // "?"] = $m.message.usage)
    | [.[]] | select(length > 0)
    | {input: (map(.input_tokens // 0) | add), output: (map(.output_tokens // 0) | add),
       cache_write: (map(.cache_creation_input_tokens // 0) | add), cache_read: (map(.cache_read_input_tokens // 0) | add)}' 2
>/dev/null || true
}

record_ticket() {
  local result="$1" reason="${2:-}" file="$RECORDS_FILE" spec   # RECORDS_FILE은 시작 때 절대 경로로 굳혔다
  [[ "$LOG_FILE" != "/dev/null" ]] || return 0
  phase ""   # 열린 단계(멈춘 때의 구현·CI 등)를 지금 닫는다
  mkdir -p "$(dirname "$file")"
  spec="$(spec_of_ticket "$CUR_N")"
  local tokens="" paths=() f
  if [[ "$AGENT" == claude ]]; then
    while IFS= read -r f; do [[ -z "$f" ]] || paths+=("$f"); done <<<"${CUR_AGENT_PATHS:-}"
    tokens="$(ticket_session_dirs "$CUR_N" ${paths[@]+"${paths[@]}"} | session_tokens "$CUR_STARTED_ISO")"
  fi
  jq -cn --argjson spec "${spec:-0}" --argjson issue "$CUR_N" --arg title "$CUR_TITLE" --arg result "$result" --arg reason "$re
ason" \
    --arg pr "$CUR_PR" --arg pickup "${CUR_PICKUP_SEC:-}" --arg started "$CUR_STARTED_ISO" --arg finished "$(date -u '+%Y-%m-%d
T%H:%M:%SZ')" \
    --argjson wall "$(( $(date +%s) - CUR_STARTED ))" --arg ci "$CUR_CI_SEC" --argjson repairs "$CUR_ROUND" \
    --argjson idle "$(idle_fp_count)" --arg post "${POST_MERGE_RESULT:-}" --arg model "$CUR_MODEL" --arg agent "$AGENT" \
    --arg pmwhy "${POST_MERGE_WHY:-}" --arg pmspec "${POST_MERGE_SPEC_COMPLETE:-}" \
    --arg triage "${TRIAGE_RESULT:-}" --arg log "${LOG_FILE#"$ROOT"/}" \
    --argjson impl "${PH_IMPL:-0}" --argjson repair "${PH_REPAIR:-0}" --argjson ciwait "${PH_CI:-0}" --argjson postsec "${PH_PO
ST:-0}" \
    --arg tokens "$tokens" '
    {spec: $spec, issue: $issue, title: $title, result: $result, started: $started, finished: $finished, wall_sec: $wall,
     impl_sec: $impl, repair_sec: $repair, ci_wait_sec: $ciwait, post_merge_sec: $postsec,
     repairs: $repairs, idle_false_positives: $idle, agent: $agent, log: $log}
    + (if $reason != "" then {reason: $reason} else {} end)
    + (if $triage != "" then {triage: $triage} else {} end)
    + (if $pr != "" then {pr: ($pr | tonumber)} else {} end)
    + (if $pickup != "" then {pr_pickup_sec: ($pickup | tonumber)} else {} end)
    + (if $ci != "" then {ci_sec: ($ci | tonumber)} else {} end)
    + (if $post != "" then {post_merge: $post} else {} end)
    + (if $pmwhy != "" then {post_merge_why: $pmwhy} else {} end)
    + (if $pmspec != "" then {post_merge_spec_complete: ($pmspec | split(" ") | map(select(. != "") | tonumber))} else {} end)
    + (if $tokens != "" then {tokens: ($tokens | fromjson)} else {} end)
    + (if $model != "" then {model: $model} else {} end)' >> "$file" 2>/dev/null \
    || log warn "#${CUR_N}: failed to write the ticket record (${file})"
}

# 스펙 리포트 HTML — jq 하나로 그린다(의존 도구를 늘리지 않는다). 표: 이슈·티켓·상태·결과 브리프(PR 제목, 본문의
# 인수 조건 체크 수, 중단 사유, 수리·병합 뒤 검증 결과)·구현·CI·병합 뒤 검증·경과 시간(단계 막대)·모델·마친 시각. 문자열은 전부
@html로 이스케이프한다.
# 리포트의 고정 문구는 영어, GitHub에서 가져온 값(이슈·PR 제목, 본문)과 기록의 중단 사유는 그대로(한국어면 한국어) 둔다.
# 기록의 post_merge는 0.8.7부터 영어("passed in Ns"·"failed in Ns"·"skipped")다. 그 전 기록의 한국어 값("통과 N초"·"실패 N
초"·"건너뜀")은
# 표시할 때만 영어로 바꾼다(기록 파일은 그대로).
read -r -d '' REPORT_JQ <<'JQ' || true
def dur: if . == null then "—" elif . >= 3600 then "\(. / 3600 | floor)h \((. % 3600) / 60 | floor)m"
         elif . >= 60 then "\(. / 60 | floor)m" else "\(. | floor)s" end;
def epoch: if . == null or . == "" then null else (sub("\\.[0-9]+Z$"; "Z") | fromdate) end;
def when: epoch | if . == null then "—" else strflocaltime("%Y-%m-%d %H:%M") end;
def order: (.title | capture("^T(?<n>[0-9]+)") | .n | tonumber) // 999999;
# jq(Oniguruma Perl 문법)의 ^는 줄 머리가 아니라 문자열 머리다 — 앞에 \n을 붙여 줄 머리를 \n으로 잡는다
def checks: ("\n" + .) | {checked: ([scan("\n[ \t]*[-*] \\[x\\]"; "i")] | length), total: ([scan("\n[ \t]*[-*] \\[[ x]\\]";
```

```
"i")] | length)};
def badge: {merged: ["Merged", "ok"], "merged-after-stop": ["Merged after stop", "warn"], "closed-without-pr": ["Closed without
PR", "muted"], "pr-open": ["PR open", "warn"],
           "in-progress": ["In progress", "warn"], stopped: ["Stopped", "bad"], pending: ["Not started", "muted"]}[.];
# 0.8.6 이하 기록의 한국어 post_merge 값을 표시용 영어로. 그 밖(이미 영어)은 그대로.
def post: if . == "건너뜀" then "skipped"
          else (capture("^(?<k>통과|실패) (?<s>[0-9]+)초$") // null) as $m
          | if $m == null then . else "\(if $m.k == "통과" then "passed" else "failed" end) in \($m.s)s" end end;
# 기록 하나의 병합 뒤 검증이 받은 env(#32) — " (every, spec #60 complete)". 그 값이 없는 기록(미룬 티켓, 그 전 기록)은 빈 문자
열
def pmenv: [.post_merge_why // empty, ((.post_merge_spec_complete // []) | if length > 0 then "spec \(map("#\(.)") | join(" "))
complete" else empty end)]
           | if length > 0 then " (\(join(", ")))" else "" end;
def pos: if . != null and . > 0 then . else null end;
def tok: def d1($u; $s): (. / ($u / 10) | floor) as $d | "\($d / 10 | floor).\($d % 10)\($s)";   # 소수 한 자리(2.0M)
         if . == null then "—" elif . >= 1e9 then d1(1e9; "B") elif . >= 1e6 then d1(1e6; "M") elif . >= 1e3 then "\(. / 1e3 |
floor)k" else "\(.)" end;
def tcell($v; $note): "<td class=\"nw\">" + (if $v == -1 then "none" else ($v | dur) end) + (if $note == "" then "" else "<div
class=\"muted small\">\($note)</div>" end) + "</td>";
def seg($v; $c; $den): if ($v // 0) > 0 then "<i class=\"\($c)\" style=\"width:\($v * 1000 / $den | floor / 10)%\"></i>" else
"" end;
# PR 본문에서 제목이 $h와 글자 그대로 같은 마크다운 절(#~######)의 본문 — 다음 같은·상위 수준 제목 앞까지(하위 절은 포함). 제목
이 비었거나
# 절이 없거나 본문이 비면 null. 리포트 머리의 "사람 확인 필요" 목록이 쓴다(REPORT_DECISIONS_HEADING, 이슈 #25)
def section($h): if $h == "" or . == null then null else
  split("\n") as $lines
  | ($lines | map(capture("^(?<l>#{1,6})[ \t]+(?<t>.*?)[ \t]*#*[ \t]*$") // null)) as $hs
  | ($hs | to_entries | map(select(.value != null and .value.t == $h)) | first) as $start
  | if $start == null then null else
      ($start.value.l | length) as $lvl
      | (($hs | to_entries | map(select(.key > $start.key and .value != null and (.value.l | length) <= $lvl)) | first | .key)
// ($lines | length)) as $end
      | $lines[$start.key + 1:$end] | join("\n") | gsub("^\\s+|\\s+$"; "") | if . == "" then null else . end
    end end;
# 절 본문이 "없음" 표시뿐인가 — 목록 기호(-·*)와 끝 마침표를 걷은 본문이 $none("|" 구분, 대소문자 무시)의 하나와 같으면 참.
# pr 스킬이 "없으면 '없음'"이라 쓰게 하는 저장소에서 그런 절을 빈 절처럼 뺀다(REPORT_DECISIONS_NONE, 이슈 #39)
def noneonly($none): (sub("^[-*][ \t]+"; "") | sub("\\.+$"; "") | gsub("^\\s+|\\s+$"; "") | ascii_downcase) as $b
  | $none | split("|") | map(gsub("^\\s+|\\s+$"; "") | ascii_downcase) | any(. != "" and . == $b);
($ARGS.named.heading // "") as $heading   # REPORT_DECISIONS_HEADING — --arg heading 이 없으면(지난 호출 꼴) 비움
| ($ARGS.named.none // "") as $none       # REPORT_DECISIONS_NONE — --arg none 이 없으면(지난 호출 꼴) 빈 절만 뺀다
| ($issues | sort_by(order, .number) | map(. as $i
  | ($recs | map(select(.issue == $i.number)) | sort_by(.finished)) as $rs | ($rs | last) as $r
  | ($prefix + ($i.number | tostring)) as $b
  | ($prs | map(select(.headRefName == $b or (.headRefName | startswith($b + "-"))))
     | (map(select(.state == "MERGED")) + .) | (map(select(.number == $r.pr)) + .) | first) as $pr
  # 루프가 멈춘 뒤 루프 밖(사람)에서 병합된 티켓 — 마지막 기록이 stopped이고 PR이 그 뒤에 병합됐다. 루프가 병합하고 나서 멈췄으
면 병합이 먼저다
  | (if $pr.state == "MERGED" and $r.result == "stopped" then (($pr.mergedAt | epoch) // 0) - (($r.finished | epoch) // 0) else
0 end) as $gap
  | {issue: $i, pr: $pr, rec: $r, attempts: ($rs | length), stop_gap: $gap,
     status: (if $pr.state == "MERGED" then (if $gap > 0 then "merged-after-stop" else "merged" end) elif $i.state == "closed"
then "closed-without-pr"
              elif $pr.state == "OPEN" then "pr-open" elif $r.result == "stopped" then "stopped"
              elif ($i.assignees // 0) > 0 then "in-progress" else "pending" end),
     wall: (if $r.wall_sec != null then {sec: $r.wall_sec, src: "loop"}
            elif $pr.mergedAt != null then {sec: (($pr.mergedAt | epoch) - ($pr.createdAt | epoch)), src: "pr"} else null end),
     # 경과 시간 나누기 — 0.9.8부터 기록에 단계별 시간이 있다(record_ticket·phase). 그 전 기록은 어림한다: 구현은 PR 생성 시각
- 시작(수리
     # 시간은 모른다), CI는 마지막 회차(ci_sec), 병합 뒤 검증은 결과 문자열의 초를 더한 값. PR 없이 멈춘 기록은 경과 전부가 구
현이다
     t: (if $r == null then null
         elif ($r | has("impl_sec")) then
           {impl: (($r.impl_sec // 0) + ($r.repair_sec // 0) | pos), repair: ($r.repair_sec | pos),
            ci: (if $r.ci_sec == -1 then -1 else ($r.ci_wait_sec | pos) end), post: ($r.post_merge_sec | pos), est: false}
         else
           {impl: (if $r.pr == null and $r.result == "stopped" then ($r.wall_sec | pos)
                   else (($pr.createdAt | epoch) as $c | ($r.started | epoch) as $s
                         | if $c == null or $s == null or $c < $s then null else ([$c - $s, $r.wall_sec // ($c - $s)] | min | p
os) end) end),
            repair: null, ci: (if $r.ci_sec == -1 then -1 else ($r.ci_sec | pos) end),
            post: ($r.post_merge // "" | post | [scan(" in ([0-9]+)s") | .[0] | tonumber] | add | pos), est: true} end),
     # 토큰 — 기록의 tokens(0.9.8부터), 없으면 write_spec_report가 세션 기록에서 메운 값. total은 네 값의 합
```

```
     tk: ($r.tokens | if type == "object" then . + {total: ((.input // 0) + (.output // 0) + (.cache_write // 0) + (.cache_read // 0))} else null end),
     model: ($r.model // "not recorded"),
     # 그 PR 본문의 REPORT_DECISIONS_HEADING 절(없거나 비었거나 없음 표시뿐이면 null) — 머리 목록과 행 표시가 쓴다
     dec: ($pr.body // "" | section($heading) | if . != null and noneonly($none) then null else . end)})) as $rows
| ($rows | map(select(.dec != null))) as $decs
| ($rows | map(select(.status == "merged" or .status == "merged-after-stop")) | length) as $merged
| ($rows | map(select(.issue.state == "closed")) | length) as $closed
| ($rows | map(select(.wall.src == "loop") | .wall.sec) | add // 0) as $total
| ([$rows[].tk.total | pos] | add) as $tok_total | ([$rows[].tk.output | pos] | add) as $tok_out
| ([$rows[].t.impl | pos] | add) as $impl_total | ([$rows[].t.ci | pos] | add) as $ci_total | ([$rows[].t.post | pos] | add) as $post_total
| ($rows | map(.model) | map(select(. != "not recorded")) | unique | join(", ")) as $models
| "<!doctype html>\n<html lang=\"en\"><head><meta charset=\"utf-8\"><meta name=\"viewport\" content=\"width=device-width, initial-scale=1\">"
  + @html "<title>#\($spec.number) \($spec.title) — Loop report</title>"
  + "<style>:root{--ink:#1c1f24;--muted:#5b6270;--line:#d9dde3;--soft:#f4f6f8;--bg:#fff;--accent:#2457c5;--ok:#1f7a4d;--ok-soft:#e6f4ec;--warn:#9a6700;--warn-soft:#fff6d6;--bad:#b4441c;--bad-soft:#fbeee8}"
  + "@media (prefers-color-scheme:dark){:root{--ink:#e6e8eb;--muted:#9aa3ad;--line:#3a4047;--soft:#22262b;--bg:#15181b;--accent:#7ea2ff;--ok:#5fcf8f;--ok-soft:#1c3327;--warn:#e3b341;--warn-soft:#3a3012;--bad:#ff8a65;--bad-soft:#3b2219}}"
  + "*{box-sizing:border-box}body{margin:0;padding:24px 16px 48px;color:var(--ink);background:var(--bg);font-family:system-ui,-apple-system,\"Segoe UI\",\"Apple SD Gothic Neo\",\"Noto Sans KR\",sans-serif;font-size:14px;line-height:1.55}"
  + "main{max-width:1360px;margin:0 auto}h1{font-size:22px;margin:0 0 4px}a{color:var(--accent);text-decoration:none}.kicker{color:var(--accent);font-weight:700;font-size:12px;letter-spacing:.06em}"
  + ".meta{color:var(--muted);font-size:13px;margin:0 0 18px}.kpis{display:grid;grid-template-columns:repeat(auto-fit,minmax(140px,1fr));gap:10px;margin:0 0 20px}"
  + ".kpi{background:var(--soft);border:1px solid var(--line);border-radius:8px;padding:10px 12px}.kpi .v{font-size:20px;font-weight:700}.kpi .l{color:var(--muted);font-size:12px}"
  + ".wrap{overflow-x:auto}table{width:100%;border-collapse:collapse;font-size:13px}th,td{border:1px solid var(--line);padding:8px 10px;vertical-align:top;text-align:left}th{background:var(--soft);white-space:nowrap}"
  + "td.nw{white-space:nowrap}.bits{margin-top:4px;color:var(--muted);font-size:12px}.reason{margin-top:4px;color:var(--bad);font-size:12px}.muted{color:var(--muted)}.small{font-size:12px}"
  + ".badge{display:inline-block;font-size:12px;font-weight:700;border-radius:10px;padding:0 8px;border:1px solid var(--line);white-space:nowrap}.badge.ok{color:var(--ok);background:var(--ok-soft)}"
  + ".badge.warn{color:var(--warn);background:var(--warn-soft)}.badge.bad{color:var(--bad);background:var(--bad-soft)}.badge.muted{color:var(--muted);background:var(--soft)}footer{margin-top:20px;color:var(--muted);font-size:12px}"
  + ".bar{display:flex;height:6px;min-width:96px;margin-top:6px;border-radius:3px;overflow:hidden;background:var(--line)}.bar i{display:block;height:100%}"
  + ".dot{display:inline-block;width:8px;height:8px;border-radius:50%;margin-right:5px}.s-impl{background:var(--accent)}th.tm{white-space:normal}th.tm .k{white-space:nowrap}td.tk{min-width:130px}td.br{min-width:220px}.s-ci{background:var(--warn)}.s-post{background:var(--ok)}"
  + ".dec{margin:0 0 20px;padding:12px 14px;border:1px solid var(--warn);border-left-width:4px;border-radius:8px;background:var(--warn-soft)}.dec h2{font-size:15px;margin:0 0 8px}.dec .item{margin-top:10px}.dec .item:first-of-type{margin-top:0}.dec .who{font-weight:700;font-size:13px}.dec .md{margin:4px 0 0;white-space:pre-wrap;font-size:13px}"
  + ".flag{margin-top:4px;font-size:12px}.flag a{color:var(--warn);font-weight:700}</style></head>"
  + "<body><main><div class=\"kicker\">Unattended ticket loop · Spec report</div>"
  + @html "<h1><a href=\"\($spec.url)\">#\($spec.number)</a> \($spec.title)</h1>"
  + "<p class=\"meta\">Spec " + (if ($rows | length) > 0 and $closed == ($rows | length) then "complete (all sub-tickets closed)" else "in progress (\($closed)/\($rows | length) closed)" end)
  + " · generated \($now) · spec issue " + (if $spec.state == "CLOSED" then "closed" else "open" end) + "</p>"
  + "<div class=\"kpis\"><div class=\"kpi\"><div class=\"v\">\($rows | length)</div><div class=\"l\">Tickets</div></div><div class=\"kpi\"><div class=\"v\">\($merged)</div><div class=\"l\">Merged</div></div>"
  + "<div class=\"kpi\"><div class=\"v\">\($total | dur)</div><div class=\"l\">Total loop wall-clock (recorded tickets)</div></div>"
  + "<div class=\"kpi\"><div class=\"v\">\($impl_total | dur)</div><div class=\"l\"><span class=\"dot s-impl\"></span>Implementation</div></div>"
  + "<div class=\"kpi\"><div class=\"v\">\($ci_total | dur)</div><div class=\"l\"><span class=\"dot s-ci\"></span>CI</div></div>"
  + "<div class=\"kpi\"><div class=\"v\">\($post_total | dur)</div><div class=\"l\"><span class=\"dot s-post\"></span>Post-merge check</div></div>"
  + "<div class=\"kpi\"><div class=\"v\">\($tok_total | tok)</div><div class=\"l\">Tokens (output \($tok_out | tok))</div></div>"
  + @html "<div class=\"kpi\"><div class=\"v\">\(if $models == "" then "not recorded" else $models end)</div><div class=\"l\">Models used</div></div>"
  + (if $heading == "" then "" else @html "<div class=\"kpi\"><div class=\"v\">\($decs | length)</div><div class=\"l\">\($heading)</div></div>" end) + "</div>"
  # 사람이 봐야 할 절 모음 — 설정한 제목의 절이 있는 PR만, 티켓·PR 링크와 절 본문 그대로(마크다운 원문, 줄 바꿈 유지). 하나도 없으면 블록도 없다.
  # data-count는 write_spec_report가 로그 한 줄에 건수를 적을 때 읽는다
  + (if ($decs | length) == 0 then "" else
       "<section class=\"dec\" id=\"decisions\" data-count=\"\($decs | length)\">"
       + @html "<h2>\($heading) <span class=\"muted\">— \($decs | length) PR\(if ($decs | length) == 1 then "" else "s" end)</s
```

```
pan></h2>"
       + ($decs ¦ map(@html "<div class=\"item\"><div class=\"who\"><a href=\"\(.issue.url)\">#\(.issue.number)</a> \(.issue.title) · <a href=\"\(.pr.url)\">PR #\(.pr.number)</a></div><div class=\"md\">\(.dec)</div></div>") ¦ join(""))
       + "</section>" end)
  + "<div class=\"wrap\"><table><thead><tr><th>Issue</th><th>Ticket</th><th>Status</th><th>Result brief</th><th class=\"tm\"><span class=\"k\"><span class=\"dot s-impl\"></span>Implementation</span></th><th class=\"tm\"><span class=\"k\"><span class=\"dot s-ci\"></span>CI</span></th><th class=\"tm\"><span class=\"k\"><span class=\"dot s-post\"></span>Post-merge</span> check</th><th>Elapsed</th><th>Tokens</th><th>Model</th><th>Finished</th></tr></thead><tbody>"
  + ($rows ¦ map(
      @html "<tr><td class=\"nw\"><a href=\"\(.issue.url)\">#\(.issue.number)</a></td><td class=\"tk\">\(.issue.title)</td>"
      + "<td><span class=\"badge \(.status ¦ badge ¦ .[1])\">\(.status ¦ badge ¦ .[0])</span></td><td class=\"br\">"
      + (if .pr then @html "<div><a href=\"\(.pr.url)\">PR #\(.pr.number)</a> \(.pr.title)</div>"
                     + ((.pr.body // "" ¦ checks) ¦ if .total > 0 then "<div>Acceptance criteria \(.checked)/\(.total)</div>" else "" end)
                     + (if .dec != null then @html "<div class=\"flag\"><a href=\"#decisions\">⚑ \($heading)</a></div>" else "" end)
         elif .rec == null then "<span class=\"muted\">no record</span>" else "" end)
      + (if .rec.result == "stopped" then @html "<div class=\"reason\">Stop reason: \(.rec.reason // "—")</div>" else "" end)
      + (if .rec then "<div class=\"bits\">" + ([
            "\(.rec.repairs // 0) repair\(if (.rec.repairs // 0) == 1 then "" else "s" end)",
            # 0.9.6까지는 병합 전에 멈춘 기록에도 "skipped"가 남았다 — 멈춘 기록의 skipped는 그리지 않는다. 걸린 시간은 열에 있으니 결과만
            (if .rec.post_merge and (.rec.result != "stopped" or .rec.post_merge != "skipped") then @html "post-merge check \(.rec.post_merge ¦ post ¦ gsub(" in [0-9]+s"; ""))\(.rec ¦ pmenv)" else empty end),
            (if .status == "merged-after-stop" then "merged outside the loop \(.stop_gap ¦ dur) after the stop" else empty end),
            (if .rec.triage then @html "triage \(.rec.triage)" else empty end),
            (if .attempts > 1 then "\(.attempts) attempts" else empty end)] ¦ join(" · ")) + "</div>" else "" end)
      + "</td>"
      + (.t as $t ¦ .rec as $r
         ¦ tcell($t.impl; if $t.impl == null then "" elif $t.est then (if ($r.repairs // 0) > 0 then "excl. repairs" else "" end)
                          elif $t.repair then "incl. repair \($t.repair ¦ dur)" else "" end)
         + tcell($t.ci; if $t.ci == -1 then "path excluded" elif $t.ci and $t.est and ($r.repairs // 0) > 0 then "last run only" else "" end)
         + tcell($t.post; ""))
      + "<td class=\"nw\">" + (if .wall == null then "—" else (.wall.sec ¦ dur)
          + (if .wall.src == "pr" then "<div class=\"muted small\">PR created→merged</div>"
             elif .status == "merged-after-stop" then "<div class=\"muted small\">loop, until the stop</div>" else "" end)
          + (if .t == null or .wall.sec <= 0 then "" else
               (.t.impl // 0) as $i ¦ ([.t.ci, 0] ¦ max) as $c ¦ (.t.post // 0) as $p ¦ ([.wall.sec, $i + $c + $p] ¦ max) as $den
               ¦ (.wall.sec - $i - $c - $p) as $o
               ¦ ([if $i > 0 then "implementation \($i ¦ dur)" else empty end, if $c > 0 then "CI \($c ¦ dur)" else empty end,
                   if $p > 0 then "post-merge check \($p ¦ dur)" else empty end, if $o > 0 then "other \($o ¦ dur)" else empty end] ¦ join(" · ")) as $tip
               ¦ "<div class=\"bar\" title=\"\($tip)\">"
                 + seg($i; "s-impl"; $den) + seg($c; "s-ci"; $den) + seg($p; "s-post"; $den) + "</div>"
                 + (if $o >= 60 then "<div class=\"muted small\">other \($o ¦ dur)</div>" else "" end) end) end) + "</td>"
      + "<td class=\"nw\">" + (if .tk == null then "—" else
          "<span title=\"input \(.tk.input // 0) · cache write \(.tk.cache_write // 0) · cache read \(.tk.cache_read // 0) · output \(.tk.output // 0)\">\(.tk.total ¦ tok)</span>"
          + "<div class=\"muted small\">out \(.tk.output // 0 ¦ tok)<br>new in \((.tk.input // 0) + (.tk.cache_write // 0) ¦ tok)<br>cached \(.tk.cache_read // 0 ¦ tok)</div>" end) + "</td>"
      + @html "<td class=\"nw\">\(.model)</td>"
      + "<td class=\"nw\">" + (if .status == "merged-after-stop" or .rec == null and .pr.mergedAt then (.pr.mergedAt ¦ when) elif .rec then (.rec.finished ¦ when) else "—" end ¦ sub(" "; "<br>")) + "</td></tr>") ¦ join("\n"))
  + "</tbody></table></div><footer>Elapsed is wall-clock time from when the loop created the worktree to when merge and cleanup finished. The bar under it splits that time by the three columns before it: Implementation runs from the agent's start to the PR, plus the worker's repair rounds; CI runs from the PR to the CI result (after a repair, from when the loop looked again), over every round; Post-merge check is the time its command ran, retry and bisect included; when one check covers several merged tickets (POST_MERGE_EVERY, default 4), its time is on the ticket after which it ran and the tickets before it say \"post-merge check deferred\". The grey rest (\"other\") is worktree setup, noticing the PR, merge, cleanup, port waits and triage. Records from before kit 0.9.8 have no phase times, so they are estimated: Implementation is the PR's creation time minus the start, without repairs; CI is the last run only; Post-merge check is read from the result text. Tokens are the agent's usage from its Claude Code session logs, subagents and triage sessions included: out is output, new in is input not served from the prompt cache (cache writes included), cached is input served from it. Codex workers are not counted yet. Tickets with no loop record use PR created→merged time instead. For a ticket merged outside the loop after a stop, it is the loop's time until the stop, and Finished is the merge time.</footer></main></body></html>"
JQ

# 리포트를 기본 브라우저로 연다 — 리포트는 사람이 보라고 만든 것이라 경로를 찾아 여는 한 단계를 줄인다. REPORT_OPEN=0이면 끔.
```

```
# 화면이 없는 곳(SSH 세션, DISPLAY 없는 Linux)에서는 열지 않고, 열기 실패는 루프 결과를 바꾸지 않는다(notify와 같은 태도).
open_report() {
  [[ "$REPORT_OPEN" == 1 && -z "${SSH_CONNECTION:-}${SSH_TTY:-}" ]] || return 0
  if [[ "$(uname -s)" == Darwin ]] && command -v open >/dev/null 2>&1; then
    open "$1" >/dev/null 2>&1 || true
  elif [[ -n "${DISPLAY:-}${WAYLAND_DISPLAY:-}" ]] && command -v xdg-open >/dev/null 2>&1; then
    xdg-open "$1" >/dev/null 2>&1 || true
  fi
}

# 기록에 tokens가 없는 티켓(0.9.7 이하, 또는 기록할 때 세션 기록을 못 찾음)은 리포트를 그릴 때 세션 기록에서 다시 센다(그 기록의 시작~끝
# 시각). cmux·tmux의 워크트리 자리(WORKTREE_DIR)로 찾으므로 Orca로 돈 지난 티켓은 비어 있다. 기록 파일은 고치지 않는다. Claude Code는
# 오래된 세션 기록을 지운다(cleanupPeriodDays, 기본 30일) — 그래서 새 티켓은 끝날 때 기록에 남긴다. stdin 기록 배열 → stdout 같은 배열
report_tokens() {
  local recs i n since upto t
  recs="$(cat)"
  if [[ -n "${WORKTREE_DIR:-}" ]]; then
    while IFS=$'\t' read -r i n since upto; do
      t="$(ticket_session_dirs "$n" | session_tokens "$since" "$upto")"
      [[ -z "$t" ]] || recs="$(jq -c --argjson i "$i" --argjson t "$t" '.[$i].tokens = $t' <<<"$recs")"
    done < <(jq -r 'to_entries[] | select(.value.tokens == null and (.value.agent // "claude") == "claude")
                    | [.key, .value.issue, (.value.started // ""), (.value.finished // "")] | @tsv' <<<"$recs")
  fi
  printf '%s\n' "$recs"
}

# --parent 실행이 정상 종료하면 스펙 리포트 HTML을 쓰고 브라우저로 연다. 생성 실패는 루프 결과를 바꾸지 않는다(로그만).
# 리포트는 커밋하지 않는다 — 사람이 보고 PR로 올린다(루프가 도는 동안 기본 브랜치 직접 커밋 금지).
REPORT_DECISIONS=""   # 마지막으로 쓴 리포트의 "사람 확인 필요" 건수(REPORT_DECISIONS_HEADING이 있을 때) — suggest_retro가 안내 줄에 적는다
write_spec_report() {
  [[ -n "$PARENT" ]] || return 0
  local out="$REPORT_DIR/spec-${PARENT}.html" records="$RECORDS_FILE" spec issues prs recs   # 둘 다 시작 때 절대 경로로 굳혔다
  # 목록 셋은 인자가 아니라 표준 입력으로 넘긴다 — PR 본문이 쌓인 저장소에서는 PR 목록이 인자 한도를 넘는다(#37).
  # 리포트 옆 임시 파일에 써서 성공해야 옮긴다(같은 폴더라 mv가 한 번에 바꾼다) — 생성이 실패해도 이전 리포트가 남는다
  if spec="$(gh issue view "$PARENT" --json number,title,state,url)" \
    && issues="$(gh api "repos/${OWNER_REPO}/issues/${PARENT}/sub_issues?per_page=100" --paginate \
         --jq '.[] | {number, title, state, url: .html_url, assignees: (.assignees | length)}' | jq -s '.')" \
    && prs="$(gh pr list --state all --limit 300 --json number,title,body,state,url,createdAt,mergedAt,headRefName)" \
    && recs="$( { [[ -f "$records" ]] && grep '^{' "$records" || true; } | jq -s --argjson spec "$PARENT" 'map(select(.spec == $spec))' | report_tokens)" \
    && mkdir -p "$(dirname "$out")" \
    && jq -rn --argjson spec "$spec" --arg prefix "$BRANCH_PREFIX" --arg now "$(date '+%Y-%m-%d %H:%M')" --arg heading "$REPORT_DECISIONS_HEADING" \
         --arg none "$REPORT_DECISIONS_NONE" "input as \$issues | input as \$prs | input as \$recs | $REPORT_JQ" <<<"$issues $prs $recs" > "$out.tmp" \
    && mv -f "$out.tmp" "$out"; then
    # 사람이 봐야 할 절의 건수(REPORT_DECISIONS_HEADING) — 리포트 블록의 data-count에서 읽는다. 블록이 없으면 0
    REPORT_DECISIONS="$(sed -n 's/.*id="decisions" data-count="\([0-9]*\)".*/\1/p' "$out" | head -n1)"; [[ -n "$REPORT_DECISIONS" ]] || REPORT_DECISIONS=0
    log ok "spec #${PARENT} report: ${out#"$ROOT"/}$([[ -z "$REPORT_DECISIONS_HEADING" ]] || echo " — '${REPORT_DECISIONS_HEADING}': ${REPORT_DECISIONS} PR$([[ "$REPORT_DECISIONS" == 1 ]] || echo s)") — review it, then commit it via a PR"
    open_report "$out"
  else
    rm -f "$out.tmp"
    log warn "spec #${PARENT}: report generation failed — rerun ticket-loop.sh --parent ${PARENT} to regenerate it"
  fi
}

# ---------- 티켓 하나 ----------

run_ticket() {
  local n="$1" title="$2"
  local deadline=$(( $(date +%s) + TICKET_TIMEOUT_SEC )) started_at ci_sec=0 ci_started
  started_at="$(date +%s)"
  IDLE_FP_FILE="$(mktemp "${TMPDIR:-/tmp}/ticket-loop-idle-${n}.XXXXXX")"   # idle 오판 한 번에 한 줄(idle_fp_count)
  # 병합 뒤 검증 결과는 병합한 뒤에야 정한다 — 0.9.6까지는 "skipped"로 시작해, 병합 전에 멈춘 티켓의 기록이 "검증을 건너뜀"으로 남았다
  POST_MERGE_RESULT=""; POST_MERGE_LOG=""; POST_MERGE_RETRY_LOG=""; POST_MERGE_WHY=""; POST_MERGE_SPEC_COMPLETE=""
```

```
  log title "=== #${n} ${title} ==="
  CUR_N="$n"; CUR_TITLE="$title"; CUR_STARTED="$started_at"; CUR_STARTED_ISO="$(date -u '+%Y-%m-%dT%H:%M:%SZ')"; STOP_ISSUE
="$n"
  CUR_PR=""; CUR_MODEL=""; CUR_CI_SEC=""; CUR_PICKUP_SEC=""; CUR_ROUND=0; CUR_AGENT_PATHS=""
  PH_NAME=""; PH_AT="$started_at"; PH_IMPL=0; PH_REPAIR=0; PH_CI=0; PH_POST=0
  STOP_KIND=""; STOP_RECORD=""; TRIAGE_RESULT=""

  git -C "$ROOT" fetch -q origin "$BASE_BRANCH"

  local wt wt_path orca_branch handle
  WT=""; WT_PATH=""; WT_BRANCH=""; HANDLE=""
  backend_worktree_create "${n}" "$(worker_prompt "${n}")"
  wt="$WT"; wt_path="$WT_PATH"; orca_branch="$WT_BRANCH"; handle="$HANDLE"
  CUR_AGENT_PATHS+="${wt_path}"$'\n'
  backend_set_status "${wt}" in-progress ¦¦ true
  log step "#${n}: worktree ${wt} (${wt_path}) / terminal ${handle}"
  ensure_agent_started "${n}" "${handle}"
  phase impl
  CUR_MODEL="$(detect_model "${handle}")"

  local pr rc=0 model_now prfile reason
  progress_reset "${wt_path}"
  prfile="$(mktemp "${TMPDIR:-/tmp}/ticket-loop-pr-${n}.XXXXXX")"
  wait_stop_reset   # 대기가 화면 분류로 멈추면 종류·사유가 여기로 온다(서브셸을 넘어)
  bg_wait wait_for_pr "${n}" "${handle}" "$deadline" > "$prfile" ¦¦ rc=$?   # 신호에 바로 깨도록(bg_wait)
  pr="$(cat "$prfile")"; rm -f "$prfile"
  if [[ $rc -ne 0 ]]; then
    phase ""   # 구현은 여기까지 — 무인 수습 시간은 세지 않는다
    log warn "#${n}: worker output tail ---"; terminal_tail "${handle}" ¦ dim_lines ¦¦ true
    backend_set_status "${wt}" todo "" "루프 중단: PR 없이 멈춤 — 확인 필요" ¦¦ true
    [[ $rc -ne 2 ]] ¦¦ die "#${n}: ticket timed out (${TICKET_TIMEOUT_SEC}s)"
    [[ $rc -ne 3 ]] ¦¦ die "#${n}: cannot reach ${BACKEND_NAME} terminal ${handle}: $(terminal_lost_why "${handle}"). The workt
ree is still on disk. $(terminal_lost_note worker)"
    screen_stop_check "${n}"   # 사용량 한도(usage_limit — 수습 없음)·확인 창(worker_no_pr)에 멈춘 워커면 여기서 멈춘다. 아니면
전처럼 idle strike
    reason="#${n}: worker stopped without a PR (a question or a failure?). Check the ${BRANCH_PREFIX}${n} worktree in ${BACKEND
_NAME}."
    STOP_KIND=worker_no_pr
    triage worker_no_pr "$reason" ¦¦ true   # 진단만 — 요약을 사유에 붙여 멈춘다
    die "${reason}${TRIAGE_NOTE}"
  fi
  # PR을 알아채기까지(PR 생성 → 루프가 다음 단계로) — 늘어지면 그만큼 CI·병합이 늦다(0.9.1의 PR 뒤 10분 대기를 손으로 대조해 찾
았다)
  local created pickup=""
  created="$(pr_created_epoch "${pr}")"
  if [[ -n "$created" ]]; then pickup=$(( $(date +%s) - created )); [[ $pickup -ge 0 ]] ¦¦ pickup=0; fi
  CUR_PICKUP_SEC="$pickup"
  phase "" "$created"   # 구현은 PR이 열린 때까지(GitHub 시각)
  log step "#${n}: PR #${pr} opened${pickup:+ (created $(fmt_epoch "$created"), picked up after ${pickup}s)}"
  progress_sync
  CUR_PR="$pr"; model_now="$(detect_model "${handle}")"; [[ -z "$model_now" ]] ¦¦ CUR_MODEL="$model_now"
  backend_set_status "${wt}" in-review "${pr}" "PR #${pr} — CI 대기" ¦¦ true

  # 수리를 다 쓰면 멈추기 전에 무인 수습을 한 번 띄운다(ci_failed) — 수습 세션이 PR 브랜치에 고쳐 push하면 이 고리의 처음(충돌·
CI 확인)부터
  # 이어 간다. 그래도 실패하면 수습 횟수를 이미 써서 그대로 멈춘다
  local round=0 state ci_from="$created"
  while :; do
    state="$(mergeable_state "${pr}")"
    if [[ "$state" == "CONFLICTING" ]]; then
      if [[ ${round} -ge $REPAIR_ROUNDS ]]; then
        reason="#${n}: PR #${pr} conflicts with ${BASE_BRANCH} (${REPAIR_ROUNDS} repair round(s) used up)"
        STOP_KIND=ci_failed; triage ci_failed "$reason" && continue
        die "${reason}${TRIAGE_NOTE}"
      fi
      round=$((round + 1)); CUR_ROUND=$round; log warn "#${n}: conflict → asking the worker to repair (${round})"
      phase repair
      repair "${handle}" "PR #${pr} 이 ${BASE_BRANCH}과 충돌한다. \`git fetch origin && git merge origin/${BASE_BRANCH}\`으로
충돌을 해결하고 테스트·lint를 다시 돌린 뒤 같은 브랜치에 push해라. 질문하지 말고 끝까지 해라." "$deadline" "${pr}" \
        ¦¦ die "#${n}: timed out while repairing the conflict"
      phase ""
      screen_stop_check "${n}" " while repairing PR #${pr}"   # 수리 중 사용량 한도·확인 창이면 여기서 멈춘다
```

```
      continue
    fi
    if ! ci_expected "${pr}"; then
      log step "#${n}: PR excluded by the repository CI path filter (${CI_SKIP_CMD:+CI_SKIP_CMD}${CI_SKIP_CMD:-CI_SKIP_REGEX}) — merging without checks (PR #${pr})"
      ci_sec=-1
      break
    fi
    log step "#${n}: waiting for CI (PR #${pr})"
    ci_started="$(date +%s)"
    phase ci "$ci_from"; ci_from=""   # 첫 회는 PR이 열린 때부터(PR을 알아채기 전에도 CI는 돈다)
    if ci_passed "${pr}"; then
      phase ""
      ci_sec="$(ci_run_sec "${pr}")"; [[ -n "$ci_sec" ]] || ci_sec=$(( $(date +%s) - ci_started ))
      break
    fi
    phase ""
    if [[ ${round} -ge $REPAIR_ROUNDS ]]; then
      reason="#${n}: PR #${pr} CI failed (${REPAIR_ROUNDS} repair round(s) used up). See gh pr checks ${pr}."
      STOP_KIND=ci_failed; triage ci_failed "$reason" && continue
      die "${reason}${TRIAGE_NOTE}"
    fi
    round=$((round + 1)); CUR_ROUND=$round
    if [[ "${CI_NO_CHECKS:-0}" -eq 1 ]]; then log warn "#${n}: no CI checks were registered → asking the worker to diagnose (${round})"
    else log warn "#${n}: CI failed → asking the worker to repair (${round})"; fi
    if ! repair_ci "${n}" "${pr}" "${handle}" "$deadline"; then
      reason="#${n}: no new commit was pushed after the repair, and CI still fails"
      STOP_KIND=ci_failed; triage ci_failed "$reason" && continue
      die "${reason}${TRIAGE_NOTE}"
    fi
  done
  log ok "#${n}: CI passed"
  CUR_CI_SEC="$ci_sec"

  if [[ $NO_MERGE -eq 1 ]]; then
    log step "#${n}: --no-merge — merge PR #${pr} yourself. The worktree is kept (${wt})."
    record_ticket "no-merge"; CUR_N=""; STOP_ISSUE=""; rm -f "$IDLE_FP_FILE"
    return 0
  fi

  # 일시 장애가 아닌 거절(필수 리뷰 같은 브랜치 보호)은 사람에게 넘기는 자리라 무인 수습을 띄우지 않는다(종류만 merge_failed)
  if ! merge_pr "${pr}"; then
    STOP_KIND=merge_failed
    die "#${n}: failed to merge PR #${pr} after CI passed${MERGE_ERR:+ (${MERGE_ERR})}. Check gh pr view ${pr} — if it is mergeable, merge it and rerun."
  fi
  log ok "#${n}: PR #${pr} merged"
  sleep 5
  if [[ "$(gh issue view "${n}" --json state -q .state)" != "CLOSED" ]]; then
    gh issue close "${n}" --comment "PR #${pr} 병합으로 완료 (ticket-loop)" || true
  fi
  gh issue edit "${n}" --remove-label "$IN_REVIEW_LABEL" >/dev/null 2>&1 || true
  gh issue edit "${n}" --remove-label "$STOPPED_LABEL" >/dev/null 2>&1 || true   # 지난 실행이 멈추며 붙인 라벨(코멘트는 기록으로 남는다)
  local spec; spec="$(spec_of_ticket "${n}")"
  close_spec_if_done "$spec"

  # 병합된 기본 브랜치로 무거운 검증 — CI가 안 붙는 문서 PR은 창에 넣지 않는다. 단 스펙을 끝낸 문서 PR은 넣는다 — 창이 비어 있어도
  # (POST_MERGE_EVERY=1) 스펙 끝의 검증이 TICKET_LOOP_POST_MERGE_SPEC_COMPLETE를 달고 돌게(#32). 창이 찼거나(POST_MERGE_EVERY, 기본 4) 스펙·실행이
  # 끝날 때만 돌리고 나머지는 미룬다(pm_due). 실패하면 기본 브랜치가 깨진 것이므로 다음 티켓을 쌓지 않고 멈춘다(워커
  # 워크트리는 이미 병합됐으니 정리하고, 검증 워크트리와 로그를 남긴다).
  local post_rc=0
  if [[ $POST_MERGE -eq 1 ]]; then
    if [[ $ci_sec -ge 0 ]] || spec_closed_this_run "$spec"; then
      git -C "$ROOT" fetch -q origin "$BASE_BRANCH" || true   # 병합 커밋의 부모(창의 시작)를 읽으려고
      pm_add "${n}" "${pr}" "$(gh pr view "${pr}" --json mergeCommit -q '.mergeCommit.oid // empty' 2>/dev/null || true)"
    fi
    pm_spec_closed "$spec"   # 이 병합이 스펙을 끝냈으면 창에 — 창을 덮는 검증이 TICKET_LOOP_POST_MERGE_SPEC_COMPLETE로 받는다
    if pm_due "${n}" "$spec"; then
```

```
      run_post_merge "${n}" || post_rc=$?
    elif [[ $ci_sec -lt 0 ]]; then
      POST_MERGE_RESULT="skipped"
    else
      POST_MERGE_RESULT="deferred ($(pm_of) pending)"
      log step "#${n}: post-merge check deferred — $(pm_of) merged tickets waiting ($(pm_list))$([[ $POST_MERGE_EVERY -ne 0 ]]
|| echo ", until the spec or the run ends")"
    fi
  else
    POST_MERGE_RESULT="skipped"
  fi

  # 이미 병합됐다 — 조회가 실패해도 멈추지 않고 브랜치 삭제만 건너뛴다
  local branch; branch="$(gh pr view "${pr}" --json headRefName -q .headRefName || true)"
  [[ -n "$branch" ]] || log warn "#${n}: could not read the head branch of PR #${pr} — delete it yourself (local and origin)"
  backend_set_status "${wt}" completed "${pr}" "PR #${pr} 병합 완료" || true
  backend_worktree_remove "${wt}" "${wt_path}" || log warn "#${n}: failed to remove the worktree — clean up manually (${wt},
${wt_path})"
  git -C "$ROOT" worktree prune || true
  [[ -z "$branch" ]] || git -C "$ROOT" branch -D "${branch}" >/dev/null 2>&1 || true
  if [[ -n "$orca_branch" && "$orca_branch" != "HEAD" && "$orca_branch" != "$BASE_BRANCH" ]]; then
    git -C "$ROOT" branch -D "${orca_branch}" >/dev/null 2>&1 || true
  fi
  [[ -z "$branch" ]] || git -C "$ROOT" push -q origin --delete "${branch}" >/dev/null 2>&1 || true
  # 병합 뒤 검증이 실패했으면 멈추기 전에 무인 수습을 띄운다(post_merge_recover). 이 티켓은 이미 병합됐으므로 수습 결과와 무관
하게
  # 기록은 merged다 — 멈추면 die가 알린다
  if [[ $post_rc -ne 0 ]]; then
    post_rc=0; post_merge_recover "${n}" || post_rc=$?
    reason="$PM_REASON"
  fi
  log ok "#${n} summary: wall-clock $(( ($(date +%s) - started_at) / 60 ))m, PR #${pr}${CUR_PICKUP_SEC:+ (picked up after ${CUR
_PICKUP_SEC}s)}, CI $([[ $ci_sec -ge 0 ]] && echo "${ci_sec}s" || echo "none (path excluded from CI)"), repairs ${round}, idle
false positives $(idle_fp_count), post-merge check ${POST_MERGE_RESULT}${TRIAGE_RESULT:+, triage ${TRIAGE_RESULT%%: *}}"
  record_ticket "merged"; CUR_N=""; rm -f "$IDLE_FP_FILE"
  [[ $post_rc -eq 0 ]] || die "$reason"
  notify "Ticket done" "#${n} ${title} — PR #${pr}, $(( ($(date +%s) - started_at) / 60 ))m" done
  STOP_ISSUE=""
}

# ---------- 메인 ----------

KIT_VERSION="$(kit_version)"
kit_banner "ticket-loop-kit ${KIT_VERSION}"
backend_check
pm_load
# 시작 전에는 사람이 앞에 있다 — 조회가 실패하면 바로 멈추고, 후보 0행(2)은 재확인 없이 빈 목록 그대로다(재확인은 티켓 사이 — p
ick_next_targets).
# 번호 지정이면 먼저 번호마다 상태·스펙 여부를 본다(닫힌 번호는 빼고, 스펙 이슈면 멈춘다 — #46·#53)
[[ $TICKETS_GIVEN -eq 0 ]] || given_tickets_check
pick_rc=0; targets="$(pick_targets)" || pick_rc=$?
[[ $pick_rc -ne 1 ]] || die "could not list tickets (GitHub)"
log "repository ${OWNER_REPO} / backend ${BACKEND_NAME} / base branch ${BASE_BRANCH} / label ${READY_LABEL}${SCOPE_LABEL:+ +
${SCOPE_LABEL}}${PARENT:+ / sub-issues of spec #${PARENT}}"
[[ -z "$AGENT_MODEL" ]] || log "worker model: ${AGENT_MODEL} (${AGENT_MODEL_FLAG} on the agent command)"
log "post-merge check: $([[ $POST_MERGE -eq 1 ]] && echo "on$(case "$POST_MERGE_EVERY" in 1) ;; 0) echo ", when a spec or the r
un ends" ;; *) echo ", every ${POST_MERGE_EVERY} merged tickets and when a spec or the run ends" ;; esac) — ${POST_MERGE_CMD}
${POST_MERGE_RETRY_CMD:+ (retry once on failure: ${POST_MERGE_RETRY_CMD})}" || echo "off")"
if [[ -n "$PM_PENDING" ]]; then
  log warn "post-merge check: $(pm_count) merged ticket$([[ "$(pm_count)" -eq 1 ]] || echo s) from an earlier run not checked y
et ($(pm_list)) — $([[ $POST_MERGE -eq 1 ]] && echo "the next check covers them" || echo "left for a run with the check on")"
elif [[ -n "$PM_SPEC_COMPLETE" ]]; then   # 지난 실행의 스펙 끝 검증이 실패해 멈춤(#34) — 대상이 없어도 루프 끝에 돈다
  log warn "post-merge check: spec $(pm_spec_list) completed in an earlier run, not covered by a passed check yet — $([[ $POST_
MERGE -eq 1 ]] && echo "the next check covers it" || echo "left for a run with the check on")"
fi
log "notifications: desktop$([[ -z "$SLACK_WEBHOOK_URL" ]] || echo " + Slack")"
log "triage on stop: $([[ "$LOOP_TRIAGE" == 1 ]] && echo "on — up to ${TRIAGE_MAX} per run, $((TRIAGE_TIMEOUT_SEC / 60))m each"
|| echo "off")"
if [[ "$LOOP_TRIAGE" == 1 ]] && ! triage_skill_on_base; then
  log warn "note: the loop-triage skill is not on origin/${BASE_BRANCH} — stops are not triaged until the kit install is merge
d"
fi
```

```
if [[ -f "$LOOP_ENV" ]] && grep -q 'hooks\.slack\.com' "$LOOP_ENV"; then
  log warn "note: a Slack webhook URL is in .ticket-loop/loop.env, which is committed — move SLACK_WEBHOOK_URL to ${USER_LOOP_E
NV} (anyone with the URL can post to the channel)"
fi
if [[ $POST_MERGE -eq 1 && -n "$POST_MERGE_PORT" ]] && port_in_use "$POST_MERGE_PORT"; then
  log warn "note: port ${POST_MERGE_PORT} in use ($(port_holder "$POST_MERGE_PORT" || true)) — keep it free while the loop run
s. Otherwise each merge waits up to $((PORT_WAIT_SEC / 60))m for it (or use --skip-post-merge)"
fi
# 티켓이 더 없어도 --parent면 스펙 닫기·리포트는 다시 한다(사람이 마지막 티켓을 닫은 스펙, 끝난 스펙의 리포트를 다시 뽑는 길)
[[ -n "$targets" ]] || { log "No tickets to process."; [[ $DRY_RUN -eq 1 ]] || { close_spec_if_done "$PARENT"; pm_spec_closed
"$PARENT"; pm_flush_end; write_spec_report; suggest_retro; }; exit 0; }
log "targets ($([[ $TICKETS_GIVEN -eq 1 ]] && echo "in the given order — blockers and open PRs are checked before each starts"
|| echo "all blocking issues currently closed")):"
printf '%s\n' "$targets" | dim_lines "  #"
[[ $DRY_RUN -eq 0 ]] || exit 0
target_count="$(grep -c . <<<"$targets")"
slack_post "Ticket loop started" "${target_count} ticket$([[ $target_count -eq 1 ]] || echo s) · backend ${BACKEND_NAME} · agen
t ${AGENT}$([[ $MAX -eq 0 ]] || echo " · max ${MAX}") · kit ${KIT_VERSION}
$(head -n 10 <<<"$targets" | awk -F'\t' '{print "#" $1 " " $2}')$([[ $target_count -le 10 ]] || printf '\n+%d more' $((target_c
ount - 10)))" start

# 목록을 stdin으로 물리지 않는다 — 루프 안의 gh·백엔드 CLI·git이 stdin을 읽으면 다음 티켓이 사라진다.
# 자동 선택이면 티켓 하나가 끝날 때마다 대상을 다시 계산해, 막 풀린 선행 관계를 반영한다.
done_count=0
TICKET_IDX=0   # 번호 지정의 목록 자리 — 처리 건수(done_count)와 나눈다: 도중에 닫혀 건너뛴 번호는 --max에 세지 않는다(#46)
PROCESSED=""
while :; do
  if [[ ${#TICKETS[@]} -gt 0 ]]; then
    [[ $TICKET_IDX -lt ${#TICKETS[@]} ]] || break
    n="${TICKETS[$TICKET_IDX]}"
    # 시작 검사 뒤에 바뀐 것을 티켓마다 다시 본다 — 그사이 닫혔으면(다른 PR의 Closes·사람) 건너뛰고, 스펙 이슈가 되었으면 멈춘
다
    given_ticket "${n}"; title="$GIVEN_TITLE"
    [[ -n "$GIVEN_STATE" ]] || die "#${n}: could not look up the issue (GitHub)"
    if [[ "$GIVEN_STATE" == closed ]]; then log "skip #${n} (${title}): closed since the run started"; TICKET_IDX=$((TICKET_IDX
+ 1)); continue; fi
    [[ "$GIVEN_STATE" != spec ]] || die "$GIVEN_WHY"
    blocked="$(blocked_by "${n}")" || die "#${n}: could not check blocking issues (GitHub)"   # 조회 실패를 통과시키지 않는다
    [[ "$blocked" == "0" ]] || die "#${n}: blocked by an open issue"
    pr="$(open_pr_for "${n}")" || die "#${n}: could not check for an open PR (GitHub)"
    [[ -z "$pr" ]] || die "#${n}: a PR is already open"
  else
    if [[ $NEXT_TARGETS_SET -eq 1 ]]; then targets="$NEXT_TARGETS"; NEXT_TARGETS_SET=0   # 병합 뒤 검증을 미룰 때 pm_due가 계산
해 둔 것
    elif [[ $done_count -ne 0 ]]; then
      # 목록 조회가 gh 재시도까지 실패하거나(GitHub 장애가 길다, 1), 재확인까지 빈 목록인데 스펙에 열린 하위 이슈가 남으면(성공
한 빈 응답, 2 — #22)
      # PICK_RETRY_SEC 동안 PICK_RETRY_EVERY_SEC마다 다시 해 보고(2면 한 바퀴에 재확인 PICK_RECHECK_SEC가 더 든다), 그래도 안
되면 그 사유로
      # 멈춘다 — 조용히 "No more tickets"로 끝내지 않는다.
      # 처리 중인 티켓이 없으므로(CUR_N·STOP_ISSUE 비어 있음) 이슈 코멘트·무인 수습 없이 알림과 멈춤 기록(other)만 남는다
      pick_deadline=$(( $(date +%s) + PICK_RETRY_SEC ))
      while :; do
        pick_rc=0; targets="$(pick_next_targets </dev/null)" || pick_rc=$?
        [[ $pick_rc -ne 0 ]] || break
        if [[ $pick_rc -eq 2 ]]; then pick_why="ticket list empty but spec #${PARENT} has open sub-issues"; else pick_why="coul
d not list tickets (GitHub)"; fi
        [[ $(date +%s) -lt $pick_deadline ]] || { STOP_KIND=""; die "$pick_why"; }
        log warn "${pick_why} — retrying in ${PICK_RETRY_EVERY_SEC}s (for up to $((PICK_RETRY_SEC / 60))m)"
        sleep "$PICK_RETRY_EVERY_SEC"
      done
    fi
    # 이번 실행에서 이미 처리한 티켓은 다시 집지 않는다(이슈 닫힘 반영이 늦는 경우 대비)
    n=""
    while IFS=$'\t' read -r cand ctitle; do
      [[ -n "${cand}" && " ${PROCESSED} " != *" ${cand} "* ]] || continue
      n="${cand}"; title="${ctitle}"; break
    done <<<"$targets"
    [[ -n "${n}" ]] || { log "No more tickets to process."; break; }
  fi
  run_ticket "${n}" "${title}" </dev/null
  PROCESSED="${PROCESSED} ${n}"
```

```
  done_count=$((done_count + 1)); TICKET_IDX=$((TICKET_IDX + 1))
  [[ $MAX -eq 0 ¦¦ $done_count -lt $MAX ]] ¦¦ { log "--max $MAX reached"; break; }
done

# 스펙을 닫은 뒤 남은 창을 검증한다 — 여기서 처음 닫히는 스펙(사람이 마지막 티켓을 닫음·하위 이슈 반영 지연)도 그 검증이 SPEC_COMPLETE로 받게.
# 창이 비었어도(POST_MERGE_EVERY=1) 그 스펙 번호를 싣고 통과한 검증이 이번 실행에 없었으면 그 번호로 한 번 돈다(#34)
close_spec_if_done "$PARENT"; pm_spec_closed "$PARENT"
pm_flush_end
write_spec_report
suggest_retro
log ok "done: ${done_count} ticket$([[ $done_count -eq 1 ]] ¦¦ echo s)"
notify "Ticket loop finished" "${done_count} done$([[ $TRIAGE_USED -eq 0 ]] ¦¦ echo " · ${TRIAGE_USED} triaged")$([[ -z "${CLOSED_SPECS// /}" ]] ¦¦ echo " · spec${CLOSED_SPECS// / #} complete — consider /retro")" finish
```

E. 개정 이력

3판(2026-10-12)은 두 번째 스펙의 첫 티켓 2건을 돌린 실측에서 나왔다 — 묶음 결과를 한 표로 보고 싶다는 요구와, 포트를 기다리는 루프가 멈춘 것처럼 보인 일.

바뀐 곳	내용	계기
바뀐 곳	**내용**	**계기**
묶어 돌리는 병합 뒤 검증 (6장 변수 표, 7장 6·9단계, R10, 부록 D)	`POST_MERGE_EVERY`를 더했다. 병합 뒤 검증을 병합 N개마다, 그리고 스펙이 끝나거나 실행의 마지막 티켓일 때 돌린다. 그 사이 티켓은 "deferred"로 남고 다음 검증이 덮는다. 아직 검증하지 않은 병합은 `.git/ticket-loop-post-merge.json`에 남아 다시 실행해도 이어진다. 묶음이 재실행까지 실패하면 `POST_MERGE_RETRY_CMD`로 마지막 통과 커밋과 그 뒤 병합들을 이분 탐색해 처음 실패하는 병합을 멈춤 사유·멈춤 기록(`post_merge_window`)에 싣는다. 기본값은 4다 — 이미 설치된 저장소도 키트를 올리면 병합 4개마다 묶인다(전처럼 병합마다 돌리려면 1). 0이면 스펙·실행이 끝날 때만 돈다.	키트 0.9.9 — rental-portal 스펙 #248: 통합 테스트가 티켓마다 47~52분, 12티켓에서 9.8시간(구현 시간의 85% 남짓)이었고, 워커가 PR 전에 관련 통합 테스트를 돌리기 시작한 뒤 34회 동안 잡은 회귀가 없었다
리포트의 구현·CI·병합 뒤 검증 시간과 토큰 (6장 `RECORDS_FILE`, 7장 7·9단계, 12장, 부록 D)	루프가 티켓의 경과 시간을 단계별로 세어 기록에 남긴다 (`impl_sec` · `repair_sec` · `ci_wait_sec` · `post_merge_sec`). • 구현: 에이전트가 뜬 때부터 PR이 열린 때까지. • 수리: 충돌·CI 수리. • CI: 모든 회차. • 병합 뒤 검증: 명령이 돈 시간(재실행 포함). 스펙 리포트에는 Implementation·CI·Post-merge check 열이 생기고, 경과 시간 아래 막대가 그 몫과 나머지를 보인다. 머리에 세 시간의 합계가 붙는다. 0.9.7 이하 기록은 PR 생성 시각·마지막 CI 회차·결과 문자열로 어림한다. 결과 브리프에서는 걸린 시간을 빼고 결과만 적는다. 티켓마다 에이전트 토큰(`tokens`)도 기록하고 리포트 열로 보인다. 워커·서브에이전트·무인 수습 세션의 Claude Code 세션 기록에서 더하며, 지난 기록은 리포트를 그릴 때 세션 기록에서 메운다.	키트 0.9.8 — 경과 시간 하나로는 티켓마다 실제 구현에 든 시간과 CI·통합 테스트를 기다린 시간을 가를 수 없었다(rental-portal 스펙 #248은 티켓마다 통합 테스트만 46~51분). 티켓당 토큰도 리포트에 없었다
이어 간 스펙의 회고 안내와 리포트 (6장 `RECORDS_FILE`, 7장 7·8·9단계, 부록 D)	회고 안내가 이번 실행의 로그만이 아니라 그 스펙의 티켓이 지나간 로그를 모두 적는다 — 티켓 기록에 그 실행의 로그 파일(`log`)을 남기고, 그 필드가 없는 지난 기록은 로그의 티켓 머리줄로 찾는다. 스펙 리포트는 루프가 멈춘 뒤 루프 밖에서 병합된 티켓을 "Merged after stop"으로 그리고, 멈춘 사유·멈춘 뒤 병합까지 걸린 시간을 보이며, 경과는 멈출 때까지라고 밝히고 마친 시각은 병합 시각으로 한다. 병합 전에 멈춘 티켓의 기록에 "병합 뒤 검증 skipped"가 남던 것도 고쳤다.	키트 0.9.7 — rental-portal 스펙 #248: #258에서 멈춘 루프를 다른 날 이어 갔더니, 회고 안내는 이어 간 실행의 로그만 가리키고, 사람이 병합한 #258은 리포트에 멈춘 흔적 없이 Merged·"post-merge check skipped"로 나왔다
키트 머리줄 (6장 `LOG_FILE` · `SLACK_WEBHOOK_URL`, 9장 점검 출력, 부록 D)	루프가 실행 맨 앞에 `── ticket-loop-kit 0.9.6 (커밋) ──…` 한 줄을 굵은 주황(256색 208)으로 찍는다. 버전은 설치기가 쓴 `.ticket-loop/kit.json`에서 읽고, 없으면 `unknown`. 로그 파일에는 같은 줄이 맨글자로 남는다. Slack 시작 알림에도 키트 버전이 붙는다. npx 설치기의 첫 줄도 같은 모양이다.	키트 0.9.6 — 같은 날 로그에 실행이 이어 붙어 경계와 키트 버전이 보이지 않았다(0.9.1~0.9.5 수정 전후를 가를 수 없음)

재실행만 한 CI 수리 (6장 내부 상수, 7장 4단계, 10.6, 부록 D)	CI 수리 뒤 PR head가 그대로여도 바로 멈추지 않고 CI를 한 번 더 본다 — 재실행이 도는 중이면 끝까지 보고, 통과면 이어 간다. 새 커밋도 없고 CI도 그대로 실패일 때만 "수리 뒤 새 커밋 없음"으로 멈춘다. 수리 프롬프트에 "같은 커밋의 다른 job은 통과한 러너·네트워크 오류면 코드를 바꾸지 말고 `gh run rerun <run-id> --failed` " 한 줄을 더했다 — 워커가 CI를 통과시키려고 불필요한 커밋을 만들지 않게.	키트 0.9.5 — 이슈 #9: rental-portal #258(cmux)의 CI가 러너 설치 경합으로 실패했고, 워커는 재실행만 해 1분 뒤 통과했는데 루프가 멈췄다
GitHub 일시 장애 재시도 (6장 내부 상수, 7장 4·5단계, 10.2, 10.6, R3, 부록 D)	루프의 `gh` 조회는 5xx·연결 오류면 15초·30초 쉬며 세 번까지 다시 해 본다(`gh` 를 함수로 감쌌다 — 쓰기는 다시 하지 않는다). 병합은 실패할 때마다 먼저 이미 병합됐는지 보고, 일시 장애와 "Base branch was modified"만 다시 한다. 브랜치 보호 같은 거절은 전처럼 바로 멈추되 멈춤 종류를 `merge_failed` 로 남기고 사유에 `gh` 오류 첫 줄을 싣는다. CI 대기의 `--watch` 가 실패로 끝나면 체크 상태를 다시 읽어 실패한 체크가 있을 때만 실패로 본다. 병합 뒤 브랜치 조회가 실패해도 멈추지 않는다.	키트 0.9.4 — 이슈 #7: CI 통과 직후 병합 한 번이 공지 없는 502를 받아 루프가 멈췄다(0.9.2, cmux)
수리 요청 직후의 묵은 대기 상태 (2.5, 6장 내부 상수, 7장 4단계, 부록 D)	cmux 보내기가 프롬프트 수락을 확인한다 — 붙여 넣은 뒤 15초(`SEND_ACCEPT_SEC`) 안에 `lifecycle` 이 idle에서 벗어나거나 프롬프트가 화면에 찍혀야 한다(tmux의 `sent` →running 확인과 같은 뜻, 상한도 같은 상수). 수리 대기는 대기 상태여도 PR에 새 커밋이 아직 없으면 화면이 그대로인 대기를 연달아 3번 볼 때만 끝으로 본다.	키트 0.9.3 — rental-portal #251(cmux): CI 수리 요청 5초 뒤 "새 커밋 없음"으로 멈췄고 워커는 2분 뒤 push했다. 수습 세션이 남긴 키트 버그 보고
PR 뒤 대기·CI 소요·idle 오판·수리 끝 (6장 내부 상수, 7장 2·4·7단계, 9장 예시 로그, 부록 D)	PR을 찾은 뒤에는 대기 상태 하나만 10분까지 기다리지 않고, 이슈에 `IN_REVIEW_LABEL` 이 붙거나 대기 상태면 바로, 아니면 2분에서 넘어간다. CI 소요는 GitHub 체크 시각으로 재고, 요약·기록에 PR을 알아채기까지(`pr_pickup_sec`)를 싣는다. idle 오판은 일하던 워커를 대기로 본 것만 센다(PR을 연 뒤의 대기는 세지 않고, 다시 일한 워커의 앞선 대기는 센다). 수리는 대기가 끝내 안 잡혀도 새 커밋이 5분 그대로이고 워크트리가 깨끗하면 끝난 것으로 본다.	키트 0.9.2 — rental-portal 스펙 #141(cmux) 13건 중 3건이 PR 뒤 10분을 기다렸고 CI가 2~3초로 찍혔다(이슈 #2)
수정 PR이 티켓을 닫던 것 (10.6, 부록 D)	준비 실패의 수정 PR 제목이 `fix: #17 …` 이었고, 병합 커밋(PR 제목이 들어간다)의 닫기 키워드로 GitHub가 아직 구현 전인 #17을 닫았다. `loop-triage` 스킬은 수정 PR 제목을 `fix: <요약> — #N …` 으로 바꾸고 제목·본문·커밋 메시지에서 닫기 키워드 바로 뒤에 `#N` 을 두지 않게 했다. 루프는 병합 뒤 검증 실패가 아닌 수습에서 수정 PR을 병합한 뒤 티켓 이슈를 몇 초 지켜보고, 닫혔으면 코멘트와 함께 다시 연다.	키트 0.9.1 — 0.9.0 더미 저장소 실측(#17 → 수정 PR #18), 워커가 알아채 다시 열었다
무인 수습과 멈춤 기록 (6장 변수 표, 10.2, 10.6, 11장, 부록 D)	멈출 때마다 멈춤 기록(`.ticket-loop/stops/<id>.json`)을 남기고, 병합 뒤 검증 실패·수리를 다 쓴 CI 실패와 충돌·워크트리 준비 실패·PR 없이 멈춘 워커에서는 멈추기 전에 수습 세션(`loop-triage` 스킬)을 띄워 판정(`resume` / `halt`)을 기다린다. 고쳐지면 멈춘 자리에서 이어 가고(수정 PR은 CI 뒤 병합, 병합 뒤 검증은 다시 통과해야 한다), 아니면 판정 요약을 사유에 붙여 멈춘다. 한 실행에 한 번이 기본(`LOOP_TRIAGE` , `TRIAGE_MAX` , `TRIAGE_TIMEOUT_SEC` , `TRIAGE_LABEL`). 사람은 `/loop-triage <id>` 로 기록부터 이어받는다.	키트 0.9.0 — rental-portal spec #96: 병합 뒤 검증 실패에서 루프가 다시 돌기까지 3시간 25분, 수정은 3줄
병합 뒤 검증 재실행 (6장 변수 표, 7장 6단계, R10, 부록 D)	`POST_MERGE_RETRY_CMD` 를 더했다. 병합 뒤 검증이 실패하면 포트 대기를 다시 지나 같은 워크트리에서 그 명령을 딱 한 번 돌린다(첫 실행 로그 경로는 `POST_MERGE_FAILED_LOG` , 출력은 `post-merge-N-retry.log`). 통과하면 계속 가고 이슈 코멘트와 Slack(멘션 없음)에 플레이크 의심을 남긴다. 재실행도 실패하면 두 로그를 적고 멈춘다. 비워 두면 전과 같다. 7장에 vitest 예.	키트 0.8.11 — rental-portal #106 병합 뒤 검증이 무관한 테스트 하나의 시간 초과로 멈춤(7회 중 2회)
idle 오판 수 (부록 D)	티켓 요약의 "idle false positives"와 기록의 `idle_false_positives` 가 늘 0이던 것을 고쳤다. 오판은 PR 대기 함수 안에서 세는데 그 함수가 서브셸에서 돌아 변수가 부모에 돌아오지 않았다. 티켓마다 임시 파일에 한 줄씩 쓰고 줄 수를 센다. 0.8.9까지의 기록에 남은 0은 실제 값이 아니다.	키트 0.8.10 — 0.8.8 실측 중 코드를 읽다 발견
멈춤 보고 — Slack 알림과 GitHub 기록 (6장 변수 표, 10.2, 부록 D)	멈추면 어떻게 끝나든(멈춤 조건, 터미널 창 닫힘, Ctrl-C, `kill` , `set -e` 에 걸린 명령, 로그를 받던 `tee` 가 죽은 뒤의 SIGPIPE) 한 번 보고한다 — 티켓 기록 `stopped` , 그 이슈에 코멘트와 `loop-stopped` 라벨(기록), 데스크톱과 Slack(알림). Slack에는 루프 시작·티켓 완료·루프 종료도 간다. 전에는 `die` 만 데스크톱 알림을 냈고, `set -e` ·신호로 끝나면 기록도 알림도 없었다. PR·CI 대기와 검증 명령은 백그라운드로 돌리고 `wait` 로 기다려, 다른 터미널의 `kill <루프 pid>` 에도 바로 멈춘다(포그라운드 자식을 기다리는 동안 bash가 trap을 미뤄 PR이 열릴 때까지 먹히지 않던 것 — 실측에서 찾음). 비밀 값을 둘 기계별 설정 파일 `~/.config/ticket-loop/loop.env` 를 더했다. 0.8.9: 보고에 싣는 로그 끝은 이 실행이 쓴 줄만이다(로그는 날짜별 한 파일이라 같은 날 앞 실행의 줄이 섞였다).	키트 0.8.8 — 자리를 비운 사이 루프가 끊기면 밖에서 알 길이 없음

터미널 로그 영어화 (4·7·9·10장 예시 로그와 인용 문구, 부록 D)	루프가 터미널·로그 파일에 찍는 문구(단계·건너뜀·요약·ERROR·안내)와 데스크톱 알림을 영어로 바꿨다. 워커 프롬프트·수리 프롬프트·GitHub 코멘트·백엔드 상태 코멘트는 그대로다. 티켓 기록(`RECORDS_FILE`)의 `post_merge` 값도 영어("passed in Ns"·"failed in Ns"·"skipped")가 되고, 옛 기록의 한국어 값은 리포트가 전처럼 표시할 때 바꾼다. 요약 줄은 `grep "summary:"` 로 찾는다.	키트 0.8.7 — 터미널 로그를 영어로 보고 싶다는 요구
티켓 PR 찾기 (5장 집는 조건, 6장 변수 표, 7장 2단계, 10.2 진단 함수·10.3·R2-b·R3·R5, 부록 D)	루프·스펙 리포트· `ticket-state.sh` 가 티켓 N의 PR을 head 브랜치가 `issue-N` 과 같거나 `issue-N-` 로 시작하는 것으로 찾는다(`issue-1020` 은 #102가 아니다). 전에는 `issue-N-` 만 봐서 워커가 워크트리 브랜치 `issue-N` 에 그대로 커밋해 연 PR을 못 찾고 "PR 없이 멈춤"으로 죽었다. 원격 브랜치 조회도 둘 다 본다. `worker-rules.md` 에 첫 커밋 전 `issue-N-<slug>` 를 만들라는 한 줄.	키트 0.8.6 — rental-portal #102: 워커가 PR #120을 `issue-102` 로 열었는데 루프가 idle 3회 뒤 멈춤
PR 대기 중 진행 로그 (6장 변수 표·내부 상수, 7장 2단계, 9장 예시 로그, 부록 D)	워크트리를 만든 뒤 PR까지 루프 터미널이 조용하던 구간에 `PROGRESS_SEC` (1분)마다 워크트리를 git으로 보고 브랜치 생성·첫 파일 수정·새 커밋을 바로, 커밋 없이 변경 파일 목록만 달라졌으면 `HEARTBEAT_SEC` (5분)에 한 번까지 "작업 중 — 경과·커밋 안 된 변경 파일" 한 줄을 찍는다. 같은 내용은 되풀이하지 않는다(변화가 없으면 조용하다). 수리 대기에도 같다. `PROGRESS_SEC=0` 이면 전처럼 조용하다.	키트 0.8.6 — 첫 PR이 열릴 때까지 상태창에 아무 변화가 없어 멈춘 것처럼 보임
병합 뒤 검증 로그 경로 (6장 변수 표, 7장 6단계, R10, 부록 D)	`LOG_FILE` · `RECORDS_FILE` · `REPORT_DIR` 를 시작 때 저장소 루트 기준 절대 경로로 굳힌다. 전에는 `post-merge-N.log` 가 `cd` 한 서브셸에서 상대 경로로 열려 검증 워크트리 안에 생겼고, 루프는 저장소 루트에서 꼬리를 찾지 못해 빈 꼬리로 멈췄다. 시작 줄과 ERROR 줄에 실제 로그 경로를 적는다.	키트 0.8.5 — rental-portal #101 병합 뒤 검증 실패 때 로그가 "No such file"
터미널 로그 색 (6장 변수 표, 부록 D)	루프가 터미널에 찍는 로그를 단계별 색으로 가른다 — 티켓 제목(앞에 빈 줄), 단계 전환, 성공, 되풀이·수리·건너뜀, 중단, 흐린 시각과 워커 화면 꼬리. `NO_COLOR` 나 파이프면 맨글자, 로그 파일에는 색 코드가 들어가지 않는다.	키트 0.8.4 — 긴 로그에서 지금 어느 단계인지 한눈에 안 보임
cmux·tmux 백엔드와 Windows (표지, 1·2·3·4·6·7·10·12장, 부록 A)	2.5절에 cmux·tmux 백엔드를 넣었다 — 루프가 백엔드에 시키는 아홉 가지 일의 대응표, 실행 조건(cmux 터미널 안, 메인 체크아웃 신뢰), `WORKTREE_SETUP_CMD` , 훅 기반 idle의 성질, 실측. 본문의 Orca 전용 표현을 백엔드 기준으로 고치고 3장 상태표에 cmux lane·tmux 창 이름을 더했다. 4장 끝에 Windows 절(WSL2 + tmux 백엔드, W1~W5)을 넣고 Orca는 선택으로 내렸다.	키트 0.8.2 — cmux(0.3)·tmux(0.8) 백엔드가 책에는 없었음, Windows 설치 질문
리포트 자동 열기 (7장 9단계, 6장 변수 표, 부록 D)	리포트를 쓴 뒤 기본 브라우저로 연다. `REPORT_OPEN=0` 으로 끄고, SSH 세션·화면 없는 Linux에서는 열지 않는다. 열기 실패는 루프 결과를 바꾸지 않는다.	키트 0.8.2 — 리포트 경로를 찾아 여는 한 단계를 줄임
스펙 리포트 (7장 9단계, 9·11·12장, 부록 A·D)	`--parent` 실행이 정상 종료하면 `REPORT_DIR/spec-<N>.html` (티켓별 상태·결과 브리프·경과 시간·모델·마친 시각)을 jq로 그린다. 티켓마다 `RECORDS_FILE` (JSONL)에 기록 한 줄(모델 포함, 중단은 `stopped`)을 남기고, 기록이 없는 티켓은 PR 시각으로 대신한다. `MODEL_REGEX` 추가.	묶음 결과를 로그 줄이 아니라 한 표로 보고 싶음
포트 대기 (7장 6단계, R10, 11장, 부록 D)	포트가 차 있으면 점유 프로세스와 기한을 한 줄 적고 30초마다 다시 본다. 기다리는 중과 멈춤을 문서에서 가른다. 시작 전 `lsof` 확인을 수칙에 넣었다.	사람이 띄운 개발 서버 때문에 4분을 조용히 기다려 멈춘 것으로 보임
`terminal_idle` (2.4, 7장 2단계, 부록 D)	타임아웃이 아닌 이상 응답은 터미널 읽기 재시도로 가른다 — 사람이 Orca에서 워커 워크트리를 열면 세션이 다시 붙으며 `wait` 가 그렇게 끝난다.	일하던 워커를 두고 루프가 죽음(Orca 1.4.224)
이름과 백엔드 (전체, 부록 D, 키트)	제목에서 Orca를 뺐다 — 설치 경로 `.agent-loop/` , 스킬 `setup-agent-loop` , 파일 `ticket-loop-guide.pdf` . 스크립트는 워크트리·에이전트 터미널에 하는 일을 백엔드 함수(`orca_*` , `cmux_*`)로 뽑고 `loop.env` 의 `BACKEND` 로 고른다. 이 책의 운영 설명은 Orca 백엔드 기준이다. cmux 백엔드의 실측 사실(명령 형태·lane·소켓 조건)은 2.5절과 키트 README에 있다.	키트 0.3 — Orca 외 터미널에서도 같은 루프
키트 이름 (전체, 부록 D, 키트)	키트 이름을 `agent-loop-kit` 에서 `ticket-loop-kit` 으로 — 설치 경로 `.ticket-loop/` , 스킬 `setup-ticket-loop` , 설정 브랜치 `chore/ticket-loop-setup` . 설치기가 0.3 이하 설치본(`.agent-loop/` · `.orca-loop/`)을 `git mv` 로 옮긴다.	키트 0.4 — "에이전트"보다 "티켓"이 이 루프의 단위라서
시작 전 포트 점검 (부록 D, 키트)	`POST_MERGE_PORT` 가 시작 시점에 이미 차 있으면 점유 프로세스와 함께 "주의" 한 줄을 적는다. 멈추지는 않는다 — 11장 수칙(`lsof` 확인)을 로그가 대신 상기시킨다.	키트 0.2.0
스펙 이슈 자동 닫기 (7장 5단계, 11장, 부록 D, 키트)	티켓을 닫은 뒤와 `--parent` 실행이 끝날 때, 스펙의 하위 이슈 목록을 세어 모두 닫혔으면 스펙 이슈를 코멘트와 함께 닫는다. 사람의 사후 검수에서 "스펙 이슈 닫기"가 빠졌다.	키트 0.5 — 회고: 하위 이슈가 다 끝나도 스펙이 열려 있음

스킬 묶음과 충돌 검사 (3장, 4장 5·6단계, 키트)	키트가 들이는 mattpocock 스킬을 26개 전부에서 두 묶음(core: `tdd` · `ticket-review` 항상, workflow: 기획·회고 7개 선택)으로 줄였다. 새 설치에만 적용되며, 지난 설치가 놓은 나머지는 kept로 유지·동기화한다(`--prune-skills` 로만 삭제). 설치 전에 플러그인· `~/.claude/skills` · `npx skills` · `code-review` 디렉터리와의 중복을 검사한다. `/setup-matt-pocock-skills` 는 더 필요 없다 — `docs/agents/issue-tracker.md` 는 키트 템플릿.	키트 0.6 — "이슈 구현 자동화 키트가 다른 사람의 방법론 스킬 26개를 강요하는 게 맞나"
회고 안내 (7장 "다음 티켓", 11장, 부록 D, 키트)	이번 실행에서 끝난 스펙이 있으면 루프가 끝날 때 `/retro` 로 돌아보기를 권하는 안내 한 줄과 읽을 자료를 남긴다. 회고를 자동으로 돌리지는 않는다 — `/retro` 는 사람이 후보를 보고 고르는 스킬이고, 환경 수정은 PR로 가야 하며, 워커 세션 기록은 다른 경로·머신에 있을 수 있어서다.	키트 0.6 — "루프가 끝났을 때 /retro로 최종 점검하는 단계를 넣으면 어떤가"
에이전트 프리셋 (3장 표, 12장 FAQ, 부록 D, 키트)	`AGENT` 가 `claude` | `codex` 프리셋이 되어 기동 명령·배너·모델 정규식·스킬 접두사(`SKILL_PREFIX` : `/implement` ↔ `$implement`)의 기본값을 정한다. 워커 프롬프트에서 Claude Code 도구 이름(`AskUserQuestion`)을 뺐다. 저장소 배치(`.claude/skills/`)는 그대로라 Codex는 아직 스킬을 못 본다 — 배치와 실측은 다음 단계.	키트 0.7 1단계 — "이 kit을 claude만이 아닌 codex에서도 사용할 수 있게"
스킬 배치 (3장 표·note, 4장, 12장 FAQ, 부록 B, 키트)	스킬 정본을 `.agents/skills/<이름>` 으로 옮기고 `.claude/skills/<이름>` 은 링크로 둔다(`npx skills` 배치). 설치기·동기화 스크립트가 0.6 이하 설치본을 `git mv` 로 옮긴다. `## Agent skills` 블록은 `AGENTS.md` 가 정본, `CLAUDE.md` 는 `@AGENTS.md` . `.codex/rules/ticket-loop.rules` 템플릿(기본 브랜치는 설치기가 채움), 키트 스킬 셋에 `agents/openai.yaml` . 설치기 `--agent` . 충돌 검사는 우리 링크를 정상으로 보고 다른 곳을 가리키는 링크만 차단한다.	키트 0.7 2단계
손댄 무거운 테스트 파일 1회 (4장 11단계, 7장 6단계, 12장, 부록 B, 키트)	워커 규칙과 `implement` · `pr` 이 "무거운 테스트는 돌리지 마라"에서 "자기가 추가·수정한 그 파일만 PR 직전 1회, 전체는 루프가 병합 뒤"로 바뀌었다. 키트 템플릿의 자리표는 `__HEAVY_TESTS__` (스위트)와 `__HEAVY_TEST_FILE__` (파일 하나 실행 명령)이다.	키트 0.5 — 회고: 워커가 새로 쓴 통합 테스트가 병합 뒤 검증에서 처음 깨져 루프가 멈춤

2판은 실제 저장소(Next.js 웹 앱, GitHub Free 비공개 저장소)에서 스펙 하나의 티켓 19건을 이 루프로 돌린 실측과 그 회고에서 나왔다.

바뀐 곳	내용	계기
절차의 위치 (3·8장, 부록 B)	워커 프롬프트의 7개 규칙을 저장소 소유 `implement` 스킬로 옮기고, 프롬프트에는 무인 전용 문장만 남겼다. `pr` 양식도 루프 PR에 맞춘 저장소 소유 사본으로 바꿨다.	대화형과 무인 절차가 갈라짐
리뷰 스킬 이름 (3·4장)	`code-review` → `ticket-review`	Claude Code 내장 `/code-review` 와 충돌
`terminal_idle` (2.4, 부록 D)	타임아웃 응답(`error.code: "timeout"` , exit 1)을 "닿지 못함"과 구별한다.	일하던 워커를 두고 루프가 죽음
병합 뒤 검증 (7장 6단계, R10)	`POST_MERGE_CMD` · `POST_MERGE_COPY` · `POST_MERGE_PORT` · `--skip-post-merge` 추가. 실패하면 멈춘다.	통합 테스트가 어디서도 돌지 않아 기본 브랜치가 깨진 채 배포됨
CI 제외 경로 (4장 8단계, R7)	`CI_SKIP_REGEX` 추가. 문서만 바뀐 PR은 체크 없이 병합한다.	워크플로 경로 필터 도입
대상 선택 (5장)	`--parent` · `--label` 추가. 담당자가 있는 이슈와 하위 이슈를 가진 스펙 이슈를 건너뛴다(스펙 라벨을 뗄 필요가 없어졌다).	스펙 단위 운영
로그와 요약 (6·7장)	`LOG_FILE` 기본값 `.ticket-loop/loop-MMDD.log` . 티켓마다 `#N 요약:` 한 줄(벽시계·CI·수리·idle 오판·병합 뒤 검증).	회고 자료가 없었음
워커 비용 (4장 11단계, FAQ)	검증 2층(수시 검사 + 정해진 두 번의 전체 묶음), 작은 스크린샷, 짧은 `CLAUDE.md` .	티켓당 전체 테스트 1~9회·build 2~8회
환경 (4장)	`git push` 자격 증명(키체인 대기) 주의, Orca 훅 설정 메뉴 경로, `deny` 중심 권한, 스킬 벤더링 시 커밋 기록, `.ticket-loop/*.log` 무시.	설치·운영 중 실측
복구 (10장)	판정표를 담당자 기준으로, R2에 push 대기 원인 추가, R10 신설, 진단 함수에 열린 선행 이슈 목록.	위 변경 반영
회고 (1·9·11장)	운영 리듬 끝에 `/retro` 와 환경 수정 PR을 넣었다.	—